



НТО

МАТЕРИАЛЫ ЗАДАНИЙ

Всероссийской междисциплинарной олимпиады школьников

«Национальная технологическая олимпиада»

по профилю

«Автономные транспортные системы»

2023/24 учебный год

<http://ntcontest.ru>

УДК 373.5.016:[656:681.5]

ББК 74.263.0

A22

Авторы:

В.С. Белкин, И.Е. Даниленко, М.В. Даньшина, Д.А. Казаков, А.А. Киселев,
И.А. Кобец, А.В. Коротицкий, М.В. Корячко, Н.Ю. Кузнецов, А.С. Суров,
А.В. Харитонов

A22 Всероссийская междисциплинарная олимпиада школьников 8-11 класса
«Национальная технологическая олимпиада». Учебно-методическое пособие
Том 2 **Автономные транспортные системы**
—М.: ООО «ВАШ ФОРМАТ», 2024. —218 с.

ISBN 978-5-00147-592-7

Данное пособие разработано коллективом авторов на основе опыта проведения всероссийской междисциплинарной олимпиады школьников 8-11 класса «Национальная технологическая олимпиада» в 2023/24 учебном году, а также многолетнего опыта проведения инженерных соревнований для школьников. В пособии собраны основные материалы, необходимые как для подготовки к олимпиаде так и для углубления знаний и приобретения навыков решения инженерных задач.

В издании приведены варианты заданий по профилю Национальной технологической олимпиады за 2023/24 учебный год с ответами, подробными решениями и комментариями. Пособие адресовано учащимся 8–11 классов, абитуриентам, школьным учителям, наставникам и преподавателям учреждений дополнительного образования, центров молодежного и инновационного творчества и детских технопарков.

Методические материалы также могут быть полезны студентам и преподавателям направлений, относящихся к группам:

01.00.00 Математика и механика

02.00.00 Компьютерные и информационные науки

09.00.00 Информатика и вычислительная техника

11.00.00 Электроника, радиотехника и системы связи

15.00.00 Машиностроение

23.00.00 Техника и технологии наземного транспорта

24.00.00 Авиационная и ракетно космическая техника

25.00.00 Аэронавигация и эксплуатация авиационной и ракетно-космической техники

27.00.00 Управление в технических системах

ISBN 978-5-00147-592-7

УДК 373.5.016:[656:681.5]

ББК 74.263.0



9 785001 475927 >

Оглавление

1 Введение	5
2 Автономные транспортные системы	17
I Работа наставника НТО на первом отборочном этапе	19
II Первый отборочный этап	20
II.1 Предметный тур. Информатика и информационные технологии	20
II.1.1 Первая волна. Задачи 8–11 класса	20
II.1.2 Вторая волна. Задачи 8–11 класса	32
II.1.3 Третья волна. Задачи 8–11 класса	43
II.2 Предметный тур. Физика	57
II.2.1 Первая волна. Задачи 8–9 класса	57
II.2.2 Первая волна. Задачи 10–11 класса	62
II.2.3 Вторая волна. Задачи 8–9 класса	67
II.2.4 Вторая волна. Задачи 10–11 класса	72
II.2.5 Третья волна. Задачи 8–9 класса	77
II.2.6 Третья волна. Задачи 10–11 класса	83
II.3 Инженерный тур	89
III Работа наставника НТО на втором отборочном этапе	104
IV Второй отборочный этап	105
V Работа наставника НТО при подготовке к заключитель-	

ному этапу	128
VI Заключение этап	129
VI.1 Предметный тур	129
VI.1.1 Информатика и информационные технологии. 8–11 классы	129
VI.1.2 Физика. 8–9 классы	145
VI.1.3 Физика. 10–11 классы	157
VI.2 Инженерный тур	174
VI.2.1 Общая информация	174
VI.2.2 Сюжет задачи	174
VI.2.3 Требования к команде и компетенциям участников	175
VI.2.4 Оборудование и программное обеспечение	175
VI.2.5 Описание задачи	177
VI.2.6 Материалы для подготовки	213
VII Критерии определения победителей и призеров	214
VIII Работа наставника после НТО	216

Введение

Национальная технологическая олимпиада

Всероссийская междисциплинарная олимпиада школьников «Национальная технологическая олимпиада» (далее — НТО) проводится в соответствии с распоряжением Правительства Российской Федерации от 10.02.2022 № 211-р при координации Министерства науки и высшего образования Российской Федерации и при содействии Министерства просвещения Российской Федерации, Министерства цифрового развития, связи и массовых коммуникаций Российской Федерации, Министерства промышленности и торговли Российской Федерации, Ассоциации участников технологических кружков, Агентства стратегических инициатив по продвижению новых проектов, АНО «Россия — страна возможностей», АНО «Платформа Национальной технологической инициативы».

Проектное управление Олимпиадой осуществляет структурное подразделение Национального исследовательского университета «Высшая школа экономики» — Центр Национальной технологической олимпиады. Организационный комитет по подготовке и проведению Национальной технологической олимпиады возглавляют первый заместитель Руководителя Администрации Президента Российской Федерации С. В. Кириенко и заместитель Председателя Правительства Российской Федерации Д. Н. Чернышенко.

Всероссийская междисциплинарная олимпиада школьников 8–11 класса «Национальная технологическая олимпиада» — это командная инженерная Олимпиада, позволяющая школьникам работать в 41-м инженерном направлении. Она базируется на опыте Олимпиады Кружкового движения НТИ и проводится с 2015 года, а с 2016 года входит в перечень Российского совета олимпиад школьников и дает победителям и призерам льготы при поступлении в университеты.

Всего заявки на участие в девятом сезоне (2023–24 гг.) самых масштабных в России командных инженерных соревнованиях подали более 141 тысячи школьников и студентов из всех регионов страны и семи зарубежных государств: Азербайджана, Белоруссии, Казахстана, Киргизии, Молдовы, Узбекистана и Черногории. Общий охват олимпиады с 2015 года превысил 660 000 участников. <https://journal.kruzhok.org/tpost/pggs3bp7y1-tehnologicheskaya-podgotovka-inzhenernih>



НТО способствует формированию профессиональной траектории школьников, увлеченных научно-техническим творчеством:

- определить свой интерес в мире современных технологий;
- получить опыт решения комплексных инженерных задач;
- осознанно выбрать вуз для продолжения обучения и поступить в него на льготных условиях.

Кроме того, НТО позволяет каждому участнику познакомиться с перспективными направлениями технологического развития и ведущими экспертами, а также найти единомышленников.

Ценности НТО

Национальная технологическая олимпиада — командные инженерные соревнования для школьников и студентов. Особое пространство Олимпиады создают общие ценности и смыслы, которые предлагается разделять всем: участникам, организаторам, наставникам, экспертам.

Основа всей олимпиады — это современное технологическое образование как новый уклад жизни в современном мире. Этот уклад подразумевает доступность качественного образования для каждого заинтересованного человека, возможность постепенно и непрерывно учиться и развиваться, совместно создавать среду, в которой гуманитарное знание и новые технологии взаимно дополняют друг друга. Это идеал будущего общества. Участники Олимпиады уже сейчас попадают в такое будущее.

Как организаторы мы надеемся, что принципы, заложенные в основу НТО, станут общими принципами для всех, кто имеет отношение к Олимпиаде.

Решать прикладные задачи, нацеленные на умножение общественного блага

В соревнованиях и подготовке к ним мы адаптируем реальные задачи современной науки и производства к знаниям и навыкам, которые могут освоить школьники и студенты. Задачи имеют прикладное значение для людей и не оторваны от реальности. Мы стремимся к тому, чтобы участники понимали, для чего нужно решать такие задачи, кому в мире станет лучше, если они будут решаться системно и профессионально. Ценность Олимпиады заключается в том, что здесь можно попробовать себя в этом, и найти единомышленников для решения подобных задач в будущем.

Создавать, а не только потреблять

Создание новых решений мы ставим выше стремления потреблять уже созданное. Создание ценности для других ставим выше поиска личной выгоды. Это не значит, что нужно забыть о себе и самоотверженно посвятить всю свою жизнь делу технологического прогресса. Но творчество всегда приносит большую радость, чем потребление. Это относится и ко всей олимпиаде.

Олимпиада — это общее дело организаторов, партнеров и участников. Способность принимать проблемы олимпиады как свои и пытаться решить их ценнее для творческого человека, чем желание найти недостатки в работе других.

Работать в команде

Способность работать в команде — это не только эффективная стратегия действия в современном мире. Работа в команде не отрицает наличия свободной воли каждого конкретного участника, его значимости и права на собственное мнение. Но в сообща мы стремимся достигнуть общей цели, опираясь на взаимное уважение всех участников, учитывая интересы и слабые и сильные стороны каждого.

Команды формируют целые сообщества, которые имеют сходные цели и ценности и могут очень многое, поскольку сильные горизонтальные связи помогают реализовывать самые дерзкие и амбициозные задачи. Это то, что нужно для технологического развития. Мы заняты построением такого сообщества и надеемся, что вы захотите стать его частью.

Осваивать и ответственно развивать новые технологии

Сообщество Национальной технологической олимпиады — часть Кружкового движения НТИ. Это прежде всего сообщество людей, увлеченных современными технологиями. Нас всех объединяет стремление разобраться в них, создать что-то новое и найти таких же увлеченных единомышленников.

Мы — часть сообщества технологических энтузиастов, и для нас границы возможностей технологий всегда подвижны. Именно поэтому просим не забывать об этике инженера и ученого, ответственности за свои изобретения перед людьми, которых это касается. Творя новое, не навреди!

Играть честно и пробовать себя

Мы признаем, что победа в соревнованиях важна и нужна. Но утверждаем, что для победы не все средства хороши и цель не является оправданием для грязной игры. Победа должна быть заслужена в рамках правил, единых для всех. Человек, который играет честно, не будет списывать, интриговать, подставлять других и заниматься прочей нездоровой конкуренцией.

Человек, который играет честно, — уважает себя, свою команду и соперников. Он принимает правила игры и в заданных рамках доказывает право на победу.

Мы бережем пространство Олимпиады как безопасное для всех участников. Это помогает искать себя, и при этом не бояться пробовать новые задачи, определять свой дальнейший путь, учиться на ошибках и каждый год становиться более сильным и подготовленным.

Быть человеком

Соревнования — это очень сложный и эмоционально насыщенный процесс. Что-бы он приносил радость и пользу всем, мы призываем всех участников вести себя порядочно и думать не только о себе.

Вежливость, эмпатия и забота — вот что сделает процесс комфортным и полезным для всех. Мы ценим уважение труда каждого человека и его позиции, бережное отношение к работе и жизни каждого. И просим отказаться от токсичной оценочной критики — она не решит ваши проблемы, а сделает хуже вам, другому и всей

Олимпиаде в целом.

Человек, который остается человеком, умеет признавать ошибки и отвечать за слова и дела перед другими. Здесь это ценят. Встав перед альтернативой между сиюминутной выгодой, капризом и общей целью соревнования — человек выберет последнее и поможет другим, организаторам и участникам, поддержать эту цель.

Важное замечание. Этот текст — живое выражение смыслов и ценностей Национальной технологической олимпиады. Он будет меняться вместе с развитием нашего сообщества. Авторы с благодарностью примут помощь от всех, кто чувствует сопричастность ценностям и готов включиться в их доработку.

Организационная структура НТО

НТО — межпредметная олимпиада. Спектр соревновательных направлений (профилей НТО) сформирован на основе актуального технологического пакета и связан с решением современных проблем в различных технологических отраслях. С полным перечнем направлений (профилей) можно ознакомиться на сайте НТО: <https://ntcontest.ru/tracks/nto-school/>.



Соревнования в рамках НТО проводятся по четырем направлениям:

1. НТО Junior для школьников (5–7 классы).
2. НТО школьников (8–11 классы).
3. НТО студентов.
4. Конкурс цифровых портфолио «Талант НТО».

В 2023/24 учебном году 28 профилей НТО включены в Перечень олимпиад школьников, утверждаемый Приказом Министерства науки и высшего образования Российской Федерации, а также в Перечень олимпиад и иных интеллектуальных и (или) творческих конкурсов, утверждаемый приказом Министерства просвещения Российской Федерации, что дает право победителям и призерам профилей НТО поступать в вузы страны без вступительных испытаний (БВИ), получить 100 баллов ЕГЭ или дополнительные 10 баллов за индивидуальные достижения. Преимущества при поступлении победителям и призерам НТО предлагают более 100 российских вузов.

НТО для старшеклассников проводится в три этапа:

- Первый отборочный этап — заочный индивидуальный. На данном этапе участникам предлагаются задачи по двум предметам, соответствующим тому или

иному профилю, а также задания, формирующие теоретические знания и представления по направлениям выбранных профилей.

- Второй отборочный этап — заочный командный. На данном этапе участникам предлагаются индивидуальные компетентностные и командные задачи, связанные с направлением выбранного профиля.
- Заключительный этап — очный командный. Этап представляет собой очные соревнования длительностью 5–6 дней, куда приезжают команды со всей страны, успешно справившиеся с двумя отборочными этапами, и решают комплексные прикладные инженерные задачи.

Профили НТО 2023/24 учебного года и соответствующий уровень РСОШ

Профили II уровня РСОШ

- Автоматизация бизнес-процессов
- Беспилотные авиационные системы
- Водные робототехнические системы
- Инженерные биологические системы
- Интеллектуальные робототехнические системы
- Нейротехнологии и когнитивные науки
- Технологии беспроводной связи

Профили III уровня РСОШ

- Автономные транспортные системы
- Анализ космических снимков и геопространственных данных
- Аэрокосмические системы
- Большие данные и машинное обучение
- Геномное редактирование
- Интеллектуальные энергетические системы
- Информационная безопасность
- Искусственный интеллект
- Летающая робототехника
- Наносистемы и наноинженерия
- Новые материалы
- Передовые производственные технологии
- Разработка компьютерных игр
- Спутниковые системы
- Технологии виртуальной реальности
- Технологии дополненной реальности
- Технологическое предпринимательство
- Умный город
- Фотоника
- Цифровые технологии в архитектуре
- Ядерные технологии

Профили без уровня РСОШ

- Научная медиакommunikация
- Программная инженерия в финансовых технологиях
- Современная пищевая инженерия
- Технологическое мейкерство
- Урбанистика
- Цифровое производство в машиностроении
- Цифровой инжиниринг в строительстве
- Цифровые сенсорные системы

Новые профили без уровня РСОШ

- Инфохимия
- Квантовый инжиниринг
- Технологии компьютерного зрения и цифровые сервисы
- Цифровая гидрометеорология
- Цифровое месторождение

Обратите внимание, что в олимпиаде 2024/25 года список профилей, в т.ч. входящих в РСОШ, и уровни РСОШ — могут поменяться.

Участие в НТО может принять любой школьник, обучающийся в 8–11 классе. Чаще всего Олимпиада привлекает:

- учащихся технологических кружков, любители инженерных и робототехнических соревнований;
- олимпиадников, которым интересны межпредметные олимпиады;
- фанатов и адептов передовых технологий;
- школьников, участвующих в хакатонах, проектных конкурсах и школах;
- будущих предпринимателей, намеревающихся найти на Олимпиаде единомышленников для будущего стартапа;
- увлекающихся школьников, которые хотят видеть предмет шире учебника.

Познакомить школьников с НТО и ее направлениями, замотивировать принять участие в НТО можно с помощью специальных мероприятий: Урок НТО и Дни НТО. Как педагогу провести Урок НТО, или как в образовательном учреждении организовать День НТО можно познакомиться в методических рекомендациях на сайте НТО. Там же можно выбрать и скачать необходимые уроки и подборки материалов по направлениям <https://nti-lesson.ru/>.



Участвуя в НТО, школьники получают возможность работать с практикоориентированными задачами в области прорывных технологий, собирать команды единомышленников, включаться в профессиональное экспертное сообщество, а также заработать льготы для поступления в вузы.

У НТО есть площадки подготовки по всей стране, которые занимаются привлечением участников и проводят мероприятия по подготовке к соревнованиям. Они могут быть открыты:

- в организациях общего и дополнительного образования;
- на базе частных кружков в области программирования, робототехники и иных технологий;
- в вузах;
- технопарках

и других организациях.

Каждое образовательное учреждение, ученики которого участвуют в НТО или НТО Junior, может стать площадкой подготовки к олимпиаде, что дает возможность включиться в Кружковое движение НТИ.

На сайте НТО размещены инструкции о том, как организация может стать площадкой подготовки: <https://ntcontest.ru/mentors/stat-ploshadkoi/>. Условия регистрации и требования к работе площадок подготовки обновляются вместе с развитием олимпиады. Обновленная версия размещается на сайте перед началом нового цикла олимпиады.



Наставники НТО

В НТО большое внимание уделяется работе с наставниками. Наставник НТО оказывает всестороннюю поддержку участникам Олимпиады, помогая решать организационные вопросы и развивать как технические знания и компетенции, так и социальные навыки, связанные с работой в команде.

Наставником может стать любой человек, которому интересно сопровождать участников и помогать им формировать необходимые для решения технологических задач компетенции и готовиться к соревнованиям. Это может быть преподаватель школы или вуза, педагог дополнительного образования, руководитель кружка, эксперт в технологической области, представитель бизнеса и т. п. Если наставнику не хватает собственных знаний, он может привлекать коллег и внешних экспертов и

поддерживать усилия и мотивацию учеников, которые разбирают задачи самостоятельно. На данный момент сообщество наставников НТО включает в себя более 7 тысяч человек.

Главная задача наставника — выстроить комплексную структуру подготовки к Олимпиаде в течение всего учебного года. В области ответственности наставника находится поддержка мотивации участников и помощь в решении возникающих проблем. Не менее важно зафиксировать цели и ожидания от предстоящих соревнований, что поможет оценить прирост профессиональных компетенций, личных и командных навыков за время подготовки.

Примеры организационных задач, которые стоят перед наставником НТО:

- Информирование и работа с мотивацией. На этапе регистрации на Олимпиаду наставник привлекает участников, рассказывая, что такое НТО и какие преимущества она предлагает. Наставнику необходимо разобраться в устройстве НТО, этапах и расписании этапов, а также изучить профили, чтобы помочь каждому ученику выбрать наиболее перспективные и интересные для него направления.
- Формирование программы подготовки. Наставник составляет график подготовки к НТО и следит за его реализацией, руководя процессом подготовки учеников.
- Отслеживание сроков. Наставник следит за сроками проведения этапов НТО и напоминает участникам о необходимости своевременной загрузки решений на платформу.

Примеры задач наставника, связанных с непосредственной подготовкой к соревнованиям:

- Анализ компетенций участников. Наставник вместе с учениками оценивает компетенции, которые необходимы для успешного участия в НТО, выявляет нехватку знаний и навыков и отбирает материалы и задачи, которые ученикам нужно изучить и решить.
- Содержательная подготовка к первому и второму отборочному этапу. Наставник вместе с учениками изучает материалы для подготовки, рекомендованные разработчиками выбранных профилей, а также разбирает и решает задачи НТО прошлых сезонов. Рекомендуется использовать записи вебинаров, материалы и онлайн-курсы профилей.
- Содержательная подготовка к заключительному этапу. Наставник может использовать разборы задач заключительного этапа прошлых лет, а также следить за расписанием подготовительных очных и дистанционных мероприятий и рекомендовать ученикам их посещать.

Примеры задач наставника в области развития социальных навыков, связанных с развитием личной эффективности и взаимодействия с другими участниками:

- Формирование команд. Второй отборочный этап НТО проходит в командном формате. Наставник помогает ученикам сформировать эффективную команду с оптимальным распределением ролей. В ряде случаев он может содействовать в поиске недостающих участников команды, в том числе в других городах и стать наставником такой команды, коммуникация в которой осуществляется через web-сервисы.
- Отслеживание прогресса и анализ полученного опыта. Наставник проводит ре-

флексию прогресса отдельных участников и команды по результатам каждого этапа НТО и после завершения участия в соревнованиях. Это помогает участникам оценить свое движение по траектории соревнований, сильные и слабые стороны, сформулировать, каких компетенций не хватило для более высокого результата и как их можно улучшить в будущем.

- Поддержка и мотивирование участников. Наставник поддерживает интерес учеников к соревнованиям, а также помогает им сохранять высокую мотивацию, что особенно важно, если команда показала результаты хуже, чем ожидалось.
- Выстраивание индивидуальной образовательной траектории. Наставник может помочь ученикам осознанно создать собственную траекторию развития, в том числе вне НТО: подбор обучающих курсов и соревнований, выбор вуза и направления дальнейшего обучения.

Поддержка наставников НТО

Работе наставников посвящен отдельный раздел на сайте НТО: <https://ntcontest.ru/mentors/>.



Для систематизации знаний и подходов к работе наставников в рамках инженерных соревнований разработан курс «Дао начинающего наставника: как сопровождать инженерные команды»: <https://stepik.org/course/124633/promo>. Курс формирует общие представления о работе наставников в области подготовки участников к инженерным соревнованиям.



Для совершенствования профессиональных компетенций по направлениям профилей разработан курс «Дао наставника: как развивать технологические компетенции»: <https://stepik.org/course/186928/promo>.



Наставникам для ведения занятий с учениками предлагаются образовательные программы, разработанные на основе восьмилетнего опыта организации подготовки к НТО. В настоящий момент такие программы представлены по 10-ти передовым технологическим направлениям:

- компьютерное зрение;
- геномное редактирование;
- водная, летающая и интеллектуальная робототехника;
- машинное обучение и искусственный интеллект;
- нейротехнологии;
- беспроводная связь, дополненная реальность:

и др.

<https://ntcontest.ru/mentors/education-programs/>.



Регистрируясь на платформе НТО, наставники получают доступ к личному кабинету, в котором отображается расписание отборочных соревнований и мероприятий по подготовке, требования к знаниям и компетенциям при решении задач отборочных этапов.

Формируется сообщество наставников НТО. Ежегодно Кружковое движение НТИ проводит Всероссийский конкурс технологических кружков: <https://konkurs.kruzhok.org>, принять участие в котором может каждый наставник. По итогам конкурса кружки-участники размещаются на Всероссийской карте кружков: <https://map.kruzhok.org>.



В 2022 году был разработан Навигатор для наставников команд или отдельных участников НТО: <https://www.notion.so/bdlv/5a1866975c2744728c2bd8ba80d21ec2>.



Навигатор ориентирован на начинающих наставников и помогает погрузиться в работу с НТО. Опытным наставникам Навигатор может быть полезен как сборник важных рекомендаций и статей:

- Смогут ли мои ученики принять участие в НТО.
- Как наставнику зарегистрироваться в НТО.
- Как помочь участникам выбирать профили.
- Что можно успеть сделать, если я и мои ученики начнем участвовать с нового учебного года.
- Как убедить руководство включиться в НТО.
- Что важно знать, начиная подготовку школьников.
- Как организовать подготовку.
- Как проводить рефлексию.
- Как мотивировать участников.
- Как работать с командой участников НТО.

Организаторы Олимпиады также оказывают экспертно-методическую поддержку сообществу наставников. Были разработаны методические рекомендации для наставников: «Технологическая подготовка инженерных команд»: <https://journal.kruzhok.org/tpost/pggs3bp7y1-tehnologicheskaya-podgotovka-inzhenernih>. Рассмотрены особенности подготовки к 5-ти направлениям:

- Большие данные.
- Машинное обучение.

- Искусственный интеллект.
- Спутниковые системы.
- Летаящая робототехника.



Для наставников НТО разработан и постоянно пополняется страница с материалами для профессионального развития: <http://clc.to/for-mentor>.



Автономные транспортные системы

Организаторы профиля: ООО «Академия Высоких Технологий», Московский политехнический университет и ГК «Геоскан».

Профиль «Автономные транспортные системы» посвящен применению технологий искусственного интеллекта в задачах беспилотного транспорта. Внедрение беспилотников в информационную инфраструктуру города позволяет тонко управлять ресурсом техники, предсказывать возможные развития событий и своевременно реагировать на события в городе.

Финалистам на заключительном этапе предстоит решить комплексную инженерную задачу: создать, отладить и внедрить в интеллектуальную городскую инфраструктуру (представленную специальным полигоном с моделью города) полностью автономную мультимодальную транспортную систему для доставки груза от сервисного центра до конечного адресата без вмешательства человека. Данная транспортная система включает в себя три взаимодействующих друг с другом автономно устройства, программируемые участниками: беспилотный автомобиль, автоматизированный сервисный центр, квадрокоптер. Наземный и воздушный беспилотный транспорт не только перевозит грузы, но и собирает информацию о текущей ситуации в городе, самостоятельно рассчитывает маршрут, insпектирует состояние инфраструктуры, реагирует на происшествия на дорогах города.

Сквозная технология профиля — компьютерное зрение. В течение всех этапов участники учатся применять ее в различных отраслях:

- локальное и глобальное позиционирование беспилотного автомобиля, детектирование и взаимодействие с объектами городской среды — дорожными знаками, светофорами, пешеходами, дорожной разметкой;
- навигация квадрокоптера по графическим меткам и различным визуальным маркерам;
- анализ больших данных, собираемых беспилотниками;
- распознавание маркировки для решения задач промышленной робототехники.

В первом отборочном дистанционном этапе участники решают задачи инженерного и предметного туров. Задачи первого тура знакомят участников с тематикой профиля и вовлекают в изучение образовательной программы профиля.

Задачи предметного тура определяют общий уровень подготовки участников по школьным предметам: информатика и физика. Задачи по информатике проверяют у участников знания по комбинаторике, теории вероятностей, алгоритмизации, теории игр и программированию. Задачи по физике относятся к разделам механика, оптика и электричество.

В ходе решения задач инженерного тура школьники должны продемонстрировать умение применять знания, полученные в первой половине образовательной программы профиля. Данные задания посвящены представлению изображения в памяти компьютера, использованию различных цветовых пространств, пороговой и адаптивной бинаризации, поиску и анализу контуров, детектированию объектов по цветам, распознаванию, сравнением с шаблоном, обработке видеопотока. Количество попыток сдачи решения задач не ограничено. Таким образом, в ходе первого отборочного

этапа участники профиля «Автономные транспортные системы» получают основополагающие знания и умения по технологиям профиля, а также готовятся к решению задач следующего этапа.

Второй отборочный этап посвящен отработке навыков программирования, созданию и обучению нейросетей, работе с алгоритмами компьютерного зрения для детектирования, отслеживания и распознавания объектов. В ходе решения задач второго этапа в распределенных командах учащиеся сталкиваются с отдельными элементами комплексной инженерной задачи заключительного этапа и таким образом могут освоить умения, необходимые для ее решения. Работа на втором этапе помогает участникам лучше понять сильные стороны каждого члена команды и наиболее эффективно распределять обязанности и зоны ответственности.

Организаторы разработали для участников программу подготовки к профилю «Автономные транспортные системы»: https://avt.global/nto_program.

Программа включает в себя следующие материалы:

- курс по основам программирования на Python: <https://stepik.org/course/67/promo>;
- продвинутый курс по программированию на Python: <https://stepik.org/course/512/promo>;
- урок НТО «Введение в компьютерное зрение»: <https://avt.global/nto-lesson-cv>;
- видеокурс по компьютерному зрению: <https://avt.global/cv>;
- видеокурс по нейронным сетям: <https://avt.global/neuralnets>.

Программа для подготовки выложена в открытый доступ. Организаторы профиля проводят регулярные консультации с участниками в формате онлайн и отвечают на возникающие вопросы в чате участников в Телеграм и в сообществе в Discord.

Призеры, победители и финалисты профиля поступают в ведущие ВУЗы страны и ведут в них проектную деятельность по инженерным направлениям как в области искусственного интеллекта, так и в смежных направлениях. Компетенции, полученные за время участия в профиле, востребованы в дальнейшей проектной работе в ВУЗах, а также применяются в отрасли — команда молодых ученых разработала беспилотного робота-курьера. Примером применения полученных знаний и умений в области, не связанной с беспилотным транспортом, является COVID Challenge. В рамках мероприятия участники обучили нейронную сеть, определяющую наличие инфекции COVID-19 у сотен пациентов города Москвы по трехмерным снимкам компьютерной томографии легких.

Работа наставника НТО на первом отборочном этапе

На первом отборочном этапе НТО участникам предлагаются задачи по предметам, соответствующим выбранным профилям. Для подготовки к первому отборочному этапу Олимпиады наставник может использовать следующие рекомендуемые форматы и мероприятия:

- Разбор задач первого отборочного этапа НТО прошлых лет.
- Мини-соревнования по решению задач предметных олимпиад муниципального уровня.
- Углубленные занятия по разделам предметов в соответствии с рекомендациями разработчиков профилей.

Для проверки, самостоятельного решения или проведения мини-соревнований могут использоваться предметные курсы НТО на платформе Stepik. Также возможно привлечение других преподавателей-предметников для проведения занятий в случае, если у наставника недостаточно компетенций в области предметных олимпиад.

Инженерный тур состоит из курса или теоретических материалов, погружающих участников в тематику профиля, и теоретических и практических заданий, как правило связанных с теорией.

Первый отборочный этап

Предметный тур. Информатика и информационные технологии

Первая волна. Задачи 8–11 класса

Задача II.1.1.1. Авиакомпания (9 баллов)

Темы: базы данных.

Условие

Даны фрагменты двух таблиц базы данных некоторой авиакомпании. Исходя из информации данных таблиц, определите, сколько человек вылетели из Москвы в интервале от 12 до 18 часов за 05.07.2023 и 06.07.2023.

Обратите внимание, что в разные даты один и тот же номер рейса может иметь разные пункты вылета и пункты прилета.

Таблица II.1.1: passengers

id	first_name	last_name	birth	document	flight_num	flight_date	status
1	Ivan	Ivanov	25.05.1999	*****	104	05.07.2023	True
2	Anna	Smirnova	24.05.2002	*****	104	05.07.2023	False
3	Ekaterina	Kuznetsova	04.02.1996	*****	105	05.07.2023	True
4	Aleksandr	Popov	06.04.1994	*****	103	05.07.2023	True
5	Elena	Vasilieva	03.11.1994	*****	104	05.07.2023	False
6	Sergei	Petrov	25.06.1984	*****	103	05.07.2023	False
7	Daniil	Sokolov	07.12.2000	*****	101	06.07.2023	True
8	Anastasia	Mikhailova	15.12.2002	*****	103	05.07.2023	True
9	Mikhail	Novikov	05.02.1993	*****	105	05.07.2023	True
10	Elizaveta	Fedorova	18.05.2004	*****	102	05.07.2023	True
11	Evgeniy	Morozov	26.09.2001	*****	101	05.07.2023	True
12	Semen	Volkov	16.08.1988	*****	103	05.07.2023	True
13	Vladislav	Alekseev	18.07.1981	*****	102	05.07.2023	True
14	Maksim	Lebedev	20.03.1988	*****	104	05.07.2023	False
15	Aleksandra	Semenova	27.06.1998	*****	102	05.07.2023	True
16	Kristina	Egorova	03.06.1999	*****	101	05.07.2023	True
17	Arina	Pavlova	21.05.1983	*****	102	05.07.2023	True
18	Dmitriy	Kozlov	07.05.1982	*****	101	06.07.2023	False
19	Danil	Stepanov	02.08.1986	*****	101	06.07.2023	True
20	Anna	Nikolaeva	20.04.1981	*****	101	05.07.2023	True
21	Rostislav	Orlov	27.03.1987	*****	101	06.07.2023	False

Таблица II.1.2: `flights`

id	flight_num	departure	arrival	flight_date	flight_date	status
1	101	Moscow	Kazan	05.07.2023	14:00	True
2	102	Moscow	Sochi	05.07.2023	15:30	False
3	103	Vladivostok	Novosibirsk	05.07.2023	09:00	True
4	104	Moscow	Ufa	05.07.2023	17:20	True
5	105	Moscow	Saint Petersburg	05.07.2023	19:00	True
6	101	Kazan	Kaliningrad	06.07.2023	11:15	True

Таблица `passengers` является информацией о пассажирах, которые приобрели билеты на рейсы данной авиакомпании.

В колонках:

- `id` — номер записи в таблице;
- `first_name` — имя пассажира;
- `second_name` — фамилия пассажира;
- `birth` — дата рождения;
- `document` — номер документа, по умолчанию в авиакомпании он скрыт;
- `flight_num` — номер рейса, на который пассажир приобрел билет;
- `flight_date` — дата вылета рейса;
- `status` — активен ли статус пассажира на данный рейс, если `True` — пассажир полетит (или уже полетел), `False` — билет был сдан.

Таблица `flights` является информацией о рейсах авиакомпании.

В колонках:

- `id` — номер записи в таблице;
- `flight_num` — номер рейса;
- `departure` — город вылета;
- `arrival` — город прилета;
- `flight_date` — дата вылета рейса;
- `departure_time` — время вылета рейса;
- `status` — активен ли статус рейса, если `True` — будет выполнен (или уже выполнен), `False` — рейс отменен.

Решение

Исходя из условия задачи, выберем те рейсы, которые подходят, их всего два.

101	Moscow	Kazan	05.07.2023	14:00	True
104	Moscow	Ufa	05.07.2023	17:20	True

Далее идем по таблице и ищем всех людей, которые летят 05.07.2023 номерами рейсов 101 или 104 со статусом `True`.

Людей с номером рейса 101, но датой вылета 06.07.2023 в расчет не берем, так как этот рейс не вылетает из Москвы.

1	Ivan	Ivanov	25.05.1999	*****	104	05.07.2023	True
11	Evgeniy	Morozov	26.09.2001	*****	101	05.07.2023	True
16	Kristina	Egorova	03.06.1999	*****	101	05.07.2023	True
20	Anna	Nikolaeva	20.04.1981	*****	101	05.07.2023	True

Ответ: 4.

Задача II.1.1.2. Вечный XOR (9 баллов)

Темы: алгебра логики.

Условие

Дано число 11011001 в двоичной системе счисления. К данному числу применяется операция XOR на другое, неизвестное нам, восьмизначное число в двоичной системе счисления. После операции выполняется проверка: если результат операции меньше восьмизначного, к нему дописываются незначащие нули. Такой проверкой мы поддерживаем восьмизначный формат числа. После этого операция XOR и проверка выполняются снова в той же последовательности так до бесконечности...

Определите восьмизначное неизвестное число, которое применяется в операции XOR, если известно, что на 127 применении операции в этом алгоритме результат до проверки был равен 1100011.

Решение

Заметим одну интересную особенность функции XOR: если взять результат операции XOR числа 217 и любого числа x (допустим 3) и к результату вновь применить операцию XOR с числом x (в нашем случае 3), то мы вернемся к исходному числу.

$$\begin{array}{rcl}
 11011001 & = & 217 \\
 \wedge & & \\
 1100011 & = & 99 \\
 \hline
 10111010 & = & 186
 \end{array}$$

Получается, что на 127-й по счету операции XOR, то есть нечетной, будет получено промежуточное число, которое по условию равно 1100011 или 99.

Осталось лишь узнать неизвестное число x , которое будет давать 99 в результате XOR с исходным числом 217.

Для этого можно узнать результат XOR между числами 217 и 99.

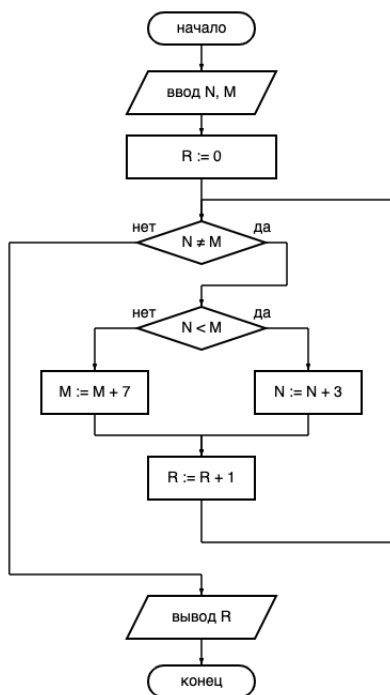
Ответ: 186.

Задача II.1.1.3. Сколько раз (11 баллов)

Темы: анализ алгоритмов.

Условие

Дана блок-схема алгоритма. Какое число будет выведено, если на вход были поданы $N = 41$ и $M = 57$.

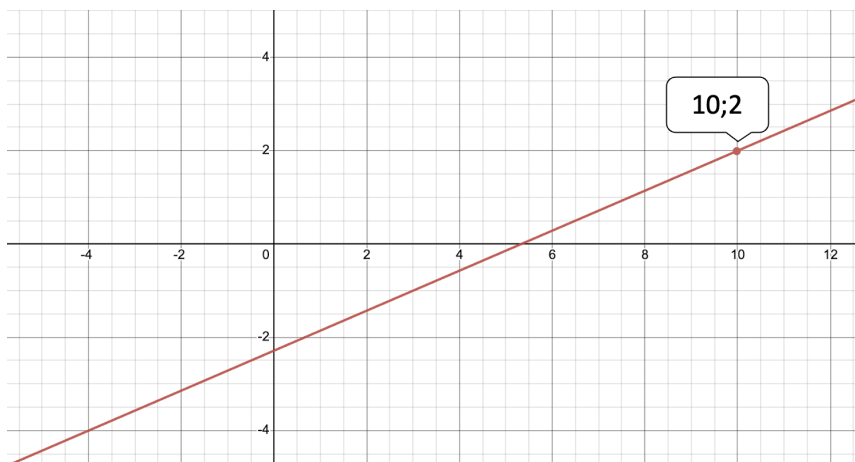


Решение

Начальные значения чисел $n = 41$ и $m = 57$. Как видно из алгоритма, программа будет прибавлять 3 к числу n (если $n < m$) и прибавлять 7 к числу m (если $m < n$) до тех пор, пока эти числа не станут равны. Значит сумма, прибавленная к числу n должна быть больше суммы прибавленной к числу m на $57 - 41 = 16$, из чего можно составить уравнение:

$$3x - 7y = 16.$$

Отсюда можно подобрать два таких целых, минимальных x и y , при которых это уравнение будет верно. Также можно построить график и найти, где он впервые проходит через целые положительные координаты.



Раз $x = 10$, а $y = 2$ то суммарное количество операций будет равно 12.

Ответ: 12.

Задача II.1.1.4. Дорога до работы (11 баллов)

Темы: графы.

Условие

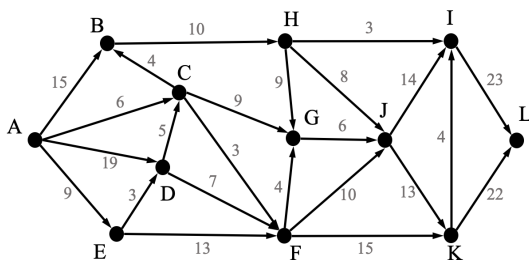
На рисунке приведена схема района «Северный», где каждая вершина графа, показанная латинскими буквами от A до L , обозначают объекты его инфраструктуры, а ребра — дороги между ними.

Гарантируется, что никаких других путей в этом районе нет и что двигаться можно лишь по направлению ребер, которое указано стрелками.

Рядом с каждой дорогой указана ее пропускная способность, которая показывает предельное количество машин, проходящих через эту дорогу за единицу времени.

Буквой A обозначен новый жилой комплекс, а буквой L — IT-парк, в который все ездят на работу с утра.

Ваша задача узнать — какое максимальное количество машин может проходить утром по дорогам этого района в единицу времени или же максимальную пропускную способность данного графа.

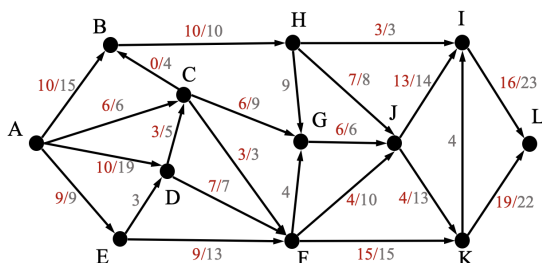


Решение

Для решения этой задачи воспользуемся теоремой **max-flow min cut** о максимальном потоке и минимальном разрезе, которая утверждает, что в сети потоков максимальный объем потока, проходящего от истока к стоку, равен общему весу ребер в минимальном разрезе, т. е. наименьший общий вес ребер, удаление которых отключило бы исток от стока.

Самым минимальным разрезом является удаление ребер BH , CF , DF , AE и GJ с суммой $10 + 3 + 7 + 9 + 6 = 35$, все другие разрезывающие исток от стока будут иметь большую сумму.

Стоит отметить, что данную задачу можно было решить и используя алгоритм Форда-Фалкерсона.



Ответ тоже получится $16 + 19 = 35$.

Ответ: 35.

Задача II.1.1.5. Уличный транспорт (14 баллов)

Темы: кодирование.

Условие

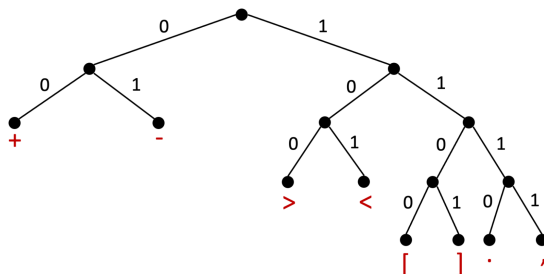
Даня и Ваня на уроке информатики получили очень странное задание. В нем им необходимо определить, какое минимальное количество информации будет содержать сообщение о способе перемещения случайного прохожего. Для этого они

простояли целые сутки на улице и поняли, что прохожие в основном передвигаются одним из пяти следующих вариантов: пешком, на самокате, велосипеде, скейтборде или роликах. Так как Ваня опаздывал на свидание, он решил, что все варианты транспорта равновероятны и убежал. Но Дания заметил одну особенность: пешеходы встречаются с вероятностью 50%, а прохожие на самокате, велосипеде, роликах и скейтбордах с вероятностью 12,5% каждый.

На сколько бит количество информации, содержащееся в сообщении о транспорте прохожего, которое посчитает Ваня, будет отличаться от количества информации, рассчитанного Данией?

Решение

При равномерном посимвольном кодировании Вани: мощность алфавита равна 8, так как всего 8 команд, для кодирования которых по формуле Хартли потребуется 3 бита. Всего в программе 100 символов, а значит, вся программа будет весить $3 \cdot 100 = 300$ бит при неравномерном кодировании Дани.



Тогда получается, что программа будет весить $2 \cdot 2 \cdot 32 + 3 \cdot 2 \cdot 6 + 4 \cdot 6 = 260$ бит, что на 40 меньше веса, полученного Ваней.

Ответ: 40.

Задача II.1.1.6. Нули и единицы (14 баллов)

Темы: системы счисления.

Условие

Существует два целых числа x и y , удовлетворяющих выражению $x = 2^i - 1$, $y = 2^j - 1$, где $x \in [1; 64]$.

Определите, сколько существует вариантов выбрать x и y при следующих условиях:

1. $x > y$.
2. Произведение данных чисел в двоичной записи содержит хотя бы одну единицу и хотя бы один ноль.

Зная, что любое число $z = 2^n - 1$, где $n \in \mathbb{N}$, выглядит как n единиц:

$$2^4 - 1 = 1111_2$$

Кроме того, заметим одно интересное свойство, что если перемножать числа та-

Например:

Получается, что для $i \equiv 2$ в паре можно взять любое $i \in [3; 15]$, и такое правило

Выходит, что на 50 вариантов взятия j существует по 13 вариантов взятия i , а это уже $50 \cdot 13 = 650$ пар.

Для каждого $j \geq 52$ количество возможных i будет уменьшаться, так как i не может быть больше 64, а значит, для $j = 52$ будет всего 12 возможных вариантов i , для $j = 53$ будет всего 11 возможных вариантов i и так далее, что выливается в арифметическую прогрессию: $12 + 11 + \dots + 2 + 1 = 78$ вариантов.

Всего получается $650 + 78 = 728$ пар.

Ответ: 728.

Задача II.1.1.7. Кольцевой сборщик (17 баллов)

Темы: программирование.

Условие

На некотором заводе решили расфасовать детали. Каждая деталь имеет свой размер, выраженный как целое число. Для фасовки сотрудники взяли кольцевой сборщик. Кольцевой сборщик — это некий механизм с ячейками разного размера, в которые можно положить деталь. Изначально выбрана для приема детали ячейка под номером 1, каждую секунду она сдвигается на следующую: через секунду будет выбрана для приема ячейка под номером 2, через две секунды — под номером 3 и так далее... Если сборщик дойдет до последней ячейки, он на следующем шагу окажется на ячейке под номером 1. Чтобы разместить деталь в сборщик, необходимо, чтобы размер выбранной для приема ячейки был равен размеру детали. Всего необходимо погрузить n деталей, каждая — имеет свой уникальный размер от 1 до n включительно. Сами они загружаются в сборщик по возрастанию, сначала с размером 1, потом с размером 2, и так далее до размера n включительно. Работники завода попросили у Вас помощи. Они сообщили вам, сколько у них деталей, а также порядок ячеек в кольцевом сборщике, и просят Вас написать программу, которая рассчитает, через сколько все детали будут погружены в кольцевой сборщик. Считайте, что деталь укладывается в кольцевой сборщик моментально.

Формат входных данных

В первой строке входных данных записано число n ($1 \leq n \leq 10^5$) — количество деталей.

Во второй строке записано n целых чисел s_i ($1 \leq s_i \leq n$) — последовательность размеров ячеек в кольцевом сборщике. Все размеры ячеек являются уникальными числами.

Выбранной при старте ячейкой считать первое число последовательности.

Формат выходных данных

Выведите одно число — количество времени, затраченное на расфасовку всех деталей.

Методика проверки

Программа проверяется на 20 тестах. Прохождение каждого теста оценивается в 0,5 балла. Тесты из условия задачи при проверке не используются.

Примеры

Пример №1

Стандартный ввод
4
3 1 4 2
Стандартный вывод
6

Пояснения к примеру

Первая на вход идет деталь с размером 1, чтобы добраться до ячейки с размером 1 необходимо затратить одну су: $3 \rightarrow 1$. Следующая на вход идет деталь с размером 2, чтобы добраться до ячейки с размером 2 необходимо затратить две сы: $1 \rightarrow 4 \rightarrow 2$. Следующая на вход идет деталь с размером 3, чтобы добраться до ячейки с размером 3 необходимо затратить одну су: $2 \rightarrow 3$. Последняя на вход идет деталь с размером 4, чтобы добраться до ячейки с размером 4 необходимо затратить две сы: $3 \rightarrow 1 \rightarrow 4$. Итого было затрачено на расфасовку всех деталей 6 с.

Решение

Заведем отдельный список/словарь, который в качестве индексов будет использовать размеры деталей, а в качестве значений — индексы ячеек для деталей на ленте. Так как детали укладываются последовательно, пройдем циклом по деталям размерами от 1 до N включительно. На момент начала укладки мы расположены над ячейкой под номером 1. Для вычисления времени до нужной нам ячейки, зная, что лента меняет ячейку каждую секунду, воспользуемся следующей формулой «точка расположения ячейки для нужной детали — наше нынешнее положение». Тем самым мы вычислим расстояние до ячейки, что и будет эквивалентно в рамках нашей задаче времени до ячейки.

Если точка расположения нашей ячейки находится позади нашей позиции, проверен полный круг по ленте, вернувшись в стартовое положение, и добавим расстояние до нужной ячейки: « N — наше нынешнее положение + точка расположения ячейки для нужной детали». После каждого перемещения по ленте обновляем нынешнюю позицию на точку, до которой мы дошли на этом шагу: «наше нынешнее положение = точка расположения ячейки для нужной детали». Суммируем все рассчитанные расстояния и получаем полное время, за которое мы обойдем всю ленту и уложим все детали.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  n = int(input())
2  indexes = dict()
3  arr = list(map(int, input().split()))
4  for i in range(n):
5      indexes[arr[i]] = i
6      current_index = result = 0
7  for i in range(n):
8      if current_index < indexes[i + 1]:
9          result += indexes[i + 1] - current_index
10     else:
11         result += n + indexes[i + 1] - current_index
12     current_index = indexes[i + 1]
13  print(result)

```

Задача II.1.1.8. Кредиты в банке (17 баллов)

Темы: программирование.

Условие

В некотором банке регулярно проходит огромное количество транзакций в сутки. Все эти транзакции (без указания личных данных клиентов) отображаются в логах банка. Это сделано для того, чтобы можно было анализировать количество денег, которые клиенты внесли в банк. Как вы знаете, банки обладают возможностью выдавать кредиты своим клиентам, но они их выдают из денег, которые вложили другие клиенты. И, естественно, чтобы выдать кредит, банк должен иметь в наличии ту сумму, на которую он это хочет сделать. Нормальной системы контроля денег у банка, о котором у нас в задаче идет речь, нет, поэтому они это делают через логи. Они узнают по ним гарантированное количество уникальных денежных единиц, которое было зафиксировано, и тем самым определяют гарантированную сумму, которую могут выдать в кредит.

Вам был дан некий отрезок из логов этого банка. Каждый клиент закодирован уникальным номером. Определите, какое гарантированное количество уникальных денежных единиц есть у банка на кредит. Для подробного понимания, как высчитывается гарантированная величина уникальных денежных единиц, смотрите пояснение к примеру.

Формат входных данных

На вход программе в первой строке поступает целое число n $1 \leq n \leq 10^5$ — количество операций в логах. В следующих n строках записано по три целых числа *from* ($1 \leq \text{from} \leq 500$), *to* ($1 \leq \text{to} \leq 500$), $\text{from} \neq \text{to}$, и *amount* ($1 \leq \text{amount} \leq 10^9$) — клиенты, которые отправили и получили деньги соответственно, а также количество денежных единиц.

Формат выходных данных

Программа должна вывести одно число — гарантированное количество уникальных денежных единиц, которые были зафиксированы по логам.

Примеры

Пример №1

Стандартный ввод
3
1 2 50
2 3 30
3 1 40
Стандартный вывод
60

Пояснение к примеру

В примере первая транзакция производится между клиентами 1 и 2 на величину 50 денежных единиц. До этого эти деньги не были в логах, а значит, это 50 уникальных денежных единиц. Далее идет транзакция между клиентами 2 и 3 на величину 30 денежных единиц. Как мы знаем из первой транзакции, у клиента под номером 2 есть 50 денежных единиц, и, соответственно, эти 30 денежных единиц могли быть пересланы из этих 50, поэтому мы не можем заявлять, что это гарантировано уникальные денежные единицы. В случае, если клиент 2 отправит 30 денежных единиц клиенту 3, то у него может остаться $50 - 30 = 20$ денежных единиц. Следующая транзакция происходит между клиентами 3 и 1 на величину 40 денежных единиц. Так как у клиента 3 нам известно только 30 денежных единиц, которые были отправлены от клиента 2, то оставшиеся $40 - 30 = 10$ будут уникальными единицами денег, так как до этого о них речь нигде в логах не шла. Итого, у нас получается $50 + 10 = 60$ гарантировано уникальных денежных единиц.

Решение

Заведем некий список/словарь, который будет хранить, сколько на данный момент у клиентов денег, которые нам известны, а также переменную, в которую будем записывать количество уникальных денег. Изначально мы не знаем ни одной транзакции, следовательно, про каждого клиента мы знаем о наличии 0 денег. Запускаем цикл, в котором обрабатываем каждую транзакцию следующим образом: от отправителя мы вычитаем сумму денег, которая указана в переводе, которую он отправил, а получателю их начисляем. Если счет отправителя становится отрицательным, следовательно, были отправлены деньги, о которых мы ранее не знали, следовательно, обновляем значение уникальных денег, добавляя модуль отрицательного баланса (той части денег, о которых мы ранее не знали). После этого запишем на баланс отправителя, что у него 0 денег, так как больше нет неизвестных денег. Обработав все транзакции таким образом, в конце выводим переменную с количеством уникальных денег.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  n = int(input())
2  bank_accounts = dict()
3  unique_moneys = 0
4  for i in range(n):
5      from_user, to_user, amount = map(int, input().split())
6      if to_user not in bank_accounts:
7          bank_accounts[to_user] = 0
8      bank_accounts[to_user] += amount
9      if from_user not in bank_accounts:
10         bank_accounts[from_user] = 0
11     bank_accounts[from_user] -= amount
12     if bank_accounts[from_user] < 0:
13         unique_moneys += abs(bank_accounts[from_user])
14         bank_accounts[from_user] = 0
15 print(unique_moneys)

```

Вторая волна. Задачи 8–11 класса

Задача II.1.2.1. Жилой дом (7 баллов)

Темы: базы данных.

Условие

Дан фрагмент таблицы базы данных некоторого жилого дома.

Таблица II.1.3: `livers`

id	first_name	last_name	birth	sex	flight_num
1	Ivan	Ivanov	25.05.1999	male	101
3	Ekaterina	Kuznetsova	04.02.1996	female	103
4	Aleksandr	Popov	06.04.1994	male	102
5	Elena	Vasilieva	03.11.1994	female	103
6	Sergei	Petrov	25.06.1984	male	102
7	Daniil	Sokolov	07.12.2000	male	102
8	Anastasia	Mikhailova	15.12.2002	female	103
9	Mikhail	Novikov	05.02.1993	male	103
10	Elizaveta	Fedorova	18.05.2004	female	104
11	Evgeniy	Morozov	26.09.2001	male	105
12	Semen	Volkov	16.08.1988	male	106
13	Vladislav	Alekseev	18.07.1981	male	104
14	Maksim	Lebedev	20.03.1988	male	106
15	Aleksandra	Semenova	27.06.1998	female	105
16	Kristina	Egorova	03.06.1999	female	107
17	Arina	Pavlova	21.05.1983	female	107
18	Dmitriy	Kozlov	07.05.1982	male	107
19	Danil	Stepanov	02.08.1986	male	108
20	Anna	Nikolaeva	20.04.1981	female	109
21	Rostislav	Orlov	27.03.1987	male	109

Таблица `livers` является информацией о пассажирах, которые проживают в доме.

В колонках:

- `id` — номер записи в таблице;
- `first_name` — имя проживающего;
- `second_name` — фамилия проживающего;
- `birth` — дата рождения;
- `sex` — пол проживающего: `male` — мужчина, `female` — женщина;
- `flat_num` — в какой квартире проживает человек.

Исходя из информации данной таблицы, определите, сколько есть потенциальных пар/семей в доме. Потенциальной парой/семьей будем называть таких проживающих, которые живут в одной квартире, имеют разный пол, а также разница их возрастов не превышает пять лет. В каждой квартире может проживать только одна пара, но не обязательно только два человека.

Решение

Учитывая, что в каждой квартире может проживать только одна пара, но не обязательно только два человека, надо проверить каждую квартиру на наличие хотя бы одной такой пары, удовлетворяющей условию задачи:

- 101: жители 1 и 2 разных полов с разницей в возрасте менее пяти лет — подходит;
- 102: жители 4, 6 и 7 одинаковых полов — не подходит;
- 103: жители 3, 5, 8 и 9, при этом у жителей 5 и 9 разный пол с разницей в возрасте менее пяти лет — подходит;
- 104: жители 10 и 13 разных полов с разницей в возрасте более пяти лет — не подходит;
- 105: жители 11 и 15 разных полов с разницей в возрасте менее пяти лет — подходит;
- 106: жители 12 и 14 одинаковых полов — не подходит;
- 107: жители 16, 17 и 18, при этом у жителей 17 и 18 разный пол с разницей в возрасте менее пяти лет — подходит;
- 108: житель 19 — не подходит;
- 109: жители 20 и 21 разных полов с разницей в возрасте более пяти лет — не подходит.

Итого получается четыре пары.

Ответ: 4.

Задача II.1.2.2. Десятки (9 баллов)

Темы: системы счисления.

Условие

Назовите максимальную систему счисления, где для чисел 10^i ($1 \leq i \leq 9$) при переводе в выбранную систему счисления их длина равна i .

Решение

Чтобы выполнялось условие, описанное в задаче, необходимо подставить под i максимальное значение (в рамках задачи это 9), и выбирать систему счисления до того момента, пока длина числа 10^i в некоторой системе счисления равна i . После того, как условие не будет выполняться, число никак не увеличится в размере, а, следовательно, не будет больше систем счисления, удовлетворяющих условию.

Так как длина числа 10^9 в десятичной системе счисления больше 9, начнем с 11-ричной системы счисления:

11-ричная система счисления — $10^9 = 47352388a$ (длина 9).

12-ричная система счисления — $10^9 = 23aa93854$ (длина 9).

13-ричная система счисления — $10^9 = 12c23a19c$ (длина 9).

14-ричная система счисления — $10^9 = 96b4b6b6$ (длина 8) — условие не выполнено.

Максимальная система счисления 13-ричная.

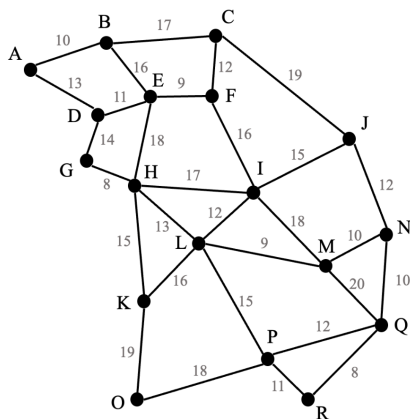
Ответ: 13.

Задача II.1.2.3. Дорожные работы (11 баллов)

Темы: теория графов.

Условие

Министерству транспорта некоторого города поступил запрос с обновлением асфальтоукладочного покрытия между важными элементами инфраструктуры. Однако совсем скоро зима, поэтому автомагистрали и дороги нужны проложить как можно скорее. Все возможные варианты прокладки дорог с требуемым для этого временем указаны на рисунке.



Решение

Из условия известно, что консерж проходит номера последовательно их кратности (сначала номера с кратностью 1, затем с кратностью 2 и т. д.).

Сделаем вывод: сколько раз номер в отеле кратен некоторым числам, столько раз его и посетят. Все числа, на которые номер комнаты кратен — это делители нашего номера, следовательно, сколько делителей у номера комнаты — столько раз ее и посетят.

Определим, в каком порядке происходят действия с дверьми:

- каждую нечетную операцию дверь меняет свое состояние с открытой на закрытую;
- каждую четную операцию дверь меняет свое состояние с закрытой на открытую (дверь будет закрыта, так как до этого была нечетная операция).

Следовательно, номер с последней закрытой дверью — это самое большое число номера, над которым проведено нечетное количество операций, или, исходя из ранее выведенного условия, нечетное количество делителей.

Нечетное количество различных делителей имеют только числа, которые являются квадратами (например, $4 = 2^2$, $81 = 9^2$). Следовательно, найдем самый большой квадрат, который меньше 9999. Ближайший полный квадрат к 9999, это $10000 = 100^2$. 10000 является большим результатом, поэтому возьмем меньший на единицу (минимальный шаг) квадрат: $(100 - 1)^2 = 99^2 = 9801$. $9801 < 9999$, а также является наибольшим квадратом, так как следующий квадрат уже превышает номер последней комнаты.

Ответ: 9801.

Задача II.1.2.5. Футбольный турнир (14 баллов)

Темы: кодирование.

Условие

В этом году проходит ежегодный футбольный турнир среди Assembler-программистов. Ежегодно это соревнование объединяет миллионы людей со всего мира, каждый с нетерпением ждет его проведения. Сейчас на соревнование было зарегистрировано 512 команд. Все соревнование проходит в три этапа: отборочный этап, групповой этап и финальный этап. Во время отборочного этапа проходит четыре стадии турнира: $\frac{1}{256}$, $\frac{1}{128}$, $\frac{1}{64}$ и $\frac{1}{32}$. Все матчи проходят по 90 мин основного времени, и, в случае ничейного результата, добавляется дополнительное время 30 мин. Если после 120 мин матча не удастся выявить победителя, проходит серия пенальти.

После отборочного этапа остается 32 команды, и они попадают в групповой этап. Все эти команды случайным образом распределяются по восьми группам, и в процессе этапа они сыграют каждый с каждым по два раза, то есть любая команда на этой стадии сыграет 6 матчей. Во время группового этапа матчи проходят только по 90 мин, независимо от результата.

По итогам группового этапа в финальную стадию проходит 16 лучших команд, и они начинают играть за кубок футбольного ассемблера. Всего проходит 4 стадии: $\frac{1}{8}$,

$\frac{1}{4}$, $\frac{1}{2}$ и финал, матча за третье место нет. Во время финальной стадии сохраняются те же правила проведения матчей, что и в отборочном этапе: 90 + 30 + серия пенальти.

Данный турнир проводится не первый год, и организаторы прекрасно знают из своей статистики, что в дополнительное время в отборочном этапе заканчивается не более 10% матчей, а также не более 20% матчей в финальном этапе.

Организаторы хотят, чтобы весь турнир прошел на высшем уровне и без нареканий, но у них возник вопрос: сколько памяти надо выделить, чтобы гарантировано сохранить все результаты матча. Организаторы хотят хранить отчеты по матчам поминно, выделяя на каждую мину по 2 байта. Серию пенальти они решили не хранить, а записывать гол на счет победителя на 120 минут. Каждый этап соревнования хранится отдельно, независимо от других, в килобайтах. Исходя из статистических данных процентов матчей, заканчивающихся в основное или дополнительное время, а также формата турнира, рассчитайте, какое минимальное целое количество памяти нужно выделить в килобайтах, чтобы гарантировано удалось сохранить все результаты турнира поминно.

Считать, что 1 Кбайт равен 1024 байтам.

Решение

Для сохранения матча длительностью 90 мин потребуется 180 байт, для 120-минутного матча — 240 байт.

Отборочный этап

В $\frac{1}{256}$ стадии пройдет $\frac{512}{2}$ (количество команд, участвующих в матче) = 256 матчей, и, следовательно, в следующую стадию пройдет 256 команд.

В $\frac{1}{128}$ стадии пройдет $\frac{256}{2}$ (количество команд, участвующих в матче) = 128 матчей, и, следовательно, в следующую стадию пройдет 128 команд.

В $\frac{1}{64}$ стадии пройдет $\frac{128}{2}$ (количество команд, участвующих в матче) = 64 матча, и, следовательно, в следующую стадию пройдет 64 команды.

В $\frac{1}{32}$ стадии пройдет $\frac{64}{2}$ (количество команд, участвующих в матче) = 32 матча, и, следовательно, в следующую стадию пройдет 32 команды.

Всего за отборочную стадию пройдет $256 + 128 + 64 + 32 = 480$ матчей, не более 10% из которых могут закончиться в дополнительное время с серией пенальти: $480 \cdot 0,1 = 48$ матчей; в основное время закончится: $480 - 48 = 432$ матча.

Следовательно, для хранения данных о матчах в отборочном этапе потребуется: $432 \cdot 180 + 48 \cdot 240 = 89280$ байт.

Групповой этап

Всего будет 32 команды, поделенных равномерно на 8 групп, следовательно, в каждой группе по 4 команды. Каждая команда сыграет друг против друга по два раза, следовательно, всего будет 6 туров между командами, а в каждом туре будет по 2 матча. Посчитаем, сколько матчей будет проведено всего: $6 \cdot 2 \cdot 8 = 96$ матчей. Все матчи пройдут только в основное время: $96 \cdot 180 = 17280$ байт.

Финальный этап

В $\frac{1}{8}$ стадии пройдет $\frac{16}{2}$ (количество команд, участвующих в матче) = 8 матчей, и, следовательно, в следующую стадию пройдет 8 команд.

В $\frac{1}{4}$ стадии пройдет $\frac{8}{2}$ (количество команд, участвующих в матче) = 4 матча, и, следовательно, в следующую стадию пройдет 4 команды.

В $\frac{1}{2}$ стадии пройдет $\frac{4}{2}$ (количество команд, участвующих в матче) = 2 матча, и, следовательно, в следующую стадию пройдет 2 команды.

В финале пройдет всего 1 матч.

Всего за финальную стадию пройдет $8 + 4 + 2 + 1 = 15$ матчей, не более 25% из которых могут закончиться в дополнительное время с серией пенальти: $15 \cdot 0,25 = 3,75$ матча. Так как в условии задачи указано **не более**, то округляем в меньшую сторону: 3 матча; в основное время закончится: $15 - 3 = 12$ матчей.

Следовательно, для хранения данных о матчах в отборочном этапе потребуется: $12 \cdot 180 + 3 \cdot 240 = 2880$ байт.

Переведем все значения из байт в КБайт:

- для хранения отборочного этапа потребуется $89280/1024 = 87,1875 = 88$ Кбайт;
- для хранения группового этапа потребуется $17280/1024 = 16,875 = 17$ Кбайт;
- для хранения финального этапа потребуется $2880/1024 = 2,8125 = 3$ Кбайт.

Всего для хранения таблицы потребуется: $88 + 17 + 3 = 108$ Кбайт.

Ответ: 108.

Задача II.1.2.6. Фиктивные переменные (14 баллов)

Темы: алгебра логики.

Условие

Дана логическая функция, состоящая из семи переменных:

$$(((a \wedge b) \vee (\neg a \wedge (\neg a \vee e) \wedge b)) \rightarrow (c \wedge (d \vee e)) \vee (\neg c \wedge d) \vee (e \wedge \neg c)), \wedge (\neg f \vee (g \wedge f) \vee \neg g).$$

Фиктивными переменными называются те переменные, которые не влияют на результат функции. Выясните, какие переменные являются фиктивными. В ответе укажите их в любом порядке слитно, без пробелов, запятых и иных знаков. Гарантируется, что есть минимум две фиктивные переменные, а также существует хотя бы одна переменная, от которой зависит результат функции.

Решение

Преобразуем выражение:

$$\neg a \wedge (\neg a \vee e) = \neg a.$$

Рассмотрим левую часть, заметим, что:

$$(((a \wedge b) \vee (\neg a \wedge b)) \rightarrow (c \wedge (d \vee e)) \vee (\neg c \wedge d) \vee (e \wedge \neg c)) \wedge (\neg f \vee (g \wedge f) \vee \neg g).$$

Теперь обратим внимание на $(a \wedge b) \vee (\neg a \wedge b)$, потому что по свойству склеивания это будет просто b :

$$((b \rightarrow (c \wedge (d \vee e)) \vee (\neg c \wedge d) \vee (e \wedge \neg c)) \wedge (\neg f \vee (g \wedge f) \vee \neg g)).$$

Во второй скобке можно заметить общий множитель $\neg c$, который можно вынести за скобки (свойство дистрибутивности):

$$((b \rightarrow (c \wedge (d \vee e)) \vee (\neg c \wedge (d \vee e)))) \wedge (\neg f \vee (g \wedge f) \vee \neg g).$$

Далее общий множитель $(d \vee e)$, который тоже можно вынести за скобки (свойство дистрибутивности):

$$((b \rightarrow (d \vee e) \wedge (c \vee \neg c)) \wedge (\neg f \vee (g \wedge f) \vee \neg g)).$$

Здесь $c \vee \neg c = 1$, а значит функция принимает вид:

$$((b \rightarrow (d \vee e)) \wedge (\neg f \vee (g \wedge f) \vee \neg g)).$$

Уберем лишние скобки:

$$(b \rightarrow (d \vee e)) \wedge (\neg f \vee (g \wedge f) \vee \neg g).$$

Теперь преобразуем правую часть, по закону поглощения:

$$\neg f \vee (g \wedge f) = \neg f \vee g,$$

после этого логическое выражение имеет следующий вид:

$$(b \rightarrow (d \vee e)) \wedge (\neg f \vee g \vee \neg g).$$

Так как $g \vee \neg g = 1$, то и вся скобка тоже превращается в 1, следовательно, функция принимает вид:

$$\neg b \vee d \vee e.$$

Значит, фиктивными переменными являются a , c , f , g .

Ответ: $acfg$.

Задача II.1.2.7. Прогнозирование (17 баллов)

Темы: программирование.

Условие

Сегодня проходит финал по перетягиванию каната. В нем принимают участие две команды: синих и красных. Обе команды проделали большой путь до этого финала ради призового фонда с конфетами. Но на днях команда красных предложила главному тренеру команды синих конфеты за то, чтобы они проиграли. И те и другие будут в плюсе, ведь тогда команда красных заберет призовой фонд, а команда синих получит гарантированные конфеты за проигрыш.

После того как он получил конфеты, руководители команды красных попросили узнать, сколько матчей они смогут гарантированно проиграть. Они дали ему один день на обдумывание, чем он и занялся. Тренер помнит, что финал проходит по следующим правилам: от команды представляются n человек, и в рамках финала

проходит также n матчей. В первом матче канат тянут по одному человеку с каждой стороны, во втором матче канат тянут по два человека с каждой стороны, на третий три, и так далее до того, пока канат не будут тянуть с каждой стороны по n человек. Побеждает в матчах та команда, у которой больше суммарная сила на сторону. Если силы равны, объявляется ничья. Тренер знает силы и своей команд, и команды соперника, и вправе на каждый матч сам решать, кто участвует за команду синих. Также он знает, что команда красных будет ставить максимально оптимально своих участников на матч.

Исходя из этого, он просит вас написать программу, которая посчитает, какое максимальное количество матчей он может проиграть, если будет сам решать кто в каком матче участвует.

Формат входных данных

В первой строке входных данных записано целое число n ($1 \leq n \leq 2 \cdot 10^5$) — количество участников в каждой команде и одновременно количество матчей в финале. Во второй строке записано n целых чисел a_i ($1 \leq a_i \leq 10^9$) — силы участников команды синих. В третьей строке записано n целых чисел b_i ($1 \leq b_i \leq 10^9$) — силы участников команды красных.

Формат выходных данных

Выведите одно целое число — максимальное количество матчей, которое команда синих может проиграть.

Методика проверки

Программа проверяется на 20 тестах. Прохождение каждого теста оценивается в 0,5 балла. Тесты из условия задачи при проверке не используются.

Примеры

Пример №1

Стандартный ввод
5
2 3 1 4 3
1 2 1 2 2
Стандартный вывод
2

Пояснения к примеру

Команда синих может проиграть первых два матча. В первом матче они поставят участника с силой 1 против участника команды красных с силой 2.

Во втором матче они поставят участников с силой 1 и 2 против участников команды красных с силами 2 и 2.

В третьем матче можно сделать ничью, но проиграть не получится. В четвертом и пятом матче команда синих может только выиграть.

Решение

Отсортируем силы участников обеих команд. Также создадим две переменные, в которых будут храниться суммарные силы участников команд на определенный раунд. Эти суммы на каждый раунд будут наполняться следующим образом: в команду синих мы будем добавлять самого слабого свободного участника из команды, в то время как в команду красных мы будем добавлять самого сильного свободного участника из команды. Тем самым мы постоянно будем задавать команде синих наиболее слабый состав на каждый раунд, а команде красных, наоборот, наиболее сильный. Посчитаем, в скольких случаях команда синих была слабее команды красных, и выведем данный результат.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  n = int(input())
2  blue_team = sorted(list(map(int, input().split())))
3  red_team = sorted(list(map(int, input().split())))
4  blue_sum = 0
5  red_sum = 0
6  res = 0
7  for i in range(n):
8      blue_sum += blue_team[i]
9      red_sum += red_team[-i - 1]
10     if blue_sum < red_sum:
11         res += 1
12 print(res)
13
```

Задача II.1.2.8. Магические ключи (17 баллов)

Темы: программирование.

Условие

Даня попал в магический коридор, в котором он видит n дверей с разными замочными скважинами. Незвестный голос говорит ему повернуть голову влево, что он без каких-либо сомнений делает. Перед ним открылась следующая картина: стол, а на нем — неограниченное количество m видов ключей, а также карта, на которой расписано, какая дверь каким ключом открывается.

Все бы ничего, но Даня снова услышал неизвестный голос, который произнес следующие слова: «Эти ключи не простые, а магические. Как только ты используешь ключ, у тебя есть k у. е. времени, чтобы воспользоваться им повторно, иначе он разрушится. Но если ты повторно воспользуешься ключом, он обновится, и у тебя снова будет k у. е. времени, чтобы им воспользоваться повторно. Каждая дверь

открывается ключом за 1 у. е. времени. Если ты хочешь выбраться из этого коридора, воспользуйся картой и собери все ключи, которые тебе нужны, иначе ты здесь останешься на века».

В этой ситуации каждый будет брать все и как можно больше, но не Дания. Он решил быть рациональным и не забивать все карманы ненужными ключами. Он отправил вам по «аске» карту и информацию про все магические свойства ключей, и просит написать программу, которая рассчитает минимальное количество ключей каждого вида, которые должен взять Дания, а также их суммарное количество.

Формат входных данных

В первой строке входных данных записано три целых числа n ($1 \leq n \leq 10^6$), m ($1 \leq m \leq 1000$) и k ($1 \leq k \leq 2000$) — количество дверей, ключей и время действия ключа после первого использования соответственно.

Во второй строке записано n целых чисел a_i ($1 \leq a_i \leq m$) — номер ключа, которым можно открыть дверь под номером i . Гарантируется, что на каждый вид ключа будет не более 1000 дверей, которые им открываются.

Формат выходных данных

Выведите в первой строке одно число — общее количество ключей, которое необходимо с собой взять. Во второй строке выведите n чисел — сколько ключей надо взять на каждый вид по отдельности. Вывод количества ключей идет по порядку: сначала количество ключей с номером 1, затем — с номером 2, и так далее.

Примеры

Пример №1

Стандартный ввод
5 2 2
2 1 2 2 1
Стандартный вывод
3
2 1

Пояснение к примеру

Для открытия первой двери нужен новый ключ с номером 2. Для открытия второй нужен новый ключ с номером 1. Для открытия третьей двери мы можем воспользоваться ранее взятым ключом 2, так как его время действия еще не закончилось еще. Для открытия четвертой двери воспользуемся ранее взятым ключом 2, так как мы его на предыдущей двери обновили, и теперь отсчет его времени действия начался снова. Для открытия пятой двери нужен новый ключ с номером 1, так как предыдущий ключ потерял свое действие. Итого нам нужно два ключа с номером 1 и один ключ с номером 2.

Решение

Создадим отдельный список/словарь, в который будем записывать в качестве индексов/ключей номера ключей от дверей, а в качестве значений под индексами/ключами будет храниться список индексов дверей, которые открываются этими ключами. После этого запускаем цикл, доставая индексы дверей по определенному ключу и вычисляем, сколько нужно ключей определенного типа, чтобы открыть все двери, которые подходят под него. Для того чтобы понимать, нужен новый ключ или нет, воспользуемся следующим условием: если разница между позицией двери и предыдущей двери, открываемой данным ключом, больше времени активности ключа, то требуется новый ключ, в ином случае нет. Если у нас есть хотя бы одна дверь, которая открывается определенным типом ключа, нужно взять минимум один ключ, в ином случае ключи не нужны. Суммируем количество раз, сколько раз, исходя из условия, потребовалось взять еще ключей для дверей, а также добавляем один (чтобы взять первый ключ для дверей). Сохраняем для двери в списке данный результат. В итоге пройдемся по всем ключам и дверям для них и выведем сумму всех ключей, а также по отдельности необходимое количество ключей.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  n, m, active_time = map(int, input().split())
2  keys = list(map(int, input().split()))
3  arr = [[] for i in range(m)]
4  for i in range(n):
5      arr[keys[i] - 1].append(i)
6  res = 0
7  count_keys = []
8  for doors_by_someone_key in arr:
9      count_keys.append(0)
10     if len(doors_by_someone_key) == 0:
11         continue
12     prev_door = doors_by_someone_key[0]
13     res += 1
14     count_keys[-1] += 1
15     for door in doors_by_someone_key:
16         if door - prev_door > active_time:
17             res += 1
18             count_keys[-1] += 1
19     prev_door = door
20  print(res, count_keys, sep='\n')
```

Третья волна. Задачи 8–11 класса

Задача II.1.3.1. Аренда авто (7 баллов)

Темы: базы данных.

Условие

Даны фрагменты двух таблиц базы данных некоторой каршеринговой компании.

Таблица II.1.4: Операции

id	Имя	Фамилия	Пол	Дата аренды	id авто	Сумма аренды	Штраф
1	Данил	Смирнов	м	03.08.2023	104	242	Есть
2	Екатерина	Кузнецова	ж	04.08.2023	106	314	Нет
3	Сергей	Попов	м	06.08.2023	105	147	Есть
4	Анастасия	Васильева	ж	08.08.2023	103	150	Нет
5	Елизавета	Штольц	м	10.08.2023	103	219	Есть
7	Дмитрий	Солоков	м	10.08.2023	10	300	Нет
8	Елена	Новикова	ж	12.08.2023	103	258	Есть
9	Михаил	Федоров	м	17.08.2023	10	294	Есть
10	Филипп	Морозов	м	18.08.2023	102	190	Нет
11	Евгений	Волков	м	20.08.2023	101	178	Нет
12	Владислав	Алексеев	м	25.08.2023	103	218	Нет
13	Максим	Лебедев	м	25.08.2023	102	176	Нет
14	Александра	Семенова	ж	28.08.2023	104	315	Есть
15	Арина	Егорова	ж	01.09.2023	102	233	Есть
16	Кристина	Павлова	ж	03.09.2023	101	166	Есть
17	Даниил	Казаченко	м	03.09.2023	102	252	Нет
18	Иван	Козлов	м	04.09.2023	101	323	Есть
19	Агата	Орлова	ж	06.09.2023	106	181	Нет
20	Владимир	Николаев	м	06.09.2023	101	271	Нет
21	Ростислав	Никифоров	м	07.09.2023	106	199	Есть

Таблица II.1.5: Автомобили

id	id авто	Марка	Модель	Номер	Год выпуска	Тип двигателя
1	101	Renault	Kaptur	K123ДЖ 50	2019	бензиновый
2	102	Renault	Logan	K015ТИ 50	2019	бензиновый
3	103	Skoda	Octavia	K329ЮТ 50	2019	дизельный
4	104	Skoda	Octavia	K841ГМ 50	2018	бензиновый
5	105	Audi	A3	K418ДВ 50	2013	дизельный
6	106	Renault	Kaptur	K641ЛТ 50	2017	бензиновый

Таблица **Операции** является информацией о арендаторах, которые воспользовались услугами каршеринговой компании.

В колонках:

- id — номер записи в таблице;
- имя — имя клиента;
- фамилия — фамилия клиента;
- пол — пол клиента;
- дата аренды — дата, когда клиент арендовал автомобиль;
- id авто — номер автомобиля, который арендовал клиент;
- сумма аренды — итоговая сумма аренды автомобиля клиентом;
- штраф — имеет ли клиент штраф за поездку.

Таблица **Автомобили** является информацией об автомобилях компании.

В колонках:

- id — номер записи в таблице;
- id авто — номер автомобиля, который арендовал клиент;
- марка — марка автомобиля;
- модель — модель автомобиля;
- номер — серийный номер автомобиля;
- год выпуска — год, когда был выпущен автомобиль;
- тип двигателя — тип двигателя автомобиля (бензиновый или дизельный).

Исходя из информации данных таблиц, определите, на сколько больше денег заработала компания на мужчинах, которые арендовали бензиновые автомобили, по сравнению с женщинами, арендовавшими дизельные?

Решение

Автомобили с бензиновыми двигателями имеют id 101, 102, 104, 106.

Автомобили с дизельными двигателями имеют id 103, 105.

Найдем всех мужчин, которые арендовали автомобили с id 101, 102, 104, 106:

id	Имя	Фамилия	Пол	Дата аренды	id авто	Сумма аренды	Штраф
1	Данил	Смирнов	м	03.08.2023	104	242	Есть
7	Дмитрий	Солоков	м	10.08.2023	101	300	Нет
10	Филипп	Морозов	м	18.08.2023	102	190	Нет
11	Евгений	Волков	м	20.08.2023	101	178	Нет
13	Максим	Лебедев	м	25.08.2023	102	176	Нет
17	Даниил	Казаченко	м	03.09.2023	102	252	Нет
18	Иван	Козлов	м	04.09.2023	101	323	Есть
20	Владимир	Николаев	м	06.09.2023	101	271	Нет
21	Ростислав	Никифоров	м	07.09.2023	106	199	Есть

Суммарно получается 2131 руб. Теперь найдем сколько компания заработала на девушках, арендовавших машины с id 103 и 105.

id	Имя	Фамилия	Пол	Дата аренды	id авто	Сумма аренды	Штраф
4	Анастасия	Васильева	ж	08.08.2023	103	150	Нет
8	Елена	Новикова	ж	12.08.2023	103	258	Есть

Итого выходит 408 руб. А значит компания заработал на мужчинах на $2131 - 408 = 1723$ руб. больше.

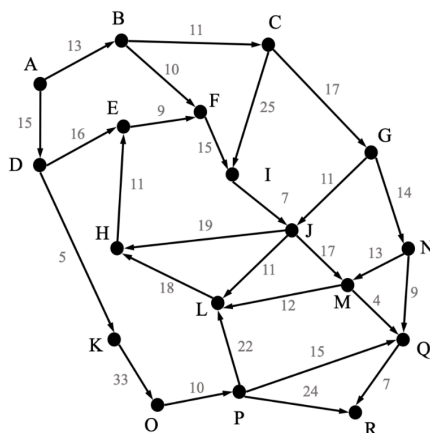
Ответ: 1723.

Задача II.1.3.2. Гонщик (9 баллов)

Темы: теория графов.

Условие

Даня и Ваня играют в одну известную видеоигру «Нужна скорость». Они прошли ее практически всю, за исключением последней сложной миссии. Им нужно как можно дольше скрываться от преследования на время. После многих безуспешных попыток они решили нарисовать карту гоночной локации, которая приведена на рисунке.

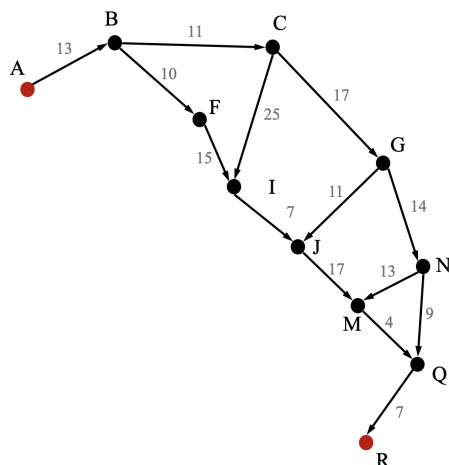


Стартом заезда считается пункт A , а финишем пункт Q . Они замерили максимальное время, которое им удастся продержаться на каждом дорожном участке. После замеров Даня и Ваня просят у вас помощи. Найдите максимально возможное время заезда при условии, что через каждый пункт можно проезжать только один раз и двигаться разрешено только в том направлении, куда указана стрелка.

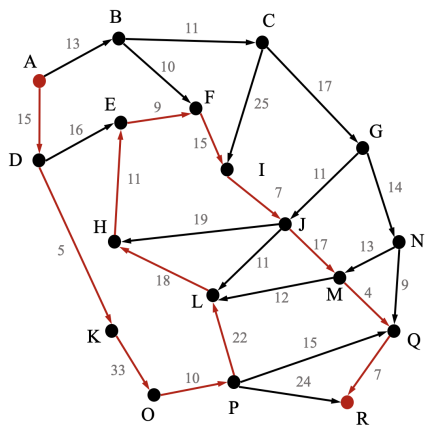
Решение

Заметим, что в графе присутствуют циклы, поэтому попытаемся их разобрать. Очевидно, что после старта есть два основных направления: либо в пункт D (направо), либо в пункт E (вниз).

Если мы идем в пункт B , то мы никогда не сможем попасть в пункт L и H , потому что они вынудят нас пройти через один пункт несколько раз, поэтому картина возможных путей там выглядит очень просто.



Однако если мы идем в пункт D, то мы можем захватить пункт L и H при этом также захватывая и другие пункты из правой части графа.



Применяя обратный алгоритм Дейкстры, сравниваем оба варианта и получаем, что самый долгий путь был на второй картинке, и он составляет 173.

Ответ: 173.

Задача II.1.3.3. Киновечер (11 баллов)

Темы: кодирование.

Условие

Недавно Даня и Ваня ходили в кино на показ новой короткометражки «Опенгеймер».

Они, конечно, были впечатлены актерской игрой и сюжетом, но больше всего им стало интересно, какое же максимальное количество цветов используется в картине. Они решили воспользоваться приложением для скачивания фильмов «Толлент». Из него они узнали, что суммарно произведение весит около 17 Гб при разрешении 1280×720 и частоте кадров 25 к/с, при этом сама картина длится приблизительно 20 мин, а звук кодировался отдельно и весит 1 Гб памяти.

Из этих данных определите, какое максимальное количество цветов могло использоваться в кадре.

Учтите, что в данной задаче:

- 1 Гб = 1024 Мб;
- 1 Мб = 1024 Кб;
- 1 Кб = 1024 Бита.

Решение

Первым делом определим общий объем памяти, в который необходимо уложиться для кодирования визуальной составляющей фильма. Если звук занимает 1 Гб, а весь фильм целиком — 17 Гб, то на кодирование картинки остается 16 Гб.

Вес одного любого кадра фильма будет составлять $1280 \cdot 720 \cdot i$, где i — глубина цвета. Видео — это набор картинок, которые показываются с частотой 25 кадров в секунду (по условию) на протяжении 20 мин (также по условию), вес всего видеофайла можно записать как $V = 1280 \cdot 720 \cdot i \cdot 25 \cdot 20 \cdot 60$.

Выражаем отсюда неизвестную i , а вместо V подставляет найденные 16 Гб, предварительно переведенные в биты:

$$i \leq \frac{16 \cdot 2^{33}}{1280} \cdot 720 \cdot 25 \cdot 20 \cdot 60,$$

$$i \leq 4,971.$$

Очевидно, что глубина цвета быть дробной не может, и округлить в большую сторону ее тоже нельзя, т.к. мы превысим наш размер видео, значит, максимальное количество бит на кодирование цвета, которые мы можем взять, равно 4.

Количество цветов можно легко найти по формуле $N = i^2$, откуда следует, что оно равно 16.

Ответ: 16.

Задача II.1.3.4. Кубическая разница (14 баллов)

Темы: системы счисления.

Условие

Существует некоторое четырехзначное число $x = abcd$ в четверичной системе счисления.

Кроме этого, есть его копия, записанная в обратном порядке, назовем ее $y = dcba$.

Сколько можно выбрать пар чисел x и y так, чтобы модуль их разности являлся кубом какого-либо целого числа?

Решение

По условию задачи имеется четверичное число x , которое можно так и представить $x = abcd$, где переменные $a, b, c, d \in [0; 3]$, так как являются цифрами четверичного алфавита.

Кроме того имеется число y — инвертированная запись числа x , которая равна $y = dcba$, где переменные $a, b, c, d \in [0; 3]$, так как являются цифрами четверичного алфавита.

Представляя числа в десятичной системе счисления, запишем уравнение, что разность чисел x и y равна кубу некоторого числа e :

$$(a \cdot 4^3 + b \cdot 4^2 + c \cdot 4^1 + d \cdot 4^0) - (d \cdot 4^3 + c \cdot 4^2 + b \cdot 4^1 + a \cdot 4^0) = e^3,$$

$$(64a + 16b + 4c + d) - (64d + 16c + 4b + a) = e^3,$$

$$64a + 16b + 4c + d - 64d - 16c - 4b - a = e^3,$$

$$63a + 12b - 12c - 63d = e^3,$$

$$63(a - d) + 12(b - c) = e^3.$$

Учитывая, что $a, b, c, d \in [0; 3]$, так как являются цифрами четверичного алфавита, как можно получить куб в разнице 63 и 12?

Такой вариант всего 1 и это: $63 - 12 \cdot 3 = 27$ Значит $a - d = 1$, $b - c = -3$.

Тогда подходит пара чисел (2031; 1302). И еще одна пара (3032; 2303).

Ответ: 2.

Задача II.1.3.5. Кубическая разница (14 баллов)

Темы: алгебра логики.

Условие

Даны две логической функции:

$$F_1 = (\neg y \vee (y \wedge \neg z) \wedge (y \vee \neg e)) \rightarrow (x \wedge w \vee w \wedge x),$$

$$F_2 = ((\neg x \vee \neg y \vee \neg z) \wedge (x \vee y \wedge z)) \wedge (\neg w \vee (e \wedge w \vee w \wedge \neg e)).$$

Определите, в скольких из всех возможных значений пяти переменных x, y, z, w, e результаты двух функций будут отличаться друг от друга?

Решение

Упростим обе функции.

Функцию $F_1 = (\neg y \vee (y \wedge \neg z) \wedge (y \vee \neg e)) \rightarrow (x \wedge w \vee \neg w \wedge x)(x \wedge w \vee \neg w \wedge x)$ можно упростить по свойствам дистрибутивности: $(x \wedge (w \vee \neg w))$ и $w \vee \neg w$ всегда будет истинно: $x \wedge 1 = x$.

Получим

$$F_1 = (\neg y \vee (y \wedge \neg z) \wedge (y \vee \neg e)) \rightarrow x.$$

$(y \wedge \neg z) \wedge (y \vee \neg e)$ можно расширить по свойствам дистрибутивности, приняв, что $(y \wedge \neg z) = a$, тогда получим:

$$a \wedge (y \vee \neg e) = (y \wedge a) \vee (\neg e \wedge a) = (y \wedge (y \wedge \neg z)) \vee (\neg e \wedge (y \wedge \neg z)).$$

Передвинем в левой части скобки по свойству ассоциативности:

$$((y \wedge y) \wedge \neg z) \vee (\neg e \wedge (y \wedge \neg z)).$$

Упростим $y \wedge y$ по свойству идемпотентности: $(y \wedge \neg z) \vee (\neg e \wedge (y \wedge \neg z))$.

Вернув $a = (y \wedge \neg z)$, упростим выражение по свойству поглощения:

$$a \vee (\neg e \wedge a) = a = (y \wedge \neg z),$$

$$F_1 = (\neg y \vee (y \wedge \neg z)) \rightarrow x.$$

В левой части выражения разложим выражение по свойству дистрибутивности:

$$\neg y \vee (y \wedge \neg z) = (\neg y \vee y) \wedge (\neg y \vee \neg z);$$

$(\neg y \vee y)$ всегда будет истинно:

$$1 \wedge (\neg y \vee \neg z) = \neg y \vee \neg z.$$

$$F_1 = (\neg y \vee \neg z) \rightarrow x.$$

Разложим импликацию:

$$(\neg y \vee \neg z) \rightarrow x = \neg(\neg y \vee \neg z) \vee x.$$

Применим на скобку закон Де Моргана: $y \wedge z \vee x$.

$$F_1 = y \wedge z \vee x,$$

$$F_2 = ((\neg x \vee \neg y \vee \neg z) \wedge (x \vee y \wedge z)) \wedge (\neg w \vee (e \wedge w \vee w \wedge \neg e)).$$

$(e \wedge w \vee w \wedge \neg e)$ можно упростить по свойствам дистрибутивности:

$$(w \wedge (\neg e \vee e)); \neg e \vee e \text{ всегда истина: } w \wedge 1 = w.$$

$$F_2 = ((\neg x \vee \neg y \vee \neg z) \wedge (x \vee y \wedge z)) \wedge (\neg w \vee w) \neg w \vee w \text{ всегда истина: } 1.$$

$$F_2 = ((\neg x \vee \neg y \vee \neg z) \wedge (x \vee y \wedge z)).$$

Выражения упрощены до трех переменных, следовательно, две переменные не влияют на результат.

Также если менять значения этих переменных, то ответ, зависимый от трех других, будет повторяться.

Следовательно, ответы будут повторяться в 2^2 (выборка вариантов переменной $(0, 1)$ в степени количества переменных) = 4 раза.

Составим таблицу истинности.

x	y	z	F_1	F_2
0	0	0	0	0
0	0	1	0	0
0	1	0	0	0
0	1	1	1	1
1	0	0	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	1	0

Результат функций различается только в одном случае.

Так как у нас есть переменные, не влияющие на результат, но повторяющиеся значения функций четыре раза, умножим количество повторений на количество различающихся значений функций: $1 \cdot 4 = 4$.

Ответ: 4.

Задача II.1.3.6. Трасса (14 баллов)

Темы: программирование.

Условие

В новом современном городе строят новую современную скоростную трассу длиной s м. Ее необходимо оборудовать так, чтобы она могла выдерживать большое количество машин и чтобы она не создавала больших пробок и аварийных ситуаций. Поэтому было принято решение посмотреть на другой, аналогичный город с такой же успешной трассой и запросить с камер записи о том, сколько машин там фиксируется за день.

Всего с камер было получено n машин, и по каждой была информация во сколько она заезжает на трассу и с какой скоростью ехала в м/с. После получения этой информации было решено узнать максимальную нагрузку в какую-то из секунд на трассе. От этого значения они и хотят понимать, какую нагрузку должна выдерживать трасса. Вы, как опытный программист и сотрудник ИТ-отдела города, взялись за эту задачу.

Напишите программу, которая по этим данным опередит максимальную нагрузку на трассу в какую-то из секунд.

Формат входных данных

В первой строке входных данных записано два целых числа n ($1 \leq n \leq 10^5$) и s ($1 \leq s \leq 10^6$) — количество зафиксированных машин и длина трассы. В следующих n строках по два целых числа t ($1 \leq t \leq 10^6$) и v ($1 \leq v \leq s$) — время заезда на трассу и скорость на трассе в м/с соответственно.

Гарантируется, что длина трассы кратна каждой скорости во входных данных.

Формат выходных данных

Выведите одно число — максимальное количество машин на трассе в некоторую секунду.

Методика проверки

Первая машина заедет на третьей секунде и выйдет на 5: [3, 5).

Вторая машина заедет на второй секунде и выйдет на 8: [2, 8).

Третья машина заедет на первой секунде и выйдет на 13: [1, 13).

Четвертая машина заедет на пятой секунде и выйдет на 6: [5, 6).

Итого максимальное количество машин будет замечено на четвертой секунде. Одновременно на трассе будет первая, вторая и третья машины.

Примеры

Пример №1

Стандартный ввод
4 60
3 30
2 10
1 5
5 60
Стандартный вывод
3

Решение

Определим два события, которые у нас возможны в задаче:

- машина заехала на трассу в определенный момент времени, обозначим это как +1 машина;
- машина выехала с трассы в определенный момент времени, обозначим это как −1 машина.

Каждое событие мы можем без особых проблем сохранять в массив и после работать с ним.

Первое событие мы можем сохранить в массив как пару (время заезда, 1), где время заезда — параметр из входных данных, а 1 — это аналог +1, дающий нам сигнал, что на трассе появилась новая машина.

Второе событие мы можем сохранить в массив как пару (время выезда, -1), где время выезда — это сумма времени заезда на трассу и длины трассы, поделенной на скорость машины, а -1 — сигнал о том, что машина выехала с трассы (-1 машина).

Отсортируем массив по первому параметру пар чисел: временам заезда и выезда с трассы. Тем самым мы получим последовательность действий на трассе. Запускаем цикл по массиву и, если действие равняется заезду машины, увеличиваем количество машин на трассе, в ином случае — уменьшаем.

Так как у нас действия помечены как 1 (+1) и -1, можем в количество машин добавлять именно их. Заведем отдельно переменную, в которой будем хранить максимальное количество машин, которое было за все время на трассе. Его мы будем обновлять после каждого действия следующим способом: если количество машин на трассе в определенный момент времени больше, чем записано в переменной, то обновляем ее значение.

По окончании цикла выводим максимальное количество машин, которое было зафиксировано.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  n, s = map(int, input().split())
2  arr = []
3  for i in range(n):
4      time_in, speed = map(int, input().split())
5      arr.append((time_in, 1))
6      arr.append((time_in + s // speed, -1))
7  arr.sort()
8  max_cars_per_sec = 0
9  cur_cars_per_sec = 0
10 for i in range(2 * n):
11     cur_cars_per_sec += arr[i][1]
12     max_cars_per_sec = max(max_cars_per_sec, cur_cars_per_sec)
13 print(max_cars_per_sec)

```

Задача II.1.3.7. Игра +1 (14 баллов)

Темы: программирование.

Условие

Игра +1 — это современная, набирающая популярность игра в просторах интернета. Она привлекает всех своей простотой и желанием добиваться высоких результатов за минимальное количество действий.

Давайте немного познакомимся с ее сутью. Нам выложено некое поле размером $1 \times n$ клеток. В каждой клетке записано некоторое число.

Если на поле есть два одинаковых числа, то их можно объединить. Операция объединения удаляет два числа, над которыми была произведена операция, а также создает новое число (на одной из освободившейся клетке), которое на единицу больше удаленных.

Например, если была объединена пара двоек, то они будут удалены, а новым числом будет 3.

Игра считается законченной, если было получено некоторое загаданное число m или на поле больше нет одинаковых чисел.

Как мы обсудили ранее, игроки хотят побеждать за минимальное количество действий. Так как единственное действие, которое существует — это объединение, то, соответственно, побеждать за минимальное количество объединений. Один из игроков решил считать и попросил вас написать ему программу, которая, исходя из поля, будет определять, сколько минимально чисел с первоначального поля надо объединить между собой, чтобы закончить игру, или выведите -1 , если невозможно собрать нужное число.

В ответе не учитывайте объединения между новыми числами, которые получаются после объединения.

Формат входных данных

В первой строке входных данных записано два целых числа n ($1 \leq n \leq 10^6$) и m ($2 \leq m \leq 100$) — количество чисел и цель, которую надо получить.

Во второй строке записано n целых чисел a_i ($1 \leq a_i < 100$, $\max(a) < m$) — числа на поле.

Формат выходных данных

Выведите одно число — минимальное количество чисел из первоначального поля, которое надо объединить для получения нужного результата.

Примеры

Пример №1

Стандартный ввод
6 4
1 2 3 1 1 2
Стандартный вывод
3

Пояснение к примеру

Для получения результата 4 достаточно выбрать $[2, 3, 2][2, 3, 2]$:

- четверки суммируются как одинаковая пара чисел, получая новое число: $[3, 3][3, 3]$;

- восьмерки суммируются как одинаковая пара чисел, получая новое число: $[4][4]$.

Решение

Создадим список/словарь, в котором подсчитаем количество каждого числа, которые нам даны на вход.

Подсчет будем ввести следующим образом: в качестве индекса/ключа будем использовать само число, а в качестве значения — сколько раз оно встретилось.

После этого заведем переменную, в которой будем хранить число, которое мы хотим достичь на определенном шагу, а также необходимое количество этих чисел.

В начальный момент времени у нас значение этой переменной равно конечному результату, который дан во входных данных, а необходимое количество — 1 (само число).

Запускаем цикл, который будет работать до тех пор, пока не соберем все числа, либо пока число, которое мы хотим достичь, не дойдет до нуля (несуществующего числа).

На каждом шагу проверяем через список/словарь, есть ли у нас необходимое количество выбранного числа. Если их достаточно, добавляем недостающее количество чисел и указываем, что собрали все числа (указывает, что нужно 0 чисел). В ином случае отнимаем часть, которую мы можем покрыть, и оставшееся необходимое количество чисел умножаем на два (так как чтобы собрать число x , необходимо два числа $x - 1$, описано подробнее в условии), а также меняем нынешнее число, которые нам нужно собрать, уменьшая его значение на 1.

Помимо этого мы ведем на каждом шагу подсчет того, сколько чисел мы взяли, для этого заранее заведем переменную.

После окончания работы цикла проверяем: если остались числа, которые мы не смогли набрать, выводим -1 , в ином случае выводим переменную, в которой мы ввели подсчет, сколько чисел взято на каждом шагу.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1 n, goal = map(int, input().split())
2 arr = list(map(int, input().split()))
3 counted = [0] * 101
4 for value in arr:
5     counted[value] += 1
6 current_need = 1
7 current_goal = goal
8 res = 0
9 while current_need > 0 and current_goal > 0:
10     res += min(current_need, counted[current_goal])
11     current_need = max(0, current_need - counted[current_goal]) * 2
12     current_goal -= 1
13 if current_need > 0:
14     print(-1)

```

```
15 else:
16     print(res)
```

Предметный тур. Физика

Первая волна. Задачи 8–9 класса

Задача II.2.1.1. Конвейер (13 баллов)

Темы: кинематика.

Условие

Робот-доставщик по ошибке заехал на один из концов конвейерной ленты длиной L , движущуюся с постоянной скоростью v в противоположном ее движению направлению. Продолжая двигаться с постоянной относительно ленты скоростью u , робот сумел покинуть конвейер через время t . Определите u . Ответ дайте в м/с, округлив до сотых.

Решение

Скорость робота относительно земли $v_1 = L/t$ складывается (с правильным учетом знаков) из его скорости относительно поверхности ленты и скорости ленты относительно земли:

$$\frac{L}{t} = v_1 = u - v.$$

Откуда простыми алгебраическими преобразованиями получим:

$$u = v + \frac{L}{t}.$$

Погрешность 0,02 м/с.

Диапазоны

Величина	min	max	Шаг
L , м	19	23	1
v , м/с	1,6	1,9	0,05
t , с	150	250	10

Ответ: $u = v + \frac{L}{t}$.

Задача II.2.1.2. Выжигатель (17 баллов)

Темы: плотность, тепловые явления.

Условие

Температура кипения сплава на t выше его текущей температуры. Его удельная теплоемкость c , удельная теплота возгонки (испарения из твердого состояния) L , плотность ρ . Какой должна быть минимальная энергия E лазерного импульса, чтобы он был способен испарить кубик сплава со стороной a при условии полного поглощения энергии импульса веществом? Считайте, что импульс настолько кратковременный, что сплав не успевает пройти жидкую фазу и испаряется непосредственно из твердой. Ответ дайте в Дж, округлив до десятых.

Решение

Теплота, необходимая для нагрева и последующего испарения сплава массы m , находится по формуле:

$$Q = (ct + L)m.$$

По условиям вся энергия импульса поглощается веществом, то есть переходит в тепло. Следовательно, $E = Q$. Масса сплава может быть выражена через его плотность и объем испаренной порции: $m = \rho a^3$. Подставляя, получим:

$$E = (ct + L)\rho a^3.$$

Погрешность 0, 2 Дж.

Диапазоны

Величина	min	max	Шаг
t , °C	2200	2700	10
c , Дж/(кг·°C)	400	500	10
L , кДж/кг	5500	7000	100
ρ , кг/м ³	6000	8000	100
a , мм	0, 6	1	0, 1

Ответ: $E = (ct + L)\rho a^3$. С учетом порядков: $E = (ct + L[\cdot 10^3])\rho a^3[\cdot 10^{-9}]$.

Задача II.2.1.3. Катушка (20 баллов)

Темы: закон Ома.

Условие

При изготовлении реостата на диэлектрическую бобину диаметром D был намотан в N одинаковых, плотно прилегающих к бобине витков, константовый провод в лаковой изоляции. Площадь поперечного сечения провода S , удельное сопротивление константана $\rho = 0,4 \text{ Ом}\cdot\text{мм}^2/\text{м}$. Номинальное сопротивление резистора было вычислено и указано в паспорте устройства, исходя из этих параметров, однако от износа n последних витков проволоки перетерлось и отвалилось. Найдите максимальное значение n , при котором сопротивление реостата отличается от номинального не более, чем на ΔR .

Решение

Один виток провода имеет длину $l = \pi D$ и электрическое сопротивление:

$$r = \frac{\rho l}{S} = \frac{\pi \rho D}{S}.$$

Каждый отвалившийся виток уменьшает общее сопротивление резистора на величину r . Таким образом,

$$n = \left[\frac{\Delta R}{r} \right] = \left[\frac{\Delta R S}{\pi \rho D} \right].$$

Погрешность 1.

Диапазоны

Величина	min	max	Шаг
D , см	3	5	0,1
S , мм ²	0,1	0,2	0,02
N	100	250	10
ΔR , Ом	3	5	0,5

Ответ: $n = \left[\frac{\Delta R S}{\pi \rho D} \right]$. С учетом порядков: $n = \left[\frac{\Delta R S}{\pi \rho D} [\cdot 10^2] \right]$.

Задача II.2.1.4. Болт (20 баллов)

Темы: золотое правило механики, работа.

Условие

При болтовом соединении деталей болт немного удлиняется, работая как растянутая пружина, прижимающая детали друг к другу с некоторой описанной в технической документации силой F . Определите, какую работу A нужно совершить, чтобы растянуть болт на величину Δl и создать таким образом силу F с помощью ключа длиной L , если диаметр резьбы болта равен d , диаметр шляпки D , шаг резьбы — b ? Трение и деформацию самих соединяемых деталей считайте пренебрежимо малым. Ответ дайте в Дж, округлив до целого.

Решение

Резьба болта и гаечный ключ представляют собой простые механизмы, дающие выигрыш в силе, но не изменяющие, согласно золотому правилу механики, работу. Поэтому общая работа, которая нужна, чтобы растянуть болт, вне зависимости от способа, считается как работа переменной силы или как разность потенциальных энергий деформированного тела (болта). При этом малость деформации позволяет применить здесь закон Гука:

$$A = \frac{F \Delta l_{\text{к}}}{2} - \frac{F \Delta l_{\text{н}}}{2}.$$

Учитывая $\Delta l_n = 0$, получим окончательно:

$$A = \frac{F \Delta l}{2}.$$

Погрешность 1 Дж.

Диапазоны

Величина	min	max	Шаг
F , кН	200	250	10
Δl , мм	0,25	0,35	0,01
L , см	40	70	5
d , мм	20	25	1
D , мм	30	40	1
b , мм	2	3	0,1

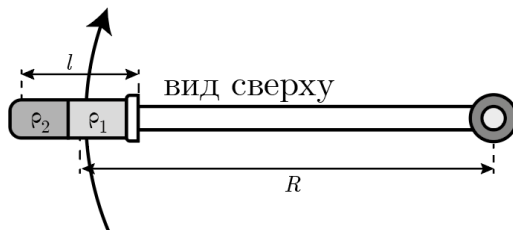
Ответ: $A = \frac{F \Delta l}{2}$.

Задача II.2.1.5. Центрифуга (25 баллов)

Темы: центростремительное ускорение, гидростатическое давление.

Условие

Для разделения жидких смесей в лаборатории используется центрифуга, представляющая собой горизонтальное колесо радиуса R , на ободе которого закрепляются пробирки с жидкостью, ориентированные дном строго от центра колеса. Найдите давление, оказываемое на дно пробирки длиной l при ее вращении в центрифуге, если пробирка совершает один оборот за время T и ровно на половину своего объема пробирка заполнена составом с плотностью ρ_1 и ровно на половину — другим составом с плотностью ρ_2 . Считайте $R \gg l$, а влияние силы тяжести пренебрежимо малым. Ответ дайте в кПа, округлив до целого. Длина окружности в 2π раз больше ее радиуса.



Решение

Линейная скорость пробирки может быть найдена по формуле $v = 2\pi R/T$, поскольку путь, проходимый пробиркой за время T , равен длине окружности с радиусом R . Соответствующее ей центростремительное ускорение a равно:

$$a = \frac{v^2}{R} = \frac{4\pi^2 R}{T^2}.$$

Это ускорение играет роль ускорения свободного падения в формуле гидростатического давления ρgh , что может быть доказано из условий равновесия элемента жидкости. Поскольку жидкости в пробирке две и каждая из них создает столб «высотой» $l/2$, для общего давления получим:

$$p = \frac{2\pi^2 R l}{T^2} (\rho_1 + \rho_2).$$

Погрешность 3 кПа.

Диапазоны

Величина	min	max	Шаг
ν , об/с	5	7	0,5
R , см	90	120	5
l , см	6	8	1
ρ_1 , кг/м ²	800	980	20
ρ_2 , кг/м ²	1020	1200	20

Ответ: $p = \frac{2\pi^2 R l}{T^2} (\rho_1 + \rho_2)$. С учетом порядков $p = \frac{2\pi^2 R l}{T^2} (\rho_1 + \rho_2) [\cdot 10^{-7}]$.

Задача II.2.1.6. (5 баллов)

Темы: физики России.

Условие

Этого выдающегося ученого, обучавшегося еще в технологическом институте Николая I, но удостоенного и ленинской, и сталинской премий, нередко называют родоначальником советской физики. Такие известные физики как Капица и Курча-тов достигли своих выдающихся результатов под его руководством, а сам он был воспитан под руководством первооткрывателя «икс-лучей», используемых теперь в каждой поликлинике.

1. Петр Николаевич Лебедев.
2. Абрам Федорович Иоффе.
3. Николай Алексеевич Умов.
4. Александр Александрович Фридман.
5. Сергей Александрович Ахманов.

6. Рем Викторович Хохлов.
7. Владимир Александрович Фок.
8. Михаил Васильевич Остроградский.

Ответ: 2.

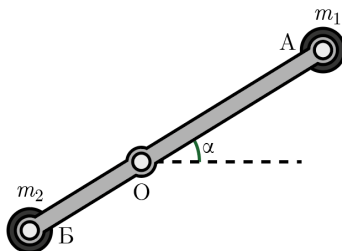
Первая волна. Задачи 10–11 класса

Задача II.2.2.1. Противовес (15 баллов)

Темы: кинематика, импульс.

Условие

Легкий рычаг АОВ, способный вращаться вокруг неподвижной точки О, где $BO = l$, $OA = 2l$ используется на производственной линии для перемещения грузов. В его точке А закреплен груз m_1 , в точке Б — противовес m_2 . В некоторый момент модуль импульса груза в точке А равен p_1 . Найдите модуль импульса противовеса в этот же момент времени. Дайте ответ в $\text{кг} \cdot \text{м/с}$, округлив до целого.



Решение

Поскольку $BO : OA = 1 : 2$, скорости двух масс всегда связаны соотношением $v_1 = 2v_2$. Тогда их импульсы связаны соотношением:

$$\frac{p_2}{p_1} = \frac{m_2 v_2}{m_1 v_1} = \frac{m_2}{2m_1}.$$

Отсюда окончательно получим:

$$p_2 = p_1 \frac{m_2}{2m_1}.$$

Погрешность $1 \text{ кг} \cdot \text{м/с}$.

Диапазоны

Величина	min	max	Шаг
l , м	1	1,5	0,1
m_1 , кг	50	80	5
m_2 , кг	50	80	5
p_1 , кг · м/с	50	160	10

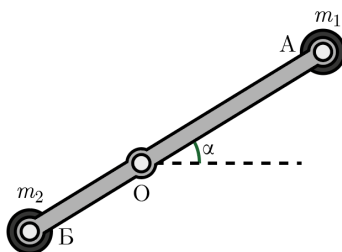
Ответ: $p_2 = p_1 \frac{m_2}{2m_1}$.

Задача II.2.2.2. Качели (20 баллов)

Темы: законы сохранения.

Условие

Легкий рычаг АОВ, способный вращаться вокруг неподвижной точки О, где $BO = l$, $OA = 2l$ используется, на производственной линии для перемещения грузов. В его точке А закреплен груз m_1 , в точке В — противовес m_2 . В некоторый момент электропривод в точке О удерживал рычаг под углом α к горизонту. В этот момент произошла авария, в результате которой привод в точке О перестал действовать, и рычаг пришел в свободное вращение вокруг этой оси. Определите скорость v_1 точки А рычага в момент, когда она оказалась строго под точкой В. Дайте ответ в м/с, округлив до десятых. Ускорение свободного падения $g = 9,8 \text{ м/с}^2$.



Решение

Поскольку $BO : OA = 1 : 2$, скорости двух масс всегда связаны соотношением $v_1 = 2v_2$. Высоты (относительно точки О), на которых находились массы в момент аварии, равны $h_{01} = 2l \sin \alpha$, $h_{02} = -l \sin \alpha$. Аналогично высоты, на которых находились массы в момент, когда точка В оказывается строго под точкой А, равны $h_1 = -2l$, $h_2 = l$.

Тогда закон сохранения энергии для масс на концах рычага имеет вид:

$$(2m_1 - m_2)gl \sin \alpha = (m_2 - 2m_1)gl + \left(m_1 + \frac{m_2}{4}\right) \frac{v_1^2}{2}.$$

Из этого уравнения можно непосредственно выразить скорость v_1 :

$$v_1 = \sqrt{\frac{8gl(2m_1 - m_2)(1 + \sin \alpha)}{4m_1 + m_2}}.$$

Погрешность 0, 2 км/ч.

Диапазоны

Величина	min	max	Шаг
l , м	1	1, 5	0, 1
m_1 , кг	50	80	5
m_2 , кг	50	80	5
α , °	30	70	5

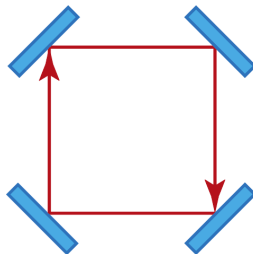
Ответ: $v_1 = \sqrt{\frac{8gl(2m_1 - m_2)(1 + \sin \alpha)}{4m_1 + m_2}}.$

Задача II.2.2.3. Резонатор (20 баллов)

Темы: оптика.

Условие

Кольцевой оптический резонатор представляет собой квадрат со стороной a , в углах которого расположены наклоненные под 45° к ходу луча зеркала. При каждом падении на зеркало $k\%$ энергии света отражается, а остальные $(100 - k)\%$ поглощаются материалом зеркала. Какая доля изначальной энергии останется у светового луча, когда он сделает N полных витков? Ответ дайте в процентах.



Решение

За один виток свет совершает 4 отражения. В ходе каждого из них его энергия изменяется в $k/(100\%)$ раз. Таким образом, общая η доля сохранившейся энергии равна:

$$\eta = \left(\frac{k}{100\%} \right)^{4N} \cdot 100\%.$$

Погрешность 1%.

Диапазоны

Величина	min	max	Шаг
a , см	10	50	5
k	98,5	99,5	0,5
N	20	30	1

Ответ: $\eta = \left(\frac{k}{100\%} \right)^{4N} \cdot 100\%.$

Задача II.2.2.4. Спидометр (20 баллов)

Темы: кинематика.

Условие

Спидометр спутниковой системы навигации, показывая текущую скорость, на самом деле определяет ее как среднюю скорость за последние t движения. Определите мгновенную скорость аппарата, если его спидометр сейчас показывает скорость v_1 , t назад показывал скорость v_2 и известно, что аппарат движется равноускоренно. Ответ дайте в км/ч, округлив до десятых.

Решение

При равноускоренном движении на протяжении некоторого времени t с начальной скоростью v_0 и ускорением a конечная скорость оказывается равна $v = v_0 + at$, а средняя

$$v_{\text{ср}} = \frac{v_0 + v}{2} = v_0 + \frac{at}{2}.$$

При этом, поскольку по прошествии дополнительного времени t и v , и v_0 вырастут на at , средняя скорость также вырастет на at . Следовательно, ускорение аппарата может быть найдено как

$$a = \frac{v_1 - v_2}{t},$$

а его мгновенная скорость:

$$v = v_1 + \frac{at}{2} = v_1 + \frac{v_1 - v_2}{2} = \frac{3v_1 - v_2}{2}.$$

Погрешность 0,1 км/ч.

Диапазоны

Величина	min	max	Шаг
t , с	1, 5	3	0, 25
v_1 , км/ч	52	56	0, 5
v_2 , км/ч	47	51	0, 5

Ответ: $v = \frac{3v_1 - v_2}{2}$.

Задача II.2.2.5. Разрядка (20 баллов)

Темы: электростатика.

Условие

Испытываемая в лаборатории антенна состоит из двух одинаковых металлических шаров, разнесенных на некоторое расстояние и соединенных перемычкой из материала с очень высоким, но конечным сопротивлением. В результате шары обмениваются зарядом таким образом, что за каждый интервал времени t разница между их зарядами сокращается вдвое. В некоторый момент первый шар был заряжен, а второй — нет. Спустя t было измерено, что сила электростатического взаимодействия между шарами антенны равна F_0 . Какой станет эта сила, спустя еще t времени? Явлением электростатической индукции пренебречь. Ответ дайте в нН (наноньютоны), округлив до целого.

Решение

Пусть в начальный момент времени заряженный шар имеет заряд q . Тогда, спустя t , заряды на шарах должны отличаться на $q/2$, что, по закону сохранения заряда, возможно только если их заряды будут равны $3q/4$ и $q/4$. Соответствующая сила взаимодействия находится по закону Кулона:

$$F_0 = k \frac{(q/4)(3q/4)}{r^2} = \frac{3}{16} \cdot \frac{kq^2}{r^2}.$$

Спустя еще t разница между зарядами должна оказаться равна $q/4$, а заряды, соответственно, $5q/8$ и $3q/8$. Соответствующая сила:

$$F = k \frac{(3q/8)(5q/8)}{r^2} = \frac{15}{64} \cdot \frac{kq^2}{r^2} = \frac{5}{4} F_0.$$

Погрешность 1 нН.

Диапазоны

Величина	min	max	Шаг
t , с	5	40	1
F , нН	10	100	2

Ответ: $F = \frac{5}{4} F_0$.

Задача II.2.2.6. (5 баллов)

Темы: физики России.

Условие

Этого выдающегося ученого, обучавшегося еще в технологическом институте Николая I, но удостоенного и ленинской, и сталинской премий, нередко называют родоначальником советской физики. Такие известные физики как Капица и Курчатов достигли своих выдающихся результатов под его руководством, а сам он был воспитан под руководством первооткрывателя «икс-лучей», используемых теперь в каждой поликлинике.

1. Рем Викторович Хохлов.
2. Абрам Федорович Иоффе.
3. Владимир Александрович Фок.
4. Александр Александрович Фридман.
5. Сергей Александрович Ахманов.
6. Петр Николаевич Лебедев.
7. Николай Алексеевич Умов.
8. Михаил Васильевич Остроградский.

Ответ: 2.

Вторая волна. Задачи 8–9 класса

Задача II.2.3.1. Катапульта (10 баллов)

Темы: кинематика.

Условие

Для запуска беспилотного летательного аппарата используется катапульта — длинная балка с установленным на ней линейным двигателем, способным создавать постоянное ускорение a . Для успешного взлета аппарат должен успеть набрать скорость v относительно воздуха до отрыва от катапульты. Вычислите минимальную необходимую длину катапульты, если взлет должен успешно осуществляться при попутном ветре со скоростью не выше u . Ответ дайте в м, округлив до десятых.

Решение

При попутном ветре скорость аппарата относительно воздуха равна $v = v_0 + u$, где v_0 — его скорость относительно катапульты. Как известно из кинематики (или законов сохранения), на расстоянии l , двигаясь с постоянным ускорением a , беспилотник может увеличить квадрат скорости на величину $v^2 = 2al_0$. Отсюда найдем изначальноную длину балки:

$$l = \frac{v^2}{2a} = \frac{(v + u)^2}{2a}.$$

Погрешность 0,1 м.

Диапазоны

Величина	min	max	Шаг
a , м/с ²	25	40	1
v , м/с	9	12	0,5
u , м/с	2	4	0,5

Ответ: $l = \frac{(v + u)^2}{2a}$.

Задача II.2.3.2. Отвердевание (15 баллов)

Темы: плотность.

Условие

Некоторый полимер был получен в жидком состоянии, в котором он целиком занимал лабораторную чашку объемом V_0 и имел плотность ρ_0 . Затем он был нагрет в специальной печи, в результате чего из состава испарились все летучие фракции общей массой Δm , а оставшееся вещество запеклось, образовав твердый сгусток с плотностью ρ . Определите объем, занимаемый твердым веществом в чашке, если известно, что пустоты в процессе запекания не образуются и никакие внешние соединения не включаются в полимер. Ответ дайте в см³, округлив до целого.

Решение

Изначальная масса полимера равна $m_0 = \rho_0 V_0$. За счет испарения она уменьшилась на Δm . Процесс отвердевания не изменяет массы вещества, поэтому объем образовавшегося твердого вещества равен $V = (m_0 - \Delta m)/\rho$. Окончательно:

$$V = \frac{\rho_0 V_0 - \Delta m}{\rho}.$$

Погрешность 5 см³.

Диапазоны

Величина	min	max	Шаг
V_0 , см ³	1200	1500	100
ρ_0 , г/см ³	0,9	1,1	0,02
ρ , г/см ³	1,4	1,7	0,02
Δm , г	100	150	10

Ответ: $V = \frac{\rho_0 V_0 - \Delta m}{\rho}$.

Задача II.2.3.3. Три образца (20 баллов)

Темы: электростатика.

Условие

В лабораторию поступило три металлических образца одинаковых размеров А, Б и В, электрически заряженных в ходе трех разных малоисследованных процессов. Лаборанты записали результаты измерений их зарядов $q_{1,2,3}$, но позже выяснилось, что процедура проведения эксперимента была нарушена и было допущено соприкосновение образцов друг с другом. В ходе анализа записей лабораторной камеры удалось установить, что вначале соприкоснулись образцы А и Б, после чего на одном из них был измерен заряд q_1 . Затем с этим образцом дополнительно соприкоснулся образец В, после чего на нем был измерен заряд q_2 . После этого все три образца соприкоснулись одновременно и на одном из них был измерен заряд q_3 . Восстановите по этим данным исходное значение заряда образца В. Считайте, что процедура измерения величины заряда на образце не меняет. Ответ дайте в нКл (нанокулонах), округлив до целого.

Решение

Когда два металлических предмета одинаковых размеров соприкасаются, на них устанавливаются одинаковые электрические заряды. По закону сохранения заряда они должны быть равны среднему арифметическому исходных зарядов.

Тогда после первого соприкосновения

$$q_1 = \frac{q_A + q_B}{2},$$

после второго

$$q_2 = \frac{q_1 + q_B}{2},$$

а после третьего

$$q_3 = \frac{q_A + q_B + q_B}{3} = \frac{2q_1 + q_B}{3}.$$

Решая эту систему уравнений, легко получим:

$$q_B = 4q_2 - 3q_3.$$

Погрешность 1 нКл.

Диапазоны

Величина	min	max	Шаг
q_1 , нКл	-20	20	4
q_2 , нКл	-20	20	4
q_3 , нКл	-20	20	4

Ответ: $q_B = 4q_2 - 3q_3$.

Задача II.2.3.4. Топливо (25 баллов)

Темы: тепловые явления.

Условие

На некоторой планете были обнаружены залежи жидкого состава, состоящего из двух трудно разделимых компонент. Первая из них химически инертна (не участвует ни в каких превращениях) и имеет теплоемкость c_1 . Вторая имеет теплоемкость c_2 и горит в атмосфере планеты с удельной теплотой сгорания q . Смесь воспламеняется при температуре θ , а окружающая среда планеты имеет температуру t . Определите, какую минимальную долю от общей массы смеси должна составлять масса горючей компоненты, чтобы горение смеси производило не меньше энергии, чем уходит на ее нагрев до температуры воспламенения. Ответ дайте в процентах, округлив до десятых.

Решение

Рассмотрим некоторую массу m смеси. Она содержит $m_2 = \alpha m$ горючей и $m_1 = (1 - \alpha)m$ негорючей жидкостей. Для воспламенения необходимо нагреть смесь на $\Delta t = \theta - t$, на что уйдет

$$Q_1 = (c_1 m_1 + c_2 m_2) \Delta t = m(c_1(1 - \alpha) + c_2 \alpha)(\theta - t)$$

теплоты. При этом от сгорания второй компоненты выделится $Q_2 = q m_2$ теплоты. Приравнявая Q_1 и Q_2 , получим:

$$\alpha q = m(c_1(1 - \alpha) + c_2 \alpha)(\theta - t),$$

откуда окончательно выразим ответ:

$$\alpha = \frac{c_1(\theta - t)}{(\theta - t)(c_1 - c_2) + q} \cdot 100\%.$$

Погрешность 0,1%.

Диапазоны

Величина	min	max	Шаг
c_1 , Дж/(кг·°C)	300	500	20
c_2 , Дж/(кг·°C)	310	510	20
q , МДж/(кг)	13	17	0,5
θ , °C	800	950	10
t , °C	-80	-50	5

Ответ: $\alpha = \frac{c_1(\theta - t)}{(\theta - t)(c_1 - c_2) + q} \cdot 100\%.$

С учетом порядков $\alpha = \frac{c_1(\theta - t)}{(\theta - t)(c_1 - c_2) + q[10^6]} \cdot 100\%.$

Задача II.2.3.5. Геккон (25 баллов)

Темы: давление, сила трения.

Условие

Робот-геккон может перемещаться по гладким вертикальным поверхностям, используя присоски. Под каждой присоской площади S при помощи системы компрессоров, откачивающих из-под присосок воздух, создается давление p , в то время как атмосферное давление $p_0 = 100$ кПа.

Какую максимальную массу может иметь такой робот, чтобы не соскальзывать с вертикальной поверхности, если он одновременно использует n одинаковых присосок (коэффициент трения резины присосок равен μ)?

Ускорение свободного падения принять равным $g = 9,8$ м/с². Ответ дайте в кг, округлив до десятых.

Решение

На каждую присоску действует прижимная сила, равная произведению ее площади на разницу внешнего и внутреннего давлений:

$$N = S(p_0 - p).$$

Возникающие в этих присосках силы трения $F_{\text{тр}} = \mu N$ должны суммарно уравновешивать вес робота:

$$mg = nF_{\text{тр}} = \mu nS(p_0 - p).$$

Таким образом, окончательно

$$m = \frac{\mu nS(p_0 - p)}{g}.$$

Погрешность 0,2 кг.

Диапазоны

Величина	min	max	Шаг
S , см ²	10	35	2,5
μ	0,12	0,25	0,01
p , кПа	10	40	5
n	6	12	2

Ответ: $m = \frac{\mu nS(p_0 - p)}{g}$. С учетом порядков $m = \frac{\mu nS(p_0 - p)}{g} [\cdot 10^{-1}]$.

Задача II.2.3.6. (5 баллов)

Темы: физики России.

Условие

Один из основателей нелинейной оптики, новой ветви науки, продемонстрировавшей, что достаточно интенсивные лучи света могут взаимодействовать друг с другом и сами с собой, фокусироваться без линзы и неожиданно менять цвет. В его честь на территории Московского университета названа улица, лаборатория, спортивный клуб и несколько учебных аудиторий. Помимо выдающихся научных достижений, он также проявил себя как талантливый организатор, способствовал развитию кооперации исследователей самых разных направлений, включая биологию и экологию, а также был профессиональным альпинистом с двадцатилетним стажем. Трагедия в одном из горных походов, к несчастью, оборвала его выдающуюся жизнь.

1. Сергей Александрович Ахманов.
2. Рем Викторович Хохлов.
3. Анатолий Алексеевич Логунов.
4. Петр Николаевич Лебедев.
5. Абрам Федорович Иоффе.
6. Александр Александрович Фридман.
7. Роберт Эмильевич Ленц.
8. Николай Алексеевич Умов.

Ответ: 2.

Вторая волна. Задачи 10–11 класса

Задача II.2.4.1. Кабель (12 баллов)

Темы: закон Ома, сопротивление.

Условие

На мобильной исследовательской станции используются стандартные резервные кабели для большинства электроприборов, имеющие площадь поперечного сечения s и длину l . В документации кабеля указано, что при подключении к стандартному лабораторному источнику постоянного тока I падение напряжения на кабеле составляет U . Определите удельное сопротивление материала кабеля. Ответ дайте в Ом $\cdot$ мм²/м, округлив до тысячных.

Решение

Сопротивления r кабеля питания вычисляются по формуле:

$$r = \frac{\rho l}{s}.$$

Согласно закону Ома для участка цепи $I = U/r$, оно также может быть выражено в виде:

$$\frac{\rho l}{s} = r = \frac{U}{I}.$$

Отсюда окончательно получим:

$$\rho = \frac{Us}{Il}.$$

Погрешность $0,002 \text{ Ом} \cdot \text{мм}^2/\text{м}$.

Диапазоны

Величина	min	max	Шаг
l , м	5	8	0,5
s , мм ²	1,6	2	0,1
I , А	1	2	0,1
U , В	0,1	0,2	0,01

Ответ: $\rho = \frac{Us}{Il}.$

Задача II.2.4.2. Мороз (15 баллов)

Темы: закон Джоуля – Ленца.

Условие

На мобильной полярной станции вышла из строя основная система обогрева, и пришлось в срочном порядке подключать давно не использовавшуюся резервную. Резервный нагреватель представляет собой теплопроводящий корпус, защищающий катушку, на которую намотан провод длиной L и площадью поперечного сечения S , обеспечивающий эффективную конвекцию. Штатный кабель питания от нагревателя, к сожалению, был утерян, поэтому нагреватель пришлось подключить к сети при помощи стандартного резервного кабеля, имеющего площадь s и длину l . Определите отношение P_n/P_k тепловой мощности, выделяющейся в нагревателе к тепловой мощности, выделяющейся в кабеле питания, если токонесущие жилы кабеля питания и провода нагревателя изготовлены из одного вещества. Дайте ответ с точностью до сотых.

Решение

Сопротивления R нагревателя и r кабеля питания вычисляются по формулам:

$$R = \rho \frac{L}{S}; \quad r = \rho \frac{l}{s},$$

где ρ — (одинаковое в обоих случаях) удельное сопротивление. Их отношение равно

$$\frac{R}{r} = \frac{Ls}{lS}.$$

Поскольку кабель и нагреватель включены в цепь последовательно, проходящие через них силы тока равны и тепловую мощность удобно искать по закону Джоуля – Ленца:

$$P_n = I^2 R; \quad P_k = I^2 r.$$

Таким образом, окончательно

$$\frac{P_{\text{н}}}{P_{\text{к}}} = \frac{l^2 R}{l^2 r} = \frac{L s}{l S}.$$

Погрешность 0,05.

Диапазоны

Величина	min	max	Шаг
L , м	15	20	1
l , м	5	8	0,5
S , мм ²	2,5	3,5	0,2
s , мм ²	1,6	2	0,1

Ответ: $\frac{P_{\text{н}}}{P_{\text{к}}} = \frac{L s}{l S}.$

Задача II.2.4.3. Стопка (20 баллов)

Темы: конденсаторы.

Условие

На тонкий полимерный лист с обеих сторон напыляется металлическое покрытие. Измерения показывают, что емкость такого конденсатора равна C_0 . Затем лист разрезают на n равных частей и складывают их в стопку. Определите емкость C такой стопки при подключении источника напряжения между самым верхним и самым нижним ее слоями. Ответ дайте в пФ (пикофарадах), округлив до целого.

Решение

Емкость плоского конденсатора задается формулой:

$$C = \frac{\varepsilon \varepsilon_0 S}{d},$$

поэтому при разрезании листа на n частей площадь и, соответственно, емкость C_1 каждой из этих частей, оказывается в n раз меньше исходной: $C_1 = C_0/n$.

В то же время стопка конденсаторов представляет собой цепь из n одинаковых конденсаторов, соединенных последовательно. Поскольку эффективная емкость C такой цепи может быть найдена по формуле:

$$\frac{1}{C} = \frac{1}{C_1} + \frac{1}{C_1} + \dots + \frac{1}{C_1} = \frac{n}{C_1} = \frac{n^2}{C_0},$$

получим окончательно

$$C = \frac{C_0}{n^2}.$$

Погрешность 2 пФ.

Диапазоны

Величина	min	max	Шаг
C , нФ	2	6	0,5
n	4	10	1

Ответ: $C = \frac{C_0}{n^2}$. С учетом порядков $C = \frac{C_0}{n^2} [\cdot 10^3]$.

Задача II.2.4.4. Два колеса (23 баллов)

Темы: кинематика.

Условие

Два колеса двухколесного балансирующего робота, исследующего отдаленный астрономический объект с очень гладкой поверхностью, расположены на разных концах одной оси длиной l . Проведя сеанс связи с Землей, робот получил указание двигаться прямо и стал вращать колеса с одинаковой угловой скоростью. Однако в ходе посадки робота одно из колес было слегка повреждено, в связи с чем его радиус оказался на Δr меньше, чем радиус r другого. Найдите Δr , если известно, что вместо прямой робот начал двигаться по окружности радиуса R (измеренного по центру робота). Колеса робота не проскальзывают по поверхности, кривизна астрономического объекта пренебрежимо мала. Ответ дайте в мм, округлив до десятых.

Решение

При равных угловых скоростях ω линейные скорости колес имеют величины ωr и $\omega(r - \Delta r)$. При этом, поскольку робот движется по окружности, то за время, за которое внешнее колесо проходит дугу длиной $\alpha(R + l/2)$, внутреннее колесо проходит дугу $\alpha(R - l/2)$. Следовательно

$$\frac{\omega(r - \Delta r)}{\omega r} = \frac{R - l/2}{R + l/2}.$$

Преобразуя это выражение, получим окончательно:

$$\Delta r = \frac{2rl}{2R + l}.$$

Погрешность 0,2 мм.

Диапазоны

Величина	min	max	Шаг
r , см	40	60	4
l , см	80	120	10
R , м	80	120	5

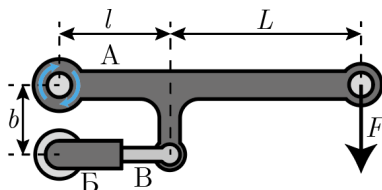
Ответ: $\Delta r = \frac{2rl}{2R+l}$. С учетом порядков $\Delta r = \frac{2rl \cdot 10^1}{2R \cdot 10^2 + l}$.

Задача II.2.4.5. Рычаг (25 баллов)

Темы: статика, давление газа.

Условие

Подъемный рычаг А, геометрические параметры b, l, L которого изображены на рисунке, шарнирно соединен с приводящим его в движение поршнем В, входящим в цилиндр Б, внутри которого содержится идеальный газ. Пренебрегая весом самого рычага, определите избыточное (над атмосферным) давление p в цилиндре, которое необходимо для удержания нагрузки F , приложенной к концу рычага, если площадь поршня равна S . Ответ дайте в МПа, округлив до десятых.



Решение

Относительно шарнира рычага сила F имеет плечо $L+l$. Сила давления поршня равна pS и имеет плечо b . Тогда условие равновесия имеет вид:

$$bpS = (l+L)F.$$

Отсюда

$$p = \frac{(l+L)F}{bS}.$$

Погрешность 0,2 МПа.

Диапазоны

Величина	min	max	Шаг
l , см	15	25	2
L , см	30	50	5
b , см	8	16	1
S , см ²	16	24	2
F , кН	2	3	0,5

Ответ: $p = \frac{(l+L)F}{bS}$. С учетом порядков $p = \frac{(l+L)F}{bS} \cdot 10^1$.

Задача II.2.4.6. (5 баллов)

Темы: физики России.

Условие

Один из основателей нелинейной оптики, новой ветви науки, продемонстрировавший, что достаточно интенсивные лучи света могут взаимодействовать друг с другом и сами с собой, фокусироваться без линзы и неожиданно менять цвет. В его честь на территории Московского университета названа улица, лаборатория, спортивный клуб и несколько учебных аудиторий. Помимо выдающихся научных достижений, он также проявил себя как талантливый организатор, способствовал развитию кооперации исследователей самых разных направлений, включая биологию и экологию, а также был профессиональным альпинистом с двадцатилетним стажем. Трагедия в одном из горных походов, к несчастью, оборвала его выдающуюся жизнь.

1. Сергей Александрович Ахманов.
2. Рем Викторович Хохлов.
3. Анатолий Алексеевич Логунов.
4. Петр Николаевич Лебедев.
5. Александр Александрович Фридман.
6. Николай Алексеевич Умов.
7. Абрам Федорович Иоффе.
8. Роберт Эмильевич Ленц

Ответ: 2.

Третья волна. Задачи 8–9 класса

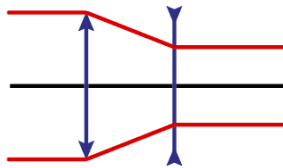
Задача II.2.5.1. Большой глаз (15 баллов)

Темы: геометрическая оптика.

Условие

Первый элемент системы ночного наблюдения имеет своей целью сузить пучок параллельных лучей, собираемых с большой площади, оставив его параллельным и неперевернутым. Он состоит из двух линз: собирающей и рассеивающей с оптическими силами D_1 и D_2 соответственно, установленных друг за другом и имеющих общую оптическую ось. Найдите расстояние между линзами. Ответ дайте в см, округлив до десятых.

Напоминание: оптической силой называется величина, обратная фокусному расстоянию линзы.



Решение

Изобразим описанную в условиях задачи оптическую схему. Из рисунка несложно видеть, что для описанного хода лучей фокусы двух линз должны совпадать. Но по определению оптической силы, фокусное расстояние линзы равно

$$F = \frac{1}{D}.$$

Подставляя в эту формулу D_1, D_2 и находя разницу между соответствующими расстояниями (верно учитывая знаки), получим:

$$l = \frac{1}{D_1} + \frac{1}{D_2}.$$

Погрешность 0, 2 см.

Диапазоны

Величина	min	max	Шаг
D_1 , дптр	3	5	0, 2
D_2 , дптр	-10	-8	0, 2

Ответ: $l = \frac{1}{D_1} + \frac{1}{D_2}$. С учетом порядков $l = \left(\frac{1}{D_1} + \frac{1}{D_2} \right) [\cdot 10^2]$.

Задача II.2.5.2. Атмосфера (15 баллов)

Темы: давление газа, гидростатика.

Условие

Оцените массу атмосферы, окружающей планету земного типа радиусом R , если ускорение свободного падения на ее поверхности равно g , а атмосферное давление — p_0 . Площадь сферы вычисляется по формуле $S = 4\pi R^2$. Ответ дайте в квинтлин (квинтиллионах) (10^{18}) кг, округлив до десятых.

Решение

Давление — это отношение силы к площади, перпендикулярно которой эта сила действует. В данном случае сила — общий вес атмосферы:

$$p_0 = \frac{mg}{S} = \frac{mg}{4\pi R^2}.$$

Для любой планеты земного типа толщина атмосферы пренебрежимо мала в сравнении с радиусом планеты, поэтому изменением ускорения свободного падения с высотой можно пренебречь. Получим окончательно

$$m = \frac{4\pi R^2 p_0}{g}.$$

Погрешность $0,1 \cdot 10^{15}$ т.

Диапазоны

Величина	min	max	Шаг
p_0 , кПа	12	24	1
g , м/с ²	2	3,5	0,1
R , км	3600	4600	100

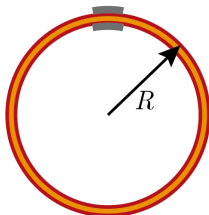
Ответ: $m = \frac{4\pi R^2 p_0}{g}$. С учетом порядков $m = \frac{4\pi R^2 p_0}{g} [10^{-9}]$.

Задача II.2.5.3. Магниты (20 баллов)

Темы: центростремительное ускорение, динамика.

Условие

К тонкому ободу медного колеса радиусом R , расположенного в горизонтальной плоскости, снаружи и изнутри прикреплены два одинаковых маленьких магнитных датчика массой m каждый. Датчики держатся только за счет притяжения друг к другу. Колесо начинают постепенно раскручивать вокруг неподвижной оси, и в момент, когда период его вращения достигает величины T , внешний датчик отлетает от колеса. Определите силу давления внутреннего датчика на обод колеса непосредственно после этого. Ответ дайте в Н, округлив до целого. Длина окружности в 2π раз больше ее радиуса.



Решение

Внешний датчик отделяется в момент, когда сила F взаимодействия между магнитами оказывается недостаточна для того, чтобы создавать центростремительное ускорение $a = v^2/R$, где скорость v может быть найдена как отношение длины окружности к периоду обращения ($v = 2\pi R/T$):

$$F = ma = \frac{mv^2}{R} = \frac{4\pi^2}{T^2}mR.$$

Поскольку размеры датчиков и толщина колеса считаются пренебрежимо малыми, внутренний датчик продолжает двигаться с тем же ускорением, а значит, давить на колесо с той же силой F :

$$F = \frac{4\pi^2}{T^2}mR.$$

Погрешность 1 Н.

Диапазоны

Величина	min	max	Шаг
R , см	36	45	3
m , г	200	300	20
T , с	0,3	0,5	0,05

Ответ: $F = \frac{4\pi^2}{T^2}mR$. С учетом порядков $F = \frac{4\pi^2}{T^2}mR[\cdot 10^{-5}]$.

Задача II.2.5.4. Пот (20 баллов)

Темы: тепловые явления, кинематика жидкости.

Условие

Для охлаждения антропоморфного робота разрабатывается система, воспроизводящая потоотделение человека. По тонким капиллярам с площадью поперечного сечения S на поверхность робота поступает вода, которая затем растекается по «коже» робота и постепенно испаряется. Определите, какой должна быть постоянная скорость v течения воды в капилляре, если один такой капилляр должен обеспечивать отведение тепловой мощности N . Считайте, что вся вода успевает испариться. Удельная теплота парообразования воды $L = 2300$ кДж/кг, ее плотность $\rho = 1000$ кг/м³. Ответ дайте в мм/с, округлив до целого.

Решение

Рассмотрим произвольный промежуток времени t . Согласно условиям, теплота Q , которую необходимо отвести от робота за это время, равна $Q = Nt$. Она может

быть определена через удельную теплоту парообразования и плотность воды как

$$Nt = Q = mL = \rho VL,$$

где объем испаренной жидкости V равен произведению расстояния d , которое прошла вода в капилляре на площадь его поперечного сечения:

$$V = dS = vtS.$$

Сопоставляя эти уравнения, получим

$$Nt = \rho vtSL,$$

откуда окончательно

$$v = \frac{N}{\rho SL}.$$

Погрешность 1 мм/с.

Диапазоны

Величина	min	max	Шаг
N , Вт	10	20	1
S , мм ²	0,1	0,2	0,01

Ответ: $v = \frac{N}{\rho SL}$. С учетом порядков $v = \frac{N}{\rho SL} [\cdot 10^6]$.

Задача II.2.5.5. Вагончик (25 баллов)

Темы: закон Ома, кинематика.

Условие

Автоматизированный вагончик движется по двум длинным рельсам, по одному из которых на него подается, а с другого — снимается электрический ток. Каждый метр одного рельса имеет сопротивление r , а двигатели вагончика — полное сопротивление R . Источник питания подает на рельсы напряжение U_0 , а для поддержания нормальной работы двигателей напряжение на них должно быть не ниже U . За какое время, стартовав от источника и двигаясь с постоянной скоростью v , вагончик сможет уехать достаточно далеко, чтобы двигатели перестали работать? Ответ дайте в с, округлив до целых.

Решение

Общее сопротивление цепи, в которую включены двигатели вагончика, равно

$$R_0 = 2r \frac{L}{l} + R,$$

где L — расстояние до источника, $l = 1$ м. Сила тока в цепи $I = U_0/R_0$, а напряжение на двигателе $U_d = IR$ (согласно закону Ома). Подставляя $U_d = U$, $L = vt$, получим:

$$U = U_0 \frac{R}{R_0} = U_0 \frac{R}{2rvt/l + R}.$$

Решая это уравнение относительно t , получим окончательный ответ:

$$t = \frac{R(U_0 - U)}{2rvU} \cdot 1 \text{ м.}$$

Погрешность 1 мин.

Диапазоны

Величина	min	max	Шаг
R , Ом	400	640	40
r , мОм	400	640	40
U , В	200	220	5
U_0 , В	320	360	5
v , м/с	6	8	0,5

Ответ: $t = \frac{R(U_0 - U)}{2rvU} \cdot 1 \text{ м.}$ С учетом порядков $t = \frac{R(U_0 - U)}{2rvU} [\cdot 10^3]$.

Задача II.2.5.6. (5 баллов)

Темы: физики России.

Условие

Вектор, описывающий плотность потока энергии в волнах, в англоязычной традиции носит фамилию английского физика, который в 1884 году вывел выражения для данного вектора в случае электромагнитных волн. Десятью годами ранее русский физик и философ вывел аналогичные уравнения для упругих волн, а потому в русской традиции вектор носит и его имя. Помимо выдающихся успехов в теории упругости, он сделал множество значимых открытий в оптике, а также выдвинул гипотезы о качественно правильном характере связи между массой и энергией, позже развитые в знаменитую формулу Эйнштейна $E = mc^2$. Выберите из приведенного списка выдающихся физиков имя этого русского ученого.

1. Владимир Александрович Фок.
2. Николай Алексеевич Умов.
3. Михаил Васильевич Остроградский.
4. Роберт Эмильевич Ленц.
5. Петр Николаевич Лебедев.
6. Александр Александрович Фридман.
7. Анатолий Алексеевич Логонов.
8. Рем Викторович Хохлов.

Ответ: 2.

Третья волна. Задачи 10–11 класса

Задача II.2.6.1. Поглотитель (15 баллов)

Темы: тепловые явления, радиоактивность.

Условие

Быстро движущиеся нейтроны, образующиеся при делении атомных ядер, взаимодействуя с молекулами воды, теряют энергию до значений, соответствующих энергии теплового движения, которая, как правило, много меньше энергии ядерного распада. Оцените, какое число нейтронов, движущихся со скоростью v , должно потерять свою энергию в кювете с водой в виде прямоугольного параллелепипеда со сторонами a, b, d , чтобы вода в этой кювете нагрелась на $\Delta t = 1^\circ\text{C}$. Удельная теплоемкость воды $c = 4,2 \text{ кДж}/(\text{кг}\cdot^\circ\text{C})$, ее плотность $\rho = 1000 \text{ кг}/\text{м}^3$. Масса нейтрона $m = 1,67 \cdot 10^{-27} \text{ кг}$. Ответ дайте в квдрлн (квадриллионах) (10^{15}) штук, округлив до целого.

Решение

Один нейтрон обладает кинетической энергией $K = mv^2/2$. Поскольку эта энергия на несколько порядков величины выше характерной энергии теплового движения, которую можно оценить как $kT \approx 4 \cdot 10^{-21} \text{ Дж}$ для комнатной температуры, можно считать, что вся его энергия передается воде. Для нагрева воды на Δt требуется энергия, равная

$$Q = cm\Delta t = c\rho abd\Delta t.$$

Приравнявая ее nK , получим ответ:

$$n = 2 \frac{c\rho abd\Delta t}{mv^2}.$$

Погрешность 3 квдрлн.

Диапазоны

Величина	min	max	Шаг
v , км/с	10 000	20 000	1000
a , см	40	60	5
b , см	10	16	2
d , см	10	16	2

Ответ: $n = 2 \frac{c\rho abd\Delta t}{mv^2}$. С учетом порядков $n = 2 \frac{c\rho abd\Delta t}{mv^2} [\cdot 10^{-24}]$.

Задача II.2.6.2. Дрейф (18 баллов)

Темы: электрический ток, МКТ.

Условие

В некотором полупроводнике концентрация подвижных электронов равна n . Какой должна быть дрейфовая (средняя) скорость электронов в элементе с площадью поперечного сечения S , изготовленном из этого полупроводника, чтобы сила электрического тока в данном сечении была равна I ? Модуль заряда электрона $e = 1,6 \cdot 10^{-19}$ Кл. Ответ дайте в см/с, округлив до целого.

Решение

Рассмотрим произвольный отрезок времени t . При силе тока I за это время через сечение проводника должен пройти заряд $q = It$. Этот заряд равен произведению числа N пересекающих сечение электронов на заряд одного из них e . В то же время при скорости дрейфа v в среднем за время t электроны проходят расстояние $l = vt$. Это значит, что через сечение проходят электроны, содержащиеся в объеме $V = lS = vtS$. Из определения концентрации следует, что их общее число $N = nV$. Совмещая все эти выражения, получим

$$It = q = enV = envtS,$$

откуда легко выражается ответ:

$$v = \frac{I}{Sne}.$$

Погрешность 2 см/с.

Диапазоны

Величина	min	max	Шаг
$n, 10^{18} \text{ м}^{-3}$	6	9	1
$S, \text{ мм}^2$	2, 4	3	0, 1
$I, \text{ мкА}$	1, 5	2, 5	0, 1

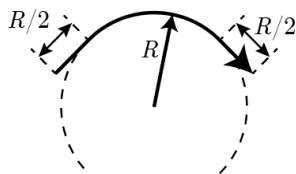
Ответ: $v = \frac{I}{Sne}$. С учетом порядков $v = \frac{I}{Sne} [\cdot 10^{-16}]$.

Задача II.2.6.3. Виразж (20 баллов)

Темы: кинематика.

Условие

Квадракopter может двигаться по любой траектории с условием, что его скорость ни в одной точке этой траектории не превышает v , а его ускорение не превышает a (при этом направления скорости и ускорения не имеют значения). За какое минимальное время он сможет пройти по траектории, изображенной на рисунке, состоящей из двух прямолинейных участков и четверти окружности радиуса R , если в начальной и в конечной точках коптер должен иметь строго нулевую скорость? Ответ дайте в с, округлив до десятых.



Решение

Общая длина траектории $(4 + \pi/2)R < 6R$. Однако, даже двигаясь все время с набором скорости на расстоянии $6R$ с ускорением a , строго сонаправленным со скоростью, коптер успел бы разогнаться только до

$$v_1 = \sqrt{3aR}.$$

Поскольку эта скорость меньше v , ограничения на максимальную скорость в задаче на самом деле несущественны. В то же время ограничения на скорость накладывает центростремительное ускорение, с которым коптер проходит изогнутый участок: поскольку оно не должно превышать a , скорость на повороте не может быть выше $v_1 = \sqrt{aR}$. Но из формулы

$$\frac{v_1^2 - v_0^2}{2} = al = \frac{aR}{2}$$

можно видеть, что v_1 в точности совпадает со скоростью, которую успевает набрать квадрокоптер на первом прямолинейном и сбросить на втором прямолинейном участках. Таким образом, он должен двигаться равноускоренно на двух участках длиной $R/2$, каждый из которых отнимет время

$$t_1 = \frac{v_1}{a} = \sqrt{\frac{R}{a}}$$

и равномерно на повороте, на что уйдет время

$$t_2 = \frac{\pi R}{2v_1} = \frac{\pi R}{2\sqrt{aR}} = \frac{\pi}{2} \sqrt{\frac{R}{a}}$$

В общей сложности, подобный полет займет время $t_2 + 2t_1$:

$$t = \sqrt{\frac{R}{a}} \left(2 + \frac{\pi}{2} \right).$$

Погрешность 0,2 с.

Диапазоны

Величина	min	max	Шаг
v , м/с	20	40	2
a , м/с ²	3	4	0,2
R , м	15	20	1

Ответ: $t = \sqrt{\frac{R}{a}} \left(2 + \frac{\pi}{2} \right).$

Задача II.2.6.4. Звездная величина (20 баллов)

Темы: фотометрия, геометрическая оптика.

Условие

Для получения качественного изображения тусклых звезд важно собрать в телескоп максимально возможную долю испущенного такой звездой излучения, что требует больших размеров основного зеркала или линзы телескопа. Яркость звезды в астрономии принято измерять в единицах видимой звездной величины (ВЗВ), увеличение ВЗВ на 1 означает уменьшение световой энергии, испускаемой звездой, в $k = \sqrt[5]{100}$ раз. При помощи телескопа-рефрактора с площадью главного зеркала s получено качественное изображение некоторой звезды. Найдите площадь S главного зеркала второго телескопа, позволяющего получить в таком же качестве изображение звезды, ВЗВ которой больше на n единиц. Ответ дайте в см^2 , округлив до целого.

Решение

Полная мощность светового потока, попадающего в объектив телескопа, определяется как произведение площади S этого объектива на плотность этого потока I . Увеличение видимой звездной величины на n единиц означает уменьшение плотности светового потока в k^n раз. Следовательно, для компенсации этого изменения площадь объектива должна быть в k^n раз увеличена:

$$\frac{S}{s} = k^n.$$

Отсюда

$$S = sk^n = 100^{n/5}s.$$

Погрешность 5 см^2 .

Диапазоны

Величина	min	max	Шаг
$s, \text{ см}^2$	20	50	10
n	2	5	1

Ответ: $S = sk^n = 100^{n/5}s$.

Задача II.2.6.5. Большая линза (22 баллов)

Темы: фотометрия, масса и плотность.

Условие

Яркость звезды принято измерять в единицах видимой звездной величины (ВЗВ), одна единица которой означает изменение световой энергии, испускаемой звездой,

в $k = \sqrt[5]{100}$ раз. Определите, какой массы линзу объектива телескопа-рефрактора пришлось бы использовать при сохранении пропорций и материала линзы, чтобы получить изображение некоторой тусклой звезды в таком же качестве, как было получено изображение на n единиц ВЗВ более яркой звезды при помощи телескопа с объективом, масса которого составляла m . Ответ дайте в т, округлив до десятых.

Решение

Как и в предыдущей задаче, площадь объектива должна быть в k^n раз увеличена. Это требует изменения радиуса объектива в $\sqrt{k^n} = k^{n/2}$ раз. Но при сохранении пропорций объем любого тела и, следовательно, масса такого объектива должны измениться пропорционально кубу радиуса:

$$M = mk^{3n/2}.$$

Погрешность 0,1 т.

Диапазоны

Величина	min	max	Шаг
m , г	100	200	20
n	7	9	1

Ответ: $vM = k^{3n/2}m = 100^{0,3n}m$. С учетом порядков $M = 100^{0,3n}m \cdot 10^{-6}$.

Задача II.2.6.6. (5 баллов)

Темы: физики России.

Условие

Вектор, описывающий плотность потока энергии в волнах, в англоязычной традиции носит фамилию английского физика, который в 1884 году вывел выражения для данного вектора в случае электромагнитных волн. Десятью годами ранее русский физик и философ вывел аналогичные уравнения для упругих волн, а потому в русской традиции вектор носит и его имя. Помимо выдающихся успехов в теории упругости, он сделал множество значимых открытий в оптике, а также выдвинул гипотезы о качественно правильном характере связи между массой и энергией, позже развитые в знаменитую формулу Эйнштейна $E = mc^2$. Выберите из приведенного списка выдающихся физиков имя этого русского ученого.

1. Владимир Александрович Фок.
2. Александр Александрович Фридман.
3. Рем Викторович Хохлов.
4. Николай Алексеевич Умов.
5. Михаил Васильевич Остроградский.
6. Роберт Эмильевич Ленц.

7. Петр Николаевич Лебедев.
8. Анатолий Алексеевич Логунов.

Ответ: 4.

Инженерный тур

Инженерные задания первого тура отобраны так, чтобы проверить компетенции, необходимые для освоения образовательной программы профиля и решения заданий следующих этапов. В первом этапе задания посвящены алгоритмам компьютерного зрения без машинного обучения: бинаризация, контурный анализ, детектирование по цветам, анализ гистограмм изображений, распознавание сравнением с шаблоном, двумерная графика, уравнения геометрических фигур на плоскости. Одновременно с этим задания были достаточно простыми, чтобы неподготовленные участники не потеряли интерес к профилю и приступили к освоению образовательной программы. Первые три шага образовательной программы профиля рассматривают все темы, необходимые для решения представленных заданий.

Без обладания вышеперечисленными компетенциями решение задачи заключительного этапа и изучение части образовательной программы, посвященной машинному обучению в компьютерном зрении, невозможно.

Задача II.3.1. Вычисление расстояний между фигурами (10 баллов)

Темы: представление изображения в памяти компьютера, декартовы координаты и евклидово расстояние, бинаризация, поиск контуров, алгоритмы перебора и сравнения элементов списка.

Условие

На изображении [II.3.1](#) представлены от трех до десяти геометрических фигур разных форм, цветов и размеров. Цвет фона на всем изображении одинаковый. Фигуры не касаются краев изображения и друг друга. Расстояние от одной фигуры до другой — наименьшая дистанция между пикселями их контуров. Расстояние между парой пикселей — евклидово расстояние.

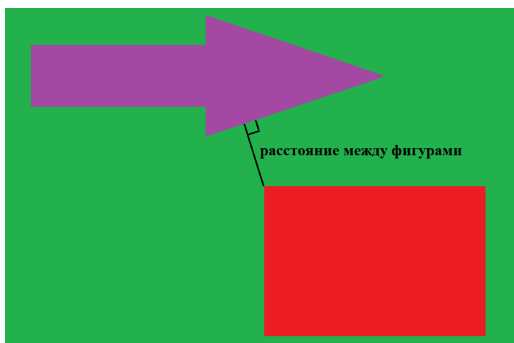


Рис. II.3.1. Пример расстояния между фигурами

Ваша задача — написать функцию, определяющую расстояние между двумя наиболее удаленными друг от друга фигурами. Верными считаются ответы, отклоняющиеся от истинного значения не более чем на 2%.

Материалы задачи для скачивания: <https://disk.yandex.ru/d/oQ4gQPhFj3LDpQ>.

Для вас подготовлены несколько файлов «.py», набор изображений и аннотации к нему. Среди подготовленных файлов есть:

- `eval.py` — файл с функцией определения расстояния между фигурами. Именно эту функцию вам необходимо дописать;
- `main.py` — файл, проверяющий точность работы вашего алгоритма. Не редактируйте его. `main.py` использует, написанные вами, функции из `eval.py` и сверяет истинные ответы с предсказанием вашего алгоритма;
- `annotations.csv` — файл, устанавливающий соотношение между изображениями и верными ответами для них. В каждой строке файла содержится путь к файлу и верный ответ.

В качестве решения необходимо сдать отредактированный файл `eval.py`. Либо архив `*.zip` с файлом `eval.py` и остальными файлами, требующимися для его работы.

Технические ограничения

Размер решения ограничен: не более 0,1 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решения можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

```
Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0;
imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2;
numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;
scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0;
torchaudio 0.13.0; torchvision 0.14.0.
```

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/oQ4gQPhFj3LDpQ>

Формат входных данных

Изображение (bgr), прочитано `cv2.imread`.

Формат выходных данных

Число типа `float`. Наименьшее расстояние между двумя самыми удаленными фигурами.

Пример выходных данных

5,435

Тесты и файл решения

<https://disk.yandex.ru/d/Q093u0I93cuvlw>

Решение

Нам необходимо найти дистанцию между двумя фигурами, наиболее удаленными друг от друга. Найдем дистанцию между всеми парами фигур на изображении и выберем максимальную.

Границами фигуры будем считать ее контур — список точек, возвращаемых функцией `cv2.findContours`. Перед тем как искать контуры бинаризуем изображение. По условию задания фигуры не касаются краев изображения. Значит, любой крайний пиксель изображения имеет цвет фона. Используем цвет фона в качестве верхнего и нижнего порога бинаризации. После такой бинаризации, пиксели фона станут белыми, а пиксели фигур — черными.

Функция поиска контуров вернет контуры всех фигур и контур всего изображения. Нам необходимо удалить из списка контур изображения. Выполняем сортировку контуров по площади в порядке убывания. После сортировки первым (индекс 0) будет контур, который необходимо удалить.

В получившемся списке попарно перебираем контуры. Вычисляем дистанцию между каждой парой фигур (контуров). Алгоритм вычисления дистанции между двумя контурами выполнен в отдельной функции, описание представлено ниже. Запоминаем максимальную дистанцию, получившуюся в ходе расчетов. Возвращаем ее в качестве ответа.

Алгоритм вычисления дистанции между фигурами. Входные данные — два контура, два списка точек. Перебираем точки первого и второго контура. Для каждой пары точек из разных контуров вычисляем дистанцию. Минимальная дистанция между точками — расстояние между фигурами.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  # -*- coding: utf-8 -*-
2  import cv2
3  import math
4
5
6  def calc_point_dist(point1, point2) -> float:
7      """
8      Вычисляем расстояние между двумя точками.
9      На вход получаем две точки в формате, который возвращает cv2.findContours.
10     Расстояние между двумя точками - евклидово расстояние (метрика),
11     вычисляем по теореме пифагора.
12     """

```

```

13
14     x1 = point1[0, 0]
15     y1 = point1[0, 1]
16     x2 = point2[0, 0]
17     y2 = point2[0, 1]
18     dist = math.sqrt((x1-x2)**2 + (y1-y2)**2)
19     return dist
20
21
22 def calc_figure_dist(cnt1, cnt2) -> float:
23     """
24     Вычисляем расстояние между двумя фигурами.
25     На вход получаем два контура - списки точек.
26     Перебираем точки первого и второго контура. Для каждой пары точек из разных
27     контуров высчитываем дистанцию.
28     Возвращаем минимальную дистанцию, которая у нас получилась.
29     Это и есть расстояние между фигурами.
30
31     Расчет дистанции между точками выполнен в отдельной функции.
32     """
33     min_dist = 10000
34     for point1 in cnt1:
35         for point2 in cnt2:
36             dist = calc_point_dist(point1, point2)
37             if dist < min_dist:
38                 min_dist = dist
39     return min_dist
40
41
42 def measure_distance(image) -> float:
43     """
44     Нам необходимо найти дистанцию между двумя фигурами,
45     наиболее удаленными друг от друга.
46     Для этого, необходимо найти дистанцию между всеми парами фигур на изображении
47     и выбрать максимальную.
48
49     Границами фигуры будем считать ее контур - список точек,
50     возвращаемых функцией cv2.findContours.
51     Перед тем, как искать контуры необходимо бинаризовать изображение.
52     По условию задания фигуры не касаются краев изображения.
53     Значит любой крайний пиксель изображения имеет цвет фона.
54     Цвет фона можно использовать в качестве верхнего и нижнего порога бинаризации.
55     После такой бинаризации, пиксели фона станут белыми,
56     а пиксели фигур - черными.
57
58     Функция поиска контуров вернет контуры фигур и контур всего изображения.
59     Нам необходимо удалить из списка контуров изображения.
60     Для этого выполняем сортировку контуров по площади в порядке убывания.
61     После сортировки первым (индекс 0) будет контур, который необходимо удалить.
62
63     В получившемся списке попарно перебираем контуры.
64     Вычисляем дистанцию между каждой парой фигур (контуров).
65     Вычисление дистанции между двумя контурами выполнено в отдельной функции.
66     Запоминаем максимальную дистанцию, получившуюся в ходе расчетов.
67     Возвращаем получившееся значение, это и есть ответ.
68     """
69     max_dist = 0 # переменная для сохранения максимальной дистанции
70     bg_color = image[0, 0]
71     binary = cv2.inRange(image, bg_color, bg_color)
72     contours, H = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_NONE)

```



```

73     if contours:
74         contours = sorted(contours, key=cv2.contourArea, reverse=True)
75         contours.pop(0)
76         for i, cnt1 in enumerate(contours):
77             for j, cnt2 in enumerate(contours):
78                 if i >= j:
79                     # либо фигуры в списках совпадают,
80                     # либо такая комбинация уже рассматривалась ранее
81                     # пропускаем эту итерацию цикла - не дублируем расчеты
82                     continue
83                 figure_dist = calc_figure_dist(cnt1, cnt2)
84                 if figure_dist > max_dist: # запоминаем максимальную дистанцию
85                     max_dist = figure_dist
86     return max_dist

```

Задача II.3.2. Подсчет пересекающихся контуров (30 баллов)

Темы: представление изображения в памяти компьютера, бинаризация, цветовые пространства, поиск контуров, алгоритмы перебора и сравнения элементов списка, цветовая идентификация.

Условие

На изображении представлены несколько контуров. Все контуры имеют разные цвета и представляют геометрические фигуры. Цвет фона одинаковый на всем изображении. Контуры пересекаются друг с другом. В местах пересечения контуров пиксели принимают цвет, равный сумме цветов пересекающихся контуров. В одной точке могут пересекаться лишь два контура. Цвета в точках пересечения контуров гарантированно не совпадают с цветами контуров. Ваша задача — написать функцию, определяющую количество контуров на изображении.

Материалы задачи для скачивания: https://disk.yandex.ru/d/c-V2Pk_td56ugA.

Для вас подготовлены несколько файлов «.py», набор изображений и аннотации к нему.

- **eval.py** — файл с функцией подсчета контуров на изображении. Именно эту функцию вам необходимо дописать.
- **main.py** — файл, проверяющий точность работы вашего алгоритма. Не редактируйте его. **main.py** использует, написанные вами, функции из **eval.py** и сверяет истинные метки с предсказанием вашего алгоритма.
- **annotations.csv** — файл, устанавливающий соотношение между изображениями и количеством контуров на них. В каждой строке файла содержится путь к файлу изображения и числу контуров на нем.

В качестве решения необходимо сдать отредактированный файл **eval.py**. Либо архив *.zip с файлом **eval.py** и остальными файлами, требующимися для его работы.

Технические ограничения

Размер решения ограничен: не более 1 МБ.

Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение

об ошибке, то следующее решения можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

```
Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0;
imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2;
numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;
scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0;
torchaudio 0.13.0; torchvision 0.14.0.
```

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

https://disk.yandex.ru/d/c-V2Pk_td56ugA

Формат входных данных

Изображение (bgr), прочитано `cv2.imread`.

Формат выходных данных

Число типа `int`. Количество контуров на изображении.

Пример выходных данных

8

Тесты и файл решения

<https://disk.yandex.ru/d/3m27sCovsSAw9w>

Решение

Количество контуров на изображении соответствует количеству уникальных цветов на изображении, не являющимся суммой двух других цветов. Посчитаем уникальные цвета на изображении и исключим из списка тот цвет, которого больше всего на изображении, это цвет фона.

Переберем пары уникальных цветов. Посчитаем число пар, при суммировании которых, получается цвет из списка уникальных. Такие цвета «суммы» — это цвета точек пересечения контуров. Вычтем из общего числа уникальных цветов число цветов «сумм». Это и будет число контуров. Только цвета контуров не являются суммой двух других цветов.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  # -*- coding: utf-8 -*-
2  import cv2
3  import numpy as np
4
5  def count_contours(image) -> int:
6      """
7      Количество контуров на изображении соответствует
8      количеству уникальных цветов на изображении,
9      не являющимся суммой двух других цветов.
10     Посчитаем уникальные цвета на изображении, исключим из списка цветов тот,
11     которого больше всего, это цвет фона.
12     Переберем пары уникальных цветов. Посчитаем число пар,
13     при суммировании которых, получается цвет из списка уникальных.
14     Посчитаем число таких цветов "сумм" - это цвета пересечений контуров.
15     Вычтем из общего числа уникальных цветов число цветов "сумм".
16     Это и будет число контуров - только цвета контуров не являются
17     суммой двух других цветов.
18     """
19
20     h = image.shape[0] # высота изображения
21     w = image.shape[1] # ширина изображения
22
23     # Превращаем изображение в двумерный массив в двумерный,
24     # для удобства поиска уникальных цветов
25     image_pixels = image.reshape((h*w, 3))
26     # Первая ось - номер пикселя, вторая - составляющие цвета
27
28     # Ищем уникальные цвета на изображении
29     colors, count_color = np.unique(image_pixels, return_counts=True, axis=0)
30     # Т.е. находим все цвета на изображении
31
32     # удаляем из списка уникальных цветов тот,
33     # которого больше всего на изображении - это цвет фона
34     indx_bg_color = np.argmax(count_color) # получаем индекс макс. элемента
35     colors = np.delete(colors, indx_bg_color, axis=0) # удаляем по индексу
36
37     # перебираем пары цветов и подсчитываем те цвета,
38     # которые получаются сложением двух других цветов
39     # т.е. подсчитываем число цветов - пересечений контуров
40     number_of_sum_colors = 0 # счетчик цветов, получающихся сложением.
41     for i, color1 in enumerate(colors):
42         for j, color2 in enumerate(colors):
43             if i >= j:
44                 # такое сочетание цветов мы уже проверяли в предыдущих итерациях
45                 continue
46                 clr_sum = color1 + color2
47                 for clr in colors:
48                     if (clr_sum[0] == clr[0] and
49                        clr_sum[1] == clr[1] and clr_sum[2] == clr[2]):
50                         number_of_sum_colors += 1
51
52     # число контуров совпадает с количеством уникальных цветов,
53     # не являющихся суммой двух других цветов
54     # вычтем из количества всех цветов число цветов "сумм" и
55     # получим число контуров
56     number_of_contours = len(colors) - number_of_sum_colors
57
58     return number_of_contours
59

```

Задача II.3.3. Автоматическое определение направления движения автомобиля (20 баллов)

Темы: представление изображения в памяти компьютера, бинаризация, численные алгоритмы аппроксимации, поиск контуров, тригонометрические функции, анализ контуров, двумерная графика.

Условие

На фото II.3.2 представлены параллельные друг другу прерывистая и сплошная линия разметки, ограничивающие полосу движения. Движение правостороннее, сплошная линия всегда является правым краем дороги.

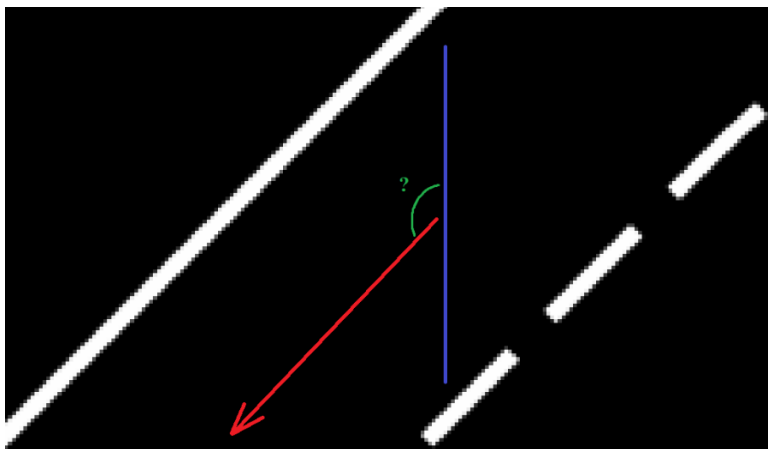


Рис. II.3.2. Угол направления движения

Напишите функцию, определяющую угол между направлением движения по полосе и вертикалью. При направлении движения вертикально вверх, искомый угол равен нулю. Если направление движения влево от центра кадра, то искомый угол отрицательный. Если движение вправо от центра кадра, то искомый угол положительный.

Материалы задачи для скачивания: <https://disk.yandex.ru/d/50Bi8-oEoKnYIA>.

Для вас подготовлены несколько файлов «.py», набор изображений и аннотации к нему.

- `eval.py` — файл с функцией определения угла направления движения. Именно эту функцию вам необходимо дописать.
- `main.py` — файл, проверяющий точность работы вашего алгоритма. Не редактируйте его. `main.py` использует, написанные вами, функции из `eval.py` и сверяет верные ответы с предсказанием вашего алгоритма.
- `annotations.csv` — файл, устанавливающий соотношение между изображениями и верными ответами для них. В каждой строке файла содержится путь к файлу с изображением дороги и углом направления движения.

В качестве решения необходимо сдать отредактированный файл `eval.py`. Либо архив `*.zip` с файлом `eval.py` и остальными файлами, требующимися для его работы.

Технические ограничения

Размер решения ограничен: не более 0,1 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решения можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

```
Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0;
imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2;
numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;
scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0;
torchaudio 0.13.0; torchvision 0.14.0.
```

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/50Bi8-oEoKnYIA>

Формат входных данных

Изображение (`bgr`), прочитано `cv2.imread`.

Формат выходных данных

Число типа `int`. Угол направления движения от -180° до 180° .

Пример выходных данных

-38

Тесты и файл решения

<https://disk.yandex.ru/d/u1-SmGvPAfFmyw>

Решение

Бинаризуем изображение и находим на нем контуры. Сортируем контуры по площади. Первый контур в списке контуров — контур сплошной линии. Второй контур в списке — контур пунктирной линии.

Метод `np.polyfit` находит коэффициенты уравнения прямой, которая наиболее соответствует входящему набору точек, он подбирает уравнение прямой методом наименьших квадратов.

Входные данные для `polyfit` — x и y координаты точек контура.

Метод вернет нам k и b прямой линии, где k — тангенс угла наклона линии. Если взять арктангенс, то получим угол наклона сплошной линии разметки в координатах изображения. Ось x вертикально вниз, ось y слева направо.

Точка b — точка пересечения прямой с осью Y . Сравнивая b для двух прямых с одинаковым k , можно определить, какая прямая выше. Мы будем это использовать, чтобы определить, какая линия разметки выше — сплошная или прерывистая. От этого зависит направление движение: направо или налево от центра кадра.

Последним действием необходимо пересчитать угол наклона в соответствии с требованием условия задачи: от -180° до 180° , 0° — движение вертикально вверх, $> 0^\circ$ — движение влево.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  # -*- coding: utf-8 -*-
2  import cv2
3  import numpy as np
4  import math
5
6  def get_angle(image) -> int:
7      """
8      Бинаризуем изображение и находим на нем контуры. Сортируем контуры по площади.
9      Первый контур в списке - контур сплошной линии.
10     Второй контур в списке - контур пунктирной линии.
11     Метод np.polyfit находит коэффициенты уравнения прямой,
12     которая наиболее соответствует входящему набору точек.
13     Y = kX + b X,Y - массивы с координатами точек, найденных выше(точки контуров)
14
15     k - тангенс угла наклона линии. Если взять арктангенс,
16     то получим угол наклона сплошной линии разметки.
17
18     b - точка пересечения прямой с осью Y.
19     Сравнивая b для двух прямых с одинаковым k,
20     можно определить какая прямая выше. Мы будем это использовать,
21     чтобы определить какая линия разметки выше сплошная или прерывистая.
22     От этого зависит направление движение: направо или налево от центра кадра.
23     Последним действием необходимо пересчитать угол наклона в соответствии
24     с требованием условия задачи.
25     """
26     # бинаризуем изображение так, линии разметки стали белыми пикселями
27     binary = cv2.inRange(image, (150, 150, 150), (255, 255, 255))
28     # находим контуры
29     contours, H = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_NONE)
30     if contours:
31         # сортируем контуры по площади
32         contours = sorted(contours, key=cv2.contourArea, reverse=True)
33         cnt = contours[0] # контур сплошной линии разметки
34         X = cnt[:, 0, 0]
35         Y = cnt[:, 0, 1]

```

```

36     k, b1 = np.polyfit(X, Y, 1) # находим k и b для прямой,
37                                # соответствующей контуру
38     angle = math.degrees( math.atan(k) ) # вычисляем угол наклона прямой,
39                                           # в координатах изображения
40
41     cnt = contours[1] # контур прерывистой линии разметки
42     X = cnt[:, 0, 0]
43     Y = cnt[:, 0, 1]
44     k, b2 = np.polyfit(X, Y, 1) # находим k и b для прямой,
45                                # соответствующей контуру
46
47     if b1 > b2: # Если сплошная линия ниже прерывистой,
48                # то движение вправо от центра кадра. От 0 до 180 градусов
49         angle = 90 + angle # пересчитываем угол под требования задачи
50     else: # Если сплошная линия выше прерывистой,
51           # то движение влево от центра кадра. От 0 до -180 градусов
52         angle = -90 + angle # пересчитываем угол под требования задачи
53
54     return int(angle)

```

Задача II.3.4. Отслеживание маршрута перемещения объекта (40 баллов)

Темы: представление изображения в памяти компьютера, обработка видеопотока, цветовые пространства, бинаризация, поиск контуров, детектирование объектов по цветам, распознавание сравнением с шаблоном, отслеживание обнаруженных объектов в видеопотоке.

Условие

На видео представлено перемещение беспилотного автомобиля по трассе в форме восьмерки. В начале видео беспилотный автомобиль неподвижен, затем он движется по восьмерке и несколько раз преодолевает перекресток, поворачивая в случайную сторону. Напишите функцию, определяющую последовательность направлений поворотов на перекрестках.

Материалы задачи для скачивания: <https://disk.yandex.ru/d/9Hf9BZZtS83SSQ>.

Для вас подготовлены несколько файлов «.py», набор видео-файлов и аннотации к нему.

- `eval.py` — файл с функцией определения направлений поворотов автомобиля. Именно эту функцию вам необходимо дописать.
- `main.py` — файл, проверяющий точность работы вашего алгоритма. Не редактируйте его. `main.py` использует, написанные вами, функции из `eval.py` и сверяет верные ответы с предсказанием вашего алгоритма.
- `annotations.csv` — файл, устанавливающий соотношение между видео-файлами и последовательностями поворотов на них. В каждой строке файла содержится путь к видео-файлу и последовательность направлений поворотов для него.

В качестве решения необходимо сдать отредактированный файл `eval.py`. Либо архив `*.zip` с файлом `eval.py` и остальными файлами, требующимися для его работы.

Технические ограничения

Размер решения ограничен: не более 0,1 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решения можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0; imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2; numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3; scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0; torchaudio 0.13.0; torchvision 0.14.0.

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/9Hf9BZZtS83SSQ>

Формат входных данных

Объект, полученный `cv2.VideoCapture`, из объекта можно читать кадры методом `.read`

Формат выходных данных

Необходимо вернуть список, состоящий из строк `'r'`, `'l'`, `'s'`, где

- `'r'` — поворот направо («right»),
- `'l'` — поворот налево («left»),
- `'s'` — проезд прямо («straight»).

Пример выходных данных

```
['l', 'l', 's', 'r']
```

Тесты и файл решения

<https://disk.yandex.ru/d/yJOulxFwX1iPYQ>

Решение

Выделяем четыре области на изображении, они совпадают с начальными участками дорог, образующих перекресток. Области имеют одинаковый размер. Их можно получить операцией среза исходного изображения:

- Область 1 — $[243 : 243 + 140, 542 : 542 + 111]$ — левая верхняя дорога.

- Область 2 — [536 : 536 + 140, 542 : 542 + 111] — левая нижняя дорога.
- Область 3 — [536 : 536 + 140, 830 : 830 + 111] — правая нижняя дорога.
- Область 4 — [243 : 243 + 140, 830 : 830 + 111] — правая верхняя дорога.

Бинаризуем изображение так, чтобы автомобиль или его часть стали белыми пикселями. Когда беспилотник въезжает на перекресток или выезжает с него, автомобиль пересекает одну из четырех областей. При этом сумма пикселей в области возрастает и начинает превышать некоторый порог.

Запишем в список номера всех областей, которые пересекает автомобиль. Будем отслеживать, где сумма пикселей превысила порог и добавлять номер в список. Но только в том случае, если номер области не совпадает с предыдущим. Нам не нужно считать одну область несколько раз, автомобиль не может приехать и уехать с перекрестка по одной дороге. Осталось преобразовать полученный список в названия l , r , s . Перебираем элементы списка так, чтобы рассматривать их парами 1–2, 3–4... Если в паре одинаковая четность, то перекресток преодолен по прямой. Если в паре номер области увеличивается на 1, то был поворот направо. Однако это не сработает в случае поворота 4–1, поэтому отслеживаем такой случай отдельно. Аналогично при повороте налево номер в паре уменьшается на 1, кроме случая 1–4, его отслеживаем отдельно.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  # -*- coding: utf-8 -*-
2  import cv2
3  import numpy as np
4
5  def track_car(video) -> list:
6      """
7      Мы выделяем четыре области на изображении,
8      они совпадают с начальными участками дорог, образующих перекресток.
9      Области имеют одинаковый размер.
10     Их можно получить операцией среза исходного изображения:
11     Область 1 - [243:243 + 140, 542:542 + 111] - левая верхняя дорога
12     Область 2 - [536:536 + 140, 542:542 + 111] - левая нижняя дорога
13     Область 3 - [536:536 + 140, 830:830 + 111] - правая нижняя дорога
14     Область 4 - [243:243 + 140, 830:830 + 111] - правая верхняя дорога
15
16     Бинаризуем изображение так,
17     чтобы автомобиль или его часть стали белыми пикселями.
18     Когда беспилотник въезжает на перекресток или выезжает с него,
19     автомобиль пересекает одну из четырех областей.
20     При этом сумма пикселей в области возрастает и
21     начинает превышать некоторый порог.
22
23     Запишем в список номера всех областей, которые пересекает автомобиль.
24     Будем отслеживать где сумма пикселей превысила порог и
25     добавлять номер в список. Но только в том случае,
26     если номер области не совпадает с предыдущим.
27     Нам не нужно считать одну область несколько раз,
28     автомобиль не может приехать и уехать с перекрестка по одной дороге.
29
30     Осталось преобразовать полученный список в названия l, r, s.
31     Перебираем элементы списка так, чтобы рассматривать их парами 1-2, 3-4...

```

```

32     Если в паре одинаковая четность, то перекресток преодолен по прямой.
33     Если в паре номер области увеличивается на 1, то был поворот направо.
34     Однако это не сработает в случае поворота 4-1, будем отслеживать его отдельно.
35     Аналогично, при повороте налево номер в паре уменьшается на 1,
36     кроме случая 1-4, его отслеживаем отдельно.
37
38     """
39
40     crossing_list = [] # список для записи номеров областей
41     number_crossing = 0 # номер области, в которой превышен порог
42
43     while True: # цикл чтения кадров
44         status, frame = video.read() # читаем кадр
45         if not status: # выходим из цикла, если видео закончилось
46             break
47         else:
48             binary = frame[:, :, 2] # оставляем R-составляющую изображения
49
50             # считаем суммы пикселей в четырех областях
51             sum1 = np.sum(binary[243:243 + 140, 542:542 + 111])
52             sum2 = np.sum(binary[536:536 + 140, 542:542 + 111])
53             sum3 = np.sum(binary[536:536 + 140, 830:830 + 111])
54             sum4 = np.sum(binary[243:243 + 140, 830:830 + 111])
55
56             # запоминаем номер области, в которой превышен порог
57             if sum1 > 650000:
58                 number_crossing = 1
59             elif sum2 > 650000:
60                 number_crossing = 2
61             elif sum3 > 650000:
62                 number_crossing = 3
63             elif sum4 > 650000:
64                 number_crossing = 4
65
66             # если номер области не совпадает с предыдущим запомненным,
67             # либо порог превышен впервые, то добавляем номер в список
68             if crossing_list == []:
69                 if number_crossing != 0:
70                     crossing_list.append(number_crossing)
71             elif number_crossing != crossing_list[-1]:
72                 crossing_list.append(number_crossing)
73
74     result = [] # список для формирования ответа из l, r, s.
75     # перебираем элементы парами 1-2, 3-4, 5-6...
76     for i, number_crossing in enumerate(crossing_list):
77         # последний элемент нас не интересует - выходим из цикла
78         if i+1 == len(crossing_list):
79             break
80
81         if i % 2 == 1: # элементы с нечетным индексом нас не интересуют
82             continue
83
84         # проверяем какой поворот кодирует пара и
85         # добавляем в список соответствующую букву
86         if number_crossing % 2 == crossing_list[i+1] % 2:
87             # одинаковая четность
88             result.append("s")
89         elif number_crossing + 1 == crossing_list[i+1]:
90             # увеличение номера области на 1
91             result.append("r")

```

```
92     elif number_crossing - 1 == crossing_list[i+1]:
93         # уменьшение номера области на 1
94         result.append("1")
95     elif number_crossing == 4:
96         result.append("x") # поворот с 4 на 1
97     elif number_crossing == 1:
98         result.append("1") # поворот с 1 на 4
99
100     return result
```

Работа наставника НТО на втором отборочном этапе

На втором отборочном этапе участникам предлагаются индивидуальные и командные задачи в рамках выбранных профилей. Для подготовки к нему наставник может использовать следующие рекомендуемые форматы и мероприятия:

- Подготовка по образовательным программам НТО по ряду технологических направлений.
- Разбор задач второго отборочного этапа НТО прошлых лет.
- Прохождение онлайн-курсов по разбору задач НТО прошлых лет.
- Прохождение онлайн-курсов, рекомендованных разработчиками профилей.
- Разбор материалов для подготовки к профилям.
- Практикумы. Для организации практикумов возможно использовать разные подходы или их комбинации:
 - Проведение практикумов по описаниям на страницах профилей и материалов для подготовки.
 - Декомпозиция задач заключительных этапов прошлых лет для выделения наиболее актуальных элементов и их изучения.
 - Анализ технических знаний и навыков (hard skills), требуемых для конкретного профиля, и самостоятельная разработка или поиск занятия для развития наиболее актуальных из них.
 - Посещение практикумов на площадках подготовки и онлайн-мероприятий от разработчиков профилей. Объявления о таких мероприятиях публикуются в группах НТО в VK и в телеграм-канале для наставников НТО (https://t.me/kruzhok_association).

Второй отборочный этап

Представленные задачи являются результатом декомпозиции задачи заключительного этапа. Они направлены на формирование знаний и компетенций, требуемых на заключительном этапе. Участники создают систему, в которой квадрокоптер патрулирует улицы города и детектирует разнообразные происшествия, например, пожар или несчастный случай на улице, о которых в информационный центр отсылаются сообщения. На основе этих данных центр формирует список заданий для беспилотных автомобилей. Перед тем как выполнять задание беспилотный автомобиль должен загрузить на борт тип оборудования, соответствующий заданию и связанному с ним происшествию. Экипировкой оборудования занимается сервисный центр.

Решение представленных задач требует командного подхода: распределения ролей, определения участков работы и разграничения ответственности. Освоение компетенций, необходимых для решения задач одним человеком, невозможно в течении второго этапа. Для решения всех пяти задач требуются слаженное взаимодействие всех участников команды, имеющих следующие компетенциями: обучение и применение нейросетевых детекторов, знание ROS, создание полетных программ для квадрокоптера Пионер, чтение технической документации, поиск объектов по цветам, построение алгоритмов и их реализация в программном коде, теория графов, реализация обмена данными через сокеты.

Представленные ниже задачи связаны с операциями, которые будут выполнять устройства автономной транспортной системы на заключительном этапе.

Задача IV.1. Обнаружение транспортных средств (35 баллов)

Темы: *нейросетевые детекторы, обнаружение объектов в видеопотоке, отслеживание объектов в видеопотоке.*

Условие

На видео запечатлено движение автомобилей по кольцевой развязке. Ваша задача: написать функцию, подсчитывающую общее число автомобилей на видео. Необходимо учитывать и неподвижные автомобили, и движущиеся. Не различайте транспортные средства по типу.



Рис. 1. Кольцевая развязка

Выполнение

1. Скачайте материалы задания: <https://disk.yandex.ru/d/DebI1aRQyS5QXw>.
2. Ознакомьтесь с материалами задания.
Для вас подготовлены несколько файлов `.py`, набор видеофайлов и аннотации к нему.
В списке подготовленных файлов содержатся:
 - `eval.py` — файл с функцией обнаружения и подсчета транспортных средств; именно эту функцию вам необходимо дописать;
 - `main.py` — файл, проверяющий точность работы вашего алгоритма; не редактируйте его. `main.py` использует написанные вами функции из `eval.py` и сверяет истинные метки с предсказанием вашего алгоритма;
 - `annotations.csv` — файл, устанавливающий соотношение между видеофайлами и количеством; в каждой строке файла содержится путь к видеофайлу и числу транспортных средств на нем.В качестве решения необходимо сдать отредактированный файл `eval.py`. Либо архив `*.zip` с файлом `eval.py` и остальными файлами, требующимися для его работы.
3. Прочитайте файл `eval.py`.
В файле содержится функция, которую вам необходимо дописать.
4. Допишите в файле `eval.py` функцию `load_models`.
Эта функция отвечает за загрузку из файлов всех моделей машинного обучения, которые вы собираетесь использовать.
5. Допишите в файле `eval.py` функцию `count_vehicles`.
Эта функция получает на вход объект `videoCapture`, связанный с видеофайлом. Усовершенствуйте функцию так, чтобы она возвращала число транспортных средств запечатленных на видео.
6. Запустите файл `main.py` и проверьте свой алгоритм.
Если программа выдала ошибку, то найдите и исправьте ее в файле `eval`, вновь

запустите файл `main.py`.

7. Пришлите решение на онлайн платформу для проверки.

Упакуйте файл `eval.py` и дополнительные файлы в архив `*.zip`. В архиве должны находиться файлы, а не одноименная архиву папка.

Технические ограничения

Размер решения ограничен: не более 30 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решение можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

```
Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0;
imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2;
numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;
scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0;
torchaudio 0.13.0; torchvision 0.14.0.
```

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/DebI1aRQyS5QXw>.

Формат входных данных

Текстовая строка.

Пример входных данных

```
medic_aid.330.114,fire.78.64,crash.31.57;122;198;gardening_tools;1;
bransboit.
```

Формат выходных данных

Последовательность текстовых сообщений отправленных через сокет.

Файл решения

<https://disk.yandex.ru/d/88m9N7uv5bDIhg>.

Тесты

<https://disk.yandex.ru/d/NjtSsi-hicEF0w>.

Решение

Для решения задания необходимо обучить нейросетевой детектор. Данных, предоставленных участникам, достаточно для разметки собственного датасета.

После того как модель обучена, необходимо детектировать автомобили на каждом кадре видеофайла.

Положение автомобилей на текущем кадре необходимо сравнить с положением на предыдущем кадре. Если в окрестности текущего положения на прошлом кадре не было автомобилей, то считаем автомобиль впервые обнаруженным и увеличиваем счетчик автомобилей на видео. Размер окрестности определяется экспериментально.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  from ultralytics import YOLO
2  import math
3
4  # Функция загрузки нейросетевой модели из файла
5  def load_models():
6      models = [YOLO('car_detector.pt')]
7      return models
8
9  # Основная функция, вызывается для каждого видеофайла отдельно.
10 def count_vehicles(video, models) -> int:
11     car_count = 0 # Счетчик автомобилей на видео.
12     old_box_list = [] # Список автомобилей на прошлом кадре.
13     while True:
14         # Читаем кадры, пока они не закончатся.
15         ret, img = video.read()
16         if not ret:
17             break
18
19         # Передаем детектору кадр и получаем результаты.
20         results = models[0].predict(img, verbose=False)
21
22         new_box_list = [] # Список автомобилей на текущем кадре.
23
24         # Перебираем ограничивающие рамки для объектов,
25         # детектированных с уверенностью > 0.5
26         # Сравниваем каждый обнаруженный автомобиль с автомобилями
27         # на предыдущем кадре.
28         # Если новый автомобиль не совпадает по координатам ни с одним из
29         # "прошлых" автомобилей,
30         # то увеличиваем счетчик автомобилей на видео.
31         for box in results[0].boxes:
32             if box.conf > 0.5:
33                 # Точка (x, y) - верхний левый угол рамки объекта.
34                 x = int(box.xyxy[0, 0])
35                 y = int(box.xyxy[0, 1])
36                 # Добавляем в список объектов на кадре.
37                 new_box_list.append((x, y))
38                 # Проверим есть-ли объекты на предыдущем кадре.
39                 if old_box_list != []:
40                     # Перебираем объекты на предыдущем кадре.
41                     # Если находим объект, в +- тех же координатах, то

```



```

42     # считаем автомобиль ранее детектированным,
43     # т.е. не увеличиваем счетчик
44     for old_box in old_box_list:
45         if math.sqrt((old_box[0]-x)**2 +
46                     (old_box[1]-y)**2) <= 120:
47             # Удаляем объект из старого списка
48             old_box_list.remove(old_box)
49             break
50     else:
51         # Операторы в else выполняются, если цикл прошел полностью,
52         # т.е. выхода через break не произошло.
53
54         # Если не нашлось ни одного подходящего объекта, то
55         # считаем авт. впервые появившимся - увеличиваем счетчик.
56         car_count += 1
57     else: # Если на предыдущем кадре нет объектов, то
58         # считаем авт. найденным впервые - увеличиваем счетчик.
59         car_count += 1
60
61     # Обновляем список автомобилей на предыдущем кадре.
62     old_box_list = new_box_list
63
64     video.release()
65     return car_count

```

Задача IV.2. Обмен данными (15 баллов)

Темы: коммутация программ через сокеты, протоколы передачи данных.

Условие

Необходимо написать функцию, реализующую обмен сообщениями с двумя устройствами: сервисным центром и информационным центром. Сервисный центр формирует список заданий, которые необходимо выполнить. Задание характеризуется двумя параметрами: координатами и типом задания. Чтобы приступить к его выполнению, необходимо зарезервировать его за собой и соответствующим образом экипироваться, сообщить информационному центру о получении необходимой экипировки. Типов экипировки больше, чем типов заданий, все возможные варианты представлены ниже.

Тип экипировки	Тип задания
bransboit	fire
medical_kit	med_aid
repair_tools	crush
cleaner_kit	—
gardening_tools	—
comfortable_salon	—

Функция, которую вам необходимо написать, получает на вход ваши текущие координаты и тип экипировки, которой вы располагаете. Функция должна запросить список заданий, выбрать ближайшее задание и зарезервировать его за собой. В общении с сервисным центром получить необходимую экипировку и сообщить об этом. После этого функция должна вернуть 0.

Прием и передача данных происходят через TCP-сокеты. Передаваемые сообщения текстовые, протокол общения представлен в материалах к заданию. Обмен сообщениями ведется по принципу запрос-ответ. Запрос всегда отправляет вы.

Выполнение

1. Скачайте материалы задания: https://disk.yandex.ru/d/SrAP_m99W6LaUQ.

2. Ознакомьтесь с материалами задания.

Для вас подготовлены несколько файлов `.py`, набор входных данных и верных ответов к ним и описание протокола общения.

В список подготовленных файлов входят:

- `eval.py` — файл с функцией создания сокета и функцией обмена сообщениями; именно функцию обмена сообщениями вам необходимо дописать;
- `main.py` — файл, проверяющий точность работы вашего алгоритма; не редактируйте его. `main.py` использует написанные вами функции из `eval.py` и сверяет результат работы вашего алгоритма с правильным ответом;
- `annotations.csv` — файл с набором входных данных и верными ответами к ним; в каждой строке файла содержатся: стартовые координаты, тип стартовой экипировки, список заданий в информационном центре, задание, которое вам необходимо зарезервировать и тип экипировки, которую необходимо получить;
- `protocol.txt` — описание сообщений, которые вы можете отправлять и сообщений, которые будут отправлять вам.

В качестве решения необходимо сдать отредактированный файл `eval.py`.

3. Прочитайте файл `eval.py`.

В файле содержится функции `setup_socket` и `communication_cycle`.

4. Допишите в файле `eval.py` функцию `communication_cycle`.

Эта функция отвечает за обмен данными с информационным и сервисным центрами.

5. Запустите файл `main.py` и проверьте свой алгоритм.

Если программа выдала ошибку, то найдите и исправьте ее в файле `eval`, вновь запустите файл `main.py`.

6. Пришлите решение на онлайн-платформу для проверки.

В качестве решения необходимо сдать отредактированный файл `eval.py` или `*.zip` архив с ним. В архиве должны находиться файлы, а не одноименная архиву папка.

Технические ограничения

Размер решения ограничен: не более 30 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решение можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0; imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2;

numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;
scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0;
torchaudio 0.13.0; torchvision 0.14.0.

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

https://disk.yandex.ru/d/SrAP_m99W6LaUQ.

Формат входных данных

Текстовая строка.

Пример входных данных

medic_aid.330.114,fire.78.64,crash.31.57;122;198;gardening_tools;1;
bransboit.

Формат выходных данных

Последовательность текстовых сообщений отправленных через сокет.

Файл решения

<https://disk.yandex.ru/d/IoYsjcBV9yMsRA>.

Тесты

<https://disk.yandex.ru/d/sIa2YlhlgiQziQ>.

Решение

1. Инициализировать сокет для получения и отправки сообщений (реализовано в отдельной функции).
2. Получить список заданий от сервера и извлечь из него данные — названия заданий и их координаты.
3. Найти ближайшее задание и зарезервировать его за собой.
4. Запросить экипировку под задание, если мы не располагаем ей.
5. Сообщить, что экипированы для задания и оборвать соединение.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```
1 import socket
2 import math
3
```

```

4 def setup_socket(ip_address, port):
5     """ Функция инициализирует сокет.
6         Вводные параметры: ip-адрес и порт сервера
7         Выводные параметры: инициализированный сокет
8
9     """
10    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11    server.connect((ip_address, port))
12    return server
13
14 def communication_cycle(conn: socket.socket, start_pos, start_equipment):
15     """ Эта функция отвечает за обмен данными с информационным и сервисным
16                                         центрами..
17
18     Вводные параметры:
19         conn - сокет из функции setup_socket,
20         start_pos - стартовая позиция беспилотного автомобиля (x, y),
21         start_equipment - начальное снаряжение
22     """
23    conn.send('server,give_tasks'.encode()) # запрашиваем у сервера список задач
24    msg = '' # полученные от сервера данные, будем записывать в переменную msg
25    while True:
26        symbol = conn.recv(1).decode() # принимаем ответ сервера по одному
27                                         символу
28        if symbol in ('|', ','):
29            break # если мы получили символ конца сообщения "/" или
30                  # читать нечего, то выходим из цикла
31        msg += symbol
32
33    # Далее необходимо выбрать ближайшее к нам задание
34    # Для этого преобразуем полученную от сервера строку с заданиями и их
35                                         координатами в два списка.
36    # В первом списке будут названия заданий, во втором дистанция до задания.
37    tasks_name_list = [] # Список для имен заданий.
38    distance_list = [] # Список для дистанции до заданий.
39
40    task_and_xy_list = msg.split(",") # Делим список заданий на части,
41                                         разделителем считаем запятую.
42    for text in task_and_xy_list: # Перебираем получившиеся части.
43        task_x_y = text.split(".") # Каждую часть делим на подчасти,
44                                         разделителем считаем точку.
45
46        # Подчасть с индексом 0 - имя задания.
47        # Подчасть с индексом 1 - X координата задания.
48        # Подчасть с индексом 2 - Y координата.
49
50        tasks_name_list.append(task_x_y[0]) # Добавляем имя задания в список.
51        task_x = int(task_x_y[1]) # Получаем X ко-ту задания в виде целого числа.
52        task_y = int(task_x_y[2]) # Получаем Y ко-ту задания в виде целого числа.
53        # Рассчитываем дистанцию до задания (евклидово расстояние)
54                                         по теореме пифагора.
55        distance = math.sqrt((start_pos[0] - task_x) ** 2 +
56                               (start_pos[1] - task_y) ** 2)
57        distance_list.append(distance) # Добавляем дистанцию в список.
58
59    """У нас есть два списка.
60        В первом названия задач - tasks_name_list.
61        Во втором дистанция до задач - distance_list.
62        Если найти индекс минимального элемента во втором массиве,
63        то получим и индекс названия ближайшей задачи в первом массиве.
64    """

```

```

64     # Находим индекс минимального элемента в списке дистанций.
65     min_dist = distance_list[0]
66     for i, dist in enumerate(distance_list):
67         if min_dist >= dist:
68             min_dist = dist
69             indx_nearest_point = i
70
71     # indx_nearest_point - индекс ближайшей задачи в списке tasks_name_list и
72     в списке, заданий полученном с сервера.
73     # tasks_name_list[indx_nearest_point] - название ближайшей задачи.
74     nearest_point_name = tasks_name_list[indx_nearest_point]
75
76     # Отправляем серверу индекс задачи, которую резервируем за собой.
77     for_server_message = "server_reserve_task," + str(indx_nearest_point) + "|"
78     conn.send(for_server_message.encode())
79     while True: # Считываем ответ сервера.
80         symbol = conn.recv(1).decode()
81         if symbol in ('|', ','): break
82         msg += symbol
83
84     # Выясняем какая экипировка нужна для выполнения нашего задания.
85     if nearest_point_name == "fire":
86         required_equipment = "bransboit"
87     elif nearest_point_name == "medic_aid":
88         required_equipment = "medical_kit"
89     elif nearest_point_name == "crash":
90         required_equipment = "repair_tools"
91
92     """Теперь необходимо необходимо получить нужную экипировку, если мы ей еще
93     не располагаем.
94     Если у нас уже имеется нужная экипировка, то ничего делать не нужно.
95     Если у нас есть экипировка, но она не подходящая под задание,
96     то нужно ее сдать и запросить новую.
97     Если у нас вообще нет экипировки, то нужно запросить новую.
98     """
99
100     if start_equipment != required_equipment: # Если наша экипировка отличается
101     # от требуемой...
102     if start_equipment != "no_equipment": # Проверяем, что у нас есть
103     # какая-то экипировка.
104         conn.send('hub,offload|'.encode()) # Если есть, сдаем ее.
105         msg = ''
106         while True:
107             symbol = conn.recv(1).decode()
108             if symbol in ('|', ','):
109                 break
110             msg += symbol
111
112     # Запрашиваем новую экипировку, соответствующую взятому заданию.
113     equipment_code = {"bransboit": "1",
114                       "medical_kit": "2",
115                       "repair_tools": "3"}
116     conn.send(f'hub,give,{equipment_code[required_equipment]}|'.encode())
117     msg = ''
118     while True:
119         symbol = conn.recv(1).decode()
120         if symbol in ('|', ','):
121             break
122         msg += symbol
123

```

```

124     # Сообщаем, что мы получили необходимую экипировку и готовы выполнять задание
125     conn.send('equip_ready!'.encode())
126     msg = ''
127     while True:
128         symbol = conn.recv(1).decode()
129         if symbol in ('|', ' '):
130             break
131         msg += symbol
132     # Разрываем соединение
133     conn.close()

```

Задача IV.3. Отслеживание маршрута беспилотного автомобиля (15 баллов)

Темы: построение и хранение графов, обнаружение объектов в видеопотоке, отслеживание объектов в видеопотоке.

Условие

На видео представлено перемещение беспилотного автомобиля по дорожной сети с несколькими перекрестками. Каждая дорога двунаправленная и имеет две полосы движения, каждая в свою сторону. Движение правостороннее. Городскую дорожную сеть можно представить в виде графа, где перекрестки — вершины, а полосы движения — ребра графа. Перекрестки и соответствующие им вершины графов пронумерованы слева направо.

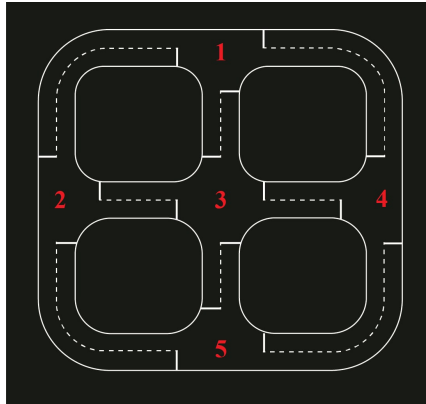


Рис. 2. Городская дорожная сеть

Ваша задача написать функцию, отслеживающую перемещения беспилотника и заполняющую матрицу. Матрица, которую нужно заполнить — аналог матрицы смежности графа. В нашем случае значения записанные в матрице обозначают число раз, которое автомобиль проехал по ребру графа.

Элемент матрицы с индексами $[2, 0]$ соответствует вертикально направленной полосе движения от третьего перекрестка к первому. Первый индекс — стартовая вершина, второй индекс — конечная вершина.

Помните, что в матрице нумерация строк и столбцов начинается с 0, а не с 1. Если значение элемента в матрице 0, значит, полосы движения, соответствующей такому ребру графа не существует, либо беспилотник еще не проезжал по этой полосе. Изначально матрица заполнена нулями.

От видео к видео схема дорожной разметки не изменяется, изменяется маршрут следования беспилотника.

Выполнение

1. Скачайте материалы задания: <https://disk.yandex.ru/d/mBYdqmLvAlY-vw>.
2. Ознакомьтесь с материалами задания.

Для вас подготовлены несколько файлов .py, набор видеофайлов и аннотации к нему. Среди подготовленных файлов есть:

- `eval.py` — файл с функцией, отслеживающей перемещения беспилотного автомобиля; именно эту функцию вам необходимо дописать;
- `main.py` — файл проверяющий точность работы вашего алгоритма; не редактируйте его. `main.py` использует, написанные вами, функции из `eval.py` и сверяет верные ответы с предсказанием вашего алгоритма;
- `annotations.csv` — файл, устанавливающий соответствие между видеофайлами и матрицами, которые должны получиться; в каждой строке файла содержится путь к видеофайлу и соответствующая ему матрица.

В качестве решения необходимо сдать отредактированный файл `eval.py`, либо архив *.zip с файлом `eval.py` и остальными файлами, требующимися для его работы.

3. Прочитайте файл `eval.py`.
В файле содержатся функции `track_movement` и `load_tools`.
4. Допишите в файле `eval.py` функцию `load_tools`.
Функция используется для загрузки моделей машинного обучения из файлов или загрузки эталонных изображений. Если вы не используете ее, пусть возвращает пустой список.
5. Допишите в файле `eval.py` функцию `track_movement`.
Функция получает на вход объект, возвращаемый `cv2.VideoCapture`, и список от `load_tools`. Функция должна возвращать массив `numpy 5 × 5` с элементами типа `np.uint8`.
6. Запустите файл `main.py` и проверьте свой алгоритм. Если программа выдала ошибку, то найдите и исправьте ее в файле `eval`, вновь запустите файл `main.py`.
7. Пришлите решение на онлайн-платформу для проверки. В качестве решения необходимо сдать отредактированный файл `eval.py` или *.zip архив с ним. В архиве должны находиться файлы, а не одноименная архиву папка.

Технические ограничения

Размер решения ограничен: не более 30 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решение можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0; imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2; numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3; scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0; torchaudio 0.13.0; torchvision 0.14.0.

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/mBYdqmLvAlY-vw>.

Формат входных данных

Видеофайл (объект `cv2.VideoCapture`).

Формат выходных данных

Матрица 5×5 в виде `numpy` массива `np.uint8`.

Файл решения

<https://disk.yandex.ru/d/08KKZ-RM3s0aaQ>.

Тесты

<https://disk.yandex.ru/d/--xCth7CZR2lcQ>.

Решение

Для решения данной задачи не обязательно определять координаты автомобиля на каждом кадре. Достаточно из кадра в кадр проверять, не находится ли автомобиль на том или ином перекрестке. Если составить список перекрестков, по которому проехал автомобиль, будет нетрудно заполнить требуемую матрицу.

Для локализации положения автомобиля изображение бинаризуется и выполняется операция заполнения пустот в белых объектах.



Рис. 3. Автомобиль после бинаризации и заполнения пустот

На изображении выбраны несколько ключевых пикселей, каждая соответствует конкретному перекрестку.

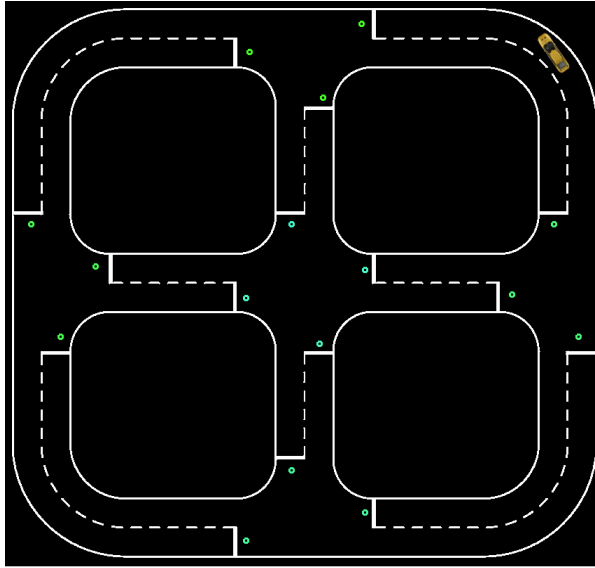


Рис. 4. Ключевые пиксели изображения

Если пиксель не черный, значит, автомобиль находится на его перекрестке. Проверяя состояние ключевых пикселей, определяем, через какие перекрестки проехал беспилотник. Остается заполнить требуемую матрицу, этот алгоритм вынесен в отдельную функцию.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  import cv2
2  import numpy as np
3  import math
4
5  # Не используется
6  def load_tools():
7      tools = []
8      return tools
9
10 # Функция возвращает номер перекрестка, на котором сейчас находится автомобиль,
11 # либо 0.
12 def intersection_number(img):
13     # Если хотя бы один из ключевых пикселей не черный, то
14     # возвращаем номер перекрестка, соответствующего не черному пикселю.
15     if (np.any(img[95, 360] != [0, 0, 0]) or
16         np.any(img[160, 465] != [0, 0, 0]) or
17         np.any(img[55, 520] != [0, 0, 0])):

```

```

18     return 1
19
20     if (np.any(img[340, 48] != [0, 0, 0]) or
21         np.any(img[400, 140] != [0, 0, 0]) or
22         np.any(img[500, 90] != [0, 0, 0])):
23         return 2
24
25     if (np.any(img[340, 795] != [0, 0, 0]) or
26         np.any(img[500, 830] != [0, 0, 0]) or
27         np.any(img[440, 735] != [0, 0, 0])):
28         return 4
29
30     if (np.any(img[790, 355] != [0, 0, 0]) or
31         np.any(img[690, 420] != [0, 0, 0]) or
32         np.any(img[750, 525] != [0, 0, 0])):
33         return 5
34
35     if (np.any(img[445, 355] != [0, 0, 0]) or
36         np.any(img[510, 460] != [0, 0, 0]) or
37         np.any(img[340, 420] != [0, 0, 0]) or
38         np.any(img[405, 525] != [0, 0, 0])):
39         return 3
40     return 0
41
42     # Функция заполняет требуемую матрицу на основе списка посещенных перекрестков.
43     def creating_graph(crossroad):
44         # Создаем матрицу 5x5, заполненную нулями.
45         result = np.zeros((5, 5), dtype=np.int8)
46         # Перебираем все посещенные перекрестки, кроме последнего.
47         for n, i in enumerate(crossroad[:-1]):
48             # n+1 - номер следующего посещенного перекрестка.
49             # crossroad[n + 1] - 1 - индекс в матрице,
50                 следующего посещенного перекрестка
51             # [i - 1] - индекс в матрице, текущего посещенного перекрестка
52             result[i - 1][crossroad[n + 1] - 1] += 1
53         return result
54
55     # Основная функция
56     def track_movement(video, tools) -> int:
57         """ Функция для отслеживания маршрута автомобиля.
58             Входные данные: видео-объект (cv2.VideoCapture)
59             Выходные данные: матрица смежности графа в виде numpy массива
60                 (dtype=np.uint8)
61
62             Примеры вывода:
63                 [[0, 0, 0, 1, 0],
64                 [0, 0, 1, 0, 0],
65                 [1, 0, 0, 0, 0],
66                 [0, 0, 1, 0, 0],
67                 [0, 0, 0, 0, 0]]
68             """
69
70         crossroad = [0] # Список посещенных перекрестков.
71         # Ядро свертки для операции заполнения внутренних пустот на черно-белом
72         ↳ изображении
73         st1 = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
74
75         while True: # Цикл чтения кадров.
76             ret_val, img = video.read()
77             if not ret_val:
78                 break # Выходим из цикла, если кадры закончились.

```

```

77
78     # Для удобства бинаризации, переводим изображение в формат HSV
79     hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
80     # Бинаризуем изображение так, чтобы белыми остались только
81                                     # пиксели автомобиля
82     maks_e = cv2.inRange(hsv, (10, 100, 100), (56, 246, 255))
83     cv2.imshow("mask_e_1", maks_e)
84     # Заполняем темные промежутки, внутри черно-белого изображения автомобиля
85     img_car = cv2.morphologyEx(maks_e, cv2.MORPH_CLOSE, st1, iterations=3)
86     cv2.imshow("img_car", img_car)
87
88     # Проверяем не проехала ли машина, по какому то из перекрестков.
89     # Запоминаем значение последнего посещенного перекрестка.
90     # Добавляем значение в список только тогда,
91                                     # когда оно не совпадает с ранее записанным.
92     # Т.е. при первом посещении нового перекрестка.
93     old = intersection_number(img_car)
94     if crossroad[-1] != old and old:
95         crossroad.append(old)
96
97     # Заполняем матрицу посещений ребер графа.
98     # Первый элемент списка - 0, не учитываем.
99     result = creating_graph(crossroad[1:])
100     return result

```

Задача IV.4. Подсчет грузов (20 баллов)

Темы: детектирование объектов по цветам, сравнение с шаблоном, контурный анализ, бинаризация.

Условие

Есть набор изображений с кубиками. На каждом изображении кубики разложены определенным образом, отличным от остальных изображений. Кубик может занимать одну из 18 позиций. По девять позиций в каждом слое. Соседние кубики всегда соприкасаются гранями полностью — угол в угол. Кубик второго слоя не может висеть в воздухе, он обязательно стоит на кубике первого слоя.

Ваша задача написать функцию, которая определит количество кубиков и их типы. Кубики отличаются друг от друга маркировкой. Всего четыре типа кубиков.



Рис. 5. Типы маркировок кубиков

Функция должна вернуть массив $3 \times 3 \times 2$, где значение элемента определяет тип кубика на изображении.

Возможные значения: 0 — позиция пуста, 1 — невидимый кубик, 2 — красный кубик, 3 — зеленый кубик, 4 — серый кубик, 5 — оранжевый кубик.

Выполнение

1. Скачайте материалы задания: <https://disk.yandex.ru/d/0Flrb4devulnKg>.

2. Ознакомьтесь с материалами задания.

Для вас подготовлены несколько файлов `.py`, набор изображений и аннотации к нему. Среди подготовленных файлов есть:

- `eval.py` — файл с функцией, определяющей количество кубиков на изображении и их типы; именно эту функцию вам необходимо дописать;
- `main.py` — файл проверяющий точность работы вашего алгоритма; не редактируйте его. `main.py` использует написанные вами функции из `eval.py` и сверяет верные ответы с предсказанием вашего алгоритма;
- `annotations.csv` — файл, устанавливающий соответствие между изображениями и массивами для них; в каждой строке файла содержится путь к файлу с изображением и соответствующий ему правильный ответ.

В качестве решения необходимо сдать отредактированный файл `eval.py`. Либо архив `*.zip` с файлом `eval.py` и остальными файлами, требующимися для его работы.

3. Прочитайте файл `eval.py`.

В файле содержатся функции `count_the_types_of_cubes` и `load_tools`.

4. Допишите в файле `eval.py` функцию `load_tools`.

Функция используется для загрузки моделей машинного обучения из файлов или загрузки эталонных изображений. Если вы не используете ее, пусть возвращает пустой список.

5. Допишите в файле `eval.py` функцию `count_the_types_of_cubes`.

Функция получает на вход BGR-изображение и список от `load_tools`. Функция должна возвращать массив `numpy 3 × 3 × 2` с элементами типа `np.uint8`.

6. Запустите файл `main.py` и проверьте свой алгоритм.

Если программа выдала ошибку, то найдите и исправьте ее в файле `eval`, вновь запустите файл `main.py`.

7. Пришлите решение на онлайн-платформу для проверки.

В качестве решения необходимо сдать отредактированный файл `eval.py` или `*.zip` архив с ним. В архиве должны находиться файлы, а не одноименная папка.

Технические ограничения

Размер решения ограничен: не более 30 МБ. Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в ходе проверки выдал сообщение об ошибке, то следующее решение можно прислать сразу.

Пакеты, ориентированные на работу с изображениями и данными, использующиеся на платформе проверки:

Python 3.8.10; catboost 1.1.1; dlib 19.24.0; gast 0.4.0; h5py 3.7.0; imutils 0.5.4; keras 2.9.0; Keras-Preprocessing 1.1.2; matplotlib 3.6.2; numpy 1.23.2; opencv-python 4.6.0.66; pandas 1.5.1; scikit-image 0.19.3;

scikit-learn 1.1.3; scipy 1.9.3; tensorflow-cpu 2.9.2; torch 1.13.0; torchaudio 0.13.0; torchvision 0.14.0.

Используйте совместимые пакеты.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/0Flrb4devulnCc>.

Формат входных данных

Изображение `np.uint8` в формате BGR.

Формат выходных данных

Массив `numpy` $3 \times 3 \times 2$ элементами `np.uint8`.

Файл решения

<https://disk.yandex.ru/d/zvxHIthEh8DnfQ>.

Тесты

<https://disk.yandex.ru/d/Gx4VIqYo6bs3tg>.

Решение

Решить задачу можно множеством способов, в том числе и с применением машинного обучения. Но из-за небольшого количества предоставленных данных, постоянного освещения, простоты и ограниченности вариантов расположения объектов задачу целесообразно решать без применения машинного обучения.

После анализа предоставленных данных можно выделить несколько областей на изображении, соответствующие граням кубиков на нем.

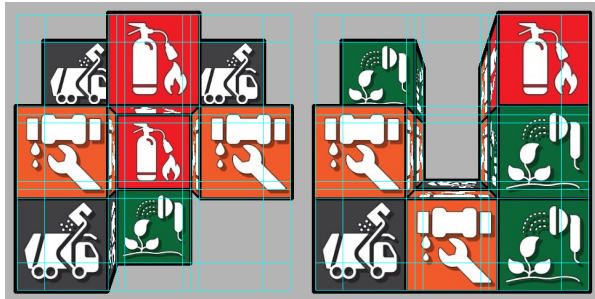


Рис. 6. Области, соответствующие граням кубиков

В каждой области подсчитывается количество пикселей цветов кубиков и черного цвета. Анализ цветов областей позволяет однозначно определить статус кубика в них. Крайние области позволяют определить статус кубиков верхнего слоя, кроме центрального.

Для определения статуса центрального кубика нужно проанализировать центральную область и соседние с ней. Если в них есть черный цвет, значит, центральный верхний куб отсутствует, иначе он закрывал бы черные грани кубиков под собой. Если черного нет, значит, цвет центральной области определяет цвет центрального кубка верхнего слоя.

Статус угловых кубиков нижнего слоя определяется по трем областям, соответствующим возможно видимым граням кубика. Для неугловых кубиков анализируются две области, соответствующие возможно видимым граням. Для центрального нижнего кубика анализируется одна центральная область.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  import cv2
2  import numpy as np
3
4  # Не используется
5  def load_tools() -> list:
6      tools = []
7      return tools
8
9  # Ниже будет пояснение
10 def get_color(arr):
11     return sum(sum(arr > 0))
12
13 # Определение статуса не угловых кубиков нижнего слоя
14 def get_bottom_center_color(color, top, col1, box1):
15     # color - цвет верхней грани нижнего кубика
16     # top - статус кубика над проверяемым
17     # col1 - сторона кубика смотрящая в центр
18     # box1 - статус центрального нижнего кубика
19     if top > 0: # если верхний кубик есть, то
20         if box1 == 0: # если центрального нижнего кубика нет, то
21             return col1 # принимаем за его цвет грань смотрящую в центр
22         else:
23             return 1 # иначе считаем кубик невидимым
24     else: # если верхнего кубика нет,
25         # то принимаем за цвет нижнего цвет верхней грани
26         return color
27
28 # Определение статуса угловых кубиков нижнего слоя
29 def get_bottom_corner_color(color, top, col1, box1, col2, box2):
30     # color - цвет верхней грани нижнего кубика.
31     # top - статус кубика над проверяемым.
32     # col1 - цвет первой из двух возможно видимых боковых сторон кубика.
33     # box1 - статус кубика смежного с col1. Он закрывает col1, если есть.
34     # col2 - цвет второй из двух возможно видимых боковых сторон кубика.
35     # box2 - статус кубика смежного с col2. Он закрывает col2, если есть.
36     if top > 0: # если кубика сверху есть, то...
37         if box1 == 0: # если нет первого кубика сбоку, то
```

```

38 # цвет грани смотрящей на него принимаем за цвет нашего кубика
39     return col1
40 elif box2 == 0: # если нет второго кубика сбоку, то
41     return col2 # аналогично
42 else:
43     return 1 # считаем наш кубик невидимым
44 else:
45     return color
46
47 # Подобранные координаты точек. Образованные ими прямоугольные области
48 # соответствуют пикселям конкретных граней кубиков.
49 xx = [79, 123, 226, 240, 253, 347, 360, 374, 478, 522]
50 yy = [79, 123, 226, 240, 253, 347, 360, 374, 478, 522]
51
52 # Доля пикселей, необходимых для определения цвета области изображения.
53 perc = 0.01
54
55 # Для каждой области высчитываем число пикселей равных 1% от ее площади.
56 limits = np.zeros((len(xx)-1, len(yy)-1), dtype=int)
57 for ix in range(len(xx) - 1):
58     for iy in range(len(yy) - 1):
59         # Площадь области умноженная на 0.01.
60         limits[ix, iy] = int((xx[ix+1]-xx[ix])*(yy[iy+1]-yy[iy])*perc)
61
62 # Основная функция, вызывается для каждого входящего изображения отдельно
63 def count_the_types_of_cubes(image, tools, debug=False) -> np.array:
64     # Бинаризуем изображение так, чтобы белыми стали пиксели,
65     # соответствующие конкретному типу кубиков.
66     # Сохраняем черно-белые изображения полученные в результате бинаризации.
67     cubes2_red = cv2.inRange(image, (30, 20, 230), (45, 37, 250))
68     cubes3_green = cv2.inRange(image, (50, 90, 0), (65, 110, 35))
69     cubes4_gray = cv2.inRange(image, (50, 50, 50), (70, 70, 70))
70     cubes5_orange = cv2.inRange(image, (33, 80, 190), (70, 110, 255))
71
72     cubes_black = cv2.inRange(image, (0, 0, 0), (10, 10, 10)) # Белыми станут
73     ↪ ребра кубиков.
74
75 # a - список хранящий статус каждой области.
76 a = []
77 colors = (cubes2_red, cubes3_green, cubes4_gray, cubes5_orange)
78 # Пробегаем по всем областям и считаем число цветных пикселей в них.
79 for ix in range(len(xx)-1):
80     row = [] # строка для хранения статусов областей.
81     for iy in range(len(yy)-1):
82         best_col = 0
83         best_val = 0
84         for col in range(len(colors)):
85             # get_color - считает число белых пикселей в области.
86             val = get_color(colors[col][xx[ix]:xx[ix + 1], yy[iy]:yy[iy + 1]])
87             if val > best_val: # Ищем область, в которой белого больше всего.
88                 best_val = val # Запоминаем максимум белых пикселей.
89                 best_col = col # Индекс цвета, которого больше всего.
90
91 # Для каждой области добавляем индекс цвета, 0 - если ничего нет.
92 if best_val > limits[ix, iy]: # Если белого больше чем порог, то
93     row.append(best_col+2) # запоминаем индекс +2.
94     # +2 потому, что 0 - отсутствующий кубик, 1 - невидимый кубик.
95 else:
96     row.append(0) # Если в области нет искомым цветов,
97                     # считаем что нет и кубиков.

```

```

97     a.append(row) # Добавляем строку в список.
98     a = np.array(a) # Превращаем A в массиве питру.
99
100     i_mid = len(a) // 2 # Индекс центральной области.
101
102     # Проверяем наличие черного в области вокруг центра,
103     # соответствующей боковым и нижней грани центрального нижнего куба
104     if get_color(cubes_black[xx[i_mid-1]:xx[i_mid + 2],
105                               yy[i_mid-1]:yy[i_mid + 2]]) > 30:
106         black_center = 0 # Если черное есть,
107                           # то считаем центральный куб в верхнем отсутствующим
108     else:
109         black_center = 10
110
111     # Определяем статус кубиков верхнего слоя по областям,
112     # которые однозначно позволяют идентифицировать наличие/отсутствие и
113     # цвет кубика.
114     up_layer = np.array([
115         [a[0, 1], a[0, i_mid], a[0, -2]],
116         [a[i_mid, 0], min(black_center, a[i_mid, i_mid]), a[i_mid, -1]],
117         [a[-1, 1], a[-1, i_mid], a[-1, -2]]
118     ])
119
120     # Заполняем 0-ми массив для хранения статусов кубиков нижнего слоя.
121     bottom_layer = np.zeros(9, dtype=int).reshape(3, 3)
122
123     # Определяем статус центрального кубика нижнего слоя.
124     if up_layer[1, 1] == 0: # Если нет центрального кубика верхнего слоя, то...
125         # определяем статус кубика по центральной области
126         bottom_layer[1, 1] = a[i_mid, i_mid]
127     else:
128         bottom_layer[1, 1] = 1 # невидимый
129
130     # Находим значение для кубиков нижнего слоя.
131     # Логика определения статуса не угловых кубиков нижнего слоя вынесена в
132     # отдельную функцию.
133     # Смотря get_bottom_center_color.
134     bottom_layer[0, 1] = get_bottom_center_color(a[1, i_mid],
135                                                  up_layer[0, 1], a[3, i_mid], bottom_layer[1, 1])
136     bottom_layer[2, 1] = get_bottom_center_color(a[-2, i_mid],
137                                                  up_layer[2, 1], a[-4, i_mid], bottom_layer[1, 1])
138     bottom_layer[1, 0] = get_bottom_center_color(a[i_mid, 1],
139                                                  up_layer[1, 0], a[i_mid, 3], bottom_layer[1, 1])
140     bottom_layer[1, 2] = get_bottom_center_color(a[i_mid, -2],
141                                                  up_layer[1, 2], a[i_mid, -4], bottom_layer[1, 1])
142
143     # Логика определения статуса угловых кубиков нижнего слоя вынесена
144     # в отдельную функцию.
145     # Смотря get_bottom_corner_color.
146     bottom_layer[0, 0] = get_bottom_corner_color(a[1, 1], up_layer[0, 0],
147                                                  a[3, 1], bottom_layer[1, 0],
148                                                  a[1, 3], bottom_layer[0, 1])
149     bottom_layer[2, 0] = get_bottom_corner_color(a[-2, 1], up_layer[2, 0],
150                                                  a[-4, 1], bottom_layer[1, 0],
151                                                  a[-2, 3], bottom_layer[2, 1])
152     bottom_layer[0, 2] = get_bottom_corner_color(a[1, -2], up_layer[0, 2],
153                                                  a[3, -2], bottom_layer[1, 2],
154                                                  a[1, -4], bottom_layer[0, 1])
155     bottom_layer[2, 2] = get_bottom_corner_color(a[-2, -2], up_layer[2, 2],

```



```

157         a[-4, -2], bottom_layer[1, 2],
158         a[-2, -4], bottom_layer[2, 1])
159
160     result = np.array([bottom_layer, up_layer])
161
162     return result

```

Задача IV.5. Полет квадрокоптера (15 баллов)

Темы: управление квадрокоптером в ROS, нейросетевые детекторы, обнаружение объектов в видеопотоке.

Условие

Напишите программу — полетное задание для квадрокоптера. Коптеру необходимо взлететь, выполнить перелет в точку HOME, после чего переместиться в точку номер 1. Далее необходимо выполнить перелет по маршруту на изображении ниже.

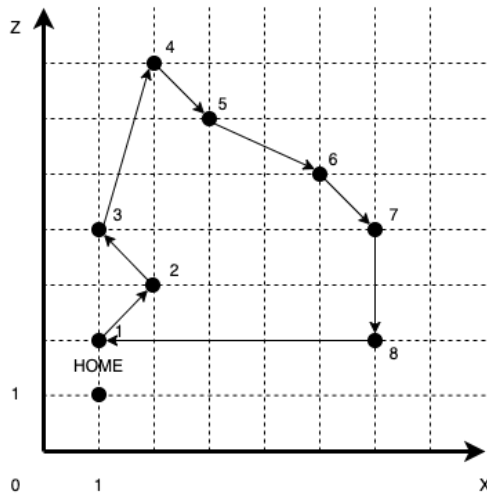


Рис. 7. Маршрут квадрокоптера

После выполнения пролета по маршруту необходимо вернуться в точку HOME и выполнить посадку.

Полетное задание задается с помощью массива `coordinates`. Точки в массиве задаются последовательно. Точка HOME задается с помощью переменной `home_point`. Для успешного выполнения задания необходимо верно изменить указанные переменные.

Полет совершается в вертикальной плоскости (координаты X и Z изменяются, Y всегда равна нулю). Для взлета используется функция `ap.takeoff()`, за посадку отвечает функция `ap.landing()`.

Выполнение: скачайте материалы задания: <https://disk.yandex.ru/d/AmtYYjNWKYB6xg>.

Для вас подготовлен файл `eval.py`. Он содержит шаблон полетного задания с некоторыми ошибками и недоработками. Вам необходимо внимательно ознакомиться с файлом, решить, какие части кода требуют изменений, и отредактировать их. Пример рабочего скрипта полета по заданной траектории представлен по ссылке: https://github.com/geoscan/gs_example/blob/noetic/src/flight_test.py.

В качестве решения необходимо сдать на проверку отредактированный файл `eval.py`.

Внимание! Для решения задачи вы вносите изменения только в файл `eval.py`. Никаких дополнительных библиотек, кроме подключенных в коде `eval.py`, использовать нельзя. Ознакомьтесь с документацией и примерами программирования Пионер Макс. https://github.com/geoscan/geoscan_pioneer_max.

Технические ограничения

Если ваш алгоритм успешно проверен платформой, то следующее решение можно прислать только через 10 мин. Если ваш алгоритм в коде проверки выдал сообщение об ошибке, то следующее решение можно прислать сразу.

Материалы задачи, предоставленные участникам

<https://disk.yandex.ru/d/AmtYYjNWKYB6xg>.

Файл решения

https://disk.yandex.ru/d/_YYCykWYZRTEVA.

Решение

Участникам необходимо добавить в массив с координатами точек назначения и исправить порядок вызова функций взлета и посадки.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```

1  #!/usr/bin/env python3
2  # -*- coding: utf-8 -*-
3
4  import rospy
5  from gs_flight import FlightController, CallbackEvent
6  from gs_board import BoardManager
7
8  rospy.init_node("flight_test_node")
9
10 home_point = [1, 0, 1]
11
```

```

12 coordinates = [
13     home_point,
14     [1, 0, 2],
15     [2, 0, 3],
16     [1, 0, 4],
17     [2, 0, 7],
18     [3, 0, 6],
19     [5, 0, 5],
20     [6, 0, 4],
21     [6, 0, 2],
22     [1, 0, 2],
23     home_point]
24
25 run = True
26 position_number = 0
27
28 def callback(event):
29     global ap
30     global run
31     global coordinates
32     global position_number
33
34     event = event.data
35     if event == CallbackEvent.ENGINES_STARTED:
36         print("engine started")
37         ap.takeoff()
38     elif event == CallbackEvent.TAKEOFF_COMPLETE:
39         print("takeoff complete")
40         position_number = 0
41         ap.goToLocalPoint(coordinates[position_number][0],
42                             coordinates[position_number][1],
43                             coordinates[position_number][2])
44     elif event == CallbackEvent.POINT_REACHED:
45         print(f"point {position_number} reached")
46         position_number += 1
47         if position_number < len(coordinates):
48             ap.goToLocalPoint(coordinates[position_number][0],
49                                 coordinates[position_number][1],
50                                 coordinates[position_number][2])
51         else:
52             ap.landing()
53     elif event == CallbackEvent.COPTER_LANDED:
54         print("programm finish")
55         run = False
56
57 board = BoardManager()
58 ap = FlightController(callback)
59 once = False
60
61 while not rospy.is_shutdown() and run:
62     if board.runStatus() and not once:
63         print("start program")
64         ap.preflight()
65         once = True
66     pass

```

Работа наставника НТО при подготовке к заключительному этапу

На этапе подготовки к заключительному этапу НТО наставник решает две важные задачи: помочь участникам в подготовке к предстоящим соревнованиям и формирование устойчивой и слаженной команды. Для подготовки рекомендуется использовать сборники задач прошлых лет. Кроме того, наставнику важно изучить организационные особенности заключительного этапа, чтобы помочь ученикам разобраться в формальных особенностях его проведения.

Наставник НТО также может познакомиться с разработчиками профилей для получения консультации о подготовке к заключительному этапу, дополнительных материалах и способах поддержки высокой мотивации участников.

При работе с командой участников рекомендуется уделить внимание следующим вопросам:

- Сплочение команды. Наставнику необходимо уделить этому особое внимание, если участники команды находятся в разных городах и не имеют возможности встретиться в очном формате. Регулярные встречи, в том числе в дистанционном формате, помогут поддержать эффективную и позитивную коммуникацию внутри команды.
- Анализ состава команды. Необходимо обсудить роли участников в команде и задачи, которые им предстоит решать в рамках выбранных ролей. Кроме того, нужно обсудить взаимозаменяемость ролей.
- Анализ знаний и компетенций участников. Необходимо убедиться, что участники обладают нужными навыками и компетенциями и продумать план по формированию и развитию недостающих навыков и компетенций.
- Составление плана подготовки. График занятий строится, исходя из даты начала заключительного этапа.
- Участие в подготовительных мероприятиях от разработчиков профилей. Перед заключительным этапом проводятся установочные вебинары, разборы задач прошлых лет, практикумы, хакатоны, мастер-классы для финалистов. Информация о таких мероприятиях публикуется в группе НТО в VK и в чатах профилей в Telegram.
- Проведение практикумов или хакатонов. Для этого наставники могут использовать материалы для подготовки к соответствующему профилю и сборники задач прошлых лет. Практикумы и хакатоны могут проводиться дистанционно, рекомендации для этого формата приведены в сборниках 2020–22 гг.

Во время заключительного этапа участников сопровождают модераторы или волонтеры, разработчики профиля и организаторы НТО. Внешнее вмешательство в ход соревнований запрещено. Участники, получившие во время проведения НТО стороннюю помощь, могут быть дисквалифицированы.

Заключительный этап

Предметный тур

Информатика и информационные технологии. 8–11 классы

Тестовые наборы для задач представлены по ссылке — https://disk.yandex.ru/d/cyYvMN3F3Q_wNw.

Задача VI.1.1.1. (50 баллов)

Условие

На робостанции появляются задания, которые отправляются на исполнение в сервисный центр. Задания бывают следующих типов: `fire`, `med_aid`, `crush`.

Известно, что за отчетный период были выполнены 40 заданий типа `fire`, 50 типа `med_aid`, 25 типа `crush`. Один робот мог исполнить только одно задание. За отчетный период невыполненных заданий нет.

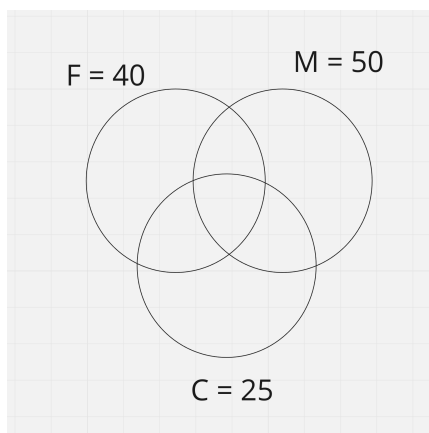
- Число роботов, которые выполняли задания типа `med_aid` или `crush`, было 60.
- Число роботов, которые выполняли только задания типа `fire` было 20.
- Число роботов, которые выполняли только задания типа `fire` и `crush`, было 5.
- Число роботов, которые выполняли все три типа заданий, было 10.

Определите, сколько роботов выполняли только задания типа `med_aid`.

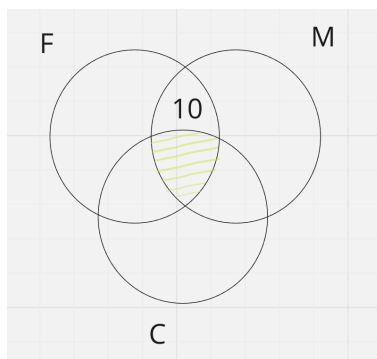
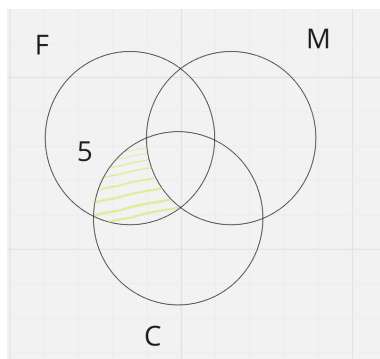
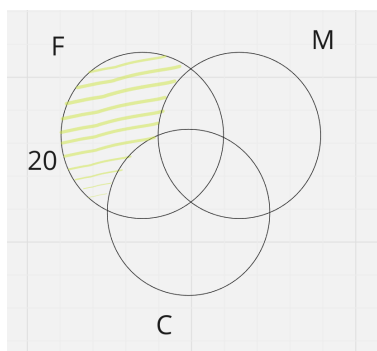
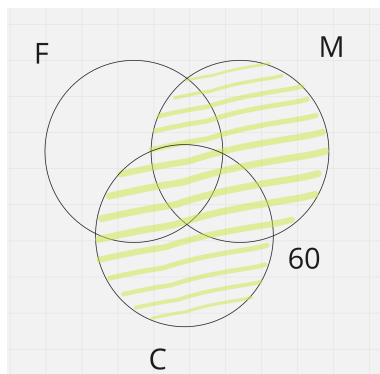
Решение

Определим множества и обозначения:

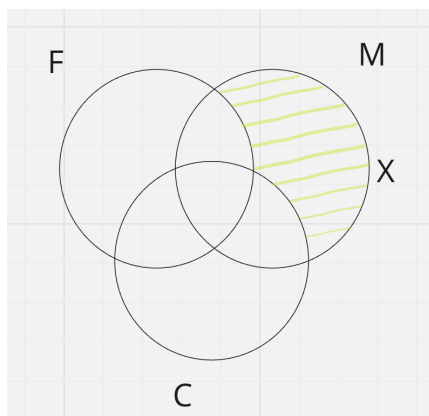
- множество роботов, которые выполняли задания типа «`fire`» — F ;
- множество роботов, которые выполняли задания типа «`med_aid`» — M ;
- множество роботов, которые выполняли задания типа «`crush`» — C .



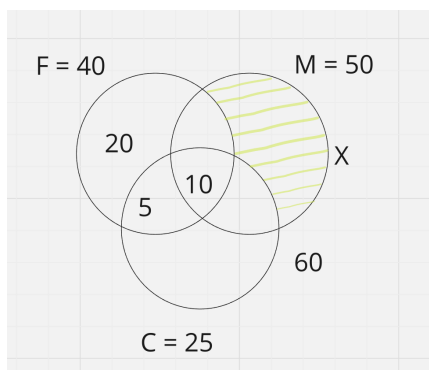
Изобразим с помощью диаграммы Эйлера – Венна данные из условия.



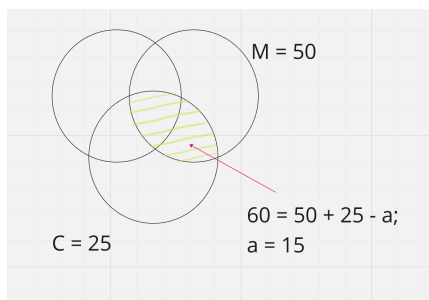
Обозначим за X область, которую необходимо найти.

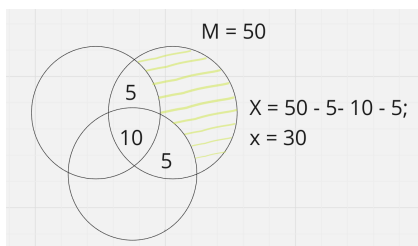
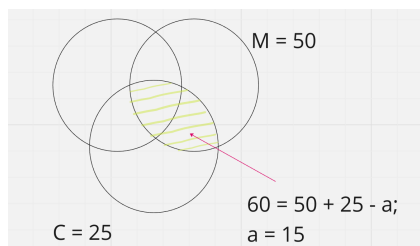


Объединим исходные данные на рисунке.

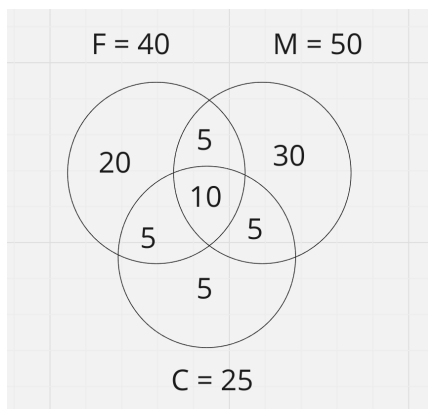


Найдем «неизвестные» значения областей диаграммы.





Итоговый вид диаграммы.



Ответ: 30.

Задача VI.1.1.2. «IP-адресация» (80 баллов)

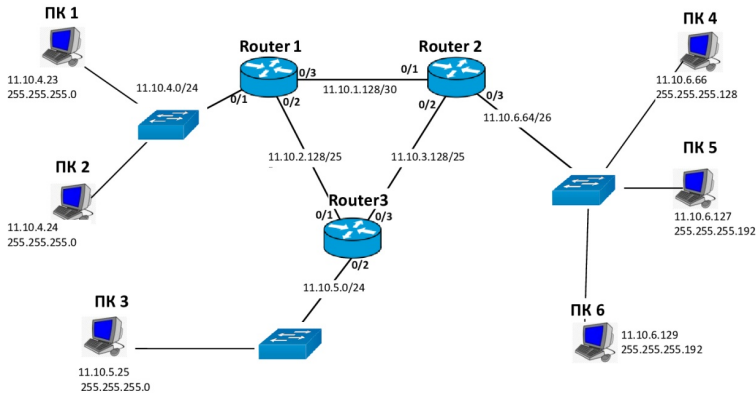
Условие

Сетевому администратору поставлена задача настроить IP-адресацию и статическую маршрутизацию сети (см. схему). Необходимо выполнить настройку сетевого оборудования таким образом, чтобы все персональные компьютеры (ПК) в сети могли обмениваться IP-пакетами. Для этого необходимо назначить IP-адреса на все компьютеры и на все интерфейсы маршрутизаторов, а также настроить шлюзы по умолчанию.

Администратору были даны шесть сетей:

- 11.10.4.0/24,
- 11.10.1.128/30,
- 11.10.6.64/26
- 11.10.3.128/25,
- 11.10.5.0/24,
- 11.10.2.128/25.

В процессе настройки сетевого оборудования и персональных компьютеров администратор допустил ошибки. Ваша задача — помочь администратору найти ошибки.



Необходимо проверить:

- 1. Правильность назначения IP-адресов на компьютеры. IP-адрес компьютера и интерфейс маршрутизатора, к которому подключен данный сегмент сети, должны находиться в одной IP-сети.
- 2. Правильность назначения шлюзов по умолчанию для компьютеров. Шлюз по умолчанию должен являться интерфейсом маршрутизатора, к которому подключен данный сегмент сети.
- 3. Правильность назначения IP-адресов на интерфейсы маршрутизаторов. Проверить с помощью маски подсети, что IP-адреса, назначенные на соседние интерфейсы маршрутизаторов, будут в одной подсети.

Укажите номера ячеек таблиц, содержащие ошибки. Ответ должен состоять из последовательно указанных адресов ячеек таблиц через пробел.

Пример формата ответа: A1 C2 E1 F1 H1 J1 W1 Z1.

Router 1

	A	B	C
	Интерфейс	IP-адрес	Маска подсети
1	FastEthernet 0/1	11.10.4.1	255.255.255.0
2	FastEthernet 0/2	11.10.2.130	255.255.255.128
3	FastEthernet 0/3	11.10.1.128	255.255.255.252

Router 2

	D	E	F
	Интерфейс	IP-адрес	Маска подсети
1	FastEthernet 0/1	11.10.1.130	255.255.255.248
2	FastEthernet 0/2	11.10.3.130	255.255.255.128
3	FastEthernet 0/3	11.10.6.65	255.255.255.192

Router 3

	G	H	I
	Интерфейс	IP-адрес	Маска подсети
1	FastEthernet 0/1	11.10.2.131	255.255.255.128
2	FastEthernet 0/2	11.10.5.1	255.255.255.0
3	FastEthernet 0/3	11.10.3.129	255.255.255.0

ПК1 «Настройка протокола IP»

	J	K	L
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.4.23	255.255.255.0	11.10.2.130

ПК2 «Настройка протокола IP»

	M	N	O
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.4.24	255.255.255.128	11.10.4.1

ПК3 «Настройка протокола IP»

	P	Q	R
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.5.25	255.255.255.0	11.10.5.1

ПК4 «Настройка протокола IP»

	S	T	U
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.6.66	255.255.255.128	11.10.6.65

ПК5 «Настройка протокола IP»

	V	W	X
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.6.127	255.255.255.192	11.10.6.65

ПК6 «Настройка протокола IP»

	Y	Z	AA
	IP-адрес компьютера	Маска подсети	Шлюз по умолчанию
1	11.10.6.129	255.255.255.192	11.10.6.65

Теоретическая справка

Вычислительная сеть (информационно-вычислительная сеть) обеспечивает обмен данными между компьютерами, серверами, маршрутизаторами и другим оборудованием или программным обеспечением.

Маршрутизатор — это сетевое устройство, которое соединяет вычислительные устройства и сети с другими сетями. Маршрутизатор работает на сетевом уровне (третий уровень сетевой модели OSI). Маршрутизаторы выполняют три основные функции:

1. Определение пути.

Маршрутизатор определяет путь данных при перемещении из источника в место назначения. Пытается найти наилучший путь, анализируя сетевые метрики, такие как задержка, пропускная способность и скорость.

2. Пересылка данных.

Маршрутизатор пересылает данные следующему устройству по выбранному пути, чтобы в конечном итоге доставить их в место назначения. Устройство и маршрутизатор могут находиться в одной или в разных сетях.

3. Балансировка нагрузки.

Маршрутизатор может отправлять копии одного и того же пакета данных по нескольким разным путям. Это позволяет уменьшить количество ошибок из-за потери данных, создать избыточность и управлять объемом трафика.

Коммутатор — устройство, предназначенное для соединения нескольких узлов компьютерной сети в пределах одного или нескольких сегментов сети. Коммутатор работает на канальном уровне (второй уровень сетевой модели OSI). Наиболее существенное различие между маршрутизаторами и коммутаторами заключается в том, что маршрутизаторы соединяют сети, а коммутаторы соединяют устройства внутри сети. Маршрутизаторы могут подключаться к Интернету и локальным устройствам, в то время как коммутаторы подключают локальные устройства.

Основные правила назначения IP-адресов на интерфейсы маршрутизаторов:

1. Соседние интерфейсы маршрутизаторов должны находиться в одной IP-сети.
2. Каждому IP-адресу соответствует маска подсети (битовая маска).

Битовая маска необходима для определения по IP-адресу, адреса подсети и адреса узла (хоста, компьютера, устройства) этой подсети. В отличие от IP-адреса маска подсети не является частью IP-пакета. Благодаря маске можно узнать, какая часть IP-адреса узла сети относится к адресу сети, а какая — к адресу самого узла в этой сети.

Например, узел с IP-адресом 11.10.2.130 и маской подсети 255.255.255.128, с длиной префикса 25 бита (/25), находится в сети 11.10.2.128.

Другой вариант определения — это определение подсети IP-адресов. Например, с помощью маски подсети можно сказать, что один диапазон IP-адресов будет в одной подсети, а другой диапазон, соответственно, в другой подсети.

Чтобы получить адрес сети, зная IP-адрес и маску подсети, необходимо применить к ним операцию поразрядной конъюнкции (побитовое И). Например, в случае более сложной маски:

IP-адрес: 00001011 00001011 00000010 10000010 (11.10.2.130).

Маска подсети: 11111111 11111111 11111110 10000000 (255.255.255.128).

Адрес сети: 00001011 00001011 00000010 10000000 (11.10.2.128).

Шлюз по умолчанию (англ. *Defaultgateway*) — узел компьютерной сети (сетевой шлюз в маршрутизируемых протоколах, интерфейс маршрутизатора), на который пакет отправляется в том случае, если маршрут к сети назначения пакета неизвестен (не задан явным образом в таблице маршрутизации хоста или не соответствует IP-адресу назначения пакета).

Шлюз по умолчанию находится в одной IP-сети, что и IP-адрес компьютера, на который он устанавливается.

Решение

Router 1

Ответ: адрес сети нельзя устанавливать на интерфейс маршрутизатора, правильный адрес 11.10.1.129.

Router 2

Ответ: неправильно указана маска подсети 255.255.255.248 т. к. префикс /30 (см. рис.) указывает на количество сетевых бит в адресе подсети, их 30 или 11111111.11111111.11111111.11111100, в десятичном виде правильная маска подсети 255.255.255.252.

Router 3

Ответ: неправильно указана маска подсети 255.255.255.0 т. к. префикс /25 (см. рис.) указывает на количество сетевых бит в адресе подсети, их 25 или 11111111.11111111.11111111.10000000, в десятичном виде правильная маска подсети 255.255.255.128.

ПК1 «Настройка протокола IP»

Ответ: неправильно указан IP-адрес шлюза по умолчанию на ПК1, должен быть указан IP-адрес интерфейса FastEthernet 0/1 маршрутизатора Router 1, 11.10.4.1.

ПК2 «Настройка протокола IP»

Ответ: неправильно указана маска подсети 255.255.255.128 т. к. префикс /24 (см. рис.) указывает на количество сетевых бит в адресе подсети, их 24 или 11111111.11111111.11111111.00000000, в десятичном виде правильная маска подсети 255.255.255.0.

ПК3 «Настройка протокола IP»

Ошибок нет.

ПК4 «Настройка протокола IP»

Ответ: неправильно указана маска подсети 255.255.255.128, правильная 255.255.255.192. Для то, чтобы пакеты с ПК4 могли маршрутизироваться в интернет, IP-адрес компьютера и шлюза по умолчанию должны находиться в одной подсети

ПК5 «Настройка протокола IP»

Ответ: неправильно указан IP-адрес компьютера, так как это широковещательный адрес, 11.10.6.01111111, маска подсети 255.255.255.192 определяет доступный диапазон адресов 11.10.6.01000000–11.10.6.01111111, первый адрес 11.10.6.01000000 (11.10.6.64 и последний 11.10.6.127) — устанавливать нельзя, можно использовать адреса в диапазоне 11.10.6.65–11.10.6.126. Обратит внимания на то, что адрес 11.10.6.66 уже используется на ПК4.

ПК6 «Настройка протокола IP»

Ответ: неправильно указан IP-адрес компьютера, так как этот адрес 11.10.6.100000001 (11.10.6.129), из другой сети, 11.10.6.100000000, (11.10.6.128) маска подсети 255.255.255.192 определяет доступный диапазон адресов 11.10.6.01000000–11.10.6.01111111, первый адрес 11.10.6.01000000 (11.10.6.64 и последний 11.10.6.127) — устанавливать нельзя, можно использовать адреса в

диапазоне 11.10.6.65–11.10.6.126. Обратить внимания на то, что адрес 11.10.6.66 уже используется на ПК4.

Ответ.

1	2	3	4	5	6	7	8
B3	F1	I3	L1	N1	T1	V1	Y1

Задача VI.1.1.3. Любимая строка робота (50 баллов)

Имя входного файла: стандартный ввод или `input.txt`.

Имя выходного файла: стандартный вывод или `output.txt`.

Ограничение по времени выполнения программы: 1 с.

Ограничение по памяти: 512 Мбайт.

Условие

Изучая новые миры, роботы обнаружили загадочный текстовый артефакт, написанный последователями древнего кода.

Текст артефакта представляет собой последовательность латинских букв и запятых. Роботам поручено обработать текст артефакта и вывести все слова строки S через запятую с пробелом, учитывая, что слово — любая непустая последовательность латинских букв без пробелов и запятых.

Эта задача стала настоящей загадкой для искусственного интеллекта, помогите ему в этом деле.

Формат входных данных

Дано число $1 \leq N \leq 1e5$ и строка S размера N .

Формат выходных данных

Выведите все слова строки S через запятую с пробелом.

Примеры

Пример №1

Стандартный ввод
15 ,asf,ga,,fs,f,
Стандартный вывод
asf, ga, fs, f

Пример №2

Стандартный ввод
28 asasf,asf,,,,fasfasf,,,saf,
Стандартный вывод
asasf, asf, fasfasf, saf

Решение

Для решения данной задачи нужно аккуратно разделить изначальную строку на несколько других, разделенных запятыми, затем убрать пустые строки, в случае их наличия, и вывести оставшиеся друг за другом через запятую и пробел.

Алгоритмическая сложность решения составляет $O(N)$.

Пример программы-решения

Ниже представлено решение на языке Python 3.

```
1 num, string = int(input()), input().split(',')
2 print(', '.join([str(i) for i in string if i != '']))
```

Задача VI.1.1.4. Матрица робо-распределения (80 баллов)

Имя входного файла: стандартный ввод или `input.txt`.

Имя выходного файла: стандартный вывод или `output.txt`.

Ограничение по времени выполнения программы: 1 с.

Ограничение по памяти: 512 Мбайт.

Условие

Существует легенда о загадочной матрице, на которой расставлены дроны (единички) и пустота (нули). Люди создали эту матрицу для создания идеального «робо-распределения», что и стало вашей задачей. Для ее успешного выполнения вам осталось лишь правильно ответить на каждый из оставшихся запросов, описанных ниже.

Для заданной матрицы существует Q запросов двух разных видов.

Вид запроса 1 типа: 1 x , где x — номер вертикали, на которой нужно найти ячейку.

Вид запроса 2 типа: 2 y , где y — номер горизонтали, на которой нужно найти ячейку.

Правильным ответом на любой запрос будет такая ячейка матрицы на заданной вертикали или горизонтали, что сумма манхэттенских расстояний (модуль между координатами x и координатами y) от этой ячейки до всех дронов будет минимальной. Выведите координаты данной ячейки для каждого запроса в виде yx .

Манхэттенское расстояние на матрице между двумя клетками с координатами (y_1, x_1) и (y_2, x_2) равно $|x_1 - x_2| + |y_1 - y_2|$, где $|x|$ = абсолютное значение числа x .

Формат входных данных

Вам даны 3 числа N , M и Q . $1 \leq Q \leq 1e5$, $1 \leq N \cdot M \leq 1e5$, матрица размером $N \cdot M$ и Q пар чисел, описывающих запросы.

Формат выходных данных

Выведите по два числа для каждого запроса, координаты Y и X необходимой ячейки. Если таких ячеек несколько, выведите такую подходящую ячейку, чтобы сумма ее координат $(Y + X)$ была минимальной.

Примеры

Пример №1

Стандартный ввод
3 3 3
1 0 1
0 1 0
0 0 0
1 2
1 3
2 3
Стандартный вывод
1 2
1 3
3 2

Пример №2

Стандартный ввод
2 4 3
0 1 1 0
1 0 0 1
1 1
2 2
1 3
Стандартный вывод
1 1
2 2
1 3

Решение

Для решения данной задачи нужно заметить, что для каждой матрицы существует ровно одна подходящая клетка, имеющая наименьшую сумму манхэттенских

расстояний до всех единиц для любого заданного столбца или строки. Она лежит на той координате, где разница расстояний до всех единиц слева и единиц справа минимальна. Аналогичным образом можно найти подобную координату для строки, наблюдая за вертикальной разницей.

Так как одну из координат ответа мы получаем в каждом запросе, для ответа остается лишь подставить подходящую вторую координату, одну из тех двух, что мы нашли ранее.

Алгоритмическая сложность решения составляет $O(NM + Q)$.

Пример программы-решения

Ниже представлено решение на языке C++.

```

1  #include "bits/stdc++.h"
2
3  #define int long long
4  #define inf (int)1e18+1
5  #define endl "\n"
6
7  using namespace std;
8
9  int minimize_dist(vector<int> &c_v, int c) {
10     int cur = 0;
11     for (int i = 0; i < c_v.size(); i++) {
12         cur += c_v[i] * i;
13     }
14     int best = cur;
15     int best_ind = 0;
16     int passed = 0;
17     for (int i = 1; i < c_v.size(); i++) {
18         passed += c_v[i - 1];
19         cur = cur - c + passed * 2;
20         if (cur < best) {
21             best = cur;
22             best_ind = i;
23         }
24     }
25     return best_ind + 1;
26 }
27
28 void solve() {
29     int n, m, q;
30     cin >> n >> m >> q;
31     vector<vector<int>>> mtrx(n);
32     vector<int> x_c(m, 0);
33     vector<int> y_c(n, 0);
34     int c = 0;
35     for (int y = 0; y < n; y++) {
36         for (int x = 0; x < m; x++) {
37             int r;
38             cin >> r;
39             if (r == 1) {
40                 x_c[x]++;
41                 y_c[y]++;
42                 c++;
43             }

```



```

44         mtrx[y].push_back(r);
45     }
46 }
47 int best_x = minimize_dist(x_c, c), best_y = minimize_dist(y_c, c);
48 for (int i = 0; i < q; i++) {
49     int a, b;
50     cin >> a >> b;
51     if (a == 1) {
52         cout << best_y << " " << b << endl;
53     } else {
54         cout << b << " " << best_x << endl;
55     }
56 }
57 }
58 }
59
60 signed main() {
61     ios::sync_with_stdio(false);
62     cin.tie(nullptr);
63     cout << fixed << setprecision(9);
64     int t = 1;
65     //  cin >> t;
66     while (t--) {
67         solve();
68     }
69 }

```

Задача VI.1.1.5. Минимальное время (100 баллов)

Имя входного файла: стандартный ввод или `input.txt`.

Имя выходного файла: стандартный вывод или `output.txt`.

Ограничение по времени выполнения программы: 1 с.

Ограничение по памяти: 512 Мбайт.

Условие

В мире чисел и роботов существует задача «Минимальное Время». В ней вам дана матрица чисел и координаты финишной клетки. Робот начинает в левом верхнем углу матрицы и может двигаться влево, вправо, вверх или вниз, тратя на каждый ход количество времени, равное числу, записанному в клетке, в которой он находится в данный момент. Нули — препятствия, в которые нельзя перейти. Определите наименьшее возможное время, достаточное для достижения роботом финишной клетки или выведите -1 , если это невозможно.

Формат входных данных

Вам даны числа $1 \leq N \cdot M \leq 1e5$, координаты $1 \leq Y \leq N$, $1 \leq X \leq M$ финишной клетки и матрица чисел размером $N \cdot M$.

Формат выходных данных

Выведите одно число T — наименьшее возможное время для достижения финишной клетки.

Примеры

Пример №1

Стандартный ввод
4 3
3 2
1 2 1
2 1 4
0 1 0
3 2 2
Стандартный вывод
4

Пример №2

Стандартный ввод
4 4
4 1
1 1 1 1
8 8 8 1
1 1 1 1
8 8 8 1
Стандартный вывод
9

Решение

Для решения данной задачи можно использовать алгоритм Дейкстры, то есть всегда двигаться из вершины с наименьшей ценой попадания туда. Для этого достаточно представить данную нам матрицу в виде графа.

Используя структуру `priority_queue`, можно добиться асимптотики $N \log N + M$, где N — количество вершин, M — ребер графа. Так как наш граф представляет собой матрицу размером $N \cdot M$, а максимальное количество ребер, которое может быть у вершины — 4, мы получаем асимптотическую сложность в $O(NM \cdot (\log(NM)))$, что достаточно быстро.

Алгоритмическая сложность решения составляет $O(NM \cdot (\log(NM)))$.

Пример программы-решения

Ниже представлено решение на языке C++.

```

1  #include "bits/stdc++.h"
2
3  #define int long long
4  #define inf (int)1e18+1
5  #define endl "\n"
6
7  using namespace std;
8
9  struct cell {
10     int time;
11     pair<int, int> coords;
12
13     cell(int t, pair<int, int> p) {
14         time = t;
15         coords = p;
16     }
17 };
18
19 bool operator<(const cell &c1, const cell &c2) {
20     return c1.time > c2.time;
21 }
22
23 void solve() {
24     int n, m, y, x;
25     cin >> n >> m >> y >> x;
26     priority_queue<cell> pq;
27     vector<vector<int>> mtrx(n);
28     vector<vector<int>> visited(n);
29     for (int i = 0; i < n; i++) {
30         for (int j = 0; j < m; j++) {
31             int num;
32             cin >> num;
33             visited[i].push_back(0);
34             mtrx[i].push_back(num);
35         }
36     }
37     int ans = inf;
38     if(mtrx[0][0]!=0){
39         pq.push(cell(0, {0, 0}));
40     }
41     while (!pq.empty()) {
42         cell c = pq.top();
43         int c_x = c.coords.second;
44         int c_y = c.coords.first;
45         pq.pop();
46         if (c_y + 1 == y && c_x + 1 == x) {
47             ans = min(ans, c.time);
48         }
49         if (visited[c_y][c_x]) continue;
50         visited[c_y][c_x] = 1;
51         if (c_x > 0 && !visited[c_y][c_x - 1] && mtrx[c_y][c_x - 1]) {
52             pq.push(cell(c.time + mtrx[c_y][c_x], {c_y, c_x - 1}));
53         }
54         if (c_x < m - 1 && !visited[c_y][c_x + 1] && mtrx[c_y][c_x + 1]) {
55             pq.push(cell(c.time + mtrx[c_y][c_x], {c_y, c_x + 1}));
56         }
57         if (c_y > 0 && !visited[c_y - 1][c_x] && mtrx[c_y - 1][c_x]) {
58             pq.push(cell(c.time + mtrx[c_y][c_x], {c_y - 1, c_x}));
59         }
60         if (c_y < n - 1 && !visited[c_y + 1][c_x] && mtrx[c_y + 1][c_x]) {

```

```
61         pq.push(cell(c.time + mtrx[c_y][c_x], {c_y + 1, c_x}));
62     }
63 }
64 cout << (ans == inf ? -1 : ans) << endl;
65 }
66
67 signed main() {
68     ios::sync_with_stdio(false);
69     cin.tie(nullptr);
70     cout << fixed << setprecision(9);
71     int t = 1;
72     //     cin >> t;
73     while (t--) {
74         solve();
75     }
76 }
```

Физика. 8–9 классы

Задача VI.1.2.1. Что за «залетная» птица? (15 баллов)

Темы: кинематика, динамика.

Условие

В вашем распоряжении имеется некоторая информация о двух дронах (коптерах). Известно, что они разные по размеру, по массе и мощности двигателей. Также они различаются по конфигурации винта и количеству лопастей. Произведены они разными фирмами, в разных странах, по разным стандартам.

Что же касается технической информации, которой вы располагаете, то она более чем скромная. Про первый коптер вы знаете лишь то, что он способен некоторое время удерживать вблизи поверхности Земли груз, масса которого в полтора раза больше массы второго коптера, и при этом не превышает массы первого коптера. Про второй коптер вам известно, что он «порожняком» (то есть не имея никакой дополнительной нагрузки), имеет снаряженную массу 4 кг, и способен за первые 3 с полета (при старте с места от поверхности Земли) набрать высоту 50 м.

От вас, как эксперта, требуют сопоставить имеющиеся данные и выяснить, какой из этих коптеров может набрать большую высоту при полете без груза, и какой из них сможет перенести больший груз, двигаясь на высоте 25 м.

Считать, что ускорение свободного падения ($g = 10 \text{ м/с}^2$), а также температура воздуха, его состав и влажность с высотой (в пределах 50 м от поверхности Земли) практически не изменяются.

Критерии оценивания

Критерий	Балл
Сформулирована мысль о необходимости сравнения тяги двигателей коптеров по абсолютной и по относительной величине	3
Определено ускорение, с которым потенциально мог подниматься первый коптер без груза	2
Записано аналитическое выражение, необходимое для нахождения ускорения, с которым поднимался второй коптер	3
Определено численное значение ускорения, с которым двигался второй коптер при взлете без груза	2
Верно сформулирован ответ на первый вопрос задачи (о том, что второй коптер сможет подняться на большую высоту)	1
Дана оценка грузоподъемностей первого и второго коптера по абсолютной величине (6,0 и 4,44 кг)	3
Верно сформулирован ответ на второй вопрос задачи (о том, что первый коптер сможет перенести больший груз)	1
Всего	15

Решение

Для того чтобы дать ответы на поставленные в задаче вопросы, необходимо каким-то образом сравнить тяги, создаваемые винтами коптеров. Притом сделать эти сравнения нужно будет не только по абсолютной величине, но и по относительной.

Про первый коптер мы знаем, что он способен удерживать вблизи поверхности Земли груз, масса которого не превышает массу самого коптера. Это означает, что его двигатели и винты способны удерживать коптер, имеющий удвоенную массу. То есть удельная (по массе) тяга двигателей и винтов первого коптера равна двум единицам, что эквивалентно грузоподъемности коптера до 6 кг. В результате можно сделать вывод о том, что в режиме максимальной тяги его двигатели и винты способны обеспечить взлет коптера (без груза) с вертикальным ускорением, равным ускорению свободного падения (10 м/с^2).

Про второй коптер нам известно, что он без груза способен набирать высоту в 50 м за первые 3 с полета. На основании этой информации можно оценить ускорение, с которым двигался коптер при таком наборе высоты. Если принять, что развиваемые коптером значения вертикальной скорости в первые секунды полета еще не очень большие (т. е. предположить, что на движение коптера действие со стороны сопротивления воздуха еще относительно мало), то коптер, находящийся под действием силы тяжести и силы тяги со стороны собственных винтов, движется (примерно) равноускоренно.

При равноускоренном характере движения пройденный путь за первые секунды со старта можно определить по формуле:

$$S = a \frac{t^2}{2}.$$

Нам известно, что за первые 3 с высота подъема составила 5 м, из чего следует, что вертикальное ускорение, с которым поднимался коптер, составляло около $11,11 \text{ м/с}^2$:

$$a = \frac{2S}{t^2} = \frac{2 \cdot 50 \text{ м}}{(3 \text{ с})^2} \approx 11,11 \text{ м/с}^2.$$

Учитывая, что ускорение свободного падения у поверхности Земли равно 10 м/с^2 , а также то, что на движение второго коптера, пусть и не очень сильно, но все же оказывалось дополнительное противодействие за счет сопротивления воздуха, можно сделать вывод о том, что удельная тяга второго коптера должна быть не менее 2,11 (единиц массы коптера).

$$\frac{11,11 + 10}{10} \approx 2,11.$$

Все вышесказанное говорит о том, что второй коптер имеет большую удельную тягу ($> 2,11$ против 2,0), и следовательно, при отсутствии груза потенциально способен подняться на большую высоту, чем первый.

Отвечая на второй вопрос задачи, необходимо учесть, что масса первого коптера — 6 кг, что в 1,5 раза больше, чем масса второго (4 кг). Поэтому, несмотря на то что винты второго коптера обеспечивают большую удельную тягу, по абсолютной величине грузоподъемность первого коптера составляет 6 кг $[(2,0 - 1) \cdot 6 \text{ кг}]$, что значительно превышает грузоподъемность (4,44 кг) второго коптера $[(2,11 - 1) \cdot 4 \text{ кг}]$.

Ответ:

1. Подняться на большую высоту способен второй коптер.
2. Перенести большой груз сможет первый коптер.

Задача VI.1.2.2. Он улетел, но обещал вернуться (15 баллов)

Темы: оптика.

Условие

Управление беспилотным летательным аппаратом осуществляется с помощью радиосвязи на ультракоротких волнах (УКВ). В отличие от коротких волн, которые благодаря многократным отражениям от ионосферы и от поверхности Земли, могут успешно распространяться далеко за горизонт, для УКВ нужна прямая видимость для обеспечения устойчивости приема сигнала. Однако, даже находясь на открытой территории (например, управляя беспилотным летательным средством, летающим над равнинной степью или над водной гладью озера) при большом удалении от точки управления, может произойти потеря управляющего сигнала, в том числе за счет кривизны Земли.

Возможно ли продолжать поддерживать связь с летательным аппаратом (не боясь потери сигнала) при удалении аппарата на 10 км от центра управления, если на всем протяжении полета аппарату было запрещено находиться на высоте более 10 м над поверхностью озера?

Принять, что система управления размещена непосредственно у поверхности Земли (практически на уровне поверхностной глади озера). Диаметр Земли считать равным 12800 км.

Критерии оценивания

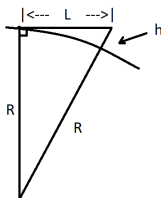
Критерий	Балл
Дано разъяснение о том, что минимальная (критическая) высота летательного аппарата определяется тем моментом, когда он уходит за горизонт (потеря радиосвязи на ультракоротких волнах произойдет при потере прямой видимости за счет кривизны Земли)	2
Сделан соответствующий пояснительный рисунок.	3
Получено аналитическое выражение для соотношения между критической дистанцией и критической высотой полета, при которой летательный аппарат уходит за линию горизонта.	6
Дан правильный и численно обоснованный ответ на второй вопрос задачи о том, что сигнал не будет утерян.	4
Всего	15

Решение

Минимальная высота полета или максимально допустимое удаление (при фиксированной высоте полета) данного летательного аппарата (при радиосвязи на ультракоротких волнах) определяется тем моментом, когда аппарат уходит за линию

горизонта. Этот момент можно оценить их геометрических соотношений, исходя из кривизны Земли.

Определим максимальную дистанцию, при которой летательный аппарат будет находиться в поле прямого зрения, если он находится на высоте 10 м. Для этого определим, на каком расстоянии будет находиться линия горизонта, если наблюдатель, располагается на высоте 10 м (т. е. посмотрим на ситуацию «глазами» летательного аппарата). Для этого сделаем вспомогательный рисунок.



При рассмотрении вспомогательного рисунка нетрудно заметить, что необходимое для решения соотношение можно получить по теореме Пифагора (можно и иначе, но это, пожалуй, самый простой способ). По теореме Пифагора: $R^2 + l^2 = (R + h)^2$, где: R — радиус Земли, а величина h — высота, на которой находится наблюдатель, для которого мы ищем l — дистанцию до линии горизонта.

$$R = \frac{D}{2} = \frac{12800 \text{ км}}{2} = 6400 \text{ км} = 6,4 \cdot 10^6 \text{ м},$$

$$h = 10 \text{ м}.$$

Из соотношения: $R^2 + l^2 = (R + h)^2$ можно выразить дистанцию до линии горизонта: $l = \sqrt{h(2R + h)}$.

В нашем случае линия горизонта будет располагаться на дистанции:

$$l = \sqrt{10 \cdot (2 \cdot 6,4 \cdot 10^6 + 10)} \approx 11314 \text{ м} \approx 11,3 \text{ км}.$$

Таким образом, высота $h = 10$ м, является критической для дистанции 11,3 км. Соответственно, на меньшей дистанции (при 10 км) критическая высота будет существенно меньше 10 м. Это означает, что дрон, летящий на высоте 10 м над поверхностью озера, даже удалившись на расстояние в 10 км, все еще будет находиться в поле зрения оператора (не «уйдет» за линию горизонта). Следовательно, сигнал с ним утерян не будет.

Ответ: да, это возможно. Сигнал на таком расстоянии утерян не будет.

Задача VI.1.2.3. Семь раз отмерь, один — отрежь! (15 баллов)

Темы: электрические явления.

Условие

Случилась небольшая авария. В электрической схеме одно из сопротивлений сгорело. В паспорте было указано, что здесь когда-то было вполне рабочее сопротивление на

10 кОм. Вам необходимо срочно сделать замену, но у вас нет в запасе такого сопротивления. Единственное, чем вы располагаете, это четыре резистора на 6 кОм каждый, немного медной проволоки, обычный паяльник, канифоль и припой. Ничего другого у вас, к сожалению, нет. Дело срочное. Нужно как-то выходить из данного затруднения.

Возможно ли, при помощи имеющихся четырех (абсолютно одинаковых) постоянных резисторов (с номинальным сопротивлением 6 кОм) собрать схему, имеющую суммарное сопротивление 10 кОм? Если вы считаете, что это возможно, тогда:

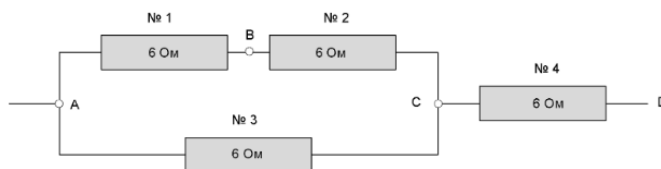
1. изобразите данную схему и поясните ее соответствующим расчетом суммарного сопротивления;
2. укажите, какой из резисторов в собранной вами схеме наиболее вероятно выйдет из строя первым, если вдруг электрическая нагрузка опять превысит допустимый уровень?

Критерии оценивания

Критерий	Балл
Верно изображена принципиальная схема, имеющая сопротивление 10 кОм.	6
Верно записана аналитическая формула для определения общего сопротивления собранной схемы.	4
Корректно рассчитано числовое значение общего сопротивления схемы (10 кОм).	2
Приведены рассуждения и правильно указан резистор, который наиболее вероятно выйдет из строя первым.	3
Всего	15

Решение

Искомая схема, составленная из имеющихся сопротивлений, будет иметь следующий вид.



Суммарное сопротивление данной схемы, составит требуемые 10 кОм.

$$R = \frac{1}{\frac{1}{R_1 + R_2} + \frac{1}{R_3}} + R_4 = \frac{1}{\frac{1}{6+6} + \frac{1}{6}} + 6 = 10.$$

Наибольшей электрической нагрузке будет подвержено сопротивление №4 (отдельно стоящее), т. к. только через него проходит весь ток, идущий по данной цепи. Поэтому, вероятнее всего, именно сопротивление №4 выйдет (при прочих равных) из строя первым.

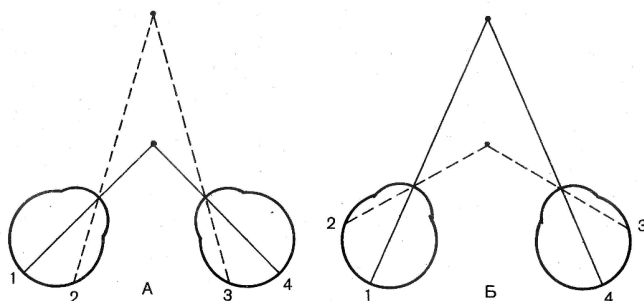
Задача VI.1.2.4. — Видишь меня?!

— Визжу, визжу... у меня стереоскопическое зрение! (25 баллов)

Темы: оптика.

Условие

Способность зрительной системы воспринимать глубину трехмерной структуры путем обработки и интерпретации отличающихся изображений достигается с помощью стереоскопического зрения.



Например, мозг человека получает первичную зрительную информацию каждым глазом, а вследствие того, что глаза находятся на некотором расстоянии, то картины, воспринимаемые ими, отличаются. Получаемые изображения немного смещены по горизонтали, а попавшие в поле зрения объекты видны под разными углами.

Для ориентировки в пространстве в качестве вспомогательного способа оценки расстояний до препятствий в системе навигации беспилотного вездехода использовали тот же принцип — принцип стереоскопического зрения. Две камеры, установленные на специальные несущие валы, были разнесены друг относительно друга на расстояние $d = 70$ см. Для поворота камер использовали приводы с шаговыми двигателями и понижающими редукторами, а взаимное положение камер (углы поворота) оценивали с помощью показаний инкрементных энкодеров — специальных датчиков угла, отслеживающих угловое перемещение в режиме, близком к реальному времени. Дискретность показаний, считываемых с помощью энкодеров, составляет 3600 единиц за один полный оборот несущих валов, к которым крепились камеры.

Оцените, с какой абсолютной погрешностью (ΔL) оператор беспилотного вездехода смог бы определить расстояние L до потенциального препятствия (например, булыжника или трещины), если действительное расстояние до препятствия на момент измерения составляет 45 м, при этом препятствие находится на оси симметрии (то есть непосредственно на пути следования беспилотного вездехода).

Приборную погрешность в определении расстояний при дискретной шкале можно принять как разность в оценке расстояний при движении энкодера на $\pm$ одну позицию, полагая, что дискретность этого прибора является основным источником

ошибки при измерении расстояний. В ответе укажите значение приборной погрешности, округленное до целого значения (в м).

Критерии оценивания

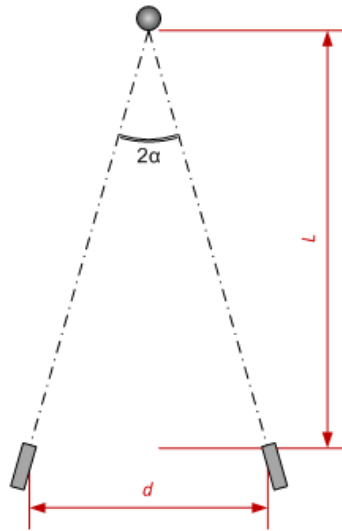
Критерий	Балл
Сделан пояснительный рисунок	2
Приведено аналитическое выражение, отражающее взаимосвязь величины угла конвергенции (половины или полного) от дистанции до наблюдаемого объекта и расстоянием между камерами $\operatorname{tg}(\alpha) = \frac{d}{2L}.$	4
Корректно вычислено значение половины угла конвергенции $\alpha = \arctg\left(\frac{d}{2L}\right).$	2
Приведена формула для определения угла $\Delta(\alpha)$, приходящегося на одно деление шкалы энкодера $\Delta(\alpha) = \frac{2\pi}{3600}.$	3
Корректно вычислено значение угла $\Delta(\alpha)$, соответствующего одному делению шкалы энкодера.	1
Сделаны рассуждения и приведены формулы для определения абсолютного отклонения (разбега возможных оценок в расстоянии) исходя из дискретности в позиционировании угла.	4
Осуществлен расчет «кажущихся» расстояний до препятствия, соответствующих верхней и нижней оценке (исходя из возможных углов позиционирования).	6
Определена абсолютная погрешность в определении расстояния до потенциального препятствия, находящегося в 45 м от беспилотного вездехода.	3
Всего	25

Решение

В момент восприятия объекта, если камеры движутся согласованно, с фиксацией взора на одном объекте, скоординированное движение называют вергентным. Движение, при котором происходит сведение оптических осей, называется конвергенцией, а разведение оптических осей называется дивергенцией.

Чем ближе объект фиксации, тем сильнее поворачиваются оптические оси друг к другу, увеличивая угол конвергенции.

Надежнее всего при таком способе определять расстояние до потенциального препятствия является случай, когда оптические оси сориентированы симметрично. Т. е. когда углы камер, взирающих на препятствие, равны друг другу.



Это как раз наш случай — препятствие расположено непосредственно на пути следования беспилотного вездехода.

При сокращении дистанции до объекта угол конвергенции (2α) увеличится. Зависимость величины α — половины угла конвергенции (точнее, тангенса угла α) от дистанции до объекта (L) можно выразить с помощью выражения:

$$\operatorname{tg}(\alpha) = \frac{d}{2L}, \text{ где}$$

d — расстояние между камерами ($d = 70 \text{ см} = 0,7 \text{ м}$).

Откуда половина угла конвергенции (значение угла между центральной осью и осью одной из камер) можно вычислить по формуле:

$$\alpha = \arctg\left(\frac{d}{2L}\right).$$

Реальная дистанции до объекта (45 м), соответствует значение угла:

$$\alpha = \arctg \frac{0,7 \text{ м}}{2 \cdot 45 \text{ м}} \approx 0,007778 \text{ рад.}$$

Нам известно, что дискретность используемых энкодеров составляет 3600 единиц за один полный оборот (т. е. на каждые 2π радиан приходится 3600 единиц шкалы). Тогда на одно деление шкалы энкодера приходится угловой шаг $\Delta(\alpha)$, равный:

$$\Delta(\alpha) = \frac{2\pi}{3600} = \frac{\pi}{1800} \approx 0,001745 \text{ рад.}$$

Такое разрешение (дискретность) по углу примерно в 4,46 раза меньше, чем текущее точное значение угла, соответствующее дистанции в 45 м. Это означает, что навигационная система, пытаясь повернуть камеры, чтобы максимально точно посмотреть на

объект, сможет выставить угол или на 4 или 5 значений $\Delta\alpha$, что будет соответствовать возможным углам позиционирования: 0,006981 или 0,008727 рад.

Величины «кажущихся» расстояний, видимых под данными углами, можно оценить по формуле:

$$L_i = \frac{\frac{d}{2}}{\operatorname{tg}(\alpha_i)} = \frac{d}{2} \operatorname{ctg}(\alpha_i).$$

В результате углам (0,006981 и 0,008727 рад) в пересчете на дистанцию будут соответствовать значения «кажущихся» расстояний в 50,133 и в 40,106 м, соответственно. Таким образом, абсолютное отклонение (разбег возможных оценок в расстоянии) может достигать чуть более 10 м.

Однако, если же речь заходит об оценке абсолютных погрешностей в определении дистанции, то корректнее будет взять половину разбега по дистанции (половину разности между верхней и нижней оценкой искомой величины). В нашем случае погрешность составит примерно 5 м, т. к. при позиционировании система выберет или положение 4 или положение 5 (какое оно посчитает более близким).

Поэтому для симметричного случая, когда препятствие находится непосредственно на пути следования вездехода и на расстоянии 45 м, ошибка (погрешность) в определении дистанции составит примерно ± 5 м.

В этой задаче об этом не спрашивали, но нужно заметить, что данную оценку мы сделали для симметричного случая, когда наблюдение является оптимальным (то есть с минимальным уровнем ошибки для фиксированной дистанции). Значит, для любого другого случая (когда удаленный на 45 м объект не будет находиться строго на оси следования) значение абсолютной погрешности в определении дистанции до объекта может заметно превысить 5 м. Т. е. найденная нами погрешность ± 5 м — это нижняя оценка возможной погрешности.

Ответ: абсолютная погрешность составит ± 5 м.

Примечание: если в ответе будет указано 10 м, т. е. будет указан разбег между показаниями (разница между верхним и нижним значением кажущейся дистанции), то и такой ответ можно принять как верный.

Задача VI.1.2.5. Любитель автогонок (30 баллов)

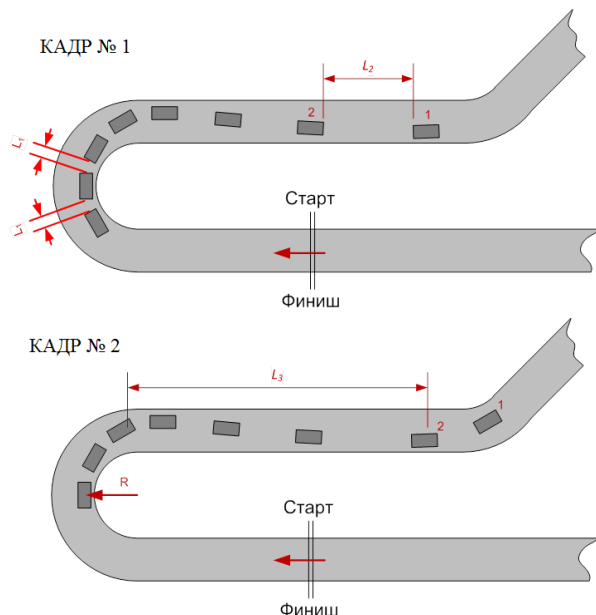
Темы: кинематика, динамика.

Условие

Довольно частым явлением в автогонках является ситуация, при которой автомобили некоторое время двигаются друг за другом «паровозиком», не имея возможности осуществить обгон. В результате «пелотон» — основная группа автомобилей — может долгое время двигаться по гоночному треку как единое целое (без смены позиций).

Посетив один из этапов чемпионата радиоуправляемых машин по кольцевым гонкам, Иван сделал несколько фотографий, опробовав быструю покадровую съемку (10 кадров в с). Придя домой, он внимательно рассмотрел и проанализировал отснятый материал.

На двух соседних кадрах (сделанных с интервалом в 0,1 с) была видна часть пелотона, преодолевающего участок, состоящий из поворота после линии «старт-финиш» и последующего прямого участка, плавно переходящего во второй поворот трека. Также Иван заметил интересную особенность — на втором кадре машина, шедшая второй, попадала ровно в то же самое место, где находилась лидирующая машина на первом кадре.



Иван, глядя на эти фотографии, решил оценить значения развиваемых радиоуправляемыми моделями мощностей. Для этого он сделал распечатку фотографий и, вооружившись линейкой, проделал нехитрые измерения, выразив дистанцию разгона (L_3) в величинах длин моделей авто. Также он измерил минимальное расстояния между машинами в первом повороте (L_1) и максимальную между ними дистанцию (L_2), наблюдаемую на прямом участке, примерно в том месте, где должен был наступать момент начала торможения. Все эти длины (L_1 , L_2 и L_3) он сначала выразил в корпусах машин, а потом, припоминая характерный размер автомобилей, пересчитал эти длины, выразив уже в метрах.

Иван помнил, что одна из машин, участвующая в соревновании, как раз уместилась на двух его ладонях. Следовательно, предположил он, длина должна получиться что-то около 30 см. Про вес моделей у него имелись резонные сомнения, но по его ощущениям, когда он держал ее в руках, она ему показалась столь же тяжелой, как полная литровая пластиковая банка с водой, которую он брал с собой.

После обработки фотографий, и учитывая размер автомобилей, Иван получил следующие значения для L_1 , L_2 и L_3 , выразив их в метрах:

- $L_1 = 0,2$ м;
- $L_2 = 1,2$ м;

- $L_3 = 10$ м.

На глаз определить радиус кривизны Ивану было сложно. Иван понимал, что автомобили, двигаясь по оптимальной траектории, переходили с внешнего радиуса трека на внутренний и затем — с внутреннего на внешний. А ему очень хотелось узнать, каков был радиус траектории в «апексе» — в точке траектории, ближайшей к внутреннему краю дороги, то есть в самой «вершине» поворота.



Классическая гоночная линия

Глядя на две фотографии (кадр №1 и кадр №2), на пометки, сделанные Иваном, ориентируясь на примерный размер и вес автомобилей и опираясь на полученные Иваном значения L_1 , L_2 и L_3 , помогите ему оценить:

1. значение коэффициента трения между покрышками и полотном трека;
2. минимальный радиус кривизны траектории в первом повороте;
3. максимальную мощность, развиваемую радиоуправляемыми машинами на участке разгона.

При расчете примите, что динамика машин при разгоне и при движении в повороте ограничена только сцепными свойствами покрышек с полотном. Все автомобили однотипны и двигаются по одной траектории. Люди, управляющие автомобилями, имеют высшую категорию пилотажа (то есть каждый из них допускает минимум ошибок и поэтому способен раскрыть максимальный потенциал вверенного ему автомобиля). Ускорение свободного падения считать равным 10 м/с^2 .

Критерии оценивания

Критерий	Балл
Из геометрических соображений и предположения о постоянстве интервала по времени между автомобилями при их движении «паровозиком» записано соотношение между максимальной и минимальными скоростями на рассматриваемом участке гоночного трека.	4
Записана формула для оценки максимальной скорости при движении на прямом участке разгона.	3
Вычислено значение максимальной скорости на прямой V_{max} .	1
Получена формула (формулы) для оценки минимальной скорости в повороте V_{min} .	2
Выведено аналитическое выражение для вычисления ускорения на прямом участке.	5
Получена численная оценка ускорения на прямом участке.	1

Критерий	Балл
Получено значение коэффициента трения между покрышками и полотном трека ($\mu = 1$).	3
Приведена формула для определения минимального радиуса кривизны траектории в повороте.	4
Получено численное значение минимального радиуса кривизны траектории в первом повороте.	1
Определена масса автомобиля (1 кг).	1
Записана формула для нахождения мощности автомобиля.	4
Получено верное численное значение мощности.	1
Всего	30

Решение

Частое явление в автогонках — когда автомобили некоторое время двигаются «паровозиком», не имея возможности осуществить обгон. Судя по всему, здесь именно этот случай. Следовательно, интервал движения (по времени) между машинами остается примерно неизменным. Из этого следует, что отношения развиваемых скоростей на отдельных участках прямо пропорциональны отношению расстояний между центрами тяжести машин, следующих друг за другом. Т. е. скорость, достигнутая автомобилем на прямой, относится к скорости автомобиля в повороте как:

$$\frac{V_{\text{на прямой}}}{V_{\text{в повороте}}} = \frac{V_{\text{max}}}{V_{\text{min}}} = \frac{L_2 + 0,3}{L_1 + 0,3}.$$

При этом максимальную скорость, достигнутую на прямой, можно оценить по расстоянию, преодолеваемому за промежуток времени между кадрами (0,1 с):

$$V_{\text{max}} = \frac{\Delta L}{\Delta t} = \frac{L_2 + 0,3 \text{ м}}{0,1 \text{ с}} = \frac{1,2 \text{ м} + 0,3 \text{ м}}{0,1 \text{ с}} = 15 \text{ м/с}.$$

Тогда минимальная скорость (скорость в повороте) можно оценить двумя способами: через пройденный путь за промежуток времени между кадрами (0,1 с):

$$V_{\text{min}} = \frac{\Delta L}{\Delta t} = \frac{L_1 + 0,3 \text{ м}}{0,1 \text{ с}} = \frac{0,2 \text{ м} + 0,3 \text{ м}}{0,1 \text{ с}} = 5 \text{ м/с}.$$

или через ранее найденное соотношение между скоростями в повороте и на прямой:

$$V_{\text{min}} = V_{\text{max}} \frac{L_1 + 0,3 \text{ м}}{L_2 + 0,3 \text{ м}} = 15 \frac{0,2 \text{ м} + 0,3 \text{ м}}{1,2 \text{ м} + 0,3 \text{ м}} = 5 \text{ м/с}.$$

Ускорение на прямом участке можно оценить по изменению скорости на участке разгона ($L_3 = 10\text{м}$). Так как ускорение на прямом участке в ходе разгона ограничено только сцепными свойствами покрышек с полотном, значит, на всем протяжении разгона ускорение (a) оставалось неизменным. Это означает, что сила, с которой автомобиль отталкивался от гоночного полотна при разгоне, была неизменной и равной:

$$F_{\text{тяги}} = \mu mg = ma, \text{ где}$$

μ — коэффициент трения, m — масса автомобиля, g — ускорение свободного падения, a — горизонтальное ускорение автомобиля на прямой разгона.

Эта сила, действующая на дистанции разгона, совершила работу, разогнав автомобиль от минимальной до максимальной скорости. Таким образом, из закона сохранения механической энергии можно выразить значение разгоняющей силы через изменение кинетической энергии автомобиля:

$$F_{\text{тяги}} = \mu mg = ma = \frac{\Delta E_{\text{кин}}}{L_3} = \frac{m(V_{\text{max}}^2 - V_{\text{min}}^2)}{2L_3}.$$

Используя ранее найденные значения скоростей, можно определить ускорение, с которыми двигались модели автомобилей при разгоне:

$$a = \frac{m(V_{\text{max}}^2 - V_{\text{min}}^2)}{2L_3} = \frac{15^2 - 5^2}{2 \cdot 10} = 10 \text{ м/с}^2.$$

Т. к. найденное горизонтальное ускорение получилось равным ускорению свободного падения, следовательно коэффициент трения (μ) равен одному.

При прохождении поворота со скоростью $V_{\text{min}} = 5 \text{ м/с}$ при условии, что коэффициент трения ($\mu = 1$), автомобиль способен двигаться по траектории с минимальным радиусом, величину которого можно оценить, исходя из величины достижимого центростремительного ускорения, которое в свою очередь зависит от сцепных характеристик покрышек с полотном. Т. к. коэффициент трения равен одному, то максимально реализуемое центростремительное ускорение составит $1g$ или 10 м/с^2 .

$$R_{\text{min}} = \frac{V_{\text{min}}^2}{a_{\text{max}}}.$$

В результате минимально возможный радиус траектории движения автомобиля, при скорости в повороте 5 м/с , составит:

$$R_{\text{min}} = \frac{(5 \text{ м})^2}{10 \text{ м/с}^2} = 2,5 \text{ м}.$$

Значение же максимальной полезной мощности (N_{max}), развиваемой электродвигателями гоночных моделей на прямой разгона, можно определить из величины ускорения и достигнутой скорости на этом участке. Вес машины Иван оценил как вес 1 л воды. То есть предположил, что масса автомобиля соответствует примерно 1 кг . Таким образом, полезная мощность в конце прямого участка должна была достичь следующего значения:

$$N_{\text{max}} = F_{\text{тяги}} V_{\text{max}} = \mu mg V_{\text{max}} = 1 \cdot 1 \text{ кг} \cdot 10 \text{ м/с}^2 \cdot 15 \text{ м/с} = 150 \text{ Вт}.$$

Ответ: коэффициент трения = 1, радиус поворота = $2,5 \text{ м}$, максимальная полезная мощность = 150 Вт .

Физика. 10–11 классы

Задача VI.1.3.1. Бабахнет или нет? (15 баллов)

Темы: тепловые явления.

Условие

Для поддержания «жизнедеятельности» некоторых исполнительных систем, входящих в состав разрабатываемого прототипа автономного беспилотного вездехода-спасателя, разработчиками было принято решение использовать пневматическую систему, где в качестве рабочего газа применяется двухатомный азот N_2 (молярная масса химического элемента азота $0,014 \text{ кг/моль}$), основной запас которого (200 г) был размещен в пятилитровом ресивере — специальном металлическом баллоне, предназначенном для хранения рабочих газов.



На тестовых испытаниях было показано, что в температурном диапазоне от $+20$ до $+150 \text{ }^{\circ}\text{C}$ ресивер был способен выдерживать избыточное внутреннее давление до 250 земных атм, а в области низких (ниже $-40 \text{ }^{\circ}\text{C}$) — вплоть до криогенных температур (около 80 K). За счет проявления хладноломкости металлов ресивер мог надежно выдерживать избыточное внутреннее давление только до 100 атм.

Принять, что давление в 1 атм соответствует давлению $0,1 \text{ МПа}$.

С учетом возможного диапазона температурных условий эксплуатации (от $-70 \text{ }^{\circ}\text{C}$ до $+140 \text{ }^{\circ}\text{C}$) необходимо выяснить, обладает ли требуемым запасом прочности материал ресивера, если согласно требованиям технического задания, все элементы конструкции должны обладать как минимум, четырехкратным запасом прочности.

Критерии оценивания

Критерий	Балл
Сформулирована мысль о том, что для установления зависимости между давлением газа, его молярным объемом и абсолютной температурой необходимо воспользоваться уравнением состояния идеального газа (уравнением Менделеева — Клапейрона).	4
Сформулирована мысль о необходимости определения давления при двух температурах, а именно при: $+140 \text{ }^{\circ}\text{C}$ (максимальная) и при температуре $-40 \text{ }^{\circ}\text{C}$ (максимальной из тех, где начинает проявляться хладноломкость металла).	3
Верно определена молярная масса ($0,028 \text{ кг/моль}$) для двухатомного газа N_2 .	1
Верно определено давление в ресивере газом для температуры $+140 \text{ }^{\circ}\text{C}$.	2
Верно определено давление в ресивере для второй температуры (при $-40 \text{ }^{\circ}\text{C}$).	1
Верно определен требуемый запас прочности ресивера для температуры $+140 \text{ }^{\circ}\text{C}$.	2

Критерий	Балл
Верно определен требуемый запас прочности ресивера для температуры $-40\text{ }^{\circ}\text{C}$.	1
Верно и полно сформулирован ответ. (при высоких температурах имеющийся запас прочности ресивера достаточен, но при низких температурах, к сожалению, нет).	1
Всего	15

Решение

Для выяснения, каким запасом прочности обладает ресивер, необходимо понять, какие величины возможного давления ему придется выдерживать при тех или иных температурных условиях эксплуатации. Для этого нужно воспользоваться уравнением состояния идеального газа (уравнением Менделеева – Клапейрона), которое устанавливает зависимость между давлением, молярным объемом, количеством вещества и абсолютной температурой идеального газа.

$$PV = \frac{m}{M}RT, \text{ где}$$

P — давление, V — объем газа, m — масса,

M — молярная масса вещества,

R — универсальная газовая постоянная, $R \approx 8,314 \text{ Дж}/(\text{моль} \cdot \text{K})$,

T — термодинамическая температура, K.

В нашем случае критическими являются две температуры.

Это $+140\text{ }^{\circ}\text{C}$ (максимальная) и температура $-40\text{ }^{\circ}\text{C}$ (максимальная из тех, где начинает проявляться хладноломкость металла). Зная молярную массу элемента азота ($0,014 \text{ кг}/\text{моль}$), можем определить соответствующую молярную массу ($0,028 \text{ кг}/\text{моль}$) для двухатомного газа N_2 как удвоенное значение молярной массы у элемента азота.

Зная общую массу газа ($0,2 \text{ кг}$) и объем ресивера ($5 \text{ л} = 0,005 \text{ м}^3$), произведем расчет давления, которые создает газ при этих температурах (разумеется, предварительно выразив значения температур по абсолютной шкале температур ($T = 273 + t$, $^{\circ}\text{C}$)).

В результате можем определить значения давлений в ресивере при соответствующих температурах.

При $+140\text{ }^{\circ}\text{C}$, давление азота составит:

$$P = \frac{m}{MV}RT = \frac{0,2 \text{ кг}}{0,028 \text{ кг}/\text{моль} \cdot 0,005 \text{ м}^3} 8,314 \text{ Дж}/(\text{моль} \cdot \text{K}) \cdot (273 + 140) \text{ K} \approx 4,9 \text{ МПа}.$$

При $-40\text{ }^{\circ}\text{C}$, давление азота составит:

$$P = \frac{m}{MV}RT = \frac{0,2 \text{ кг}}{0,028 \text{ кг}/\text{моль} \cdot 0,005 \text{ м}^3} 8,314 \text{ Дж}/(\text{моль} \cdot \text{K}) \cdot (273 - 40) \text{ K} \approx 2,77 \text{ МПа}.$$

Учитывая, что избыток давления меньше создаваемых внутри баллона давлений на $0,1 \text{ МПа}$ (с учетом давления снаружи в одну атмосферу), а также учитывая то обстоятельство, что нам требуется четырехкратный запас прочности, то «критические» давления, которые должен будет выдерживать ресивер, заправленный 200 г

азота, при температурах $+140\text{ }^{\circ}\text{C}$ и $-40\text{ }^{\circ}\text{C}$, будут составлять, соответственно:

$$P_{\text{крит}}^{t=+140\text{ }^{\circ}\text{C}} = 4 \cdot (P^{t=+140\text{ }^{\circ}\text{C}} - 0,1 \text{ МПа} \approx 4 \cdot (4,9 - 0,1) \text{ МПа} = 19,2 \text{ МПа} = 192 \text{ атм},$$

$$P_{\text{крит}}^{t=-40\text{ }^{\circ}\text{C}} = 4 \cdot (P^{t=-40\text{ }^{\circ}\text{C}} - 0,1 \text{ МПа} \approx 4 \cdot (2,77 - 0,1) \text{ МПа} = 10,68 \text{ МПа} = 106,8 \text{ атм}.$$

Это означает, что при высоких температурах имеющийся запас прочности ресивера достаточный, т. к. ресивер способен выдержать давление до 250 атм ($250 > 192$), а при низких температурах, к сожалению, нет. При низких температурах ресивер способен был выдержать давление только до 100 атм, а требуется выдержать 106,8 атм.

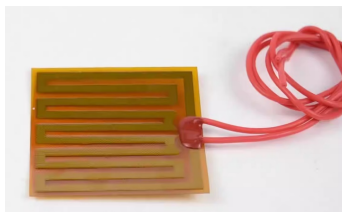
Ответ: нет. Не обладает. При высоких температурах имеющийся запас прочности ресивера достаточен, но при низких температурах, к сожалению, нет.

Задача VI.1.3.2. Вот вам горчичник для согрева (20 баллов)

Темы: электрические явления.

Условие

Нагревательные пластины, также известные как плоские нагреватели, являются полезным и эффективным решением для нагрева поверхностей, например, топливных баков с жидкостями или газами, для которых требуется поддержание определенного уровня температур при той или иной эксплуатации, в том числе в суровых климатических условиях. Нагреватели такого типа обычно представляют собой плоскую пластину, которая закрепляется на поверхности и, как правило, состоит из четырех основных компонентов: нагревательного элемента, изоляционного материала, защитного корпуса и клемм подключения к сети питания.



Нагревательный элемент (чаще всего изготовленный из нихрома — сплава, состоящего из 55–78% никеля, 15–23% хрома с добавками марганца, кремния, железа, алюминия) в виде проволоки или металлической ленты — является тем самым главным рабочим элементом, который нагревается при пропускании через него электрического тока. Он обычно расположен в центре нагревательной пластины и покрыт изоляционным слоем. В качестве изоляторов может использоваться корундовый песок, слюда, силикон и т. д.

В ходе проверки работоспособности нагревательного элемента в виде пластины, предназначенной для обогрева элементов трансмиссии и топливного бака беспилотного аппарата, был проведен небольшой тестовый эксперимент. Предварительно измерив электрическое сопротивление металлического проводника нагревательной пластины ($R = 5 \text{ Ом}$), экспериментаторы подключили пластину к источнику напряжения.

Подаваемое напряжение (U) в первые 3 с плавно и монотонно изменяли (см. таблицу), при этом начальное значение напряжения (в момент времени $t = 0$) составляло 1 В, а в моменты времени $t = 1, 2$ и 3 с напряжение достигло следующих значений: $U(1) = 2$ В, $U(2) = 5$ В и $U(3) = 10$ В, соответственно.

Время, с	Напряжение, В
0	1
1	2
2	5
3	10

На основании имеющейся информации:

1. Постройте графики изменения напряжения и тока во времени и запишите соответствующие уравнения, отражающие характер изменения напряжения и тока во времени, $U = f(t)$ и $I = f(t)$.

Определите:

2. через какое время сгорит нагреватель, если подаваемое напряжение будет продолжать изменяться, согласно установленному вами закону, при том, что критическое значение тока для проводника, используемого в нагревательном элементе, составляет 10 А;
3. какое количество теплоты выделилось на рассматриваемом участке цепи (на нагревательном элементе пластины) за первую секунду эксперимента?

Критерии оценивания

Критерий	Балл
Построен график $U(t)$.	1
Построен график $I(t)$.	2
Составлено уравнение зависимости напряжения во времени $U(t) = t^2 + 1$.	2
Составлено уравнение зависимости тока от времени $I(t) = 0,2 \cdot t^2 + 0,2$.	1
Вычислено время, через которое сгорит нагреватель (будет достигнут критический ток для данного проводника).	3
Указано, что количество теплоты, которое выделилось на нагревательном элементе пластины, определяется согласно закону Джоуля – Ленца.	2
Записано аналитическое выражение для выражения количества теплоты, которое выделяется на нагревательном элементе при указанном изменении тока и напряжения.	5
Получено численное значение количества теплоты, выделившейся за первую секунду.	4
Всего	20

Решение

Зная характер изменения $U(t)$ и зная сопротивление ($R = 5$ Ом), можем построить (в соответствии с законом Ома) график изменения $I(t)$. Он будет почти аналогичен первому графику, с той лишь разницей, что масштаб по оси $Y(I)$ будет в пять раз меньше.

Время, с	Напряжение, В	Ток, А
0	1	0,2
1	2	0,4
2	5	1
3	10	2

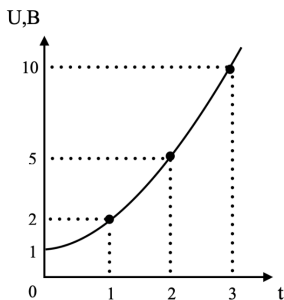


Рис. VI.1.1

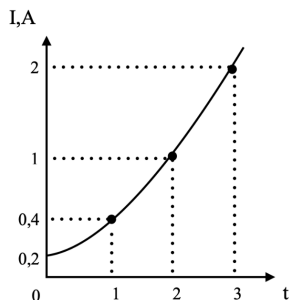


Рис. VI.1.2

Нетрудно догадаться, что уравнения, описывающие изменение напряжения и тока, будут иметь вид, подобный уравнению параболы, смещенной по оси Y (соответственно на 1 В и 0,2 А), а также разный численный коэффициент, стоящий перед t_2 :

$$U(t) = t^2 + 1,$$

$$I(t) = 0,2t^2 + 0,2.$$

Согласно выявленному уравнению, отражающему характер изменения тока во времени $I(t)$:

$$I(t) = \frac{1}{5}(t^2 + 1)$$

мы можем узнать момент времени, когда будет достигнуто критическое значение тока. Для этого нужно решить простое квадратное уравнение вида:

$$\frac{1}{5}(t^2 + 1) = 10.$$

Нетрудно увидеть, что критический ток (10 А) будет достигнут через 7 секунд после начала эксперимента. Действительно:

$$I(t = 7) = 0,2(7^2 + 1) = 10.$$

Согласно закону Джоуля – Ленца, можно вычислить количество теплоты, которое выделилось на нагревательном элементе пластины. При постоянном напряжении и токе, количество тепла определяется из соотношения вида:

$$\Delta Q = UI\Delta t = \frac{U^2}{R}\Delta t.$$

Однако в том случае, если в проводнике со временем меняется значение тока (напряжения), тогда при известном характере изменения напряжения, при постоянном

электросопротивления за время (t) в проводнике будет выделяться следующее количество тепла:

$$Q = \frac{1}{R} \int_0^t U(t)^2 dt.$$

Зная закон изменения напряжения во времени $U(t) = t^2 + 1$, определим количество тепла, выделившегося за первую секунду эксперимента.

Первообразная для функции $(t^2 + 1)^2$ или, что тоже самое, для функции: $t^4 + 2t^2 + 1$ будет функция вида:

$$\frac{t^5}{5} + \frac{2}{3}t^3 + t.$$

На основании этого можно произвести расчет

$$\begin{aligned} Q &= \frac{1}{R} \int_0^1 U^2 dt = \frac{1}{R} \int_0^1 (t^2 + 1)^2 dt = \\ &= \frac{1}{R} \int_0^1 (t^4 + 2t^2 + 1) dt = \frac{1}{R} \left(\frac{t^5}{5} + \frac{2}{3}t^3 + t \right) \Big|_0^1 = \\ &= \frac{1}{5} \left(\frac{1}{5} + \frac{2}{3} + 1 \right) = \frac{1}{5} \cdot \frac{3 + 10 + 15}{15} = \frac{28}{75} \text{ Дж} \approx 0,373 \text{ Дж}. \end{aligned}$$

Ответ: $U(t) = t^2 + 1$, $I(t) = 0, 2t^2 + 0, 2$, $t = 7$ с, $Q \approx 0,373$ Дж.

Задача VI.1.3.3. Ванька-встанька... только уже с крыльями (25 баллов)

Темы: динамика, механические колебания.

Условие

Представьте, что вы или кто-то из ваших коллег предложил внести изменения в схему несущих винтов у коптера — направить оси винтов (как у синхроптера) под углом друг к другу. Но, в отличие от синхроптера, сделать пересечение осей сверху (см. рис. VI.1.3). Такое расположение, по мнению разработчика, позволило бы без сильного ухудшения тяги получить более стабильный режим зависания.

Для подтверждения сделанного заявления была проведена демонстрация — небольшой наглядный эксперимент. Когда коптер завис над полом на высоте 1,5–2 м (позиция №0), коптер вывели из нормального положения слегка ударив по одному из держателей. В результате, коптер дал крен (см. позицию №1). Крен был небольшим, около 10°. Но это привело к тому, что коптер, немного теряя высоту, стал приобретать горизонтальную скорость. По мере своего движения коптер через некоторое время достиг положения «равновесия», пройдя через первоначальное (горизонтальное) положение (позиция №2), однако он продолжил замедленное движение вправо и стал немного клониться в противоположную сторону.

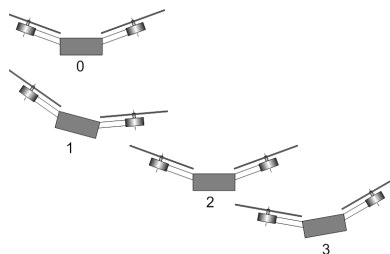
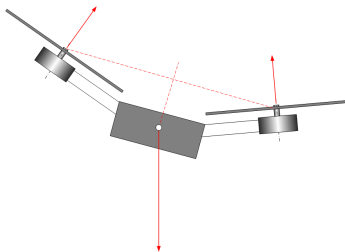


Рис. VI.1.3

Разработчик пояснил, что коптер совершал эти движения в режиме полной автономии, чисто механически, без какого-либо участия системы корректировки положения.

Если вы согласны с разработчиком и с его пояснением, попробуйте примерно оценить время, которое потребуется коптеру (с таким расположением несущих винтов), чтобы самостоятельно (только за счет применяемой в нем геометрии винтов) совершить возвратное движение от максимального крена (позиция №1) к первоначальному горизонтальному положению (позиция №2).



Считать, что ускорение свободного падения составляет 10 м/с^2 . Число винтов у данного коптера — два, как и у синхроптера. Расстояние между местами закрепления винтов — 40 см, а угол между осями винтов составляет 40° . Центр тяжести каждого из двигателей расположен на оси вращения соответствующего винта и при этом смещен на 5 см ниже точки закрепления соответствующего винта. Центр тяжести центральной части коптера находится на 15 см ниже линии, соединяющей точки (места) закрепления винтов. Масса коптера составляет 1 кг, при этом основная часть массы (около 80%) приходится на центральную часть коптера, содержащую батарею. На двигатели приходится около 20% от общей массы коптера. Доля массы, приходящаяся на лопасти и несущие штанги, пренебрежимо мала.

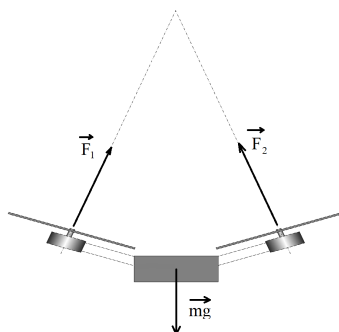
Критерии оценивания

Критерий	Балл
Сделан пояснительный рисунок с указанием действующих на коптер сил.	2

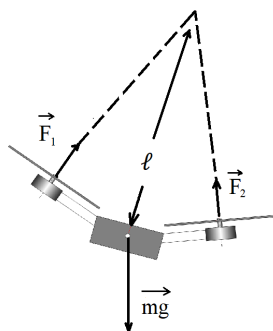
Критерий	Балл
Приведено рассуждение, согласно которому, поведение коптера можно описать как поведение выведенного из равновесия физического маятника. Сформулирована мысль, что в идеализированном приближении время возврата из положения №1 в положение №2 можно оценить как $1/4$ от периода колебания математического маятника, размеры которого можно определить по взаимной ориентировки осей винтов и распределению масс у конструктивных элементов коптера.	5
Записано аналитическое выражение для вычисления времени возврата по аналогии с $1/4$ от периода колебания математического маятника.	2
Сделан расчет, определяющий местоположение центра тяжести, для оценки эффективной длины аналога математического маятника.	5
Сделан расчет, определяющий среднюю (эффективную) удаленность массивных элементов коптера относительно оси вращения (относительно точки пересечения осей винтов).	5
Дана численная оценка времени возврата по аналогии с одной четвертью от периода колебания математического маятника.	3
Озвучены причины, из-за которых время возврата коптера в положение №2 должно отличаться от вычисленного (быть больше).	3
Всего	25

Решение

При такой конфигурации (схеме) в расположении винтов мы получаем следующие направления сил.

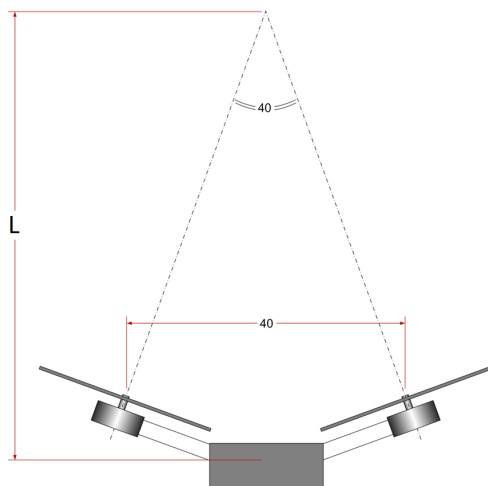


При этом сказано, что система корректировки отключена. Это означает, что при отклонении от нормального (первоначального) положения коптер приобретет возвращающий момент сил. Модуль всех трех сил не изменяется во времени, но их взаимное расположение (точнее, взаимная ориентировка) претерпевает изменения. Сила тяжести всегда направлена вниз, а винты тянут туда, куда направлены оси. Т. к. места крепления винтов лежат выше центра тяжести, то при наклонении, возникает не скомпенсированный момент сил, стремящийся вернуть коптер в первоначальное положение.



В первом приближении это равносильно тому, как если бы данный коптер тянули за нити (с неизменным уровнем натяжения). В результате такая система становится похожей на некий маятник, где вместо груза выступает коптер, центр тяжести которого смещен относительно точек крепления «нитей», а точка пересечения осей винтов выступает в роли точки общего подвеса и, при этом является подвижной. Благодаря этому «движению псевдоподвеса», коптер увлекается в сторону и как бы немного отставая, приобретает эффект раскачки. Такой псевдомаятник нельзя назвать математическим, да и физическим очень условно. Однако в силу отсутствия данных о моменте инерции коптера (он имеет вращательную составляющую движения) у нас нет возможности произвести точный расчет. Но в качестве первого приближения можно принять, что время возврата в первоначальное положение от позиции максимального отклонения является четверть периода колебаний математического маятника.

Из геометрических соображений найдем местоположение центра тяжести коптера относительно точки пересечения осей винтов, т. е. найдем эффективную длину L .



Расстояние (a) от пересечения осей до центров тяжести каждого из двигателей

коптера составляет:

$$a = \frac{20}{\sin 20^\circ} + 5 \approx 63,476 \text{ см.}$$

Расстояние (b) от точки пересечения осей до центра тяжести корпуса коптера составляет:

$$b = \frac{20}{\operatorname{tg} 20^\circ} + 15 \approx 69,95 \text{ см.}$$

Расстояние (a_y) от точки пересечения осей до центров тяжести обоих двигателей коптера найдем через проекцию расстояния a вдоль вертикальной оси:

$$a_y = a \cdot \cos 20^\circ \approx 59,65 \text{ см.}$$

В результате теперь мы можем оценить удаленность центра тяжести всего коптера от пересечения осей, учитывая относительные доли масс и их соответствующую удаленность. Центр тяжести смещен от точки пересечения осей на величину H , значение которой определим по формуле:

$$H = 0,2 \cdot a_y + 0,8 \cdot b \approx 0,2 \cdot 59,65 + 0,8 \cdot 69,95 \approx 67,89 \text{ см.}$$

Эффективная удаленность масс от оси вращения (от точки пересечения осей) будет, по-видимому, немного отличаться от удаленности точки пересечения осей от центра тяжести. Расчет эффективной удаленности (L) дал следующий результат:

$$L = 0,2 \cdot a + 0,8 \cdot b \approx 0,2 \cdot 63,476 + 0,8 \cdot 69,95 \approx 68,655 \text{ см.}$$

Действительно, получилось, что L имеет несколько большее значение, чем H . Это означает, что инерционность коптера будет чуть большей в сравнении с математическим маятником длиной H . Момент от силы тяжести, возвращающей коптер в положение №2, получается чуть меньшим за счет того, что центр тяжести коптера находится немного ближе к точке пересечения осей винтов, чем средняя (эффективная) удаленность масс от оси вращения (от точки пересечения осей винтов).

С другой стороны, эта разница не очень велика (чуть более 1%).

$$\frac{L}{H} \approx \frac{68,655}{67,89} \approx 1,01128.$$

В итоге можно сделать примерный расчет по времени возврата, используя в качестве модельной аналогии время возврата математического маятника (длиной L или H) из крайнего положения к положению равновесия. Этому времени соответствует четверть от периода колебания. Следовательно, искомое время будет равно:

$$t \geq \frac{1}{4}T = \frac{2\pi}{4} \sqrt{\frac{L}{g}} = \frac{\pi}{2} \sqrt{\frac{0,68655 \text{ м}}{10 \text{ м/с}^2}} \approx 0,4116 \text{ с} \approx 0,41 \text{ с.}$$

Ответ: коптеру потребуется не менее 0,41 с.

Примечание: в действительности, на возврат в положение №2 коптеру потребовалось бы больше времени. Можно внести поправку на отличное от 1 соотношение $\frac{L}{H}$. С учетом этого время возврата немного подрастет, достигнув значения, примерно 0,416 с. Кроме этого, возникающий момент сил (от силы тяжести) будет частично расходоваться на обеспечение вращательной составляющей движения корпуса коптера (часть энергии перейдет во вращение коптера относительно центра тяжести).

Коптер, в отличие от математического маятника, имеет определенный размер и пространственное распределение масс, поэтому обладает существенным моментом инерции. Но это выходит за рамки школьного курса физики, поэтому в качестве примерной численной оценки ранее полученный результат (по аналогии с математическим маятником) вполне достаточен.

Задача VI.1.3.4. Жесткое приземление (20 баллов)

Темы: динамика, законы сохранения в механике.

Условие

Амортизаторы — это специальные демпфирующие устройства, способные преобразовывать до 40% механической энергии движения в тепловую при ударных воздействиях. Они применяются для гашения колебаний, поглощения вибраций и уменьшения последствий механического воздействия от сильных толчков или резких ударов по подвижным элементам подвески.

Совместно с с упругими элементами (пружинами, торсионами) амортизаторы эффективного уменьшают амплитуды качания корпуса за счет подавления высоких относительных скоростей вертикального перемещения.



Поддрессоренная снаряженная масса беспилотного аппарата (с пустым грузовым отсеком) составила 40 кг. При проверке хода подвески было выявлено, что в состоянии покоя (при снаряженной массе) клиренс (расстояние от земли до днища) составляет 10 см, а при помещении в грузовой отсек 10 кг песка клиренс уменьшился на 1 см.

Считая, что ускорение свободного падения вблизи поверхности Земли составляет 10 м/с^2 , а доля неподдрессоренной массы ничтожно мала по сравнению с общей массой аппарата, определите максимально допустимую высоту падения аппарата с грузом, если падение заканчивается приземлением на твердую горизонтальную поверхность с одновременным касанием всех четырех колес.

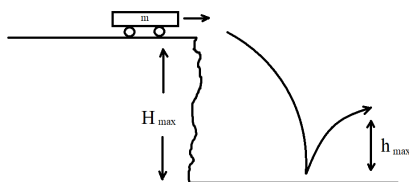
Приземление можно считать успешным, если за счет совместной работы упругих пружин и амортизаторов подвески данный аппарат избежит удара днищем о поверхность.

Критерии оценивания

Критерий	Балл
Сделан пояснительный рисунок.	2
По закону Гука: $F = -kx$ определена жесткость подвески.	3

Критерий	Балл
Определено значение полного хода подвески, которая определяется величиной клиренса в состоянии невесомости (при свободном падении).	5
Вычислен ресурс запасаемой упругой энергии в пружинах подвески при полном ходе подвески.	4
Определена полная энергия удара, которую способна поглотить подвеска за счет работы пружин и амортизаторов (демпферов).	3
Получено значение максимально допустимой высоты, с которой машине позволительно упасть, избежав удара днищем о поверхность.	3
Всего	20

Решение



При падении с высоты H автомобиль приобретает дополнительную кинетическую энергию (в вертикальном направлении) за счет потенциальной.

$$mgH = \frac{mV_y^2}{2}.$$

V_y — вертикальная составляющая скорости, которую необходимо будет погасить с помощью подвески (за счет совместной работы пружин и демпфирующих амортизаторов).

Полагая, что подвеска работает упруго (на малых скоростях движения подвески амортизаторы не оказывают существенного сопротивления движению), согласно Закону Гука ($F = -kx$) можно определить жесткость подвески. Мы знаем, что при проверке хода подвески было выявлено, что в состоянии покоя (при снаряженной массе 40 кг) клиренс автомобиля — расстояние от земли до днища, — составлял 10 см (т. е. 0,1 м), а при загрузке в грузовой отсек дополнительных 10 кг (или 100 Н) клиренс уменьшался на 1 см (т. е. на 0,01 м).

Клиренс, см	m, кг	Вес, Н
10	40	400
9	50	500

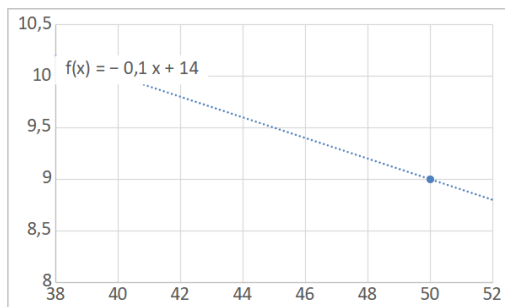
Следовательно:

$$\Delta F = k \cdot \Delta x = \Delta m \cdot g.$$

Откуда жесткость подвески:

$$k = \frac{10 \text{ кг} \cdot 10 \text{ м/с}^2}{0,01 \text{ м}} = 10^4 \text{ Н/м}.$$

При падении с обрыва автомобиль временно находится в состоянии невесомости, поэтому на подвеску автомобиля не оказывается давление. Зная характер изменения клиренса в зависимости от массы (веса) автомобиля, нетрудно установить, что в состоянии невесомости, что равносильно отсутствию подрессоренной массы, полный клиренс или полный ход подвески $\Delta X_{\max}$ должен составлять 14 см (см. график изменения хода подвески в зависимости от массы автомобиля).



Следовательно, полная запасенная энергия в пружинах подвески не может превысить:

$$E_{\text{пруж}} = k \frac{\Delta X_{\max}^2}{2} = 10^4 \text{ Н/м} \frac{(0,14 \text{ м})^2}{2} = 98 \text{ Дж.}$$

По условию задачи сказано, что в подвеске до 40% механической энергии удара могут поглотить амортизаторы (демпферы). Это значит, что на долю пружин от полной энергии удара приходится 60%. Следовательно, предельно допустимая энергия удара по полностью раскрытой подвеске (с пружинами и амортизаторами) не должна превышать:

$$E_{\text{полн. удара}} = \frac{E_{\text{пруж.}}}{0,6} = \frac{98 \text{ Дж}}{0,6} \approx 163,3 \text{ Дж.}$$

Максимальная высота, с которой позволительно упасть машине при массе 50 кг (сама машина +10 кг груза), определяется значением потенциальной энергии, не превышающей допустимую энергию удара.

$$E_{\text{полн. удара}} = mgH_{\max}$$

откуда:

$$H_{\max} = \frac{E_{\text{полн. удара}}}{mg} \approx \frac{163,3 \text{ Дж}}{50 \text{ кг} \cdot 10 \text{ м/с}^2} \approx 0,327 \text{ м} = 32,7 \text{ см.}$$

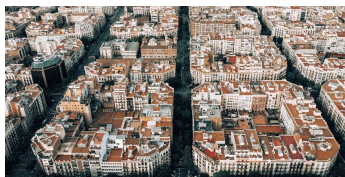
Ответ: максимально допустимая высота падения автомобиля с грузом 32,7 см.

Задача VI.1.3.5. Легко ли заблудиться в трех «соснах»? (20 баллов)

Темы: оптика.

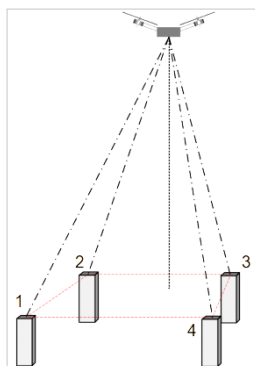
Условие

Небольшой летательный аппарат (беспилотник), специализирующийся на доставке специальной почты, осуществлял полет над городом, имеющим Гипподамову систему планировки (то есть с такой планировкой, где улицы пересекаются под прямыми углами).



Пролетая над одним из жилых кварталов, имеющих прямоугольную форму, беспилотник ориентировался на местности с помощью регистрации оптических сигналов, отраженных от угловых отражателей, расположенных на крышах девятиэтажных зданий, расположенных строго по углам этого прямоугольного квартала.

Беспилотник должен был доставить посылку на крышу здания №4. Однако, регистрируя промежутки времени между испусканием и получением отраженного оптического сигнала, он в какой-то момент перестал получать сигнал от углового отражателя, расположенного именно на том самом здании (на здании №4).



Обратный сигнал успешно вернулся только от трех точек (№1, №2 и №3) спустя время t_1 , t_2 и t_3 . По какой-то причине угловой отражатель №4 перестал срабатывать должным образом. Бортная система сбора и обработки сигналов получила следующие значения временных задержек (выраженных в микросекундах):

- $t_1 = 4,1$ мкс;
- $t_2 = 3,8$ мкс;
- $t_3 = 4,2$ мкс.

Размеры квартала и высота пролета беспилотника над ним неизвестны. Причем они неизвестны ни вам, ни беспилотнику. В распоряжении беспилотника и вас имеются только данные о временах t_1 , t_2 и t_3 . Возможно ли как-то помочь летательному

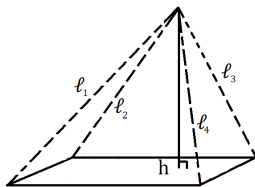
аппарату заполнить «пробел в данных», вычислив расстояние до крыши четвертого здания, ориентируясь только на полученные значения t_1 , t_2 и t_3 ? Если вы считаете, что это возможно, то оцените на каком расстоянии от крыши здания №4 сейчас находится беспилотник.

Критерии оценивания

Критерий	Балл
Сформулирована мысль, что дистанции до источников определяются по времени и скорости распространения сигнала (прямая пропорциональность). Следовательно, задача сводится к нахождению расстояния L_4 через установленные по временам задержек t_1 , t_2 и t_3 , расстояния L_1 , L_2 и L_3 .	2
По имеющимся временам задержек t_1 , t_2 и t_3 , вычислены соответствующие расстояния до зданий L_1 , L_2 и L_3 .	2
Сделаны необходимые пояснительные рисунки.	2
Введена трехмерная система координат.	2
Даны аналитические записи выражения расстояний до крыш соответствующих зданий при помощи трехмерной системы координат (составлена система уравнений).	4
Получено аналитическое решение для вычисления расстояния L_4 через расстояния L_1 , L_2 и L_3 .	6
Получен верный численный ответ для дистанции L_4 .	2
Всего	20

Решение

Оптические сигналы распространяются со скоростью света. Следовательно, расстояния до точек, пропорциональны временам задержек сигналов.



Зная скорость света $c = 3 \cdot 10^8$ м/с и понимая, что суммарный путь сигнала составляет удвоенное расстояние до опорных точек, мы можем вычислить соответствующие расстояния беспилотника до крыш первых трех зданий по соответствующим временам задержек сигнала:

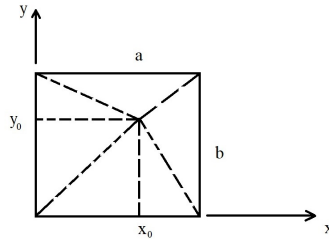
$$L_i = \frac{1}{2} c \cdot t_i.$$

Получаем $L_1 = 615$ м, $L_2 = 570$ м, $L_3 = 630$ м.

Далее введем следующие обозначения:

- пусть дрон сейчас завис на высоте h в точке с координатами: $(x_0; y_0; h)$;

- пусть стороны прямоугольника (описывающего рассматриваемый квартал) равны соответственно a и b .



Тогда по теореме Пифагора (для трехмерного случая) можно выразить расстояния через координаты:

$$\begin{cases} l_1^2 = x_0^2 + y_0^2 + h^2, \\ l_2^2 = x_0^2 + (b - y_0)^2 + h^2, \\ l_3^2 = (a - x_0)^2 + (b - y_0)^2 + h^2, \\ l_4^2 = (a - x_0)^2 + y_0^2 + h^2. \end{cases}$$

Сложив первое с третьим и второе с четвертым, получим систему из двух уравнений.

$$\begin{cases} l_1^2 + l_3^2 = x_0^2 + (a - x_0)^2 + y_0^2 + (b - y_0)^2 + 2h^2, \\ l_2^2 + l_4^2 = x_0^2 + (a - x_0)^2 + y_0^2 + (b - y_0)^2 + 2h^2. \end{cases}$$

После чего нетрудно заметить, что: $l_4^2 = l_1^2 + l_3^2 - l_2^2$.

Следовательно:

$$l_4 = \sqrt{l_1^2 + l_3^2 - l_2^2}.$$

Таким образом, текущая дистанция до крыши четвертого здания равна:

$$l_4 = \sqrt{615^2 + 630^2 - 570^2} \approx 670,98 \text{ м} \approx 671 \text{ м}.$$

В результате текущая дистанция от беспилотника до крыши четвертого здания (L_4) может быть оценена по временам t_1 , t_2 и t_3 , т. е. ориентируясь на другие три здания. Искомое значение составляет примерно 671 м.

Ответ: да. Это возможно. Дистанция составляет 671 м.

Инженерный тур

Общая информация

На заключительном этапе профиля АТС участникам необходимо запустить и отладить городскую автоматизированную транспортную систему для скорейшего обнаружения и устранения чрезвычайных ситуаций. Система состоит из трех программируемых устройств: беспилотного автомобиля, квадрокоптера и сервисного центра. Участники разрабатывают и отлаживают программы, управляющие устройствами автономной транспортной системы.

Сюжет задачи

В городе Н уже появились и используются беспилотные наземные и летающие аппараты. Они осуществляют пассажирские и грузовые перевозки. С течением времени автономная транспортная система города стала настолько отлаженной, что для части беспилотников не хватает задач, и они простаивают. Администрация города хочет максимально использовать этот ресурс — задействовать беспилотники в работе экстренных служб города. Квадрокоптеры должны наблюдать за ситуацией в городе и немедленно сообщать о зафиксированных происшествиях. Беспилотным автомобилям следует как можно быстрее добираться до мест происшествий и устранять их последствия.

Было предпринято несколько попыток реализовать задуманное, но комплексного решения так и не получилось. Ценным результатом предыдущих разработок стали сменные модули оборудования для беспилотных автомобилей. Модули содержат все необходимые инструменты для выполнения одной из следующих задач: тушение пожара, оказание медицинской помощи, устранение прорывов центрального отопления, эвакуация транспортных средств и т. п. Перед выездом к месту происшествия на беспилотник должен быть установлен модуль, соответствующий типу происшествия. Для снятия и установки модулей на наземные беспилотники построили специальный сервисный центр, но не создали для него управляющую программу.

Администрация города устраивает конкурс на разработку автономной транспортной системы для экстренных служб города. Заказываемая система должна быть полностью автономной. Она состоит из беспилотного автомобиля, автоматизированного сервисного центра и квадрокоптера.

В конкурсе участвуют несколько команд разработчиков. Команда, которая к концу конкурса продемонстрирует наиболее совершенный прототип слаженно работающей транспортной системы для экстренных служб, станет победителем. Совершенство системы определяется количеством типов чрезвычайных ситуаций, которые корректно устраняются.

Требования к команде и компетенциям участников

Количество участников в команде: 3–4 человека.

- **Программист беспилотного автомобиля:** базовые знания в электронике; работа с алгоритмами локального позиционирования беспилотных автомобилей. разработка алгоритмов обнаружения и взаимодействия с объектами воссозданной городской среды.
- **Программист квадрокоптера:** знание ROS; работа с компьютерным зрением и нейронными сетями в задачах квадрокоптеров, осуществляющих навигацию над полигоном воссозданной городской среды.
- **Программист сервисного центра:** определение трехмерных координат объектов по изображению с камеры. Работа с алгоритмами детектирования и классификации маркировок на грузах.
- **Капитан команды:** реализация обмена данными между устройствами транспортной системы; оценка сложности и декомпозиция задач, распределение задач среди участников команды, отслеживание дедлайнов.

Оборудование и программное обеспечение

Оборудование

Наименование	Описание
Полигон «АЙКАР Стенд», версия «Город НТО» и макеты зданий для полигона «Город НТО»	Интерактивный полигон городской среды со зданиями, дорожной разметкой и объектами городской среды: пешеходами, дорожными знаками, светофорами. Моделирует город, в котором необходимо разработать и запустить автономную транспортную систему. Доступ к полигону участники получают по расписанию. На полигоне участники отлаживают всю транспортную систему в целом, решают подзадачи беспилотного автомобиля и квадрокоптера.
УМК «АЙКАР»	Программируемая учебная модель беспилотного автомобиля «АЙКАР», испытательный полигон «АЙКАР Стенд», версия «Восьмерка» и объекты городской среды: пешеходы, дорожные знаки. Доступен участникам в течение всего заключительного этапа. Используется для запуска и отладки программ локального позиционирования беспилотного автомобиля.
Квадрокоптер «Пионер Макс»	Доступен участникам в течение всего заключительного этапа. Используется для запуска и отладки программ квадрокоптера.

Наименование	Описание
Сервисный центр	Устанавливается на полигоне «Город НТО». Моделирует здание с системой приема, хранения и выдачи грузов. Оснащен промышленным манипулятором, которым участники управляют программно. Сервисный центр устанавливает и демонтирует с беспилотных автомобилей оборудование, необходимое для устранения чрезвычайных происшествий. Доступ к сортировочному хабу участники получают по расписанию. Используется для запуска и отладки программ распознающих маркировки на грузах и перемещающих грузы в соответствии с их маркировкой.
Объекты чрезвычайных ситуаций	Объекты, моделирующие чрезвычайные происшествия: манекены пешеходов, парогенераторы с подсветкой, парогенераторы в форме труб, сигнальные конусы и знаки аварийной остановки.
Стационарный компьютер для обучения нейронных сетей	Доступен участникам в течение всего заключительного этапа. Используется для обучения нейронных сетей и отладки программ, использующих их.
Ноутбук для инференса нейронных сетей	Доступен участникам в течение всего заключительного этапа. Используется для взаимодействия с программируемыми устройствами на полигоне и отладки алгоритмов компьютерного зрения и инференса нейросетей.
Ноутбук для редактирования программного кода	Доступен участникам в течение всего заключительного этапа. Используется для чтения документации, коммуникации с организаторами, поиска данных в интернете и редактирования программного кода.

Программное обеспечение

Наименование	Описание
Tengine	Механизм для конвертации моделей нейросетей и их высокопроизводительного инференса в встраиваемых устройствах. Используется при квантовании нейросетевого детектора и запуске квантованного нейросетевого детектора на бортовом компьютере беспилотного автомобиля.
Python3	Основной язык для написания алгоритмов компьютерного зрения и работы с нейросетями. Используется для написания программ всех устройств транспортной системы.
OpenCV (библиотека для Python3)	OpenCV — библиотека алгоритмов компьютерного зрения, обработки изображений, численных алгоритмов и инференса нейросетей с открытым кодом. Используется для написания программ всех устройств транспортной системы.
Darknet (фреймворк)	Darknet — фреймворк для обучения нейросетевых детекторов с открытым исходным кодом, написанный на языке C с использованием программно-аппаратной архитектуры параллельных вычислений CUDA. Используется для обучения нейросетевых детекторов.

Наименование	Описание
ROS (операционная система)	ROS — экосистема для программирования роботов, предоставляющая функциональность для распределенной работы. Используется при написании и запуске программ для квадрокоптера.
LabelImg	Приложение для разметки датасетов.
PuTTY	Клиент для различных протоколов удаленного доступа. Используется для подключения к устройствам транспортной системы по SSH.

Описание задачи

Общая информация о командной задаче заключительного этапа

Участникам необходимо разработать и отладить автоматизированную транспортную систему для обнаружения и устранения чрезвычайных происшествий без вмешательства человека. Система состоит из трех устройств: беспилотного автомобиля, автоматизированного сервисного центра и квадрокоптера.

Все перечисленные устройства работают на полигоне воссозданной городской среды с макетами зданий, дорогами, перекрестками, дорожными знаками, пешеходами, светофорами и объектами, моделирующими чрезвычайные ситуации (ЧС).

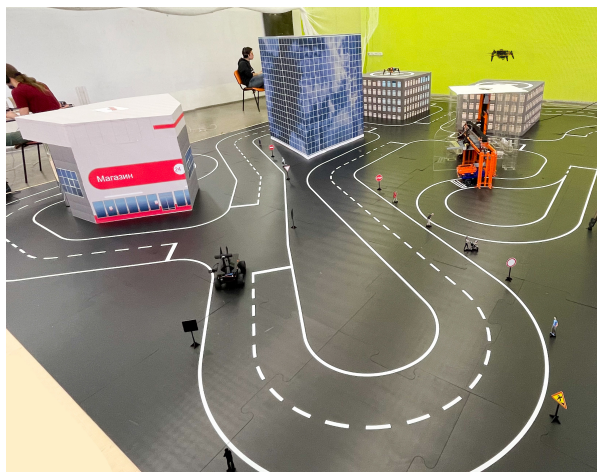


Рис. VI.2.1. Полигон «АЙКАР Стенд», версия «Город НТО»

Для каждого устройства необходимо написать несколько алгоритмов.

Квадрокоптер:

1. Взлет, полет над городом, посадка.

2. Определение по изображению с камеры тип и местоположение ЧС.
3. Передача данных о ЧС беспилотным автомобилям.

Беспилотный автомобиль:

1. Перемещение по дорожной сети города с соблюдением правил из пункта А в пункт Б.
2. Получение данных о ЧС и выбор ближайшего.
3. Остановка для замены модуля оборудования у сервисного центра.
4. Обмен данными с сервисным центром: запросы демонтажа и установки модулей оборудования.
5. Обнаружение на изображении с камеры различных видов ЧС.
6. Построение маршрута до ближайшего ЧС и остановка около него.

Автоматизированный сервисный центр:

1. Обмен данными с беспилотным автомобилем: запросы демонтажа и установки модулей оборудования.
2. Поиск по данным с камеры различных модулей оборудования их захват и перемещение промышленным манипулятором.

Задача разработки транспортной системы разбита на несколько более простых подзадач. В течение трех дней участники решают эти подзадачи и получают за них баллы.

Последний день — день финальных испытаний. На финальных испытаниях участники демонстрируют слаженную работу всех трех устройств. Если транспортная система работоспособна, участники получают баллы.

Материалы, предоставленные участникам на старте заключительного этапа, находятся доступны по ссылке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/materials>.

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions>.

Подзадачи беспилотного автомобиля (БПА)

Подзадача БПА 1. Коммутация электронных модулей беспилотного автомобиля

Условие

Необходимо изучить инструкцию по работе с моделью беспилотного автомобиля «АЙКАР», соединить электронные модули согласно схеме подключения и продемонстрировать работу базового программного кода.

Для выполнения подзадачи необходимо:

- показать разработчику профиля скоммутированные электронные модули и получить разрешение на включение питания модели беспилотного автомобиля;
- запустить на модели беспилотного автомобиля «АЙКАР» базовый программный код и продемонстрировать его работу разработчику профиля.

1 балл — базовый код работает (беспилотник движется по разметке).

Выполнив это подзадание, участники убеждаются в исправности оборудования и работоспособности базового программного кода, а также осваивают умения, необходимые для дальнейшей работы с учебным беспилотным автомобилем: установка аккумуляторов, включение питания систем беспилотника, передача файлов на бортовой компьютер, управление операционной системой бортового компьютера, запуск программ на бортовом компьютере.

Типовые ошибки при выполнении подзадания:

- невнимательное чтение инструкции;
- включение питания без разрешения разработчика профиля.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS0Qg/solutions/БПА1>.

Для решения задачи необходимо безошибочно подключить соединительные провода к электронным модулям беспилотного автомобиля, подключиться к бортовому компьютеру беспилотного автомобиля через SSH, скопировать файлы базового программного кода на бортовой компьютер и запустить их для исполнения. Вышеописанные действия выполняются согласно инструкциям, предоставленным участникам.

Подзадача БПА 2. Остановка в зоне обслуживания сервисного центра

Условие

«АЙКАР» устанавливается на одну из трех стартовых позиций по выбору участников.

По команде разработчика участники запускают программу на беспилотном автомобиле. «АЙКАР» должен преодолеть маршрут до сервисного центра и остановиться в зоне обслуживания.

Подзадача считается выполненной, если:

- при движении через город беспилотный автомобиль не выходил за пределы дорожной разметки более, чем одним колесом;
- после полной остановки беспилотника все четыре колеса находятся в пределах зоны обслуживания;
- подряд проведены два успешных испытания.

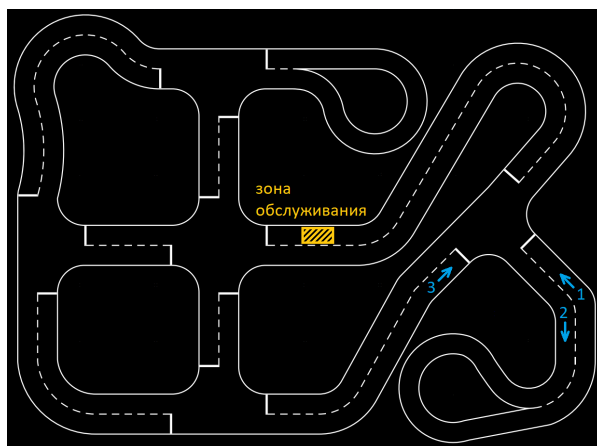


Рис. VI.2.2. Стартовые позиции и зона обслуживания на полигоне

1 балл — при старте из позиции 1.

2 балла — при старте из позиции 2.

2 балла — при старте из позиции 3.

Типовые ошибки при выполнении подзадания:

- отключение алгоритма поиска центра полосы движения при первом появлении в кадре зоны обслуживания;
- реагирование на все стоп-линии, обнаруживаемые на идущих друг за другом кадрах.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПА2>.

Для решения поставленной задачи необходимо разобраться в базовом коде и понять, как происходит переключение между алгоритмам движения по разметке и алгоритмами преодоления перекрестков. При движении по разметке на каждом кадре проводится поиск стоп-линии. При обнаружении стоп-линии происходит смена алгоритма на преодоление перекрестка — изменяется переменная **STATE**. Обратное переключение происходит при стабильном обнаружении левой и правой линий разметки. В зависимости от стартового положения необходимо указать верный вариант преодоления перекрестка.



Рис. VI.2.3. Зона обслуживания на изображениях с камер беспилотного автомобиля и СЦ

Вторая часть задания связана с остановкой в зоне обслуживания. Вариантов решения несколько. Можно было вести отсчет пунктирных линий левой линии дорожной разметки и останавливаться при обнаружении n -й линии. Встроенный энкодер позволял измерять пройденную автомобилем дистанцию, что также позволяло остановиться в нужном месте.

Ниже представлен третий вариант решения, на каждом кадре проводится подсчет желтых пикселей. Остановка производится только после того, как число желтых пикселей на кадре становится ниже определенного порога и меньше, чем на предыдущем кадре, т. е. беспилотник заехал в зону обслуживания и приблизился к ее краю.

Участок кода, представленного ниже, следует разместить в основном цикле чтения кадров. Он позволяет определить момент, когда нужно остановиться в зоне обслуживания.

```
# получаем интересный нас участок кадра
orig_frame = frame.copy()
frame = cv2.resize(frame, SIZE)
mark_frame = frame[150:200, 216:316]

# проводим бинаризацию, белыми будут только желтые пиксели
```

```

hsv_frame = cv2.cvtColor(mark_frame, cv2.COLOR_BGR2HSV_FULL)
bin_frame = cv2.inRange(hsv_frame, (28, 119, 146), (43, 255, 255))
# подсчитываем желтые пиксели
yellow = np.count_nonzero(bin_frame)
#print(f"Current yellow: {yellow}")

# если пикселей стало больше 600, значит в кадре есть участок зоны обслуживания
if yellow > 600 and not yellow_flag:
    #print("Found yellow > 2000")
    yellow_flag = True
# если пикселей становится меньше 600, а на предыдущих кадрах было больше, то
# зона обслуживания более не занимает весь кадр,
# значит беспилотник приближается к ее краю и нужно остановиться
if yellow_flag and yellow < 600:
    #print("Yellow end")
    STATE = STOP

```

Следующий фрагмент решения показывает, как выполняется переключение алгоритмов движения по разметке и преодоления перекрестков.

```

# если мы движемся по разметке и обнаружили стоп линию, то
# переключаемся на алгоритм преодоления перекрестка по прямой.
# рассчитанное на предыдущем этапе число желтых пикселей сравнивается с
# порогом и не позволяет распознать зону обслуживания как стоп линию.
if STATE == GO and detect_stop2(wrapped) and yellow < 100:
    count += 1
    print(f"Count: {count}")
    START_ACTION = False
    #STATE = ON_CROSS
    #STATE = random.choice([CROSS_RIGHT, CROSS_STRAIGHT, CROSS_LEFT])
    STATE = CROSS_STRAIGHT

# если мы проезжаем перекресток прямо и обнаруживаем линии разметки, то
# переключаемся на алгоритм движения по разметке.

if STATE == CROSS_STRAIGHT:
    if detect_return_road(wrapped, find_lines.left_side_amount,
        ↪ find_lines.right_side_amount)
    and not detect_stop2(wrapped):
        STATE = GO

```

Подзадача БПА 3. Обмен данными с сервисным центром

Условие

По команде разработчика участники запускают программу-имитатор сервисного центра на любом компьютере команды и программу на беспилотном автомобиле. «АЙКАР» должен отправлять сообщения и получать на них ответ. Последовательность сообщений следующая:

1. сообщение о прибытии в зону обслуживания;
2. запрос на демонтаж оборудования;
3. запрос на установку оборудования конкретного типа;
4. сообщение об освобождении зоны обслуживания.

Вы можете пропустить сообщения — отправлять не все, но изменять их порядок нельзя.

Подзадача считается выполненной, если:

- в терминал SSH соединения с автомобилем выводятся ответы от имитатора сервисного центра;
- в терминал компьютера с запущенным имитатором выводятся сообщения от беспилотника;
- разработчик просмотрел код запущенных программ и признал его рабочим.

1 балл за каждый тип верно обрабатываемых сообщений.

Типовые ошибки при выполнении подзадания:

- организация общения двух устройств через посредника;
- неверно определенные и указанные в коде IP-адреса устройств;
- отсутствие декодирования сообщения при его получении;
- отсутствующая обработка символа конца строки.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПА3>.

Участникам необходимо написать программу, отправляющую и принимающую сообщения от другой программы. Интерфейс обмена данными между имитатором-сервисного центра и беспилотником участники выбирают самостоятельно. Далее представлено решение, в котором используется модуль.

Socket Python.
import socket

```
HOST = "172.16.65.69" # ip сервера
PORT = 2002 # порт сервера
# устанавливаем соединение - подключаемся к сокету
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as soc:
    soc.connect((HOST, PORT))

# отправляем запросы, для каждого из них получаем и выводим ответ
for command in range(4):
    if command == 0:
        soc.sendall("arrived".encode())
        # заготовка для обработки и анализа полученного ответа
        # msg = soc.recv(1024).decode()
        # if msg == "...":
        #     pass
        # elif msg == "... ..":
        print(f"Получено: {soc.recv(1024).decode()}")
    if command == 1:
        soc.sendall("dismantling request".encode())
        print(f"Получено: {soc.recv(1024).decode()}")
    if command == 2:
        soc.sendall("equipment request".encode())
        print(f"Получено: {soc.recv(1024).decode()}")
    if command == 3:
        soc.sendall("leaving the service area".encode())
        print(f"Получено: {soc.recv(1024).decode()}")

    soc.sendall("stop".encode())
```

```
break
print("Соединение закрыто")
```

Подзадача БПА 4. Запрос данных о ЧС и определение ближайшей

Условие

По команде разработчика участники запускают программу-имитатор информационного центра на любом компьютере команды и программу на беспилотном автомобиле. «АЙКАР» должен запросить и напечатать в терминал список ЧС, а также вывести последовательность маневров на перекрестках для достижения ближайшей ЧС, например, `forward`, `left`, `right`.

Координаты ЧС указаны в формате X, Y . X и Y соответствуют координатам квадрокоптера, расположенного над ЧС, в его системе отсчета. Происшествия могут произойти в зонах, представленных на следующем изображении.

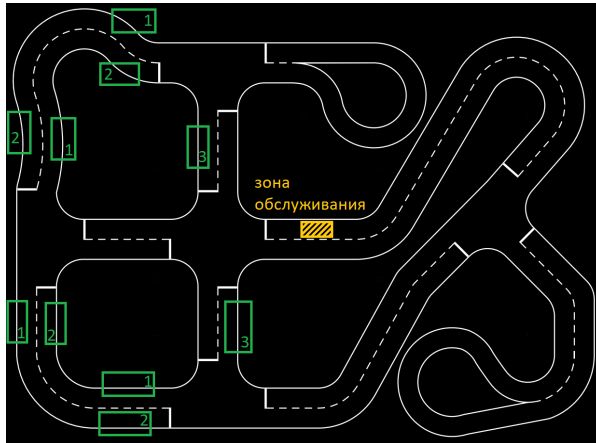


Рис. VI.2.4. Зоны возможных ЧС

Число внутри зоны обозначает ее удаленность от зоны обслуживания. В списке ЧС, который получит беспилотник, ближайшим будет только одно!

Подзадача считается полностью выполненной, если:

- в терминал SSH соединения с автомобилем выводится список ЧС, тип ближайшего из них и список маневров до него;
- испытание пройдено успешно два раза подряд;
- разработчик просмотрел код запущенных программ и признал его рабочим.

1 балл — вывод списка ЧС.

2 балла — вывод типа ближайшей ЧС.

3 балла — вывод последовательности маневров для достижения ближайшей ЧС.

Типовые ошибки при выполнении подзадания:

- неверное понимание удаленности в рамках условия задачи;
- потери времени на создание алгоритма поиска пути, вместо заранее указанных в коде последовательностей;
- неверно определенные и указанные в коде IP-адреса устройств;
- отсутствие декодирования сообщения при его получении;
- отсутствующая обработка символа конца строки.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПА4>.

Для решения задания необходимо выполнить следующие действия:

1. принять и извлечь из сообщения от программы-имитатора названия происшествий и их координаты;
2. по координатам определить ближайшее происшествие;
3. определить последовательность поворотов для достижения ближайшего происшествия.

Ниже представлен участок кода, отвечающий за прием сообщений и извлечение из них данных.

```
import socket

HOST = "172.16.65.69" # ip сервера
PORT = 5000 # порт сервера

# устанавливаем соединение - подключаемся к сокету
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as soc:
    soc.connect((HOST, PORT))
    print("Запрос точек")
    soc.sendall("request".encode())

# прием строки с списком происшествий
data = soc.recv(1024).decode()
print("Список ЧС:")
ESs = data.split(";") # ";" - отделяет одно происшествие от другого
ESss = []
for ES in ESs: # перебираем строки с данными о единственном происшествии
    if ES == ' ': # если строка пустая, переходим к следующей
        continue
    type, x, y = ES.split(':') # ":" - отделяет друг от друга данные о ЧС
    x, y = float(x), float(y) # переводим текст в вещественное число
```

Для каждого происшествия имеем его тип и координаты. Необходимо сопоставить координаты и области, отмеченные на изображении в условии задачи. Для этого необходимо использовать квадрокоптер. Помещать его в зоны ЧС и фиксировать координаты, выдаваемые системой навигации. Результаты измерений можно зафиксировать в коде или отдельном файле. Ниже представлен json-файл с номерами областей, координатами их центров и списком маневров.

```
{
  "1": [[2.3, 3.5], 3, ["straight", "right", "right"]],
  "2": [[1.55, 4.5], 2, ["straight", "right"]],
```

```

"3":[[1.1, 5.3], 1, ["right", "left_t"]],
"4":[[0.46, 3.5], 1, ["straight", "right"]],
"5":[[0.1, 3.5], 2, ["right", "left_t"]],
"6":[[0.47, 0.9], 2, ["left", "right"]],
"7":[[0.05, 1], 1, ["straight", "left_t"]],
"8":[[1.2, 0.14], 1, ["left", "right"]],
"9":[[1.4, 0.05], 2, ["straight", "left_t"]],
"10":[[3, 1], 3, ["straight", "left_t", "left_t"]]
}

```

В программе данные из этого файла извлекаются следующим образом.

```

from json import loads

with open('data.json', 'r') as f:
    coords = loads(f.read())

```

Далее необходимо сопоставить полученные от имитатора координаты с областями из json-файла.

Для каждого набора `type`, `x`, `y` выполняется следующий код.

```

min_d = 1e5
remoteness = -1
way = []
n = 0

# перебираем координаты областей из json-файла
# находим ближайшую область и сохраняем путь до нее и удаленность
for coord in coords:
    # находим евклидово расстояние до области
    d = ((coords[coord][0][0] - x)**2 + (coords[coord][0][1] - y)**2)**0.5
    if min_d > d:
        min_d = d
        remoteness, way = coords[coord][1:]
        n = coord

# добавляем в список данные о происшествии в области
ESss.append((type, remoteness, way, n))

```

Остается отсортировать области по удаленности и вывести требуемую информацию о ближайшей ЧС.

```

ESss.sort(key= lambda x: x[1]) # сортируем список по удаленности
nearest = ESss[0] # сохраняем данные о ближайшем ЧС
print("Ближняя ЧС:", nearest[0], nearest[3])
print("Путь: ", end='')
[print(node, end=' ') for node in nearest[2]]
print()

```

Подзадача БПА 5. Обнаружение конкретных типов ЧС

Условие

«АЙКАР» устанавливается на трассу. Участники решают, каких ЧС не будет вдоль маршрута. Разработчик сообщает участникам тип ЧС, у которой необходимо остановиться, и расставляет модели ЧС по трассе. По команде разработчика участники запускают программу на беспилотном автомобиле.

Подзадача считается выполненной, если:

- беспилотный автомобиль доехал и остановился у ЧС того типа, который указал разработчик. Расстояние от рулевых колес до модели ЧС должно быть меньше 15 см;
- при движении по трассе беспилотный автомобиль не выходил за пределы дорожной разметки более чем одним колесом;
- испытание пройдено успешно два раза подряд;
- разработчик просмотрел код запущенных программ и признал его рабочим.

12 баллов — вдоль трассы установлены модели ЧС четырех типов.

9 баллов — вдоль трассы установлены модели ЧС трех типов.

6 баллов — вдоль трассы установлены модели ЧС двух типов.

3 баллов — вдоль трассы установлены модели ЧС одного типа.

Типовые ошибки при выполнении подзадания:

- реагирование на единичные ложно-положительные срабатывания детектора для сложных объектов;
- остановка беспилотного автомобиля слишком далеко от объекта.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПА5>.

Для решения этой задачи участникам необходимо разработать алгоритм детектирования ЧС разных типов на изображении. Объекты моделирующие ЧС представлены ниже.



Рис. VI.2.5. Объекты, моделирующие ЧС

Объекты, моделирующие ЧС, имеют изменяющуюся форму и пересекающиеся между собой цвета. Исправные и поврежденные трубы отличаются друг от друга столбом пара и наличием черной «пробоины», исправные трубы не являются ЧС. Пешеходы, которым стало плохо, одеты в разную одежду и меняют позы. Необходимо стабильно детектировать объекты при разных уровнях освещенности, ракурсах и фонах. Для решения поставленной задачи уместно применить нейросетевой детектор. Возможно использовать алгоритмы без машинного обучения, но они будут работать менее точно и требоватькратно больше времени для отладки.

Для решения задания необходимо собрать датасет и разметить датасет со всеми видами чрезвычайных происшествий. Необходимо фотографировать объекты с разнообразных ракурсов, под которыми беспилотный автомобиль может их увидеть. Если мы хотим детектировать объекты на одной конкретной дистанции, то в датасет добавляем только фотографии с объектами на этом конкретном удалении от камеры, в настройках обучения детектора нужно отключить масштабирование при аугментации данных.

После обучения нейросетевого детектора его необходимо квантовать для запуска на бортовом компьютере беспилотного автомобиля.

Ниже приведен код, использующий детектор для обнаружения конкретного вида ЧС и остановки около него. Обратите внимание на методику, используемую участниками для борьбы с ложными срабатываниями детектора.

Перед циклом чтения кадров выполняются следующие инструкции.

```
# загрузка модели детектора из файла
model_file = 'yolov4-tiny-obj_best_ats_fin_2024_uint8_v2.tmfile'
model = yolopy.Model(model_file, use_uint8=True, use_timvx=True, cls_num=6)
print("Model loaded")

# список названий классов детектора
# конус, возгорание, поврежденная труба,
# исправная труба, знак "авария", пешеходу плохо
classes_list = ["cone", "fire", "flood", "no_flood", "triangl", "victim"]

# получаем от пользователя тип ЧС, возле которого необходимо остановиться
needed_es = input()

# счетчик детектированных труб с "пробоиной"
pipe_count = 0

# словарь для соответствия типа ЧС и классов детектора.
es = {"crush": ["cone", "triangl"],
      "fire" : ["fire"],
      "injury" : ["victim"],
      "pipe" : ["flood"]}
```

В цикле чтения кадров последовательно выполняются две операции: поиск ЧС и поиск линий дорожной разметки. Далее представлен участок кода, детектирующий ЧС.

```
# frame - изображение с камеры
# передаем кадр детектору
classes, scores, boxes = model.detect(frame)

obj = [classes_list[i] for i in classes] # формируем список из детектированных
↳ классов
print(obj)

# перебираем список обнаруженных объектов
for i in obj:
    # если класс совпадает с тем, около которого нужно остановиться, то
    if i in es[needed_es]:
        # запоминаем индекс объекта в списке обнаруженных объектов
        ind = classes_list.index(i)
        # извлекаем параметры ограничивающей объект рамки
```



```

x, y, w, h = boxes[classes.index(ind)]
if i == 'fire':
    # для возгорания, находим число оранжевых пикселей в ограничивающей рамке
    cropped = frame[y:y + h, x:x + w]
    hsv_cropped = cv2.cvtColor(cropped, cv2.COLOR_BGR2HSV_FULL)
    bin_cropped = cv2.inRange(hsv_cropped, (11, 78, 176), (39, 255, 255))
    orange = np.count_nonzero(bin_cropped)
    # если пикселей недостаточно, то игнорируем объект.
    if orange < 500:
        continue
    # эта проверка понадобилась из-за ложных срабатываний детектора,
    # он реагировал на выключенный испаритель без "огня".
    # для испарителя с огнем orange >= 500.

    # считаем число детектирований поломанных труб за пол секунды
    if pipe_count < 18:
        print(pipe_count)
        pipe_stamp = time.perf_counter()
        pipe_count += 1
        continue
    if time.perf_counter() - pipe_stamp > 0.5:
        pipe_count = 0
    # объект игнорируется если он детектируется меньше 18 раз за пол секунды.
    # эта проверка позволяет пропускать ситуации, в которых труба без
    ↪ повреждений
    # ошибочно детектируется как поврежденная

# print(y)
# чем ближе объект, тем выше его Y координата в кадре
# проверяем как далеко от нас объект. Если объект рядом - останавливаемся
if y > 125:
    STATE = STOP

```

Подзадача БПА 6. Проезд маршрута до точки происшествия

Условие

«АЙКАР» устанавливается на трассу с перекрестками. По команде разработчика участники запускают программу-имитатор информационного центра на любом компьютере команды и программу на беспилотном автомобиле. «АЙКАР» должен запросить список ЧС и ехать по трассе, пока не совершит все маневры, необходимые для достижения ближайшей ЧС.

Списки ЧС для имитатора участники получают от организаторов. В списках указаны декартовы координаты ЧС в системе отсчета квадрокоптера. Участникам необходимо самостоятельно перевести координаты в «удаленность» из БПА 4.

Подзадача считается полностью выполненной, если:

- «АЙКАР» преодолел необходимое количество перекрестков и правильно повернул на каждом из них;
- при движении по трассе беспилотный автомобиль не выходил за пределы дорожной разметки и на встречную полосу более чем одним колесом;
- испытание пройдено успешно два раза подряд;
- разработчик просмотрел код запущенных программ и признал его рабочим.

6 баллов.

Типовые ошибки при выполнении подзадания:

- неверное понимание удаленности в рамках условия задачи;
- неверно определенные и указанные в коде IP-адреса устройств;
- отсутствие декодирования сообщения при его получении;
- отсутствующая обработка символа конца строки.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS0Qg/solutions/БПА6>.

Для решения этой подзадачи участникам необходимо добавить к базовому коду обмен сообщениями и определение маршрута до ближайшего ЧС из задания БПА 4. Организовать очередь маневров, которые необходимо совершить.

Ниже представлен фрагмент кода, в котором реализованы подсчет совершенных маневров и переключение между ними.

```
# до цикла обработки кадров мы определяем список маневров,
# которые необходимо совершить. Определяем способом, изложенным в БПА 4.
way = nearest[2]
```

Внутри цикла обработки кадров отслеживается и изменяется значение перемен-ной STATE. В зависимости от ее значения включается тот или иной алгоритм пере-мещения. Далее представлен код, выполняющийся в теле цикла обработки кадров.

```
# - НАЧАЛО ПОВОРОТА НАПРАВО --- #
if STATE == CROSS_RIGHT:
    if not START_ACTION and not find_lines.left_found:
        START_ACTION = True

if STATE == CROSS_RIGHT and START_ACTION:
    # устанавливаем виртуальную левую линию разметки
    left = int(right - wrapped.shape[1] * 0.6)

    if detect_return_road(wrapped, find_lines.left_side_amount,
        ↪ find_lines.right_side_amount):
        # Конец поворота направо в момент,
        # когда мы видим правую и левую линию разметки.
        # Переход в состояние движения по разметке (дорога без перекрестков)
        STATE = GO
# --- КОНЕЦ ПОВОРОТА НАПРАВО --- #

# - НАЧАЛО ПРЕОДОЛЕНИЕ ПЕРЕКРЕСТКА ПО ПРЯМОЙ --- #
if STATE == CROSS_STRAIGHT:
    if not START_ACTION:
        START_ACTION = True
        SUBSTATE = 0

if STATE == CROSS_STRAIGHT and START_ACTION:
    """пропустим строки кода, отвечающие за навигацию на перекрестке
    ....."""

    if detect_return_road(wrapped, find_lines.left_side_amount,
        ↪ find_lines.right_side_amount)
    and not detect_stop2(wrapped):
```

```

        # Конец поворота направо в момент,
        # когда мы видим правую и левую линию разметки.
        # Переход в состояние движения по разметке (дорога без перекрестков)
        STATE = GO
# - КОНЕЦ ПРЕОДОЛЕНИЯ ПЕРЕКРЕСТКА ПО ПРЯМОЙ --- #

# - НАЧАЛО ПОВОРОТА НАЛЕВО--- #
if STATE == CROSS_LEFT:
    STATE = _CROSS_LEFT_STRAIGHT
    # В первой стадии движемся по прямой полметра, по показаниям энкодера
    meters = 0.55
    arduino.dist(int(DIST_METER*meters))

if STATE == _CROSS_LEFT_STRAIGHT_AGAIN:
    # В третьей стадии движемся на самые темные столбцы до обнаружения
    # левой и правой линий разметки
    if detect_return_road(wrapped, find_lines.left_side_amount,
        ↪ find_lines.right_side_amount)
    and not detect_stop2(wrapped):
        STATE = GO
    """пропустим строки кода, отвечающие за навигацию на перекрестке
    ....."""

if STATE == _CROSS_LEFT_LEFT:
    # Во второй стадии поворачиваем налево по показаниям энкодера
    arduino.check()
    if arduino.waiting():
        arduino_status = arduino.read_data()
        if 'end' in arduino_status:
            # как только дистанция по энкодеру пройдена,
            # переключаемся в следующую стадию
            STATE = _CROSS_LEFT_STRAIGHT_AGAIN

if STATE == _CROSS_LEFT_STRAIGHT:
    # В первой стадии проходим полметра по показаниям энкодера, рулим на
    # самые темные столбцы (центр противоположной полосы дорожного движения)
    """пропустим строки кода, отвечающие за навигацию на перекрестке
    ....."""

    if arduino.waiting():
        arduino_status = arduino.read_data()
        if 'end' in arduino_status:
            # как только дистанция по энкодеру пройдена,
            # переключаемся в следующую стадию
            STATE = _CROSS_LEFT_LEFT
            meters = 0.625
            arduino.dist(int(DIST_METER*meters))

if STATE == _CROSS_LEFT_LEFT:
    angle = 120
# - КОНЕЦ ПОВОРОТА НАЛЕВО--- #

# - ДВИЖЕНИЕ ПО ДОРОГЕ БЕЗ ПЕРЕКРЕСТКОВ--- #
# Если найдена стоп линия, то мы переключаем состояние на тот маневр,
# который указывает счетчик выполненных маневров "count"
if STATE == GO and detect_stop2(wrapped) and time.time() - stamp > 5:
    stamp = time.time()

```

```
STATE = way[count]
count += 1 # увеличиваем счетчик
print(f"Count: {count}")
START_ACTION = False
```

Подзадача БПА 7. Все вместе

Условие

«АЙКАР» устанавливается в стартовое положение 1 (см. подзадачу БПА 2). Участники сообщают, какие пять зон ЧП исключаются из подзадачи. По команде разработчика участники запускают программы-имитаторы на любом компьютере команды и программу на беспилотном автомобиле. «АЙКАР» должен получить список ЧС, выбрать ближайшую, доехать до зоны обслуживания, запросить необходимое оборудование, доехать до выбранного ранее ЧС и остановиться.

Подзадача считается полностью выполненной, если:

- при движении по трассе беспилотный автомобиль не выходил за пределы дорожной разметки и на встречную полосу более, чем одним колесом;
- испытание пройдено успешно два раза подряд;
- разработчик просмотрел код запущенных программ и признал его рабочим.

10 баллов.

Типовые ошибки при выполнении подзадания:

- невнимательное чтение инструкции;
- включение питания без разрешения разработчика профиля.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПА7>.

В решении необходимо объединить разработанные на предыдущих этапах алгоритмы и дописать алгоритм преодоления Т-образных перекрестков.

Ниже представлен участок кода, отвечающий за выполнение поворота налево на Т-образных перекрестках.

```
# --- НАЧАЛО Т-ПОВОРОТА НАЛЕВО --- #
if STATE == CROSS_LEFT_T:
    # В первой стадии движемся по прямой полметра, по показаниям энкодера
    STATE = _CROSS_LEFT_T_STRAIGHT
    meters = 0.6
    arduino.dist(int(DIST_METER * meters))

if STATE == _CROSS_LEFT_T_STRAIGHT:
    # Проверяем не проехали-ли мы отмерянную дистанцию
    arduino.check()
    if arduino.waiting():
        arduino_status = arduino.read_data()
        if 'end' in arduino_status:
            # Если дистанция пройдена - переключаемся во второе состояние
            # поворачиваем колеса налево и движемся 0,4 м по показаниям энкодера
```

```

        meters = 0.4
        arduino.dist(int(DIST_METER * meters))
        STATE = _CROSS_LEFT_T_LEFT

if STATE == _CROSS_LEFT_T_LEFT:
    arduino.check()
    if arduino.waiting():
        arduino_status = arduino.read_data()
        if 'end' in arduino_status:
            # Если дистанция пройдена - переключаемся в третье состояние
            # Опять едем по прямой 0,2 метра
            meters = 0.2
            arduino.dist(int(DIST_METER * meters))
            STATE = _CROSS_LEFT_T_STRAIGHT_AGAIN

if STATE == _CROSS_LEFT_T_STRAIGHT_AGAIN:
    arduino.check()
    if arduino.waiting():
        arduino_status = arduino.read_data()
        if 'end' in arduino_status:
            # Дистанция пройдена - переключаемся в движение по дороге без перекрестков
            STATE = GO
# --- КОНЕЦ Т-ПОВОРОТА НАЛЕВО --- #

```

Подзадачи сервисного центра (СЦ)

Захват манипулятора может перемещаться только в разрешенной зоне. На высоте манипулятора 160 разрешенная зона ограничена точками с следующими координатами, указанными на рисунке.

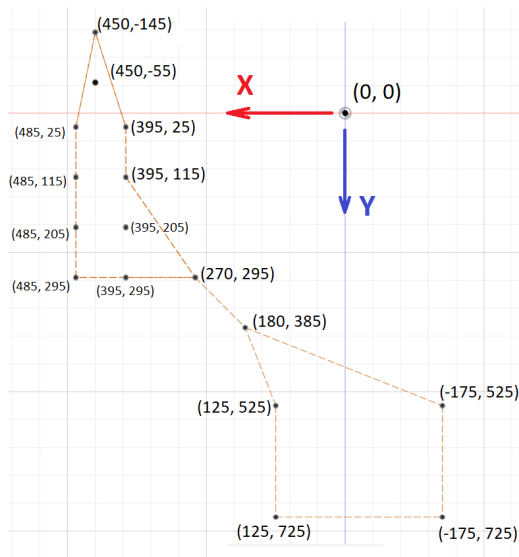


Рис. VI.2.6. Координаты точек в разрешенной зоне манипулятора

Манипулятор не может перемещаться за пределами разрешенной области. Если потребовать перемещения из точки $(395, 25)$ в точку $(270, 295)$, то манипулятор не выполнит команду! Прямой маршрут между этими точками лежит за пределами разрешенной зоны.

На любые перемещения манипулятору требуется время. Если вы введете команду до того, как манипулятор закончит перемещение по предыдущей команде, то команда будет проигнорирована. Самый простой вариант избежать этого — выделить на каждое действие некоторое стандартное количество времени.

Высота 160 подходит для перемещения грузов между зоной хранения и беспилотником.

Высота 65 подходит для захвата грузов, находящихся на хранении. На высоте 65 разрешенная зона меньше. В нее не входят пять точек с схемы выше.

Ниже представлено изображение разрешенных зон для высоты 65 и 160.

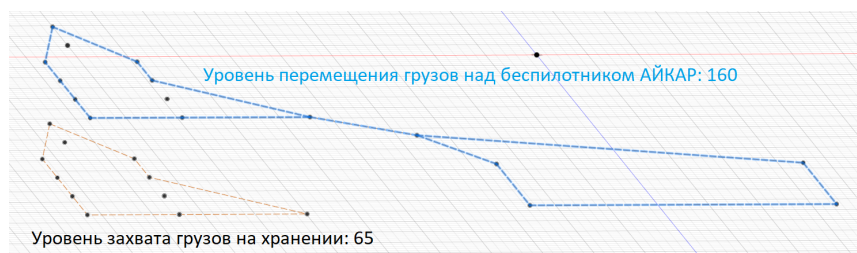


Рис. VI.2.7. Уровни перемещения в разрешенной зоне манипулятора

Грузы всегда располагаются так, что вам не нужно поворачивать захват манипулятора. Т. е. стороны грузов параллельны осям X и Y .

Центры грузов примерно совпадают с точками на изображении ниже. Две верхние точки используются только на финальном заезде. Точка $(450, -145)$ всегда свободна.

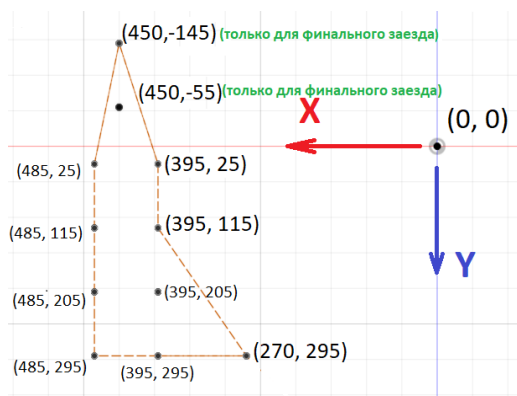


Рис. VI.2.8. Координаты точек хранения грузов

Подзадача СЦ 1. Установка модуля оборудования на беспилотник

Условие

«АЙКАР» без модуля оборудования устанавливается в зоне обслуживания. Участники сообщают, какое количество модулей находится в зоне хранения. Разработчик сообщает, модуль какого типа необходимо установить на «АЙКАР». По команде разработчика участники запускают программу на компьютере СЦ.

Задание считается полностью выполненным, если:

- манипулятор захватил модуль оборудования из зоны хранения и установил на беспилотник; после полной остановки модуля он находится в предназначенном для него креплении;
- модуль не касался других модулей;
- испытание пройдено успешно два раза подряд;
- разработчик посмотрел код и признал его рабочим.

3 балла — в зоне хранения один модуль.

5 балла — в зоне хранения 4 модуля.

7 баллов — в зоне хранения 8 модулей.

Типовые ошибки при выполнении подзадания:

- попытка перемещать захват через запрещенную область;
- одновременное понижение высоты и сжимание захвата;
- использование высоты больше 65 для захвата модулей оборудования.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS0Qg/solutions/СЦ1>.

Решение удобно разбить на три логические части:

- поиск модуля запрашиваемого типа;
- захват модуля манипулятором и перемещение в кузов беспилотника.

Области кадра, в которых могут располагаться модули хранения, заранее определены и не меняются от кадра к кадру. Из-за этого нет необходимости в детектировании модулей оборудования.

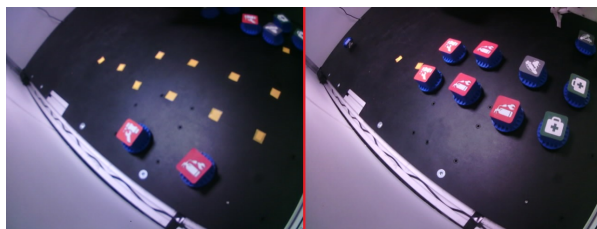


Рис. VI.2.9. Изображения зоны хранения с камер СЦ

Достаточно для каждой области, в которой может находиться модуль оборудования, провести классификацию по пяти возможным классам:

- `empty` — зона хранения пустая;
- `medical kit` — аптечка;
- `fire extinguisher` — огнетушитель;
- `repair tools` — инструменты для ремонта;
- `car repair kit` — авторемонтный набор.

Для классификации удобно использовать нейросетевой классификатор. Ниже представлен пример того, как можно выделить на изображении зоны возможного хранения грузов. По контурам этих областей можно вырезать изображения для датасета.

При сборе датасета необходимо располагать модули в разных комбинациях, чтобы каждый модуль побывал в каждой зоне. Кроме того, следует обратить внимание на левые зоны, т. к. в них модули могут закрывать друг друга. Для более точной классификации нужны все возможные комбинации перекрытия модулей друг другом.

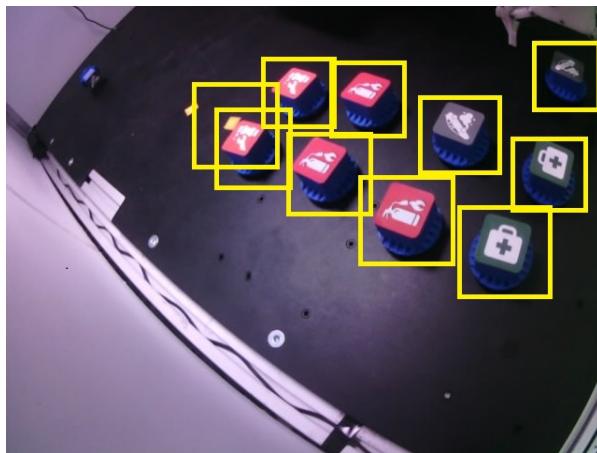


Рис. VI.2.10. Области, соответствующие точкам хранения, для классификации

Ниже представлен пример подготовки датасета для обучения нейронной сети.

```
from sklearn.model_selection import train_test_split
import tensorflow as tf
import numpy as np
import random
import pickle
import cv2
import os
import pandas as pd

csv_path = "*annots.csv" # укажите путь к файлу с аннотациями
image_path = "data_train/" # укажите путь к папке с изображениями для тренировки
table = pd.read_csv(csv_path, delimiter=';')
```



```

# число строк (первая строка с заголовками столбцов не учитывается)
row_num = len(table.axes[0])
col_num = len(table.axes[1]) # число столбцов

# пустые списки для изображений и аннотаций (меток) к ним
data = []
labels = []
# список классов
classes = ["empty", "medical_kit", "fire_extinguisher", "repair_tools",
↳ "car_repair_kit"]

for row in table.itertuples(): # перебираем строки
    image_type = row[2] # тип изображения записан в третьем столбце
    image = cv2.imread(image_path + row[1]) # читаем файл с изображением
    image = cv2.resize(image, (32, 32)) # масштабируем под вход сети
    data.append(image)
    label = [0, 0, 0, 0, 0] # 5, по числу классов
    label[classes.index(image_type)] = 1 # ставим одну единицу
    labels.append(label)

# создаем из списка массив и нормируем изображения на 1
data = np.array(data, dtype="float") / 255
labels = np.array(labels)

# делим подготовленные данные на тренировочную и валидационную часть
(trainX, testX, trainY, testY) = train_test_split(data, labels,
    test_size=0.15, # доля изображений
    ↳ для проверки
    random_state=42)

print("Dataset prepared")

```

В следующем фрагменте кода задается архитектура нейронной сети и параметры ее обучения.

```

# задаем архитектуру нейронной сети
model = Sequential()
inputShape = (32, 32, 3) # размер входящего изображения
chanDim = -1 # последний канал отвечает за цвет

model.add(Conv2D(8, (5, 5), padding="same", input_shape=inputShape))
model.add(Activation("relu"))
model.add(BatchNormalization(axis=chanDim))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(16, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(BatchNormalization(axis=chanDim))
model.add(Conv2D(16, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(BatchNormalization(axis=chanDim))
model.add(MaxPooling2D(pool_size=(2, 2)))

model.add(Conv2D(32, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(BatchNormalization(axis=chanDim))
model.add(Conv2D(32, (3, 3), padding="same"))
model.add(Activation("relu"))
model.add(BatchNormalization(axis=chanDim))
model.add(MaxPooling2D(pool_size=(2, 2)))

```

```

model.add(Flatten())
model.add(Dense(128))
model.add(Activation("relu"))
model.add(BatchNormalization())
model.add(Dropout(0.5))

model.add(Dense(128))
model.add(Activation("relu"))
model.add(Dropout(0.5))

model.add(Dense(16))
model.add(Activation("softmax"))

# Настраиваем оптимизатор
opt = Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999, amsgrad=False)

# компилируем модель
model.compile(loss="categorical_crossentropy", optimizer=opt,
              metrics=["accuracy"])
print("Model compiled")
model.summary() # выводим данные о модели

# запускаем обучение классификатора
EPOCHS = 10
checkpointer = ModelCheckpoint(filepath='*.h5', verbose=1,
                               save_best_only=True)

H = model.fit(trainX, trainY, validation_data=(testX, testY),
              epochs=EPOCHS, batch_size=32,
              shuffle=True,
              callbacks=[checkpointer])
print("Model trained")

```

В фрагменте кода ниже определяется, по каким координатам необходимо переместить манипулятор, чтобы захватить запрашиваемый модуль.

```

import tensorflow as tf
import numpy as np
import random
import cv2
from mcx import mcx
ROBOT_NAME = 'Robot12_1'
BASE_CAMERA = "Cam1"

# Загружаем модель из файла
model = tf.keras.models.load_model("Zone_MultiClass.h5")
print("Model loaded")
classes = ["empty", "medical_kit", "fire_extinguisher", "repair_tools",
          ↪ "car_repair_kit"]

# storage_areas[i][0] - левый верхний угол зоны на изображении
# storage_areas[i][1] - правый нижний угол зоны
# storage_areas[i][2] - координаты точки в системе отсчета манипулятора
storage_areas = [[(523, 248), (563, 288), (485, 295)],
                 [(560, 159), (595, 194), (395, 295)],
                 [(585, 68), (615, 98), (270, 295)],
                 [(429, 204), (467, 242), (485, 205)],
                 [(470, 122), (506, 158), (395, 205)],
                 [(335, 163), (371, 199), (485, 115)],

```

```

[(382, 95), (416, 129), (395, 115)],
[(263, 145), (295, 177), (485, 25)],
[(311, 80), (343, 112), (395, 25)]

# Получаем кадр с камеры ЦЦ
programm = mcx()
frame = programm.getImage(BASE_CAMERA)
need_equip = input("Введите название запрашиваемого модуля оборудования")
for i in range(9): # перебираем зоны хранения
    x1 = storage_areas[i][0][0]
    y1 = storage_areas[i][0][1]
    x2 = storage_areas[i][1][0]
    y2 = storage_areas[i][1][1]
    image_zone = frame[y1:y2, x1:x2] # вырезаем изображение зоны
    image_zone = cv2.resize(image_zone, (32, 32)) / 255
    # сеть принимает на вход изображение с добавленным измерением
    image_zone = np.expand_dims(image_zone, axis=0)
    pred = model.predict(image_zone)
    # интерпретируем выходные данные от сети
    class_index = np.argmax(pred[0])
    zone_status = classes[class_index]
    # проверяем какой модуль обнаружен
    if zone_status == need_equip:
        # если модуль нам подходит - останавливаем дальнейший поиск
        # сохраняем координаты модуля в системе отсчета манипулятора
        X_for_manip = storage_areas[i][2][0]
        Y_for_manip = storage_areas[i][2][1]
        break
    else:
        # если не сработал break
        print("Требуемый модуль оборудования не обнаружен")
        exit()

```

Далее осуществляется перемещение манипулятора к модулю, его захват и транспортировка в кузов беспилотника. Важно отслеживать состояние манипулятора и отдавать команды только тогда, когда манипулятор завершил перемещение.

```

# составим список точек, в которые необходимо перемещать
# манипулятор для захвата груза и его доставки в кузов беспилотника
# [X, Y, высота, состояние захвата] 1 - захват сжат
points_list = [[X_for_manip, Y_for_manip, 160, 0],
               [X_for_manip, Y_for_manip, 65, 0], # опускаем манипулятор над модулем
               [X_for_manip, Y_for_manip, 65, 1], # захватываем модуль
               [X_for_manip, Y_for_manip, 160, 1], # поднимаем модуль
               [270, 295, 160, 1], # перемещаемся к зоне обслуживания
               [180, 385, 160, 1], # перемещаемся к зоне обслуживания
               [(125 + (-175)) / 2, (525 + 725) / 2, 160, 1], # центр зоны
               ↪ обслуживания
               [(125 + (-175)) / 2, (525 + 725) / 2, 160, 0]] # отпускаем модуль

for point in points_list:
    programm.move(ROBOT_NAME, point[0], point[1], point[2], 0, point[3])

# ждем пока манипулятор не сообщит нам, что закончил перемещение
state = 1
while state > 0:
    data = programm.getData() # получаем отчет от манипулятора
    data = data.replace('"', '')
    data = data.replace('\n', '\n')
    lines = data.split('\n')

```

```

for line in lines:
    if line.endswith('#'):
        parts = line.split(':')
        # state == 0 если манипулятор закончил движение
        state = int(parts[2])

```

Подзадача СЦ 2. Демонтаж оборудования с беспилотника

Условие

«АЙКАР» с «пустым» модулем оборудования устанавливается в зоне обслуживания. Участники сообщают, сколько модулей оборудования располагается внутри СЦ. По команде разработчика участники запускают программу на компьютере СЦ.

Задание считается полностью выполненным, если:

1. манипулятор захватил модуль оборудования с беспилотника и перенес на свободное место в зоне хранения;
2. модуль не касался других модулей;
3. испытание пройдено успешно два раза подряд;
4. разработчик посмотрел код и признал его рабочим.

2 балл — зона хранения полностью свободна.

3 балла — в зоне хранения 4 модуля.

4 балла — в зоне хранения 8 модулей.

Типовые ошибки при выполнении подзадания:

- попытка перемещать захват через запрещенную область;
- попытка попасть в зону обслуживания минуя «транзитные» точки;
- захват модуля оборудования из кузова на высоте 160;
- отсутствие детектирования модуля в кузове беспилотника.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/СЦ2>.

От предыдущей эта подзадача отличается необходимостью детектировать положение беспилотника в зоне разгрузки для того, чтобы расположить захват прямо над модулем оборудования в кузове. Определять свободное место для захваченного груза можно тем же способом, что и ранее — нейросетевым классификатором.

Кроме беспилотного автомобиля в зоне обслуживания нет никаких посторонних объектов, соответственно, единственный объект синего цвета — модуль оборудования. Проще всего детектировать его по цвету, реализация детектирования представлена ниже.

```

# Получаем кадр с камеры СЦ
programm = mcx()
frame = programm.getImage(CAR_CAMERA)
frame = frame[155:469, 365:339] # вырезаем зону обслуживания из кадра
# бинаризуем картинку - белыми будут только синие пиксели

```

```

frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
binary = cv2.inRange(frame, (90, 148, 0), (156, 255, 255))
# находим контуры
contours, hierarchy = cv2.findContours(binary, cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE)
if contours:
    # сортируем контуры по их площади
    contours = sorted(contours, key=cv2.contourArea, reverse=True)
    contour = contours[0] # выбираем самый большой контур
    x, y, w, h = cv2.boundingRect(contour)
    # находим центр ограничивающего контур прямоугольника
    xc = x + w//2
    yc = y + h//2
else:
    print("Зона обслуживания пуста")
    exit(0)

```

Координаты модуля оборудования на изображении необходимо перевести в координаты манипулятора. Камера почти не искажает изображение и находится над зоной обслуживания. Перевести одни координаты в другие можно пропорционально. Погрешность из-за искажений и наклона объектива камеры будет составлять максимум 1 см, что допустимо из-за большой ширины захвата.

```

# x_img от 339 до 365 соответствует y_man от 525 до 725]
# y_img от 155 до 469 соответствует x_man от -175 до 125]
yc_equip_man = (365-339)/(xc-339)*(725-525)+525
xc_equip_man = (469-155)/(yc-155)*(125+175)-175

```

После того как координаты найдены, проводим поиск пустой точки хранения на складе. Используем метод из СЦ 1. В результате мы получим `x_empty_point` и `y_empty_point`. Как и в СЦ 1, составляем список точек для манипулятора. Перемещение по точкам осуществляется аналогично СЦ 1.

```

# составим список точек, в которые необходимо перемещать
# манипулятор для захвата груза и его доставки в кузов беспилотника
# [X, Y, высота, состояние захвата] 1 - захват сжат
# из первой точки можно попасть в любую другую не пересекая запрещенную область
points_list = [[395, 115, 160, 0],
               [270, 295, 160, 0], # перемещаемся к зоне обслуживания
               [180, 385, 160, 0], # перемещаемся к зоне обслуживания
               [-175, 525, 160, 0], # перемещаемся к дальний угол зоны
               [xc_equip_man, yc_equip_man, 160, 0], # над грузом
               [xc_equip_man, yc_equip_man, 147, 0], # опускаем захват
               [xc_equip_man, yc_equip_man, 147, 1], # захватываем модуль
               [xc_equip_man, yc_equip_man, 160, 1], # поднимаем модуль
               [-175, 525, 160, 1], # перемещаемся к дальний угол зоны
               [180, 385, 160, 1], # перемещаемся к зоне обслуживания
               [270, 295, 160, 1], # перемещаемся к зоне обслуживания
               [395, 115, 160, 1], # "центральная точка"
               [x_empty_point, y_empty_point, 160, 1], # над грузом
               [x_empty_point, y_empty_point, 65, 1], # над грузом
               [x_empty_point, y_empty_point, 65, 0]] # отпускаем груз

```

Подзадача СЦ 3. Обмен данными с беспилотником и его обслуживание

Условие

Участники сообщают, какое количество модулей находится в зоне хранения. Оператор сообщает, какой тип оборудования запрашивает беспилотник. В зону обслуживания устанавливается «АЙКАР» с «пустым» модулем. По команде разработчика участники запускают программу на компьютере СЦ и программу-имитатор беспилотника на любом компьютере команды.

Последовательность сообщений следующая:

- сообщение о прибытии в зону обслуживания;
- запрос демонтажа оборудования;
- запрос установки оборудования конкретного типа;
- сообщение об освобождении зоны обслуживания.

Вы можете пропустить сообщения — отправлять не все, но изменять их порядок нельзя.

Подзадача считается выполненной, если:

- в терминале СЦ выводятся сообщения от имитатора беспилотника;
- в терминал компьютера с запущенным имитатором выводятся сообщения от СЦ;
- запросы беспилотника сопровождаются действиями манипулятора: демонтажом и установкой оборудования;
- модули не касаются друг друга;
- разработчик просмотрел код запущенных программ и признал его рабочим.

3 балла — в зоне хранения один модуль.

6 баллов — в зоне хранения 4 модуля.

9 баллов — в зоне хранения 8 модулей.

Типовые ошибки при выполнении подзадания:

- получение всех сообщений сразу, без выполнения соответствующих запросам действий;
- обе программы пытаются подключиться к инициализированному сокету;
- отсутствие ответных сообщений на запросы.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS0Qg/solutions/СЦ3>.

В качестве имитатора беспилотного автомобиля используется доработанный участок кода из БПА 3.

```
import socket
```

```
HOST = "172.16.65.69" # ip сервера
PORT = 2002 # порт сервера
```

```

# устанавливаем соединение - подключаемся к сокету
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as soc:
    soc.connect((HOST, PORT))

    soc.sendall("arrived".encode())
    # ждем в цикле, пока не получим отзыв
    while True:
        msg = soc.recv(1024).decode()
        if msg.startswith("arrived-Ok"):
            print(f"Получено: {msg.decode()}")
            break

    soc.sendall("dismantling request".encode())
    # ждем в цикле, пока не получим отзыв
    while True:
        msg = soc.recv(1024).decode()
        if msg.startswith("dismantling-Ok"):
            print(f"Получено: {msg.decode()}")
            break

    need_equip = input("Введите тип запрашиваемого оборудования.\n" +
        "Возможные типы: medical_kit, fire_extinguisher, repair_tools,
        ↪ car_repair_kit.")
    msg = "equipment request:"+need_equip
    soc.sendall(msg.encode())
    # ждем в цикле, пока не получим отзыв
    while True:
        msg = soc.recv(1024).decode()
        if msg.startswith(need_equip+"-Ok"):
            print(f"Получено: {msg.decode()}")
            break

    soc.sendall("leaving the service area".encode())
    print(f"Получено: {soc.recv(1024).decode()}")

```

Для реагирования на сообщения имитатора к коду из СЦ 1 и СЦ 2 добавляется прием и анализ данных от имитатора. Детектирование модуля в кузове беспилотника, поиск свободных мест и модулей на хранении, захват и перемещение модулей осуществляется как и ранее.

```

import socket

# инициализируем сокет
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('0.0.0.0', 2002))
sock.listen(1)
print('Ожидание подключения...')
soc, address = sock.accept()
print('Подключен к', address)

# ждем в цикле, пока не получим сообщение о прибытии
while True:
    msg = soc.recv(1024).decode()
    if msg.startswith("arrived-Ok"):
        print(f"Получено: {msg.decode()}")
        soc.sendall("arrived-Ok".encode())
        break

# ждем следующий запрос
while True:

```

```

msg = soc.recv(1024).decode()
if msg.startswith("dismantling request"):
    print(f"Получено: {msg.decode()}")
    # здесь располагаем код из СЦ 2,
    # отвечающий за демонтаж модуля оборудования с беспилотника
    # ....
    # как только демонтаж завершен, отправляем сообщение об этом
    soc.sendall("dismantling-Ok".encode())
    break

# ждем следующие запросов
while True:
    msg = soc.recv(1024).decode()
    if msg.startswith("equipment request"):
        print(f"Получено: {msg.decode()}")
        parts = msg.split(":")
        need_equip = parts[1]
        # здесь располагаем код из СЦ 1,
        # отвечающий за установку запрашиваемого модуля оборудования на беспилотник
        # ....
        # как только установка завершена, отправляем сообщение об этом
        msg = need_equip + "-Ok"
        soc.sendall(msg.encode())
        break

# ждем сообщения о освобождении сервисной зоны
while True:
    msg = soc.recv(1024).decode()
    if msg.startswith("leaving the service area"):
        print(f"Получено: {msg.decode()}")
        print("Отключаемся")
        break

soc.close()

```

Подзадачи квадрокоптера (БПЛА)

Подзадача БПЛА 1. Перелет над городом

Условие

Квадрокоптер устанавливается на стартовую площадку. По команде разработчика участники запускают программу. Квадрокоптер должен взлететь, по порядку посетить все зоны интереса и приземлиться на посадочной площадке.

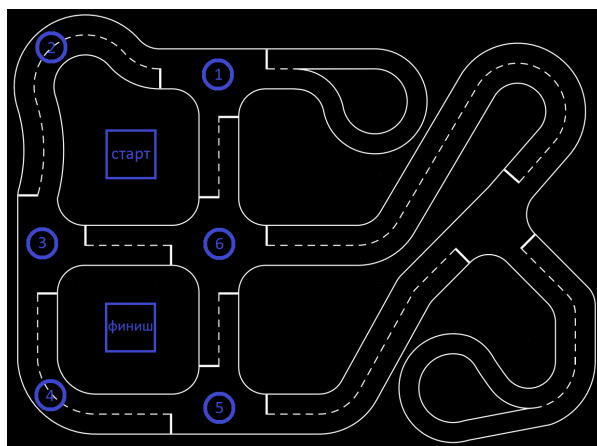


Рис. VI.2.11. Точки маршрута квадрокоптера

Подзадача считается полностью решенной, если квадрокоптер последовательно выполнил следующие действия:

1. взлетел с стартовой площадки на произвольную высоту, выполнил зависание и активировал мигающую светодиодную индикацию (цвет — зеленый, интервал мигания 1 с);
2. последовательно посетил все точки интереса, в каждой точке завис и активировал мигающую светодиодную индикацию (цвет — синий, интервал мигания 0,4 с);
3. приземлился и после полной остановки остался в пределах посадочной площадки.

1 балл — за каждый выполненный элемент до первого невыполненного: взлет и шесть точек.

5 баллов — посадка.

Типовые ошибки при выполнении подзадания:

- восприятие обработчиков событий как последовательно выполняющихся в теле цикла операторов;
- отсутствия сигналов светодиодной подсветкой.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS0Qg/solutions/БПЛА1>.

Квадрокоптер определяет свои координаты по ультразвуковой системе позиционирования, для участников взаимодействие с ней аналогично работе с GPS. Участники знают трехмерные координаты квадрокоптера в каждый момент времени, однако координаты определяются с погрешностью ± 20 см. Управление перемещением квадрокоптера осуществляется через указание координат точек, которые квадрокоптер должен посетить.

Координаты точек, составляющих маршруты перелета, изначально неизвестны. Определить эти координаты можно множеством способов. Наиболее технически верный — пронести квадрокоптер по полигону и в каждой точке маршрутов собрать несколько значений трехмерных координат. Проанализировав статистические данные, можно вывести наиболее вероятные значения координат точек, в программе необходимо указывать именно их.

Для получения координат от квадрокоптера используется следующий код.

```
import rospy
from time import sleep
from gs_navigation import NavigationManager
from gs_board import BoardManager

rospy.init_node("navigation_test_node")
board = BoardManager()
navigation = NavigationManager()

time_delta = 0.1
cnt = 10

def coords():
    x = []
    y = []
    z = []
    for i in range(cnt):
        pos = navigation.lps.position()
        x.append(pos[0])
        y.append(pos[1])
        z.append(pos[2])
        sleep(time_delta)

    return sum(x) / len(x), sum(y) / len(y), sum(z) / len(z)

n = 0
while not rospy.is_shutdown() and input() == '':
    if board.runStatus():
        print(navigation.lps.yaw())
        x, y, z = coords()
        print(x, y, z)
        with open('points.txt', 'a') as f:
            f.write(str(n) + ', ' + str(round(x, 3)) + ', ' + str(round(y, 3)) + ', ' +
                str(round(z, 3)) + '\n')
            n += 1
```

Координаты точек записываются в отдельный текстовый файл.

```
2.55 4.68
0.6 5.16
0.18 2.35
0.26 0.15
2.47 0.16
2.54 2.35
1.3 1.17
```

Далее представлена программа управления полетом квадрокоптера.

```
import rospy
from gs_flight import FlightController, CallbackEvent
```

```

from gs_board import BoardManager
from gs_module import BoardLedController
from gs_navigation import NavigationManager
from time import sleep
import math
import threading

rospy.init_node("task_1") # инициализируем ноду

height = 1.5 # высота перелетов над городом
local_height = 0.5 # высота перед приземлением

# параметры для светодиодной подсветки
color = (0, 255, 0)
period = 1
led = BoardLedController()

# функция мигания подсветкой
def blink():
    global color, period, not_finish
    while not_finish:
        led.changeAllColor(*color)
        sleep(period)
        led.changeAllColor(0, 0, 0)
        sleep(period)

# выполняем мигание в отдельном потоке!
blink1 = threading.Thread(target=blink)

# составляем список координат точек, которые необходимо посетить
coordinates = []
with open('task1_points.txt', 'r') as f:
    for line in f.readlines():
        x, y = map(float, line.split())
        coordinates.append([x, y, height])

run = True # переменная отвечающая за работу программы
position_number = 0 # счетчик пройденных точек

def callback(event): # функция обработки событий Автопилота
    global ap
    global run
    global coordinates
    global position_number
    global color, period, not_finish, blink1

    event = event.data
    if event == CallbackEvent.ENGINES_STARTED:
        # блок обработки события запуска двигателя
        print("engine started")
        ap.takeoff() # отдаем команду взлета
    elif event == CallbackEvent.TAKEOFF_COMPLETE:
        # блок обработки события завершения взлета
        print("takeoff complite")
        not_finish = 1
        blink1.start() # запускаем мигание
        sleep(6)
        color = (0, 0, 0)
        position_number = 0
        print(f'go to point 0')

```

```

# отдаем команду полета в точку
ap.goToLocalPoint(coordinates[position_number][0],
                  coordinates[position_number][1],
                  coordinates[position_number][2])

elif event == CallbackEvent.POINT_REACHED:
    # блок обработки события достижения точки
    print(f"point {position_number} reached")
    if position_number != len(coordinates)-1: # если точка не последняя - мигаем
        color = (0, 0, 255)
        period = 0.4
        sleep(4)
        color = (0, 0, 0)
        period = 1
    position_number += 1 # увеличиваем счетчик точек
    if position_number < len(coordinates):
        # если точка не последняя - летим в следующую
        print(f'go to point {position_number}')
        ap.goToLocalPoint(coordinates[position_number][0],
                          coordinates[position_number][1],
                          coordinates[position_number][2])
    else:
        # если точка последняя - приземляемся
        if not_finish:
            not_finish = 0
            print('landing')
            sleep(2)
            ap.goToLocalPoint(coordinates[-1][0], coordinates[-1][1], 1.0)
        else:
            sleep(2)
            ap.landing() # отдаем команду посадки
elif event == CallbackEvent.COPTER_LANDED:
    # блок обработки события приземления
    ap.disarm()
    print("finish programm")
    run = False # прекращаем программу

board = BoardManager() # создаем объект бортового менеджера
navigation = NavigationManager()
ap = FlightController(callback) # создаем объект управления полета

once = False # переменная отвечающая за первое вхождение в начало программы

print(navigation.lps.position(), navigation.lps.yaw())
sleep(1)
print(navigation.lps.position(), navigation.lps.yaw())
sleep(1)
print(navigation.lps.position(), navigation.lps.yaw())
sleep(8)

while not rospy.is_shutdown() and run:
    # цикл работает пока ROS включен или пока переменная тип равна True
    # цикл необходим для того чтобы можно было получать события от автопилота
    if board.runStatus() and not once: # проверка подключения RPi к Пионеру
        print("start programm")
        ap.preflight() # отдаем команду выполнения предстартовой подготовки
        once = True

lps = navigation.lps.position()

```

```

if position_number < len(coordinates):
    print(round(math.hypot(coordinates[position_number][0] - lps[0],
                        coordinates[position_number][1] - lps[1],
                        height - lps[2]), 3))

sleep(2)

```

Подзадача БПЛА 2. Обнаружение происшествий

Условие

Квадрокоптер устанавливается на стартовую площадку. В случайные зоны, обозначенные зеленым на изображении ниже, устанавливаются от одной до четырех моделей ЧС. В одной зоне не может быть более одной ЧС.

По команде разработчика участники запускают программу. Квадрокоптер должен пролететь над городом, обнаружить ЧС и вывести их координаты.

Подзадача считается выполненной, если:

- квадрокоптер взлетел с стартовой площадки на произвольную высоту, выполнил зависание и активировал мигающую светодиодную индикацию (цвет – зеленый, интервал мигания 1 с);
- при обнаружении ЧС выполнял зависание и активировал подсветку соответствующего типу ЧС цвета;
- при обнаружении ЧС в терминал SSH соединения выводил тип и координаты ЧС;
- приземлился и после полной остановки остался в пределах посадочной площадки.

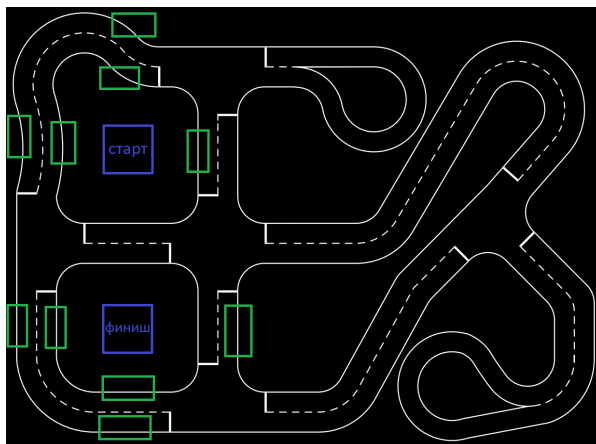


Рис. VI.2.12. Зоны возможного возникновения ЧС

3 балла — за каждый верно распознанный тип ЧС.

1 балл — за каждые верно определенные координаты ЧС.

Штраф 2 балла — за каждый ошибочно распознанный тип ЧС.

Штраф 1 балл — за каждые неверно определенные координаты ЧС.

4 балла — за успешно выполненную посадку.

Типовые ошибки при выполнении подзадания:

- сбор датасета на высоте, не соответствующей высоте полета;
- попытки детектировать ЧС сразу в нескольких зонах.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПЛА2>.

К коду, полученному в БПЛА 1, необходимо добавить алгоритм детектирования ЧС.



Рис. VI.2.13. Объекты, моделирующие ЧС

Объекты, моделирующие ЧС, имеют изменяющуюся форму и пересекающиеся между собой цвета. Исправные и поврежденные трубы отличаются друг от друга столбом пара и наличием черной «пробоины», исправные трубы не должны определяться как ЧС. Люди, которым стало плохо, одеты в разную одежду и меняют позы.

Необходимо стабильно детектировать объекты при разных уровнях освещенности, ракурсах и фонах. Для решения поставленной задачи уместно применить нейросетевой детектор. Возможно использовать алгоритмы без машинного обучения, но они будут работать менее точно и требовать кратно больше времени для отладки.

Ниже представлена функция для сохранения изображения с камеры, используется для сбора датасета.

```
def take_photo(n):
    print(f'yaw: {navigation.lps.yaw()}')
    for i in range(num_photos):
        name = f'{position_number}_{n}_{i}.jpg'

        cv2.imwrite(dir_path + '/' + name, frame)
        print(f'saved photo {name}')
```

```
sleep(period)
```

Описанная выше функция должна вызываться в обработчике завершения перелета в точку.

```
elif event == CallbackEvent.POINT_REACHED:
    # блок обработки события достижения точки
    if position_number < len(coordinates) - 1:
        if point_stage == 0:
            # ...
            take_photo(1)          # делаем снимок
            print('Foto maked')
            # ...
            # летим к следующей точке
            ap.goToLocalPoint(coordinates[position_number][0],
                               coordinates[position_number][1],
                               ↪ local_height)

            point_stage += 1
        elif point_stage == 1:
            sleep(2)
            take_photo(2)
            print('Foto maked')
            # ...
# ...
```

Как и в БПЛА 1. Необходимо вычислить координаты центров зон возможных ЧС. При достижении зоны следует передать изображение с камеры в детектор. Если обнаружено ЧС, то считаем, что его координаты совпадают с координатами квадрокоптера. Для детектирования ЧС во время полета вносим изменения в обработчик завершения перелета в точку.

```
elif event == CallbackEvent.POINT_REACHED: # блок обработки события достижения точки
# проверяем не закончились ли точки в списке
    if position_number < len(coordinates) - 1:
        sleep(0.5)
        print('classification')
        # отдаем изображение с камеры детектору
        # берем только центральную часть кадра.
        # чтобы ЧС из соседних зон не обнаруживались детектором
        img = frame[140:340, 200:440]
        classes, scores, bboxes = yoloCVmodel.detect(img, 0.8, 0.4)
        if classes:
            # выводим тип и координаты ЧС
            print("Detect :" + type_dict[classes[0]])
            print("X: " + str(coordinates[position_number][0]))
            print("Y: " + str(coordinates[position_number][1]))
            # мигаем подсветкой
            led.changeAllColor(*color_dict[classes[0]])
            sleep(5)
            # выключаем подсветку
            led.changeAllColor(0, 0, 0)
            # летим к следующей точке
            position_number += 1
            print(f'go to point {position_number}')
            ap.goToLocalPoint(coordinates[position_number][0],
                               ↪ coordinates[position_number][1], height)
            point_stage = 0
```

```

else:
    if not_finish:
        not_finish = 0
        print('landing')
        sleep(2)
        ap.goToLocalPoint(coordinates[-1][0], coordinates[-1][1], 0.8)
    else:
        sleep(2)
        ap.landing() # отдаем команду посадки

```

Подзадача БПЛА 3. Обнаружение ЧС и оповещение о них

Условие

Аналогично предыдущей подзадаче. Теперь тип и координаты обнаруженных ЧС необходимо выводить на компьютере без SSH соединения с квадрокоптером. Квадрокоптер обязательно должен приземлиться и остаться в устойчивом положении на крыше.

2 балла — за каждое полностью верное сообщение в терминале компьютера без SSH соединения с квадрокоптером.

Штраф 1 балл — за каждое неверное сообщение.

Типовые ошибки при выполнении подзадания:

- сбор датасета на высоте, не соответствующей высоте полета;
- попытки детектировать ЧС сразу в нескольких зонах.

Решение

Файлы с решениями подзадач находятся в папке: <https://disk.yandex.ru/d/BCF4uFE7BsS00g/solutions/БПЛА3>.

Метод детектирования ЧС и получение их координат остается таким же, как в БПЛА 2. Найденные ЧС и их координаты следует сохранять в список и после облета всех точек передать список беспилотному автомобилю. После приземления следует выполнить (см. ниже).

```

import socket

# инициализируем сокет
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(('0.0.0.0', 2004))
sock.listen(1)
print('Ожидание подключения...')
soc, address = sock.accept()
print('Подключен к', address)

# emergency_list - пополняется при обнаружении ЧС
# emergency = [type_dict[classes[0]],
#               str(coordinates[position_number][0]),
#               str(coordinates[position_number][1])]
# emergency_list.append(emergency)

# ждем в цикле, пока не получим запрос списка ЧС
while True:

```



```

msg = soc.recv(1024).decode()
if msg.startswith("get_emergency_list"):
    print(f"Получено: {msg.decode()}")
    msg = ""
    for i in emergency_list:
        # формируем сообщение с разделителями
        msg = i[0] + ";" + i[1] + ";" + i[2] + ";"
    soc.sendall(msg.encode())
    break
soc.close()

```

Для запроса списка ЧС и вывода его на экран используется следующий код (см. ниже).

```

import socket

HOST = "172.16.65.73" # ip сервера
PORT = 2004 # порт сервера

# устанавливаем соединение - подключаемся к сокету
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as soc:
    while soc.connect((HOST, PORT)):
        soc.sendall("get_emergency_list".encode())
        # ждем в цикле, пока не получим ответ
        while True:
            msg = soc.recv(1024).decode()
            print(f"Получено: {msg.decode()}")
            break

```

Материалы для подготовки

1. Программа подготовки к профилю Автономные транспортные системы Национальной технологической олимпиады https://avt.global/nto_program.
2. Задачи профиля Автономные транспортные системы 2022–2023 <https://ntcointest.ru/docs/ats-assignments1.pdf>.
3. Задачи профиля Автономные транспортные системы 2021–2022 <https://ntcointest.ru/docs/ats-assignments.pdf>.
4. Задачи профиля Автономные транспортные системы 2020–2021 https://drive.google.com/file/d/1jJwI_5MgX-wvmwK7WUh_mhQrEcFAhB_K/view.
5. Руководство по OpenCV https://docs.opencv.org/4.x/d9/df8/tutorial_robot.html.
6. SDK для программирования квадрокоптера «Пионер Макс» https://github.com/geoscan/geoscan_pioneer_max.

Критерии определения победителей и призеров

Первый отборочный этап

В первом отборочном этапе участники решали задачи предметного тура по двум предметам: физике и информатике и инженерного тура. В каждом предмете максимально можно было набрать 100 баллов, в инженерном туре 100 баллов. Для того, чтобы пройти во второй этап участники должны были набрать в сумме по обоим предметам не менее 5 баллов, независимо от уровня.

Второй отборочный этап

Количество баллов, набранных при решении всех задач второго отборочного этапа, суммируется. Победители второго отборочного этапа должны были набрать не менее 56 баллов, независимо от уровня.

Заключительный этап

Индивидуальный предметный тур

- физика — максимально возможный балл за все задачи — 100 баллов;
- информатика — максимально возможный балл за все задачи — 100 баллов.

Командный инженерный тур

Команды заключительного этапа получали за командный инженерный тур от 0 до 200 баллов: команда, набравшая наибольшее число баллов среди других команд, становилась командой-победителем.

Все результаты команд нормировались по формуле:

$$\frac{200 \times x}{MAX},$$

где x — число баллов, набранных командой,

MAX — число баллов, максимально возможное за инженерный тур.

В заключительном этапе олимпиады индивидуальные баллы участника складываются из двух частей, каждая из которых имеет собственный вес: баллы за индивидуальное решение задач по предметам (физика, информатика) с весом $K_1 = 0,15$ каждый предмет и баллы за командное решение задач инженерного тура с весом $K_2 = 0,7$.

Итоговый балл определяется по формуле:

$$S = K_1 \cdot (S_1 + S_2) + K_2 \cdot S_3,$$

где S_1 — балл первой части заключительного этапа по физике (предметный тур) в стобалльной системе ($S_{1 \text{ макс}} = 100$);

S_2 — балл первой части заключительного этапа по информатике (предметный тур) в стобалльной системе ($S_{2 \text{ макс}} = 100$);

S_3 — итоговый балл инженерного командного тура в двухсотбалльной системе ($S_{3 \text{ макс}} = 200$).

Итого максимально возможный индивидуальный балл участника заключительного этапа равен 170 баллов.

Критерий определения победителей и призеров

Чтобы определить победителей и призеров (независимо от класса) на основе индивидуальных результатов участников, был сформирован общий рейтинг всех участников заключительного этапа. С начала рейтинга были выбраны 2 победителя и 7 призеров (первые 25% участников рейтинга становятся победителями или призерами, из них первые 8% становятся победителями, оставшиеся — призерами).

Критерий определения победителей и призеров (независимо от уровня)

Категория	Количество баллов
Победители	52,75 и выше
Призеры	От 41,35 до 52,15

Работа наставника после НТО

Участие школьника в Олимпиаде может завершиться после любого из этапов: первого или второго отборочных либо после заключительного этапа. В каждом случае после завершения участия наставнику необходимо провести с учениками рефлексию — обсудить полученный опыт и проанализировать, что позволило достичь успеха, а что привело к неудаче.

Важная задача наставника — превратить неудачу в инструмент будущего успеха. Для этого необходимо вместе с учениками наметить план развития компетенций и подготовки к будущему сезону Олимпиады. Подробные материалы о проведении рефлексии представлены в курсе «Наставник НТО»: <https://academy.sk.ru/events/310>.



Наставнику важно проинформировать руководство образовательного учреждения, если его учащиеся стали финалистами, призерами и победителями. Публичное признание высоких результатов дополнительно повышает мотивацию.

В процессе рефлексии с учениками, не ставшими призерами или победителями, рекомендуется уделить особое внимание особенностям командной работы: распределению ролей, планированию работы, возникающим проблемам. Для этого могут использоваться опросники для самооценки собственной работы и взаимной оценки участниками других членов команды (P2P). Такие опросники могут выявить внутренние проблемы команды, для решения которых в план подготовки можно добавить мероприятия, направленные на ее сплочение.

Стоит рассказать, что в истории НТО было много примеров, когда не победив в первый раз, на следующий год участники показывали впечатляющие результаты, одержав победу сразу в нескольких профилях. Конечно, важно отметить, что так происходит только при учете прошлых ошибок и подготовке к Олимпиаде в течение года.

Еще одним направлением работы наставника после НТО может стать создание кружка по направлению профилей или по формированию необходимых компетенций: программирование, электроника, робототехника, 3D-моделирование и т. п. Формат подобного кружка может быть различным: короткие модули, дополнительные курсы, факультативы, группы дополнительного образования. Для создания кружков можно воспользоваться образовательными программами, опубликованными на сайте НТО: <https://ntcontest.ru/mentors/education-programs/>.



Важным фактором успешного участия в следующих сезонах НТО может стать поддержка родителей учеников. Знакомство с родителями помогает наставнику продемонстрировать им важность компетенций, развиваемых в процессе участия в НТО, для будущего образования и карьеры школьников. Поддержка родителей помогает мотивировать участников и позволяет выделить необходимое время на занятия в кружке.

С участниками-выпускниками наставнику рекомендуется обсудить их дальнейшее профессиональное развитие и его связь с выбранными профилями НТО. Отдельно можно обратить внимание на льготы для победителей и призеров, предлагаемые в вузах с интересующими ученика направлениями. Кроме того, ряд вузов предлагает льготы для всех финалистов НТО, а также учитывает результаты Конкурса цифровых портфолио «Талант НТО».

